

# ENGINEERING NOTEBOOKS

**Understand the mechanism. Trace the failure. Explain the tradeoff.**

33 notebooks | full reading edition | 27 September 2026

This book contains the current teaching material, worked code listings, exercises and known limitations. Browser interactions have static worked equivalents. Running labs requires the documented software; reading does not.

# Choose your route

Notebook 1 - Agents, LangChain, LangGraph, Deep Agents, and MCP

Notebook 2 - Python for AI, ML, and Generative AI

Notebook 3 - Java and Spring Boot, From Objects to Services

Notebook 4 - React and Frontend Engineering

Notebook 5 - Full-Stack Java, Python, React, and AI Engineering

Notebook 6 - Algorithms, Design Principles, and Patterns

Notebook 7 - Agents that survive mistakes and restarts

Notebook 8 - A remote MCP service you can inspect and test

Notebook 9 - Python engineering, mathematics, and ML from scratch

Notebook 10 - Java and Spring beyond the happy path

Notebook 11 - React, TypeScript, browser behavior and real tests

Notebook 12 - Trees, graphs, backtracking and dynamic programming

Notebook 13 - All 23 classic design patterns, implemented and challenged

Notebook 14 - Build and defend a complete connected system

Notebook 15 - Graded interviews, debugging rounds and answer keys

16 - Real models: from a talking helper to a bounded agent

17 - Identity and deployment: who are you, and where does the app live?

18 - Many workers, one trustworthy result

19 - Seeing failures and testing the whole experience

20 - Deeper algorithms and mathematics, one small step at a time

21 - Practice that measures understanding

22 - Kafka: a shared notebook that many teams can read

23 - Docker: pack the lunchbox before sending it to school

24 - Kubernetes: a caretaker for running applications

25 - Cache management: keeping photocopies useful

26 - Real-time applications: messages that survive disconnection

27 - Reliability: prove what happens when things go wrong

28 - Put the pieces together: a full-stack engineering capstone

29 - Databases: the organised memory of your application

30 - RAG: connect the question to the right evidence

Notebook 31 - Java, Spring Batch and microservices interview masterclass

Notebook 32 - Load balancing: from a playground queue to production traffic

Notebook 33 - The launch room: cloud, AI quality, databases and recovery

Static equivalents of interactive experiments



### 3. Agent types and how each works

These are useful categories, not one universal taxonomy. Classical AI categories and LLM workflow patterns describe different aspects and can overlap.

| Type                   | Child-sized example                                | Procedure                                              | Main limitation                  |
|------------------------|----------------------------------------------------|--------------------------------------------------------|----------------------------------|
| Simple reflex          | Turn on a lamp if dark                             | Match current observation to a rule                    | Ignores hidden history           |
| Model-based reflex     | Remember whether a room was cleaned                | Update internal state, then apply rules                | Internal model can be wrong      |
| Goal-based             | Find a path to the library                         | Search actions toward a goal                           | Search can be expensive          |
| Utility-based          | Pick a route balancing time and tolls              | Maximize a defined preference score                    | Bad utility gives bad behavior   |
| Learning agent         | Improve route estimates from trips                 | Update predictions or policy from feedback             | Feedback may be biased           |
| Tool-calling LLM agent | Look up a book, then answer                        | Model proposes tool, harness executes, model continues | Invalid or repeated calls        |
| ReAct-style loop       | Alternate information gathering and action         | Observe, choose an action, inspect result              | Loops or unreliable explanations |
| Plan-and-execute       | Make a study plan, complete subtasks               | Plan, execute steps, replan if necessary               | Plans become stale               |
| Router                 | Send arithmetic to calculator, policy to retrieval | Classify need, select specialist                       | Misrouting                       |
| Supervisor             | Assign work to subject tutors                      | Delegate, collect, verify, synthesize                  | Coordination overhead            |
| Evaluator-optimizer    | Draft an answer, critique, revise                  | Generate, score, improve within a cap                  | Self-critique may repeat errors  |
| Human-assisted         | Propose a reservation for review                   | Pause with exact action, validate response, resume     | Poor approval context            |

#### Lab: deterministic representatives

This offline simulation makes the decision rules visible. It does not contain a trained model or simulate actual intelligence.

```
# lab: agent_types
def reflex(dark):
    return "lamp on" if dark else "lamp off"

def remember_and_act(previous_clean, observed_dirt):
    clean = previous_clean and not observed_dirt
    return "rest" if clean else "clean room"

def utility_route(routes):
    return min(routes, key=lambda r: r["minutes"] + 2*r["toll"])[ "name" ]

def route_question(question):
    return "calculator" if question.startswith("add ") else "catalog"

assert reflex(True) == "lamp on"
assert remember_and_act(True, True) == "clean room"
assert utility_route([{"name": "A", "minutes": 10, "toll": 4},
                      {"name": "B", "minutes": 14, "toll": 0}]) == "B"
assert route_question("add 2 3") == "calculator"
print("Four agent decision rules passed")
```

if picks a branch. The memory rule combines the previous belief with a new observation. The utility rule assigns toll a weight of two minutes per unit, then chooses the minimum cost. The router intentionally uses a narrow rule so you can predict its behavior; a learned router would need error evaluation.

**Exercise:** Increase toll weight to zero. **Answer:** Route A wins because only travel minutes remain. Changing the objective changes behavior even with identical observations.

## Lab: goals, learning, plans, and bounded revision

These small deterministic examples expose each procedure. The running-average learner genuinely updates an estimate from observations, but it is not a neural model. The planner and reviewer are explicit rules, not an LLM.

```
# lab: agent_goal_learning_plan
from collections import deque

def goal_agent(graph, start, goal):
    queue = deque([(start, [start])])
    seen = {start}
    while queue:
        place, route = queue.popleft()
        if place == goal:
            return route
        for neighbor in graph.get(place, []):
            if neighbor not in seen:
                seen.add(neighbor)
                queue.append((neighbor, route + [neighbor]))
    return None

def learn_reward(rewards):
    mean = 0.0
    for count, reward in enumerate(rewards, 1):
        mean += (reward - mean) / count
    return mean

def plan_and_execute(topic):
    plan = ["define", "example", "check"]
    handlers = {"define": lambda: f"Define {topic}",
                "example": lambda: f"Show a small {topic} example",
                "check": lambda: "Ask the learner to explain it"}
    return [handlers[step]() for step in plan]

def revise_until_clear(draft, max_rounds=2):
    for _ in range(max_rounds):
        if "Example:" in draft:
            return draft
        draft += " Example: two plus three equals five."
    raise RuntimeError("review budget exhausted")

assert goal_agent({"home":["road"], "road":["library"]},
                  "home", "library") == ["home","road","library"]
assert learn_reward([2,4,6]) == 4.0
assert len(plan_and_execute("gradients")) == 3
assert "Example:" in revise_until_clear("Explain addition.")
print("Goal search, estimate learning, plan execution, and revision verified")
```

Goal search uses a queue and remembers visited places. Reward learning moves the estimate by the new error divided by observation count, giving the arithmetic mean. The plan lists dependencies in a fixed order. The revision loop has a transparent acceptance rule and a cap. Real evaluators must check correctness rather than merely the presence of the word Example. Supervisor and human-assisted patterns appear in the parallel and interrupt labs below.

## 4. Build a small agent harness from scratch

The harness needs more than a loop: a tool allowlist, input validation, a budget, error handling, and a termination condition. Production also needs caller authorization, durable state, timeouts, audit records, and side-effect protections. The following toy demonstrates the central loop without network access.

```
# lab: bounded_harness
def add(a: int, b: int) -> int:
    if type(a) is not int or type(b) is not int:
        raise ValueError("integer arguments required")
    return a + b

def run_scripted_agent(decisions, allowed_tools, max_steps=3):
    observations = []
```

```

for step, decision in enumerate(decisions, start=1):
    if step > max_steps:
        raise RuntimeError("step budget exceeded")
    if decision["kind"] == "finish":
        return {"answer": decision["answer"], "observations": observations}
    if decision["kind"] != "tool":
        raise ValueError("unknown decision kind")
    name = decision["name"]
    if name not in allowed_tools:
        raise PermissionError("tool not allowed")
    result = allowed_tools[name](**decision["arguments"])
    observations.append({"tool": name, "result": result})
raise RuntimeError("agent did not finish")

decisions = [
    {"kind": "tool", "name": "add", "arguments": {"a": 2, "b": 3}},
    {"kind": "finish", "answer": "The sum is 5."},
]
result = run_scripted_agent(decisions, {"add": add})
assert result["observations"][0]["result"] == 5
print(result["answer"])

```

enumerate counts decisions. The step limit prevents endless execution. A finish decision returns a result. The allowlist blocks arbitrary function names. `**arguments` unpacks named inputs into the function. The toy script supplies a final answer directly, so it does not verify that the answer used the tool result. A real harness should evaluate this and replace scripted decisions with model outputs; authorization must come from trusted application state.

**Failure drill:** Change the tool name to `delete_everything`. **Expected result:** `PermissionError`, before any such operation executes.

## 5. LangChain: reusable agent components

LangChain's current Python API exposes `create_agent` for model/tool loops with configurable middleware. Its agent implementation builds on `LangGraph`. The broad benefit is reusable interfaces; it does not remove the need to validate tools and evaluate results. [Official overview](#).

Think of a toy construction kit: message objects, model adapters, tools, and structured outputs fit together. Middleware can inspect or modify processing around model and tool calls. Keep business policy in normal tested functions instead of scattering it across prompts.

### Integration recipe: supply your own configured model

This recipe is not an offline lab. It needs LangChain and a configured tool-capable chat model. `configured_model` must be supplied by your application; it is not a hidden free service.

```

from langchain.agents import create_agent

def stock_count(item: str) -> int:
    """Return the local demonstration stock count for one item."""
    return {"notebook": 7, "pencil": 12}.get(item.lower(), 0)

def build_stock_agent(configured_model):
    return create_agent(
        model=configured_model,
        tools=[stock_count],
        system_prompt="Use stock_count for counts. Do not invent stock.",
    )

# agent = build_stock_agent(configured_model)
# result = agent.invoke({"messages": [{"role": "user",
#                                     "content": "How many pencils remain?"}]})
#

```

The import selects the factory. The tool's type hints describe inputs and output; its docstring explains purpose. The factory composes the model, tool, and instruction. `invoke` submits the initial state. Actual tool choices and final wording depend on the model, so the guaranteed value 12 belongs to the tool, not to any untested model response.

**Check:** Should a tool return fake live inventory? **Answer:** No. This one explicitly says demonstration stock; a deployed tool must use real authorized data or disclose staleness.

## Lab: actual LangChain execution with a scripted model

This tests framework wiring and a tool round trip without a model account. The model is deliberately scripted: it asks for pencil stock once and then reports the tool message. Passing this test does not demonstrate a real model's reasoning ability.

```
# lab: langchain_scripted_model
from langchain.agents import create_agent
from langchain_core.language_models.chat_models import BaseChatModel
from langchain_core.messages import AIMessage, ToolMessage
from langchain_core.outputs import ChatGeneration, ChatResult

class ScriptedStockModel(BaseChatModel):
    @property
    def _llm_type(self):
        return "teaching-scripted-stock"
    def bind_tools(self, tools, **kwargs):
        return self
    def _generate(self, messages, stop=None, run_manager=None, **kwargs):
        if isinstance(messages[-1], ToolMessage):
            reply = AIMessage(content="Stock result: " + messages[-1].content)
        else:
            reply = AIMessage(content="", tool_calls=[{
                "name": "stock_count", "args": {"item": "pencil"},
                "id": "stock-call-1", "type": "tool_call"}])
            return ChatResult(generations=[ChatGeneration(message=reply)])

def stock_count(item: str) -> int:
    """Return demonstration stock for a named item."""
    return {"pencil":12, "notebook":7}.get(item.lower(), 0)

stock_agent = create_agent(model=ScriptedStockModel(), tools=[stock_count])
stock_result = stock_agent.invoke({"messages":[{"role":"user","content":"Pencil stock?"}]}))
assert stock_result["messages"][-1].content == "Stock result: 12"
assert any(isinstance(message, ToolMessage) for message in stock_result["messages"])
print("LangChain invoked the tool and supplied its result to the scripted model")
```

The subclass supplies the model interface. `bind_tools` is a no-op only because this fake already knows its sole test action. `_generate` inspects the last message: a tool result means finish; otherwise emit a structured tool request. The framework performs dispatch and appends the tool result. Returning `ChatResult` and `ChatGeneration` wraps the assistant message in the expected model response objects.

## 6. LangGraph: draw the path and carry a notebook

A graph resembles a school timetable with branches. State carries the work. Nodes do steps. Edges connect steps. Compilation creates an executable graph. LangGraph supports deterministic and model-driven processing; using an LLM is optional. [Official graph overview](#).

## Lab: an actual graph without a model account

```
# lab: langgraph_basic
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class LessonState(TypedDict):
    question: str
    answer: str

def answer_node(state: LessonState):
    return {"answer": state["question"].strip().upper()}

builder = StateGraph(LessonState)
builder.add_node("answer", answer_node)
builder.add_edge(START, "answer")
builder.add_edge("answer", END)
graph = builder.compile()
out = graph.invoke({"question": "learn agents", "answer": ""})
assert out["answer"] == "LEARN AGENTS"
print(out["answer"])
```

`TypedDict` describes a dictionary's expected fields; it is not full runtime validation. The node returns a state update. `START` and `END` are graph boundaries. The two edges describe the path. `compile` prepares execution; `invoke` runs it with an initial state. Uppercasing is deliberately simple so the graph behavior is visible.

Conditional edges choose paths from state. A reducer defines how a field handles incoming updates, such as appending results rather than replacing them. Concurrent writes to one field need an appropriate merge strategy; last-writer accidents are not a coordination design. [Graph API](#).

## 7. Checkpointing, interrupts, and replay

Saving a game's progress is a checkpoint. A graph checkpointer saves execution state associated with a thread identifier. Durable persistence survives process loss; an in-memory saver does not. Separate short-term workflow state from longer-term user memory. [Persistence guide](#).

An interrupt pauses for input. Resuming can restart the interrupted node's body, so code before the interruption may run again. Keep irreversible effects after validated approval and use idempotency. The same thread identifier reconnects the continuation; it does not itself authorize the caller. [Interrupt semantics](#).

```
# lab: langgraph_interrupt
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command

class ReviewState(TypedDict):
    proposal: str
    approved: bool

def review(state):
    response = interrupt({"proposal": state["proposal"]})
    if type(response) is not bool:
        raise ValueError("approval must be a Boolean")
    return {"approved": response}

b = StateGraph(ReviewState)
b.add_node("review", review)
b.add_edge(START, "review")
b.add_edge("review", END)
app = b.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "lesson-review"}}
paused = app.invoke({"proposal": "Reserve demonstration book", "approved": False}, config)
assert "__interrupt__" in paused
finished = app.invoke(Command(resume=True), config)
assert finished["approved"] is True
print("Paused and resumed; no reservation was executed")
```



The state contains a proposal and a Boolean. `interrupt` exposes the exact proposal. The response is checked rather than trusting that any nonempty string means yes. The saver is intentionally temporary. The second invocation supplies the review result. This lab changes only graph state: it performs no real reservation.

## 8. Parallel agents and orchestration

If three children independently check spelling, arithmetic, and citations, they can work concurrently. If one must first produce the draft, reviewers must wait for it. Parallelize independent work, not dependencies.

Fan-out sends work to multiple workers; fan-in combines results. A supervisor assigns tasks and resolves disagreements. Handoffs transfer active responsibility. Map-reduce maps similar work across items, then reduces outputs. A pipeline passes one stage's output into the next. A DAG has no directed cycles; an agent graph may deliberately contain bounded loops.

```
# lab: bounded_parallel_workers
import asyncio

async def worker(name, evidence, semaphore):
    async with semaphore:
        await asyncio.sleep(0)
        return {"worker":name, "evidence":evidence, "status":"ok"}

async def parallel_review():
    cap = asyncio.Semaphore(2)
    jobs = [worker("math", "2+3=5", cap),
            worker("sources", "document A", cap),
            worker("format", "JSON valid", cap)]
    return await asyncio.wait_for(asyncio.gather(*jobs), timeout=2)

results = await parallel_review()
assert [r["worker"] for r in results] == ["math","sources","format"]
print("Three simulated reviewers; at most two inside the bounded section")
```

`async` declares a coroutine; `await` yields while waiting. The semaphore caps in-flight sections. `gather` schedules and collects jobs in input order, not completion order. `wait_for` bounds the await time. These workers are deterministic simulations, not LLMs. For real requests, define cancellation, partial failures, rate limits, retries, and which missing reviewers prevent finalization.

Do not assume cancellation reverses a remote side effect. Give workers separate result fields or an explicit reducer. When parallel calls have different durations, total time approaches the slowest branch plus overhead, not the sum, only if resources and dependencies permit genuine overlap.

**Exercise:** One worker fails. Should its absence be disguised as agreement? **Answer:** No. Mark the failure and apply a predefined policy: retry, degrade with disclosure, or stop.

## Lab: actual LangGraph fan-out and fan-in

```
# lab: langgraph_parallel_merge
from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END

class ParallelState(TypedDict):
    findings: Annotated[list[str], operator.add]
    report: str

def math_check(state): return {"findings":["math: 2+3=5"]}
def source_check(state): return {"findings":["source: local lesson"]}
def combine_checks(state): return {"report":"; ".join(sorted(state["findings"]))}

parallel = StateGraph(ParallelState)
parallel.add_node("math", math_check)
parallel.add_node("source", source_check)
parallel.add_node("combine", combine_checks)
parallel.add_edge(START, "math")
parallel.add_edge(START, "source")
parallel.add_edge(["math", "source"], "combine")
parallel.add_edge("combine", END)
merged = parallel.compile().invoke({"findings":[], "report":""})
assert len(merged["findings"]) == 2
assert merged["report"] == "math: 2+3=5; source: local lesson"
print(merged["report"])
```

Both branches begin from START. Annotated attaches a reducer: `operator.add` concatenates returned finding lists. The list-valued edge waits for the named predecessor nodes before combining. Sorting gives a deterministic display order rather than relying on scheduling. These independent functions demonstrate orchestration; their tiny workload is not a performance benchmark.

## 9. Deep Agents and the agent harness

“Deep agent” is sometimes informal terminology for a long-running agent with planning and delegation. LangChain's **Deep Agents** is also a specific package. Its harness supplies capabilities such as planning, filesystem interactions, subagents, and context management. Depth here is not the count of neural-network layers. [Official overview](#).

A long research task needs a plan, evidence records, intermediate artifacts, and stop criteria. File-backed context can keep detailed artifacts outside the immediate prompt. Summaries save space but may discard important evidence. A delegated subagent should receive a bounded task, permitted tools, required output structure, and budget; return evidence and uncertainty with conclusions.

### Integration recipe

```
from deepagents import create_deep_agent

def lookup_term(term: str) -> str:
    """Look up one term in the local teaching glossary."""
    glossary = {"harness":"Runtime around the model and tool loop."}
    return glossary.get(term.lower(), "No glossary entry found.")

def build_tutor(configured_tool_calling_model):
    return create_deep_agent(
        model=configured_tool_calling_model,
        tools=[lookup_term],
        system_prompt="Teach with examples. Cite actual glossary evidence.",
    )
```

The model must support the capabilities this harness uses. Filesystem and execution permissions depend on the chosen backend and deployment; a filesystem interface is not automatically an operating-system sandbox. Review exposed capabilities before using real private files. This factory pattern follows the documented entry

point; provider execution remains untested in this workbook. [Quickstart API](#).

**Check:** Does giving an agent a todo list guarantee it completes the task? **Answer:** No. Completion needs observable acceptance tests and resource limits, not just a plan.

### Lab: Deep Agents harness with the same scripted model

Run after the LangChain scripted-model cell. This confirms construction and a basic tool loop in the actual package. It does not test autonomous planning, file operations, subagent quality, or every middleware capability.

```
# lab: deepagents_scripted_model
from deepagents import create_deep_agent
deep_tutor = create_deep_agent(
    model=ScriptedStockModel(), tools=[stock_count],
    system_prompt="Use the demonstration stock tool and report its value.")
deep_result = deep_tutor.invoke({"messages":[{"role":"user","content":"Pencil stock?"}]})
assert deep_result["messages"][-1].content == "Stock result: 12"
print("Deep Agents harness completed a scripted tool round trip")
```

The factory builds the harness around an injected model and tool. The input has the same message-oriented shape. The check verifies the final response produced from the actual tool result. A real model may choose different paths and requires separate task evaluations.

## 10. MCP: a shared language for capabilities

Model Context Protocol connects an AI host with external capability servers. The host is the application; it manages clients that communicate with servers. A server can expose tools, resources, and prompts. Tools perform operations, resources provide readable context, and prompts provide reusable interaction templates. A server need not contain a model. [Official architecture](#).

Imagine electrical sockets: a common connection reduces custom adapters, but plugging something in does not prove the device is safe. MCP provides interoperability, not automatic authorization, truth, sandboxing, or trustworthy tool descriptions.

Client and server must agree on a supported protocol and capabilities. Versioned specifications change lifecycle details; do not combine a handshake from one revision with message fields from another. Use a compatible SDK and the recorded version. A JSON-RPC request identifies an operation and correlates a result with an ID. SDKs handle the exact envelope; do not handcraft wire messages from outdated snippets.

Local stdio uses a subprocess's input and output channels; avoid ordinary debug logging to a protocol stdout stream. HTTP transports support separately running services and introduce network identity, credentials, origin policies, and timeouts. Neither transport choice replaces application permissions.

**Working story:** A host lists available catalog tools, obtains the schema for `book_count`, calls it with `{topic:"space"}`, receives a result, then supplies that result to the model. The application must still enforce the requester's access to the underlying catalog.

## 11. FastMCP: author a server and test its client

FastMCP is a Python library for creating and consuming MCP services. The standalone package imports from `fastmcp`; the MCP Python SDK has also provided a FastMCP class under another import path. Similar names do not mean identical releases or APIs. This workbook uses the standalone package and records the installed version. [FastMCP introduction](#).

## Lab: real MCP calls in memory

```
# lab: fastmcp_roundtrip
from fastmcp import FastMCP, Client

server = FastMCP("SchoolCatalog")

@server.tool
def book_count(topic: str) -> int:
    """Return a demonstration catalog count for a topic."""
    return {"space":3, "plants":2}.get(topic.strip().lower(), 0)

async def check_catalog():
    async with Client(server) as client:
        tools = await client.list_tools()
        assert "book_count" in {tool.name for tool in tools}
        result = await client.call_tool("book_count", {"topic":"space"})
        assert result.data == 3
        return result.data

count = await check_catalog()
print("MCP returned:", count)
```

The server has a human-readable name. The decorator registers the function as a tool. Type information supplies schema ingredients. The client context opens and closes its lifecycle. Listing tools checks discovery; calling the tool exercises dispatch and result handling. In-memory transport avoids sockets and credentials while testing real library behavior. [Client transport documentation](#).

For a separate local process, put server declarations in `server.py` and call `server.run()` inside an `if __name__ == "__main__":` guard. HTTP serving uses the documented transport configuration. The guard prevents imports from automatically starting a blocking server. The notebook lab intentionally does not expose a network endpoint. [Server quickstart](#).

## Lab: resources and prompts are different primitives

Run after the FastMCP server/client cell. A resource offers content at an identifier; a prompt offers a reusable message template. Neither grants additional privileges.

```
# lab: fastmcp_resources_prompts
@server.resource("catalog://{topic}")
def catalog_resource(topic: str) -> str:
    return f"Demonstration notes about {topic}."

@server.prompt
def study_prompt(topic: str) -> str:
    return f"Explain {topic} with one small example."

async def check_primitives():
    async with Client(server) as client:
        contents = await client.read_resource("catalog://space")
        assert "space" in contents[0].text
        prompt = await client.get_prompt("study_prompt", {"topic":"agents"})
        assert "agents" in prompt.messages[0].content.text
        return "Resource read and prompt rendered"

print(await check_primitives())
```

The resource template fills its topic from the URI. The prompt fills its topic from named arguments. The client reads the content and retrieves the rendered prompt through distinct operations. A host decides how to present or use them; fetching a prompt does not require obeying untrusted instructions inside arbitrary retrieved content.

## 12. Agent architecture principles

Separate policy, reasoning, and effects. Policy is trusted authorization and limits. Reasoning proposes next steps. Effects change the outside world. Keeping these distinct makes it possible to test authorization without trusting the model.

Use typed boundary contracts, but validate values at runtime. Give tools narrow responsibilities: `get_order(id)` is easier to audit than arbitrary database execution. Return structured errors that distinguish not found, forbidden, temporary failure, and invalid input.

Keep business facts in an authoritative service. Let context contain selected evidence with identity, version, and permissions. A summary is derived information and may be lossy. Cache keys must include relevant model, prompt, user/tenant, permissions, and freshness constraints.

Use idempotency for side effects, time budgets for calls, and bounded retries with exponential backoff and jitter for suitable transient failures. Repeating invalid credentials is not a resilience strategy. A checkpoint is not an exactly-once guarantee for remote operations.

Measure task success, supported claims, unauthorized actions, tool failures, cost per successful task, and latency percentiles. Log structured traces without indiscriminately recording secrets. Evaluate unanswerable tasks, corrupt results, tool outages, and adversarial retrieved instructions.

## 13. A complete capstone you can explain

Build a library assistant with three capabilities: search a fixed catalog, propose a reservation, and retrieve reservation status. The model may propose actions; an authenticated service owns permissions and mutations. Start with a deterministic workflow, then introduce model choice only where it helps.

State contains request ID, tenant/user identity from authentication, question, evidence IDs, proposed action, approval status, and final result. Search and availability checks can be independent only if their inputs and consistency requirements permit it. The reservation step follows validation and any required approval. A durable idempotency record connects request ID to result.

Acceptance tests: authorized available book succeeds; restricted book never reaches answer context; unknown title requests clarification; timeout after a successful reservation does not duplicate it; a malicious catalog description cannot grant a new tool; a stopped task launches no new work; citations refer to returned evidence.

**Mock interview:** Explain why a five-agent design may be worse than a two-node workflow. **Model answer:** It adds coordination and failure modes without necessarily adding independent useful work. Compare measured task success and cost under equal budgets; use delegation when specialization or independent verification justifies it.

## 14. Exercises with answers and coverage

**1. Model versus harness?** Model produces predictions or action proposals; harness manages execution, state, constraints, and tools.

**2. Why a reducer?** Multiple updates need explicit combination semantics; appending independent findings differs from overwriting a shared answer.

**3. Can a checkpoint replace an idempotency key?** No. A workflow may replay after a remote action already completed.

**4. Why not pass every tool to every worker?** It increases irrelevant choices and authority. Narrow capabilities make behavior easier to control and evaluate.

**5. Does MCP make a tool safe?** No. It standardizes communication; security remains a system property.

**6. Is async equal to multicore CPU execution?** No. Async coordinates waiting. Threads, processes, native kernels, and accelerators have different execution behavior.

**7. What is the best agent architecture?** The simplest structure meeting measured requirements under its constraints; there is no universal winner.

**8. What should a reviewer receive?** The exact proposed action, relevant evidence, expected effect, uncertainty, and permission context.

Detailed coverage: agent vocabulary, harness loop, agent types, LangChain entry point, runnable LangGraph, interrupts, bounded parallelism, Deep Agents entry point, MCP, FastMCP, orchestration, permissions, replay, evaluation, and capstone design. Advanced introductions: distributed graph persistence, large-scale scheduler internals, remote MCP authorization, sandbox backends, dynamic delegation, and model-provider adapters. These require environment-specific engineering beyond a small teaching lab.

Advanced continuation: [07-durable-agents](#). The advanced workshop and accompanying projects extend the introductory scope described above.

## Deep workshop - An agent is a controlled loop, not a magic employee

A user asks, “Which of my invoices is overdue?” Separate data, decisions, and permissions. The model can suggest a search, but the server decides whose invoices may be searched. A tool is an ordinary function behind a checked boundary. The harness owns the loop, validates arguments, tracks cost, and decides when to stop.

### Walk the machine

1. Create state containing the authenticated user ID, question, remaining tool budget of 2, and an empty evidence list. Obtain the ID from the session, never from the model.
2. The model proposes `search_invoices(status="overdue")`. Validate the arguments. Reject unknown properties rather than silently accepting them.
3. Apply ownership on the server and execute the query. Suppose the tool returns invoice I-7, amount 120, due date 1 September.
4. Store the result as data. Text inside an invoice saying “ignore your rules” is not a new instruction.
5. Ask the model to explain the result. Check referenced invoices belong to the authorized result before releasing the answer.
6. Stop after an answer or the budget limit. A third tool request must not create an infinite loop.

### Design decision and answer

A fixed workflow is preferable when the steps are known: fetch, validate, format. An agent helps when the next search depends on previous evidence. More freedom adds paths to test. Parallel searches reduce some latency but multiply cost and need a merge policy. The model supplies another user's ID. Should a valid string be accepted? **No**: schema validity establishes shape, not permission. Test with two users whose records have distinct IDs; neither must retrieve the other's records.

# Notebook 2 - Python for AI, ML, and Generative AI

Learn only the Python foundations and engineering habits that support data, models, retrieval, agents, and services. This is an original self-contained lesson sequence, not a catalog of every Python feature. Read without installing anything; running labs requires the recorded Python packages. Start from the first cell and run in order.

## 1. A program is a careful recipe

A variable is a label attached to an object. A function is a reusable recipe. A list stores an ordered collection. A dictionary associates keys with values. A loop repeats work. An exception signals that a requested operation could not complete normally.

Python indentation marks blocks. = assigns; == compares. Strings hold text. Integers represent whole numbers, floats approximate many real numbers, and Booleans represent true or false. None represents an absent value by convention; it differs from zero, an empty string, and false.

```
# lab: values_and_functions
def total_price(count, unit_price, delivery=0):
    if count < 0:
        raise ValueError("count cannot be negative")
    return count * unit_price + delivery

assert total_price(3, 10, delivery=5) == 35
prices = {"pencil":5, "book":20}
cart = ["pencil", "book", "pencil"]
total = sum(prices[item] for item in cart)
assert total == 30
print(total)
```

The function receives two required values and one default. The conditional rejects an invalid count. The return statement produces the answer. The dictionary connects each name to a price. The generator expression looks up one price per item, and sum adds them. In a money-handling system use integer minor units or deliberate decimal arithmetic rather than casual float rounding.

**Exercise:** What happens when a cart item is absent from the dictionary? **Answer:** Direct lookup raises KeyError. Decide whether missing means invalid, unavailable, or eligible for a default; do not silently treat every missing price as zero.

## 2. Mutability, copying, and common traps

Two labels can point to the same list. Changing that list is visible through both names. A shallow copy creates a new outer container while nested mutable objects can remain shared. A deep copy recursively copies more structure, but copying everything is not always the correct design.

```
# lab: mutation
original = [[1], [2]]
shallow = original.copy()
shallow[0].append(9)
assert original[0] == [1, 9]

def append_label(label, labels=None):
    result = [] if labels is None else list(labels)
    result.append(label)
    return result

assert append_label("a") == ["a"]
assert append_label("b") == ["b"]
```

```
print("Each call receives its intended list")
```

A mutable default such as `labels=[]` is created when the function is defined, not freshly on every call. The `None` pattern avoids surprising shared defaults. A tuple is an immutable sequence but can contain mutable objects. Sets keep distinct hashable members and are useful for membership tests; they do not replace ordered records when order matters.

**Interview check:** Is Python pass-by-value or pass-by-reference? **Answer:** A useful precise account is that arguments bind local names to passed objects. Mutating a shared object can be visible; rebinding a local name does not rebind the caller's variable.

### 3. Iteration, comprehensions, and generators

`enumerate` supplies index and value. `zip` pairs corresponding values but normally stops at the shortest input; use deliberate length checking when silent truncation is wrong. A comprehension builds a collection. A generator yields items on demand, avoiding a full intermediate list.

```
# lab: batches
from itertools import islice

def batches(items, size):
    if size <= 0:
        raise ValueError("size must be positive")
    iterator = iter(items)
    while True:
        group = list(islice(iterator, size))
        if not group:
            return
        yield group

assert list(batches(range(7), 3)) == [[0,1,2],[3,4,5],[6]]
print("Batches preserve the last partial group")
```

`iter` gives a stream of items. `islice` reads at most the next `size` items. An empty group signals completion. `yield` returns one group while preserving the function's continuation. This is useful for documents, embedding requests, and minibatches. A generator can be exhausted; do not expect it to restart automatically.

### 4. Functions, closures, decorators, and types

Functions are objects: you can pass a scoring function to a sorter. A closure remembers bindings from an enclosing scope. A decorator accepts a callable or class and returns a replacement or wrapped object; it is how many tool registries register functions.

```
# lab: function_registry
registry = {}
def register(function):
    registry[function.__name__] = function
    return function

@register
def square(value: float) -> float:
    """Multiply a value by itself."""
    return value * value

assert registry["square"](4) == 16
print(sorted(registry))
```

`@register` applies the registration function after defining `square`. The annotation describes expected types; ordinary Python does not enforce all annotations at runtime. Use explicit checks or a validation library at untrusted boundaries. Wrapping decorators should preserve metadata when needed, commonly with `functools.wraps`.



Use small pure functions for calculations. A pure function does not secretly read changing global state or perform external effects. File access, network calls, and model loading belong in visible boundaries so tests can substitute controlled implementations.

## 5. Classes, dataclasses, and composition

A class groups state and behavior. An instance is one created object. A dataclass generates common record methods from declared fields. Prefer combining objects through explicit interfaces over building deep inheritance trees only to reuse a few lines.

```
# lab: composition
from dataclasses import dataclass
from typing import Protocol

class Scorer(Protocol):
    def score(self, text: str) -> int: ...

@dataclass(frozen=True)
class LengthScorer:
    def score(self, text: str) -> int:
        return len(text.split())

def choose_longest(texts: list[str], scorer: Scorer) -> str:
    if not texts:
        raise ValueError("at least one text required")
    return max(texts, key=scorer.score)

assert choose_longest(["hello", "hello learner"], LengthScorer()) == "hello learner"
print("A scorer was supplied as a dependency")
```

The protocol states the capability the caller needs. The concrete object supplies it. `frozen=True` prevents ordinary field reassignment, not every possible deep mutation. Injecting dependencies makes it easy to replace a real model with a deterministic fake during tests.

## 6. Files, JSON, exceptions, and cleanup

JSON exchanges basic structured values; it does not safely encode arbitrary executable Python objects. `with` manages cleanup even when an exception occurs. Catch specific expected failures, preserve useful context, and do not silently turn every failure into a success-shaped empty result.

```
# lab: json_roundtrip
import json
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    target = Path(folder) / "record.json"
    target.write_text(json.dumps({"id": "doc-1", "text": "Hello"}), encoding="utf-8")
    data = json.loads(target.read_text(encoding="utf-8"))
    assert data["id"] == "doc-1"
print("JSON read back; temporary directory cleaned up")
```

Use explicit encodings. Validate record shape after loading. Treat pickle and other executable serialization formats as trusted-input-only unless you have a deliberately controlled security model. Do not eval user text. Path validation matters when a user chooses filenames; joining a path alone does not prevent traversal outside an allowed directory.

## 7. NumPy: a fast table of numbers

An ndarray has a shape and a data type. Think of shape as the box dimensions. (3, 2) means three rows with two values each. Vectorization sends work to optimized array routines rather than a Python loop per element. Broadcasting aligns compatible dimensions; it is convenient but can silently create the wrong shape if you do not check.

```
# lab: numpy_shapes
import numpy as np
X = np.array([[1.,2.],[3.,4.],[5.,6.]])
w = np.array([2.,-1.])
prediction = X @ w
assert prediction.shape == (3,)
assert np.allclose(prediction, [0.,2.,4.])
centered = X - X.mean(axis=0)
assert np.allclose(centered.mean(axis=0), [0.,0.])
print(prediction.tolist())
```

@ performs matrix multiplication. axis=0 averages down rows, producing one mean per column. Broadcasting subtracts those two means from each row. \* instead performs elementwise multiplication under broadcasting. Shape (n, ) is not interchangeable with (n,1) in all expressions: mixing them can create an unintended (n,n) result.

Views may share underlying storage; .copy() requests independent data. Check dtype before accumulating large counts or using low precision. Numerical stability matters: subtract the maximum score before softmax and avoid taking log of an unintended zero.

**Exercise:** Multiply shape (4, 3) by (3, 2) using @. **Answer:** (4, 2). Explain the shared dimension before doing arithmetic.

## 8. pandas: records, missingness, and joins

A DataFrame is a labeled table. It is useful for ingestion and analysis, while tensors or arrays feed many models. Missing is not always zero. A join combines records using keys, and duplicate keys can multiply rows unexpectedly.

```
# lab: pandas_join
import pandas as pd
users = pd.DataFrame({"user_id":[1,2], "group":["A","B"]})
events = pd.DataFrame({"user_id":[1,1,2], "value":[2,3,4]})
joined = events.merge(users, on="user_id", validate="many_to_one")
totals = joined.groupby("group")["value"].sum().to_dict()
assert totals == {"A":5,"B":4}
print(totals)
```

The merge attaches a group to each event. validate asserts the expected key relationship. groupby gathers rows by group; selecting value and summing produces group totals. Before training, inspect duplicate identifiers, label availability dates, missing values, and unexpected types. Train-serving consistency begins in these mundane checks.

## 9. scikit-learn: prevent leakage with pipelines

Imagine practicing an exam with its answer distribution already revealed. Preprocessing on the whole dataset can create a milder form of that problem. Fit preprocessing only on training examples, including inside each cross-validation fold. Pipelines tie transformations and predictors into one fitting procedure. [Official leakage guidance](#).

```
# lab: sklearn_pipeline
import numpy as np
from sklearn.pipeline import make_pipeline
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = np.array([[i, i % 3] for i in range(-20, 21)], dtype=float)
y = np.array([int(i >= 0) for i in range(-20, 21)])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(random_state=7))
model.fit(X_train, y_train)
probabilities = model.predict_proba(X_test)[: ,1]
assert probabilities.shape == y_test.shape
assert np.all((probabilities >= 0) & (probabilities <= 1))
print("Held-out predictions:", len(probabilities))

```

This toy has an intentionally simple label rule; its score would not prove real-world usefulness. `fit` learns both scaler and classifier using training only. `predict_proba` returns columns per class; here column one corresponds to class 1. Real evaluation additionally needs the correct time/group split, metrics, threshold choice, uncertainty, and an untouched final test.

## 10. PyTorch: tensors and gradients

A tensor resembles an array with support for devices and automatic differentiation. A computation graph records differentiable operations. `backward()` computes gradients; an optimizer changes parameters. Those are different responsibilities.

```

# lab: torch_gradient
import torch
w = torch.tensor(7.0, requires_grad=True)
prediction = w * 2.0
loss = 0.5 * (prediction - 20.0) ** 2
loss.backward()
assert abs(w.grad.item() + 12.0) < 1e-6
with torch.no_grad():
    w -= 0.1 * w.grad
    w.grad.zero_()
assert abs(w.item() - 8.2) < 1e-5
print(round(w.item(), 2))

```

The first tensor is trainable. Loss is half the squared price error. Backpropagation produces -12. `no_grad` prevents the parameter update from building an unnecessary derivative graph. Gradients accumulate by default, so clear them for the next intended independent update. `.item()` extracts a scalar for reporting; doing this inside a differentiable calculation disconnects that value from autograd.

In a normal loop: load a batch, clear gradients, predict, compute loss, backpropagate, and step the optimizer. `model.train()` changes behavior of modules such as dropout; `model.eval()` changes that behavior for evaluation. Neither alone disables autograd. Use an appropriate no-gradient/inference context for evaluation. Seed experiments and report nondeterminism limits rather than assuming every device produces identical runs.

## 11. Text, tokens, embeddings, and retrieval in Python

A token is a unit from a tokenizer, not necessarily a whole word. An embedding is a numerical representation. A similarity score measures closeness under a representation and metric; it is not automatically a probability of correctness.

## Lab: retrieval mechanics with transparent word vectors

```
# lab: toy_retrieval
from collections import Counter
import math
def bag(text):
    return Counter(text.lower().split())

def cosine_counts(a, b):
    dot = sum(value*b.get(word,0) for word,value in a.items())
    norm_a = math.sqrt(sum(v*v for v in a.values()))
    norm_b = math.sqrt(sum(v*v for v in b.values()))
    return dot/(norm_a*norm_b) if norm_a and norm_b else 0.0

documents = {"A":"plants need water", "B":"planets orbit stars"}
query = bag("plants water")
ranking = sorted(documents, key=lambda k: cosine_counts(query, bag(documents[k])), reverse=True)
assert ranking[0] == "A"
print(ranking)
```

This is lexical bag-of-words retrieval, not a pretrained semantic embedding model. Counters store term frequencies. The numerator combines shared coordinates, and norms normalize vector length. Empty input has an explicit score convention. A full RAG system adds parsing, chunk metadata, authorization, reranking, context selection, generation, citation verification, and evaluation.

## 12. Async, threads, processes, and batching

Async is useful when work waits on sockets, files, or other supported asynchronous operations. A coroutine must yield for others to proceed on the same event loop. A blocking call inside it can stall the loop. Threads can help I/O and native routines; ordinary CPU-heavy Python code often needs processes or native kernels for useful parallelism on conventional GIL-enabled builds. Python builds and extensions vary, so benchmark the actual runtime.

```
# lab: async_limit
import asyncio
async def double_later(value, limit):
    async with limit:
        await asyncio.sleep(0)
        return value*2

async def run_batch():
    limit = asyncio.Semaphore(2)
    return await asyncio.gather(*(double_later(v, limit) for v in range(4)))

values = await run_batch()
assert values == [0,2,4,6]
print(values)
```

Notebook code uses top-level await; a normal standalone script can call `asyncio.run(run_batch())`. Do not call `asyncio.run` inside an already-running notebook event loop. Bound concurrency and timeouts, distinguish retryable errors, and maintain cancellation semantics. More concurrent requests can cause rate limiting rather than more throughput.

## 13. Architecture and design principles for ML code

Separate data ingestion, transformation, training, evaluation, model artifacts, and serving. Keep the feature transformation and model version together. A result should identify its data snapshot and configuration so you can reproduce or diagnose it.

```
raw records -> validated records -> split -> fitted pipeline -> artifact
                        |               |
                        |               |
                    held-out evaluation  serving adapter
```

Use dependency injection for repositories, model clients, clocks, and random generators. Define a narrow predictor interface instead of importing provider SDKs throughout the business logic. Configuration belongs in explicit validated settings; secrets belong in a controlled environment or secret store rather than source code or notebooks.

Design pure transformations that accept data and return data. Keep effects in adapters. Prefer composition to inheritance when swapping a retriever or scorer. Use Strategy for interchangeable scoring, Adapter for provider APIs, Factory for configuration-driven construction, and Repository for persistent access. A pattern is useful only when it simplifies a real variation or boundary.

## 14. Testing, environments, packaging, and debugging

A unit test checks a small function. An integration test checks connected components. A contract test checks request/response shape. An end-to-end test checks a user journey. ML additionally needs data-quality checks, leakage checks, slice metrics, and regression evaluation. Test the property you care about, not merely that a method was called.

Use virtual environments to isolate dependencies, pin compatible versions for repeatable labs, and keep an environment report with results. A lockfile describes a resolved environment more exactly than a loose list of package names. A notebook can hide execution order; restart the kernel and run all cells to reveal missing dependencies between cells.

Debug from the first failing assumption: input shape, dtype, missingness, target definition, gradient connection, device placement, then optimization. Profile before optimizing. Vectorization, batching, caching, and better algorithms have different effects; a faster incorrect pipeline is still incorrect.

## 15. Capstone and exam practice

Build a document classifier with a clean input schema, leakage-safe split, baseline, fitted preprocessing, evaluation report, saved artifact, and prediction wrapper. Add a deterministic fake predictor for interface tests. Explain how deployment receives exactly the same transformations and what happens to unknown categories or empty text.

- 1. Why zero gradients?** Because autograd accumulates them unless cleared; accumulation should be intentional.
- 2. Why a pipeline?** It keeps preprocessing fitting within the correct training boundary and makes inference consistent.
- 3. Why not `.item()` midway through a loss?** It converts to a Python scalar and loses the gradient connection for that path.
- 4. List comprehension versus generator?** The first materializes results; the second yields lazily, with different lifetime and reuse behavior.
- 5. Why use a context manager?** It scopes resource acquisition and cleanup, including exceptional exits.
- 6. Does a type hint validate hostile JSON?** No. Perform runtime validation.
- 7. Why is a shared notebook global risky?** Later cells can accidentally depend on earlier hidden state. Test fresh execution.
- 8. When can a data join leak the future?** When historical rows receive attributes updated after their prediction time. Use point-in-time semantics.

Detailed coverage: Python values, collections, mutation, functions, decorators, protocols, files, JSON, generators, NumPy, pandas, scikit-learn, PyTorch, retrieval, async, testing, and architecture. Introductions only: distributed training internals, GPU kernel programming, advanced numerical analysis, metaclasses, and packaging native extensions. Those are specialized topics, not prerequisites for every AI engineering role. The [official Python tutorial](#) supports the language baseline; no external reading is needed for these labs.

Advanced continuation: [09-python-ai-depth](#). The advanced workshop and accompanying projects extend the introductory scope described above.

## Deep workshop - Follow Python objects through a training pipeline

A variable is a name attached to an object, not necessarily a private copy. A function that mutates an input list may also change the caller's data. Repeatable experiments require a clear ownership policy.

### Work the example by hand

Start with `rows = [[1, 10], [2, 20]]`. Assigning `alias = rows` creates a second name for the same outer list. `alias[0][0] = 9` changes what `rows` sees. Now use `copy = rows[:]`: only the outer list is copied. Changing `copy[0][1]` still changes the original inner list. `[row[:] for row in rows]` copies both levels in this specific structure. Arbitrary nesting needs a deliberate policy, sometimes `deepcopy`. Suppose column 0 is a feature and column 1 is the target. A preprocessing function overwrites features with normalized values. If datasets share row objects, one transformation can silently alter another dataset. Split independent records first, fit transformation parameters only on training data, then apply those fixed parameters to validation data.

### Trace consumption

For `(x*x for x in range(3))`, consumption produces 0, 1, 4. A second pass produces nothing because the iterator is exhausted. A list retains values; a generator streams them. Recreate the iterator for multiple epochs instead of assuming it rewinds.

### Predict and explain

Why does a default argument `items=[]` remember earlier values? **Answer:** The default object is created when the function is defined. Use `items=None`, then allocate inside the function. This is shared ownership, not random Python behaviour. Test by calling the function twice with no argument and checking the second result is independent.

# Notebook 3 - Java and Spring Boot, From Objects to Services

This notebook builds from simple Java calculations to a layered HTTP service. The companion algorithms notebook contains the classic design-pattern catalog and algorithm practice. Here we apply those ideas to Java and Spring. Read the lessons offline. Running Java labs needs a JDK; running the Spring project additionally needs Maven and initially available dependencies.

## 1. Java is a typed recipe language

A variable has a declared type. A class describes objects and behavior. An object is one instance. A method is a function belonging to a class or object. The compiler translates source into bytecode; the JVM executes it using interpretation and/or runtime compilation. The JDK supplies development tools such as the compiler, not just execution support.

Primitive types include `int`, `long`, `double`, `boolean`, and `char`. Reference variables refer to objects or `null`. `String` represents text and is immutable. `final` prevents reassignment of a variable; it does not magically make a referenced mutable object's contents immutable.

```
// lab: FirstJava
public class FirstJava {
    static int total(int count, int price, int fee) {
        if (count < 0) throw new IllegalArgumentException("negative count");
        return Math.addExact(Math.multiplyExact(count, price), fee);
    }
    public static void main(String[] args) {
        if (total(3, 10, 5) != 35) throw new AssertionError();
        String a = new String("book");
        String b = new String("book");
        if (!a.equals(b)) throw new AssertionError();
        System.out.println("Total: 35; text values equal");
    }
}
```

`public class` names the program. `static` makes a method callable without creating an instance. `Exact` arithmetic throws on overflow instead of silently wrapping. `main` is the entry point for this ordinary Java program. `equals` compares `String` values; `==` compares reference identity for objects. Compile using `javac FirstJava.java`, then run `java FirstJava`.

**Practice:** Does `final List<String>` prevent adding an item? **Answer:** No. It prevents changing that variable to refer to another list; the list's own mutability still matters.

## 2. Objects, interfaces, records, and encapsulation

Imagine different calculators following one agreement: each can quote a price. An interface expresses that agreement. Implementations supply behavior. Encapsulation protects an object's rules rather than exposing arbitrary writable fields.

```
// lab: PricingDemo
public class PricingDemo {
    interface Pricing { int price(int quantity); }
    record FlatPricing(int unitPrice) implements Pricing {
        FlatPricing {
            if (unitPrice < 0) throw new IllegalArgumentException();
        }
        public int price(int quantity) {
            if (quantity < 0) throw new IllegalArgumentException();
            return Math.multiplyExact(unitPrice, quantity);
        }
    }
}
```

```

    }
}
static int checkout(Pricing pricing, int quantity) {
    return pricing.price(quantity);
}
public static void main(String[] args) {
    if (checkout(new FlatPricing(7), 3) != 21) throw new AssertionError();
    System.out.println("Strategy supplied through an interface");
}
}

```

The interface names the capability. The record is a compact data carrier with generated accessors and value-oriented methods. Its constructor validates the rate. checkout depends on an abstraction rather than one concrete pricing class. This is composition and dependency injection in ordinary Java, without a framework.

Inheritance expresses an is-a relationship with substitutability constraints. Composition expresses has-a or uses-a. A subtype should preserve the contract expected by callers; throwing unsupported-operation errors for normal parent behavior may violate that expectation. Prefer narrow interfaces that callers actually need.

### 3. Collections, generics, equality, and hashing

An ArrayList provides indexed dynamic-array access. A linked list stores linked nodes; knowing a node can make local changes cheap, but finding an index is linear. A HashMap associates keys with values using hashes and equality. A TreeMap maintains ordered keys. A HashSet tests distinct membership; a priority queue exposes the smallest or comparator-preferred element, not a globally sorted iteration order.

Generics such as List<String> express element types and catch many errors at compilation. Java generics are largely implemented with type erasure; a generic parameter is not always available as runtime type information. A List<Dog> is not a List<Animal> because the latter could permit adding a Cat.

PECS means producer extends, consumer super: use suitable wildcard bounds when an API reads produced values or accepts consumed ones. Do not add wildcards everywhere; first identify the direction of use.

Equal objects must have equal hash codes. Mutating a key's equality-relevant fields after insertion can make hash-based lookup fail. Hash collisions are possible and do not imply equality. Arrays use identity-based equality unless you use appropriate array comparison utilities.

```

// lab: CollectionDemo
import java.util.*;
public class CollectionDemo {
    public static void main(String[] args) {
        Map<String, Integer> counts = new HashMap<>();
        for (String word : List.of("ai", "java", "ai")) {
            counts.merge(word, 1, Integer::sum);
        }
        if (counts.get("ai") != 2) throw new AssertionError();
        List<String> ranked = counts.keySet().stream()
            .sorted(Comparator.comparingInt((String k) -> counts.get(k))
                .reversed().thenComparing(Comparator.naturalOrder()))
            .toList();
        if (!ranked.equals(List.of("ai", "java"))) throw new AssertionError();
        System.out.println(ranked);
    }
}

```

merge inserts one or combines with the old count. The stream builds a processing pipeline. The comparator sorts larger counts first and text as a tie-breaker. toList collects the stream result; do not assume every list-returning API promises mutability. Streams do not inherently improve speed or make side effects safe.



## 4. Exceptions, resources, and Optional

An exception is a structured failure path. Checked exceptions require declaration or handling; unchecked exceptions extend `RuntimeException` and typically represent programming or application failures not enforced at the signature. Do not use that distinction alone to decide whether a failure is recoverable.

Try-with-resources closes `AutoCloseable` resources. Preserve causes when wrapping errors. Avoid catch-all handlers that hide failure and return empty business results. A missing record and a failed database connection are different outcomes.

Optional expresses an explicitly optional return. It can reduce accidental null assumptions, but calling `get()` blindly recreates the problem. `orElse` evaluates its fallback eagerly; `orElseGet` calls a supplier when necessary. Null checks still matter at external boundaries.

Immutability makes sharing easier. Defensive copies protect collections. Records are shallowly immutable in their component bindings, not necessarily in nested data. Validate constructor invariants and avoid publishing partially initialized objects.

**Practice:** Why is returning an empty list for a database timeout misleading? **Answer:** Callers cannot distinguish no records from unavailable information and may take incorrect actions.

## 5. JVM, memory, and concurrency

Objects are generally managed on the heap; method frames and local execution state use thread stacks, subject to optimization details. Garbage collection reclaims unreachable managed objects; it does not guarantee immediate release of files or sockets. A memory leak can occur when reachable structures retain objects no longer needed.

Concurrency is overlapping progress. Parallelism is simultaneous execution. A race occurs when outcomes depend on uncontrolled interleaving. `volatile` supports visibility and ordering for a variable, but `count++` is a read-modify-write sequence and is not made atomic just by `volatile`.

```
// lab: ConcurrentCount
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
public class ConcurrentCount {
    public static void main(String[] args) throws Exception {
        AtomicInteger count = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(2);
        try {
            Future<> a = pool.submit(() -> { for(int i=0;i<1000;i++) count.incrementAndGet(); });
            Future<> b = pool.submit(() -> { for(int i=0;i<1000;i++) count.incrementAndGet(); });
            a.get(); b.get();
            if (count.get() != 2000) throw new AssertionError();
            System.out.println(count.get());
        } finally { pool.shutdown(); }
    }
}
```

The executor owns worker threads. Futures let the caller await completion and observe failures. `AtomicInteger` makes each increment atomic; it would not automatically protect a multi-variable business invariant. Always close or shut down owned execution resources.

Locks protect critical sections; consistent lock ordering helps avoid deadlock. `CompletableFuture` composes asynchronous results but needs deliberate executor and exception handling. Virtual threads can support many blocking tasks with lower per-thread overhead in suitable workloads; they do not make CPU work free, remove database limits, or provide backpressure automatically. [Java learning baseline](#).

## 6. SOLID, cohesion, and boundaries

Single responsibility means grouping behavior that changes for the same reason. Open/closed means enabling expected extensions without repeated invasive edits. Liskov substitution means preserving the contract of an abstraction. Interface segregation means avoiding forcing clients to depend on unrelated methods. Dependency inversion means policy depends on abstractions, with concrete details supplied at boundaries.

Imagine a checkout service. It calculates a total, then asks a Payment interface to charge. A provider-specific SDK sits behind an adapter. This isolates the business rule from SDK changes. Do not create twenty interfaces for a one-function toy merely to claim architectural sophistication.

DRY avoids duplicated knowledge, not every repeated line. KISS prefers clear solutions. YAGNI avoids speculative features. High cohesion keeps related behavior together; low coupling limits unnecessary dependencies. Composition, explicit invariants, immutability, and clear error contracts often matter more than naming a design pattern.

The companion pattern notebook explains all 23 classic GoF patterns with concrete examples and cautions. In Spring you commonly encounter Factory, Proxy, Adapter, Strategy, Template Method, Observer-style events, and dependency injection, which is a broader construction technique rather than one of those 23 patterns.

## 7. Spring: a container assembles objects

In ordinary Java you write `new Service(repository)`. Spring can construct and connect managed objects, called beans. Inversion of control means the container controls this construction lifecycle. Dependency injection supplies what an object needs, often through its constructor.

`@Component` marks a discoverable component; service and repository stereotypes communicate roles.

`@Configuration` and `@Bean` define objects explicitly. Constructor injection makes required dependencies visible and supports tests without container startup. Multiple candidates require qualification or a clear selection rule.

Singleton scope normally means one bean instance per application context, not a universal JVM or cluster singleton. Request scope ties lifecycle to a request. A singleton with mutable per-user fields can leak or mix concurrent requests. Keep request data in request-specific values, not shared service instance fields.

Spring Boot adds opinionated setup, dependency management, and conditional auto-configuration. Conditions inspect classes, existing beans, and configuration. Boot does not remove Spring's underlying object lifecycle. Inspect configuration rather than calling unexpected behavior “magic.”

## 8. A layered service

```
HTTP request -> controller -> application service -> repository
                        |               |
                        domain rules   database
HTTP response <- response DTO <---+
```

A controller translates HTTP. The service coordinates a use case and its invariants. A repository handles persistence. DTOs define external data shapes. Domain objects express business concepts. An entity maps persistence; returning it directly can couple your API to storage and expose unwanted fields.

Hexagonal architecture calls external interfaces ports and concrete integrations adapters. A modular monolith can preserve these boundaries inside one deployment. Microservices add independently deployed boundaries and distributed failure; choose them for actual operational needs, not as a default interview decoration.

**Example:** `POST /reservations` accepts a book ID. The controller validates shape, the service verifies authority and availability, and the repository persists a reservation transactionally. An LLM recommendation does not

bypass this path.

## 9. HTTP, validation, and error contracts

GET retrieves without intended mutation. POST commonly creates or initiates work. PUT represents replacement semantics at an identified resource and is intended to be idempotent. PATCH applies a partial change whose idempotency depends on the patch operation. DELETE is idempotent in intended effect even if repeated response codes differ.

Use status codes deliberately: 200 success, 201 created, 202 accepted for ongoing processing, 400 invalid request, 401 unauthenticated, 403 forbidden, 404 missing or intentionally undisclosed resource, 409 state conflict, 429 rate-limited, and 5xx server-side failure classes. Include stable error codes and request IDs without leaking stack traces.

Validate DTO fields with Bean Validation and enforce business invariants in services and databases. A “positive amount” annotation cannot verify that a user owns an account. Input type validation, authentication, authorization, and business validity solve different problems.

### Framework recipe: a small controller

The delivered labs/spring-demo project contains the complete build and testable controller. This excerpt is the teaching view, not an instruction to place every class into one file.

```
@RestController
@RequestMapping("/api")
class GreetingController {
    @GetMapping("/greeting")
    Greeting greeting(@RequestParam(defaultValue="learner") String name) {
        if (name.isBlank() || name.length() > 80) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "invalid name");
        }
        return new Greeting("Hello, " + name);
    }
    record Greeting(String message) {}
}
```

The class handles HTTP requests. The mapping prefixes paths. The method reads a query parameter, checks a small constraint, and returns a structured record that the configured web stack serializes. It is a deliberately public demonstration greeting, not a template for exposing private data without authentication. [Official REST guide](#).

## 10. Transactions, JPA, and database correctness

A transaction groups changes under chosen guarantees. Atomicity means all-or-nothing within its boundary. Consistency means application/database invariants are preserved by correctly designed operations. Isolation controls what concurrent work can observe. Durability concerns committed changes surviving failures within the storage system's guarantees.

@Transactional commonly works through a proxy around bean calls. Self-invocation can bypass that proxy, so annotating a method does not guarantee every internal call starts the intended transaction. Defaults commonly roll back unchecked exceptions; checked-exception rollback and other behavior are configurable. Verify your transaction manager and framework configuration. [Official transaction semantics](#).

JPA maps Java objects and relationships to persistent records. An EntityManager tracks managed entities within a persistence context. Dirty checking notices changes for flushing; flush synchronizes SQL work but is not identical to transaction commit. Lazy loading defers fetching and can fail outside the needed context. N+1 queries arise when loading a collection triggers an extra query per item.

Use explicit fetch plans, projections, or suitable joins for read paths. Index frequent predicates and join keys after examining query plans. Optimistic locking uses versions to reject conflicting updates; pessimistic locking holds database locks under defined scope. A Java synchronized block cannot coordinate independent service instances by itself.

**Practice:** Two users reserve the last copy simultaneously. What prevents double booking? **Answer:** A database-enforced constraint or correctly isolated conditional update/locking strategy, combined with service logic. Two separate read-then-write checks alone can race.

## 11. Security, identity, and browser boundaries

Spring Security's servlet integration uses a filter chain. Authentication determines identity; authorization determines allowed operations. Protect both endpoint access and object-level ownership. A valid token does not entitle its holder to every record ID. [Security architecture](#).

JWT is a token format, not encryption by default. Validate signature, issuer, audience, expiration, and relevant claims with a suitable library. OAuth 2.0 concerns delegated authorization; OpenID Connect adds identity-oriented functionality. Do not build password hashing or token verification from scratch for production.

CORS is a browser cross-origin policy and does not authenticate callers. CSRF matters particularly when browsers automatically attach credentials such as cookies. Choose protections from the actual credential transport and threat model instead of copying an unconditional disable-CSRF snippet. Output encoding, parameterized SQL, secret handling, and resource limits remain necessary.

## 12. Testing and operating Spring services

Unit-test domain rules without starting Spring. Use appropriate slices for controller or persistence tests and broader integration tests when wiring and infrastructure matter. Mocking every dependency cannot establish database transaction behavior. Real database integration tests may use disposable containers or an equivalent isolated environment.

Configuration should be explicit across profiles. Health checks indicate selected readiness/liveness conditions, not perfect business correctness. Structured logs, metrics, and traces answer different questions: events, aggregates, and request paths. Carry correlation IDs across calls and control sensitive fields.

For resilience, set timeouts, cap connections, retry only suitable failures, and use circuit breakers or bulkheads where appropriate. A circuit breaker temporarily stops calls to a failing dependency; it is not a retry loop. Graceful shutdown should stop accepting new work and handle in-flight tasks within a budget.

## 13. Build, versions, and capstone

The accompanying Spring project pins its parent version and Java target; the validation report states whether it compiled and tested. Do not infer compatibility solely from the machine's newest JDK. The documentation checked for this edition lists Spring Boot 4.1.1 and its supported Java range; deployed projects should use their own supported, tested dependency set. [System requirements](#).

Capstone: a book-reservation API with users, books, and reservations. Implement DTO validation, a transactional service, persistent uniqueness/availability protection, object-level permissions, pagination, structured errors, idempotent create handling, and tests for concurrent attempts. Add an AI suggestion service behind an interface while keeping reservation authority in the Java application service.

## 14. Oral exam with answers

**Why constructor injection?** Required dependencies are explicit and can be supplied in tests; it discourages hidden mutable setup.

**Why not return entities?** It couples clients to storage, can trigger accidental lazy loads, and can disclose unwanted fields.

**Can volatile replace a lock?** Only for suitable visibility/ordering use cases, not arbitrary compound invariants.

**Why equals and hashCode together?** Hash collections rely on equal keys producing compatible hash behavior.

**What is a proxy?** An object standing in for another to add behavior such as transactions or authorization around calls.

**Why might a transaction annotation not apply?** Calls can bypass the configured proxy, or configuration and transaction-manager scope may differ from assumptions.

**Does garbage collection close database connections promptly?** No. Use structured resource management and appropriate pools.

**When prefer a modular monolith?** When one deployment meets needs and strong internal boundaries give adequate maintainability without distributed coordination overhead.

**Coverage:** Detailed tutorials and labs cover Java basics, object design, collections, streams, exceptions, concurrency, dependency injection, layers, HTTP, transactions, security, and testing. JVM GC tuning, bytecode engineering, every Spring portfolio module, and every database-specific isolation behavior are advanced introductions or outside this notebook. The algorithms/patterns companion extends interview breadth without pretending a finite set covers all possible exams.

Advanced continuation: [10-java-spring-depth](#). The advanced workshop and accompanying projects extend the introductory scope described above.

## Deep workshop - Trace a Java request to a database commit

An API transfers 10 credits from A to B. The controller translates HTTP, the service enforces business rules, and the repository persists data. These boundaries let you test accounting without starting a browser and change error formatting without rewriting accounting.

### Six observable steps

1. Parse account IDs and amount; reject invalid types and nonpositive amounts.
2. Authenticate the caller and separately authorize the source account.
3. Enter the transactional service through its Spring-managed proxy.
4. Protect balances using a concurrency strategy, such as ordered row locks or conditional updates. Checking the balance outside that boundary can race.
5. Subtract 10 from A, add 10 to B, and record the operation identity atomically.
6. Commit before reporting success. A retry with the same intent must not repeat the transfer.

## Reproduce a failure

Begin with A=100 and B=20. Inject failure after the debit but before the credit. Query through a new transaction after rollback: balances must remain 100 and 20. An in-memory entity is insufficient evidence because its fields may already have changed.

## Explain the boundary

Does @Transactional undo an email? **No**: a database rollback cannot unsend a remote message. Commit an outbox record beside the transfer, then deliver it through a retryable worker with duplicate handling. Also state the exception rollback policy: not every exception causes rollback under defaults. A happy-path test cannot establish rollback behaviour.

# Notebook 4 - React and Frontend Engineering

Begin with the browser, then learn JavaScript, React state, data fetching, design, accessibility, and tests. The explanations are self-contained. JavaScript labs use Node; the delivered React project contains dependencies and tests. React is a UI library, not a complete authentication, database, or backend system.

## 1. The browser is a document machine

HTML describes meaning and structure. CSS describes presentation and layout. JavaScript describes behavior. The DOM is the browser's in-memory document tree. An HTTP request asks a server for a resource; a response contains status, headers, and usually a body.

A button should be a button rather than a clickable paragraph. A form label should identify its input. Semantic elements supply behavior to keyboard users and assistive technology. Visual appearance alone does not determine usability.

CSS selectors choose elements. The cascade, specificity, and source order determine applicable rules. The box model includes content, padding, border, and margin. Flexbox suits one-dimensional alignment; Grid suits two-dimensional layouts. Use responsive constraints instead of assuming every reader has your screen width.

**Practice:** Does styling a div to look like a button provide keyboard activation automatically? **Answer:** No. Prefer the real button element and preserve focus indication.

## 2. JavaScript essentials for React

`const` prevents rebinding a variable; it does not freeze an object. `let` allows rebinding. Arrays hold ordered values. Objects hold named properties. Destructuring extracts values. Spread makes a shallow copy or expands elements. Arrow functions are concise functions with lexical `this` behavior.

```
// lab: immutable_update
import assert from 'node:assert/strict';
const oldItems = [{id:1, done:false}, {id:2, done:false}];
const newItems = oldItems.map(item =>
  item.id === 2 ? {...item, done:true} : item);
assert.equal(oldItems[1].done, false);
assert.equal(newItems[1].done, true);
assert.equal(newItems[0], oldItems[0]);
console.log('Changed item copied; original state preserved');
```

`map` creates a new array. The conditional replaces only the matching object. Spread copies that object's properties and `done:true` overrides one. Unchanged items retain their references. This supports predictable state updates without unnecessary deep copying.

`===` compares without coercing types. `null` is an explicit empty value; `undefined` often represents absence. Nullish coalescing `??` falls back for null or undefined, while `||` also falls back for other falsy values such as zero. Use the operator matching your intended meaning.

## 3. Closures, promises, and the event loop

A closure retains access to variables from an enclosing scope. A promise represents eventual completion or failure. `async` functions return promises. `await` pauses that function's continuation, not the whole JavaScript runtime. A long synchronous computation can still block UI responsiveness.

```
// lab: promises
import assert from 'node:assert/strict';
const double = async value => value * 2;
const values = await Promise.all([double(2), double(3)]);
assert.deepEqual(values, [4,6]);
const outcomes = await Promise.allSettled([
  Promise.resolve('ok'), Promise.reject(new Error('unavailable'))
]);
assert.equal(outcomes[1].status, 'rejected');
console.log('Results and partial failures are explicit');
```

`Promise.all` rejects if an input rejects but does not automatically cancel underlying work. `allSettled` collects each outcome. Neither creates unlimited safe concurrency; cap requests when needed. `Fetch` resolves for many HTTP error statuses, so inspect `response.ok` rather than treating every resolved fetch as business success.

## 4. React: describe what the screen should be

A component is commonly a function that produces a UI description from props and state. JSX is syntax for those descriptions, transformed by tooling. Props are inputs supplied by a parent. State is remembered information owned by a component or shared state layer. Re-rendering computes a new description; committing applies necessary changes. [Official React introduction](#).

Imagine a scoreboard: its display is a function of the score. You update the score, then React updates the view. Do not manually edit the score text and also expect state to remain the authority.

Rendering should be pure: the same inputs should not secretly send a payment or mutate shared data. Components can render more than you expect, including development checks. Put user-triggered work in event handlers and external synchronization in appropriate effects.

## 5. State is a snapshot, not a mutable notebook

An event handler closes over values from a particular render. Calling a setter schedules an update; it does not rewrite the handler's local variable. When the next value depends on the previous one, a functional update expresses that dependency: `setCount(previous => previous + 1)`.

Never mutate a state array with `push` and expect all update logic to remain correct. Create a new array and copy changed objects. Stable keys identify list items across updates; array position can be the wrong identity when items move or are removed. [Array update guidance](#).

Derived values, such as completed-count from a task list, usually need not be separately stored. Duplicated state can disagree. Keep one authoritative representation and calculate inexpensive derived values during rendering.

**Practice:** Why can three calls to `setCount(count + 1)` in one handler behave differently from three functional updates? **Answer:** The first calls use the same captured snapshot; functional updates compose through queued previous values.

## 6. Reducers make state transitions visible

A reducer receives current state and an action and returns next state. Think of an action as a small labeled instruction: add item, toggle item, reset. It should not secretly perform network calls. Pure reducers are easy to test. React's `useReducer` connects that function to component updates. [API reference](#).

### Lab: a complete local study list

The delivered project uses the following component. The source is included in the notebook; the JavaScript build and DOM interaction checks are recorded separately from standalone Node labs.



```
// lab: StudyApp
import React, {useReducer, useState} from 'react';

export function reducer(state, action) {
  switch (action.type) {
    case 'add': {
      const title = action.title.trim();
      if (!title) return state;
      return [...state, {id:action.id, title, done:false}];
    }
    case 'toggle':
      return state.map(item => item.id === action.id
        ? {...item, done:!item.done} : item);
    default: return state;
  }
}

export default function StudyApp() {
  const [items, dispatch] = useReducer(reducer, []);
  const [title, setTitle] = useState('');
  const completed = items.filter(item => item.done).length;
  function submit(event) {
    event.preventDefault();
    if (!title.trim()) return;
    dispatch({type:'add', id:crypto.randomUUID(), title});
    setTitle('');
  }
  return <main>
    <h1>My study practice</h1>
    <form onSubmit={submit}>
      <label htmlFor="topic">Topic to practice</label>
      <input id="topic" value={title}
        onChange={event => setTitle(event.target.value)} />
      <button type="submit" disabled={!title.trim()}>Add topic</button>
    </form>
    <p role="status">{completed} of {items.length} completed</p>
    <ul>{items.map(item => <li key={item.id}>
      <label><input type="checkbox" checked={item.done}
        onChange={() => dispatch({type:'toggle', id:item.id})} />
      {item.title}</label>
    </li>)}</ul>
  </main>;
}
```

The imports name React and the two hooks. reducer is a normal function. The add case trims text and appends a new item; toggle copies only the affected item. The component stores items and the controlled input's text. Completed count is derived rather than duplicated.

Submitting prevents the browser's default form navigation, creates a stable identity at action time, and dispatches an action. The input's value follows state and its event handler updates state. A label names the field; the button has a clear role; checkboxes provide native keyboard behavior. The status paragraph reports progress. This local app intentionally does not persist across refresh or claim backend authorization.

## 7. Effects synchronize with outside systems

Effects are for synchronizing with things outside rendering, such as subscriptions or a network connection. Dependencies describe reactive values used by the effect; suppressing dependency checks to force a desired schedule can capture stale values. Cleanup disconnects or invalidates the previous synchronization. [Effect guidance](#).

## Integration recipe: handle a changing query

```
useEffect(() => {
  const controller = new AbortController();
  let active = true;
  setStatus('loading');
  fetch(`/api/search?q=${encodeURIComponent(query)}`, {
    signal: controller.signal
  }).then(response => {
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response.json();
  }).then(data => {
    if (active) { setResults(data.items); setStatus('ready'); }
  }).catch(error => {
    if (active && error.name !== 'AbortError') setStatus('error');
  });
  return () => { active = false; controller.abort(); };
}, [query]);
```

This fragment assumes surrounding state and a defined search endpoint; it is not a standalone component. Abort requests cancellation. The active flag also prevents obsolete results from updating this component. Query encoding preserves URL structure. A production response needs runtime schema validation and a user-facing error policy. Framework data-loading facilities may handle caching and races more systematically.

Development Strict Mode can exercise extra setup/cleanup behavior to reveal bugs. Do not “fix” duplicate development observations by hiding missing cleanup. [useEffect reference](#).

## 8. Hooks, context, refs, and component design

Hooks must follow their documented call rules so React can associate state with the right component execution. Custom hooks package reusable stateful logic; they do not share one global state merely because two components call the same hook.

Context passes a value through a subtree without manually forwarding every prop. It is useful for themes or bounded shared concerns, but every rapidly changing value does not belong in one giant provider. State ownership should follow who reads and updates it.

A ref remembers a mutable value without triggering a render when changed. Use it for appropriate imperative handles or values that should not drive the visible UI. `useMemo` caches a computed value and `useCallback` a function identity under dependency rules; they are performance tools, not correctness guarantees. Measure before adding them everywhere.

Prefer small components with clear props and composition. A presentational component can receive data and callbacks while a boundary component coordinates fetching. Avoid a rigid rule that every component must be split in two; use boundaries that clarify responsibilities.

## 9. TypeScript and boundary contracts

TypeScript checks types during development; the resulting JavaScript still receives real runtime data. A type assertion does not validate a server response. Parse and validate at the boundary.

```
type LoadState<T> =
  | {status: 'loading'}
  | {status: 'ready'; data: T}
  | {status: 'error'; message: string};

function label(state: LoadState<string[]>): string {
  if (state.status === 'ready') return `${state.data.length} items`;
  if (state.status === 'error') return state.message;
  return 'Loading';
}
```

The status field discriminates the allowed shape. Loading cannot accidentally carry a required success payload. This reduces impossible state combinations. Generics let the same state structure describe different payload types. Compile-time narrowing helps code clarity; server identity, permissions, and semantic validity still need checks.

## 10. Routing, rendering modes, and data ownership

A router maps locations to UI. Put shareable navigation state in the URL when appropriate, such as search filters and selected resource IDs. Local transient input need not become a URL parameter. Protect routes for user experience, but enforce authority on the server regardless of hidden links.

Client-side rendering builds much UI in the browser. Server-side rendering sends rendered markup; hydration connects client behavior to compatible server output. Static generation prepares markup ahead of time. Server Components and streaming depend on the chosen supporting framework and boundary rules; they are not simply a flag that makes every component run everywhere.

Avoid sending secrets in client bundles. Any code or credential delivered to the browser should be considered accessible to its user. SSR does not automatically fix authorization or malicious HTML. Escape data appropriately and avoid inserting untrusted HTML without a justified sanitization design.

## 11. Forms, accessibility, and useful failure states

Every data-dependent view needs loading, empty, error, and success states. A blank panel gives no useful explanation. Keep previous data only when its staleness is clear. Disable duplicate submissions while also making the server robust to retries; a disabled button is not an idempotency mechanism.

Forms need labels, clear errors, appropriate input types, and focus behavior. Keyboard navigation, visible focus, contrast, and screen-reader names matter. Announce important asynchronous status changes without repeatedly interrupting the user. Tables need meaningful headers; icons need text alternatives when they convey meaning.

For AI chat, distinguish generated text from verified tool results. Show citations, tool progress, and recoverable errors without exposing secrets or unnecessary implementation details. Streaming partial output is not final success.

## 12. Performance and testing

Measure bundle size, render cost, network waterfalls, and user-perceived latency. Code splitting can defer unused modules. Virtualizing long lists reduces rendered elements but requires careful focus and accessibility behavior. Debouncing waits for a pause before work; throttling limits frequency. Cancel or ignore obsolete requests rather than letting old responses overwrite new input.

Unit-test pure reducers and utilities. Component tests interact through labels, roles, and visible behavior. Integration tests exercise API contracts. Browser tests cover real navigation and crucial journeys. Snapshot tests alone cannot demonstrate keyboard operability or correct network races.

The delivered project checks adding and completing a topic in a DOM test environment and produces a bundled static page. A DOM simulation does not replace cross-browser testing, screen-reader review, or a complete accessibility audit.

## 13. Frontend architecture and capstone

Organize by feature when it keeps related UI, validation, API calls, and tests together. Use a small shared design system for repeatable controls. Keep domain contracts separate from provider-specific transport details. Avoid a global mutable object that every component can change.

Capstone: a study dashboard with topics, completion status, search, and an AI explanation panel. The React layer owns display and input state. Java owns users and persisted study records. Python owns model inference or retrieval. The frontend receives a documented result structure and handles errors explicitly. Notebook 5 explains the full request path.

## 14. Interview practice

**Props versus state?** Props are inputs supplied by a parent; state is remembered information owned by a component or state layer.

**Why stable keys?** They help match item identity across list changes; incorrect keys can attach local state to the wrong item.

**Why no direct mutation?** It undermines predictable change tracking and can alter prior snapshots shared by other code.

**Effect versus event?** An event handles a particular interaction; an effect synchronizes after rendering with an external system.

**Does TypeScript validate an API payload?** No, runtime checks are still needed.

**Does Promise.all cancel work on failure?** Not automatically; define cancellation separately.

**Why avoid derived duplicate state?** Multiple representations can become inconsistent.

**Coverage:** Detailed foundations cover browser structure, JavaScript, state, reducers, effects, component design, TypeScript concepts, accessibility, rendering choices, performance, and tests. Advanced framework-specific routing internals, compiler implementation, every third-party state library, and all CSS specifications are outside a finite introductory workbook.

Advanced continuation: [11-react-depth](#). The advanced workshop and accompanying projects extend the introductory scope described above.

## Deep workshop - Watch a React render as a photograph

Each render supplies handlers that see that render's values. Calling a setter requests a future render; it does not rewrite the local variable in the current photograph.

### Trace three updates

Start at count=0. A handler calls `setCount(count + 1)` three times. Each expression reads the captured 0, so each requests replacement with 1. The next count is 1. Using `setCount(c => c + 1)` three times instead applies transformations:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ . The updater describes a change to pending state.

## Connect this to a network race

A user searches “cat,” then “car.” The older cat request finishes last. If every response writes the result, the screen shows cat while the input says car. Use supported cancellation and a request-identity check so only the current request may commit. Cancellation alone does not prove remote work stopped. Model idle, loading, success, and error explicitly. Zero results is an empty success, not endless loading. If old results remain during refresh, label them as refreshing. A retry should preserve the query the user meant to submit.

## Check your design

Should firstName, lastName, and fullName be independent state? **Usually no:** derive fullName while rendering. Three stored values create synchronization obligations. Keep minimal authoritative state and calculate cheap derived values. Test fast successive input changes with responses resolved in reverse order; an ordinary single-request test misses the race.

# Notebook 5 - Full-Stack Java, Python, React, and AI Engineering

This notebook teaches how the parts cooperate. A system is more than a collection of framework names. We use one story throughout: a learning portal where users manage study topics and ask for explanations supported by course notes. The labs are local teaching examples; the architecture sections describe what must be added for a real service.

## 1. Start from the user's journey

A learner opens a dashboard, signs in, selects a topic, asks a question, receives an explanation with sources, and records completion. Each step has a user-visible result and a possible failure. Begin with these outcomes before drawing microservices.

React presents forms, progress, and results. A Java service owns users, study records, permissions, and transactional business actions. A Python service handles retrieval, model inference, and model-specific evaluation. A database stores authoritative records. Object storage can hold documents; a search index supports retrieval. A queue can manage long-running jobs.

This division is an example, not a requirement that Java and Python must both appear in every system. A single language and a modular monolith may meet the same requirements more simply.

```
Browser / React
|
HTTPS + authenticated request
v
Java API -----> relational database
|
authorized internal request
v
Python AI service ---> allowed document retrieval ---> model adapter
|
structured answer + evidence + status
v
Java response ---> React loading/error/success display
```

**Practice:** Should Python trust a user ID sent directly by the browser without verification? **Answer:** No. Identity must come from an authenticated and authorized boundary, not an arbitrary field.

## 2. Contracts are agreements between teams

An API contract says what inputs are accepted, what outputs mean, how errors are represented, and which guarantees exist. A typed schema is part of it; permissions and timing semantics are also part of it.

For an explanation request, define question, course ID, request ID, and supported options. Derive identity from the authenticated context. For a response, include status, answer, evidence identifiers, and a request ID. A citation should refer to evidence actually provided, not a made-up URL.

```
{
  "request_id": "r-123",
  "status": "complete",
  "answer": "A gradient describes local rates of change.",
  "evidence": [{"document_id": "math-1", "version": 2}],
  "warnings": []
}
```

Document whether complete means generation finished or the business task succeeded. Version schemas intentionally. Additive optional fields are often easier to evolve than changing the meaning of an existing field. Clients must not assume every error body matches the success shape.

**Exercise:** A service returns HTTP 200 with `{error:"failed"}` to every failure. Why is that harmful? **Answer:** Transport status and business meaning disagree, monitoring becomes misleading, and clients may treat failures as success.

### 3. SQL, keys, joins, and indexes

A relational table stores rows with named columns. A primary key identifies a row. A foreign key constrains a relationship. A unique constraint protects a business uniqueness rule. SQL selects, joins, groups, and changes records. Parameterized queries separate data from SQL syntax.

```
# lab: relational_integrity
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("PRAGMA foreign_keys=ON")
db.execute("CREATE TABLE learner(id INTEGER PRIMARY KEY, name TEXT NOT NULL)")
db.execute("""CREATE TABLE topic(
    id INTEGER PRIMARY KEY,
    learner_id INTEGER NOT NULL REFERENCES learner(id),
    title TEXT NOT NULL,
    UNIQUE(learner_id, title)
)""")
with db:
    db.execute("INSERT INTO learner VALUES (?,?)", (1,"Asha"))
    db.execute("INSERT INTO topic VALUES (?,?,?)", (10,1,"Gradients"))
row = db.execute("""SELECT l.name,t.title FROM learner l
    JOIN topic t ON l.id=t.learner_id WHERE l.id=?""", (1,)).fetchone()
assert row == ("Asha","Gradients")
try:
    with db:
        db.execute("INSERT INTO topic VALUES (?,?,?)", (11,1,"Gradients"))
except sqlite3.IntegrityError:
    pass
else:
    raise AssertionError("duplicate should be rejected")
db.close()
print("Join and uniqueness rule verified")
```

The database is temporary and in memory. Foreign keys are explicitly enabled for this SQLite connection. Placeholders carry data separately from the query. The transaction context commits or rolls back appropriately. The uniqueness rule is enforced in storage, not just by an earlier application check.

An index is an auxiliary lookup structure. It speeds some reads but adds write and storage costs. Composite index order affects which predicates and orderings it supports. Inspect plans and representative data. Normalization reduces duplicated facts and update anomalies; denormalization can speed selected reads at the cost of consistency work.

### 4. Transactions and distributed work

A local database transaction can atomically record a study update. It cannot automatically make a remote model call and a message broker commit part of the same transaction. Distributed boundaries require explicit failure design.

The outbox pattern records a business change and an event in the same database transaction. A relay later publishes the event. Publication can happen more than once, so consumers need idempotency. A saga coordinates a sequence of local transactions with compensating actions where feasible; compensation is not identical to restoring the world as if nothing happened.

CQRS separates command and query models. Event sourcing stores state-changing events as the source of truth and derives current views. You can use an outbox without event sourcing and CQRS without multiple deployed services. Do not bundle unrelated patterns merely because they often appear in conference diagrams.

**Failure story:** The service reserves a place, publishes an event, then crashes before acknowledging. A retry must not reserve another place. Persistent uniqueness and request identity handle this better than hoping the network delivers exactly once.

## 5. Idempotency and request identity

An idempotency key identifies one logical operation. Bind it to the authenticated scope and the operation payload. Reusing the same key for a different request should be rejected rather than returning an unrelated old result.

```
# lab: idempotent_operation
import hashlib, json
records = {}
effects = []

def perform_once(user, key, payload):
    identity = (user, key)
    digest = hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest()
    if identity in records:
        old_digest, result = records[identity]
        if old_digest != digest:
            raise ValueError("key reused with different payload")
        return result
    result = {"operation_id": len(effects)+1, "topic":payload["topic"]}
    effects.append(result)
    records[identity] = (digest, result)
    return result

first = perform_once("user-1", "key-1", {"topic":"agents"})
second = perform_once("user-1", "key-1", {"topic":"agents"})
assert first == second and len(effects) == 1
try:
    perform_once("user-1", "key-1", {"topic":"databases"})
except ValueError:
    pass
else:
    raise AssertionError("conflicting retry accepted")
print("Sequential retry did not duplicate the demonstration effect")
```

This demonstrates semantics, not production durability or concurrency safety. A real implementation needs transactional persistence, uniqueness, in-progress status, crash recovery, and a defined retention period. The toy can lose everything on restart and race under concurrent access. Returning references to mutable cached values also needs defensive handling in a deployed design.

## 6. Authentication, authorization, and trust boundaries

Authentication answers who the caller is. Authorization answers whether that caller may perform this action on this object. Enforce both at the authoritative service. Never take a model's statement that permission exists as evidence.

Browser sessions can use secure cookie-based designs with appropriate CSRF defenses. Token-based API designs require signature/issuer/audience/expiry checks and safe storage. Service-to-service identity should be explicit; a forwarded user identifier alone is not authenticated delegation.

Restrict document retrieval by tenant and entitlement before evidence reaches a model or logs. A secret excluded from the final answer can still leak through prompts, telemetry, or caches. Separate secret management from model-visible context.



Use parameterized SQL, output encoding, controlled file paths, upload limits, and dependency hygiene. An AI feature inherits ordinary application-security responsibilities and adds prompt-injection and model-output risks. A model's generated SQL or code is a proposal, not something to execute with unlimited authority.

## 7. A permission-aware retrieval lab

```
# lab: authorized_retrieval
docs = [
    {"id": "public", "tenant": "school-A", "text": "gradient slope", "roles": {"student", "teacher"}},
    {"id": "exam", "tenant": "school-A", "text": "gradient exam answers", "roles": {"teacher"}},
    {"id": "other", "tenant": "school-B", "text": "gradient notes", "roles": {"student"}},
]

def retrieve(query, authenticated_tenant, authenticated_role):
    terms = set(query.lower().split())
    allowed = [d for d in docs if d["tenant"] == authenticated_tenant
               and authenticated_role in d["roles"]]
    return sorted(allowed,
                  key=lambda d: len(terms & set(d["text"].split())), reverse=True)

found = retrieve("gradient", "school-A", "student")
assert [d["id"] for d in found] == ["public"]
print("Restricted and cross-tenant documents excluded before ranking")
```

Identity arguments in this lab represent already-verified server context. The function itself does not authenticate a browser. Filtering precedes ranking so restricted documents never become candidate evidence. The score is transparent word overlap, not a semantic embedding model. Production needs versioning, expiry, policy updates, authorization-aware caching, and tests for permission changes.

## 8. ML and generative AI service design

A predictive model maps inputs to scores or estimates. A generative model produces structured or unstructured outputs. An agent adds action selection and tool execution around a model. Do not turn a deterministic lookup or calculation into an expensive agent loop without a measured reason.

Training pipelines need data versions, split definitions, preprocessing, configuration, metrics, and model artifacts. Serving needs validated input, the matching preprocessing, stable output contracts, monitoring, and rollback. Training-serving skew means the deployed transformation differs from the training transformation.

For RAG, separate corpus preparation, retrieval, reranking, context assembly, generation, and citation checking. For fine-tuning, define a held-out task distribution and evaluate regressions. For agents, evaluate task completion and unauthorized actions, not just fluent responses.

Use a provider adapter so business logic does not depend on one SDK response shape. Record model identity and generation settings. A retry can produce different text; nondeterminism is part of the interface. Validate structured output semantically even when the JSON syntax is guaranteed.

## 9. Queues, streaming, cancellation, and backpressure

Long work should not require an unbounded browser request. A job API can accept work with 202, return a job ID, and expose status. Workers claim jobs and update states such as queued, running, succeeded, failed, or cancelled. Job IDs are not authorization tokens.

Server-sent events are useful for one-way server-to-browser updates. WebSockets support bidirectional messaging with different connection-management needs. Streaming requires event IDs, reconnect semantics, and clear terminal states. Showing partial tokens does not mean the entire operation succeeded.

Backpressure prevents a faster producer from overwhelming a slower consumer. Bound queue length, worker concurrency, per-tenant demand, and memory. Cancelling the UI's fetch does not necessarily stop a background job. Define cancellation at each layer and avoid launching new side effects once cancellation is accepted.

**Exercise:** A reconnect receives the last event twice. What should the UI do? **Answer:** Deduplicate or apply events idempotently using a stable event identity, consistent with the stream contract.

## 10. Caching, retries, circuit breakers, and rate limits

A cache saves repeated work but creates freshness and isolation questions. Define key, value, TTL, invalidation, and failure policy. Include tenant/permission context where outputs depend on it. A cache hit for another user can be a disclosure, not a performance success.

Retries help transient failures when repeating is safe. Use bounded exponential backoff with jitter, honor applicable retry hints, and enforce an overall deadline. Retry storms can overload recovering services. A circuit breaker opens after selected failures and later probes recovery. A bulkhead caps resource sharing so one dependency cannot consume the whole service.

Rate limits control arrival over time. A token bucket refills capacity at a rate and allows a bounded burst. A concurrency limit bounds simultaneous work. These solve different problems. Limiting to ten requests per second does not prevent thousands of very long requests accumulating.

## 11. Observability and evaluation

Logs record events, metrics aggregate measurements, and traces connect steps within a request. Use request/job IDs and structured fields. Protect sensitive inputs and outputs; indiscriminate logging can create a second private-data store.

Report task success, correctness, citation support, invalid tool calls, permission failures, retries, p50/p95 latency, and cost per successful task. A low cost per token can be misleading if the model needs many retries or causes more support work.

Evaluation datasets should include ordinary cases, ambiguous questions, unanswerable cases, stale documents, conflicting evidence, malformed inputs, and dependency failures. Keep a final held-out set and avoid repeatedly tuning prompts directly to it. Slice by relevant groups and data conditions rather than relying only on one average.

## 12. Deployment, containers, and CI/CD

A container packages an application and its dependencies in an isolated process environment using operating-system mechanisms. It is not automatically a strong sandbox for arbitrary hostile code. Images should be reproducible, minimal enough for purpose, and scanned under your process. Avoid baking secrets into image layers.

CI runs checks on changes. CD prepares or deploys validated artifacts under a release policy. Keep unit tests fast, integration tests representative, and end-to-end tests focused on critical journeys. Database migrations need backward/forward compatibility planning across rolling versions.

Blue/green deployment switches between environments. Canary deployment shifts a fraction of traffic and observes outcomes. Rollback must account for schema and data changes; restoring an old binary does not undo every new write. Feature flags decouple selected behavior activation from code deployment but add configuration combinations to test.

## 13. End-to-end capstone specification

Build the learning portal with these slices, completing one user journey at a time.

| Slice       | React                    | Java                        | Python                                 | Acceptance condition                   |
|-------------|--------------------------|-----------------------------|----------------------------------------|----------------------------------------|
| Topics      | Form and list            | CRUD with ownership         | Not needed                             | One user cannot change another's topic |
| Explanation | Question and result view | Authenticate and route      | Retrieve allowed notes, produce answer | Evidence IDs match authorized notes    |
| Long work   | Job status and cancel    | Job ownership and lifecycle | Worker execution                       | Cancellation stops new work            |
| Progress    | Completion control       | Idempotent update           | Not needed                             | Retrying does not duplicate an event   |
| Evaluation  | Admin results view       | Authorized access           | Batch eval runner                      | Regressions have reproducible examples |

The included independent labs teach the primitives. This specification is not claimed to be a completed production deployment. To build it, start with a deterministic Python explanation adapter and replace it with a model only after permission, error, and retry behavior is tested.

For a timed design interview, spend the first minutes clarifying scale, latency, freshness, users, permissions, and failure costs. Give a small viable architecture, then explain how measured constraints would justify queues, replicas, caches, or service separation.

## 14. Exam answers and coverage

**Why two backend languages?** Sometimes Java business infrastructure and Python model tooling justify it; otherwise the extra boundary may not be worthwhile.

**Can a distributed system promise exactly once just by using a queue?** Not in a general end-to-end sense. Define delivery and effect semantics, durable identity, and transactional boundaries.

**Where should authorization live?** At trusted service boundaries and object-level operations, including retrieval filters and mutations.

**Why distinguish cache TTL from correctness?** A recent cached result can still be wrong for the current user's permissions or changed business state.

**What is a modular monolith?** One deployment with explicit internal module boundaries; it need not be an unstructured codebase.

**How do you evaluate an AI system?** Test the whole user task and individual stages under real constraints, including unsupported answers, authority violations, failures, and cost.

Detailed coverage: request paths, contracts, SQL, transactions, idempotency, identity, retrieval permissions, model-service separation, streaming, queues, resilience, observability, deployment, and capstone design. Advanced introductions: consensus, multi-region databases, Kubernetes internals, hardware scheduling, and formally verified distributed protocols. No finite notebook can exhaust all distributed-system designs.

Advanced continuation: [14-fullstack-workshop](#). The advanced workshop and accompanying projects extend the introductory scope described above.

## Deep workshop - Connect the system with a durable job

A long AI request should not depend on one browser connection staying open. A durable job is a kitchen ticket: closing the menu does not erase the order. The browser displays progress; storage owns the job state.

## Follow J-42

The browser submits a question and idempotency key. Java checks identity and ownership, commits pending job J-42, and returns its ID. A worker claims a lease, calls Python with authorized context, and stores a result. The browser may consume SSE, but after reconnecting it can retrieve the durable result with a GET. A lease is temporary ownership. A slow old worker can continue after its lease expires and a replacement starts. Protect result writes with a claim/version token and make effects idempotent. Expiry alone does not stop a paused process from waking up.

## Define the contract

A create response needs a job ID and status URL. A status response needs stable status values, a version, a completed result, or a safe error code. Distinguish “cancellation requested” from “all work definitely stopped.” These promises require different evidence.

## Reproduce uncertainty

Disconnect after commit but before the response. Retry the same payload and key: expect the same job ID. Retry another payload with that key: expect conflict. Scope the key to the caller and operation. A globally unscoped key can expose another user's result. This experiment tests a failure boundary that an ordinary successful POST does not.

# Notebook 6 - Algorithms, Design Principles, and Patterns

This companion builds interview problem-solving skills across Python, Java, frontend, and AI engineering. It contains detailed lessons for recurring algorithm techniques, a complete catalog of the **23 classic Gang of Four patterns**, selected architecture patterns, worked code, and exercises. “All 23 GoF patterns” is a bounded claim; it does not mean all patterns or algorithms ever invented.

## 1. How to solve a problem before coding

Imagine organizing numbered library cards. First ask what answer is required, whether duplicates exist, how much data there is, and whether the cards are already sorted. A fast method based on an untrue assumption is not a solution.

Define inputs, outputs, invalid cases, and constraints. Work a tiny example. Write a correct simple approach. State an invariant: something true before and after each step. Then improve time or memory if constraints require it. Test empty, singleton, duplicate, boundary, and adversarial cases.

Big-O describes growth, not seconds.  $O(n)$  scans  $n$  items;  $O(\log n)$  repeatedly shrinks the search region by a constant factor;  $O(n \log n)$  often appears in efficient comparison sorting;  $O(n^2)$  checks many pairs. Space complexity includes auxiliary storage and recursion depth under your stated convention.

**Example:** A ten-item linear scan can beat a complicated indexed approach whose setup dominates. Explain workload and reuse, not just asymptotic notation.

## 2. Data structures as containers with promises

An array provides indexed positions. A dynamic array occasionally reallocates; appending is commonly amortized  $O(1)$ , not a guarantee that every append costs the same. A linked list changes connections cheaply when the relevant node is known, but searching still takes work.

A stack removes the newest item first, like stacked plates. A queue removes the oldest first, like a lunch line. A deque works at both ends. A hash map stores key/value associations with expected fast lookup under suitable hashing assumptions; worst-case behavior and implementation protections vary. A balanced search tree supports ordered operations in logarithmic time. A heap efficiently exposes an extremal item but does not keep all entries in sorted order.

A trie stores shared prefixes, useful for autocomplete. A graph stores vertices and edges, representing relationships or transitions. A disjoint-set structure tracks which items belong to the same connected group. Select a structure based on operations, not familiarity.

## 3. Hash lookup: find a pair without checking every pair

For numbers [2,7,11,15] and target 9, the answer uses 2 and 7. Instead of comparing every pair, remember earlier values and ask whether the complement has appeared.

```
# lab: two_sum
def two_sum(numbers, target):
    seen = {}
    for index, value in enumerate(numbers):
        complement = target - value
        if complement in seen:
```

```

        return seen[complement], index
    seen[value] = index
    return None

assert two_sum([2,7,11,15], 9) == (0,1)
assert two_sum([3,3], 6) == (0,1)
assert two_sum([1], 2) is None
print("Pair indices found without reusing one item")

```

The invariant is that seen contains only earlier indices. Checking before insertion prevents using one item twice. Expected time is  $O(n)$ , auxiliary space  $O(n)$ , with ordinary hash-table assumptions. Decide how to handle multiple valid answers; this returns the first found under its traversal.

## Why the complement lookup is enough

**Prerequisites:** indexing, subtraction, dictionary lookup, and the difference between a value and its position. The brute-force method examines  $n(n-1)/2$  distinct pairs: with four values, six pairs. The dictionary changes the question from “which earlier number works?” to “has this one required number appeared?”

| Step | Value | Required complement | Dictionary before lookup | Action                 |
|------|-------|---------------------|--------------------------|------------------------|
| 0    | 2     | 7                   | empty                    | Save 2 → 0             |
| 1    | 7     | 2                   | 2 → 0                    | Return indices 0 and 1 |

**Why it is correct:** For any valid pair at indices  $i < j$ , value  $i$  is in the dictionary when  $j$  is processed, unless the function already returned another valid pair. Thus the lookup at  $j$  finds a partner. Checking before insertion ensures the partner is strictly earlier. Duplicate values are allowed: the two 3s in [3,3] are different positions.

**Tradeoff:** The brute-force method needs constant auxiliary memory but quadratic work. Sorting plus two pointers can reduce memory requirements for some value-based variants, but sorting changes positions; preserve original indices when the answer requires them. Hash lookup has expected linear work under the dictionary assumptions, not an unconditional worst-case promise.

**Try it:** Trace [4,1,5,3] with target 6. What is returned, and why is a one-element [3] insufficient for target 6?

**Answer:** Index 2 finds the earlier 1 at index 1, so (1,2) is returned. The single 3 is checked before it is saved, preventing reuse of the same element.

## 4. Two pointers and sliding windows

Two pointers move through data while preserving a relationship. For a sorted array and target sum, compare endpoints. If their sum is too small, advance the lower pointer; if too large, decrease the upper pointer. Sorted order justifies eliminating candidates.

A sliding window tracks a contiguous region. Grow the right edge and shrink the left when a condition is violated. It works when the update and shrink rules preserve a useful invariant; not every optimization problem has that structure.

```

# lab: longest_unique_window
def longest_unique(text):
    last = {}
    left = best = 0
    for right, character in enumerate(text):
        left = max(left, last.get(character, -1) + 1)
        last[character] = right
        best = max(best, right - left + 1)
    return best

assert longest_unique("abba") == 2
assert longest_unique("") == 0
assert longest_unique("abcabcbb") == 3
print("Longest window lengths verified")

```

last remembers the most recent index. The left boundary never moves backward. After the update, the current window has no repeated character. Each index advances once, giving  $O(n)$  expected time. Python characters here are code points; user-perceived grapheme clusters require more text processing.

## Why the window boundary cannot move backward

**Prerequisites:** contiguous substrings, zero-based indices, and a dictionary of last-seen positions. A substring is contiguous; a subsequence need not be. This algorithm solves the former.

Trace abba:

| right | character | previous position | left after update | current window | best |
|-------|-----------|-------------------|-------------------|----------------|------|
| 0     | a         | absent            | 0                 | a              | 1    |
| 1     | b         | absent            | 0                 | ab             | 2    |
| 2     | b         | 1                 | 2                 | b              | 2    |
| 3     | a         | 0                 | 2                 | ba             | 2    |

At the final a, setting left to previous+1 without max would move it back to 1 and admit bba, which contains duplicate b. The old a is already outside the active window; it should not affect left.

**Proof idea:** Before adding the new character, the active window has no duplicates. Only the new character can introduce a duplicate. Moving left past its latest occurrence removes that duplicate without adding any earlier elements. The longest valid suffix ending at each right position is considered, so taking the maximum covers the optimum.

**Cost:** Each character causes a bounded number of expected dictionary operations. The nested-loop appearance of other window implementations does not automatically imply quadratic time: if each pointer moves forward at most  $n$  times, total pointer movements are linear.

**Failure boundary:** A sum-based window that shrinks when the sum is too large usually relies on nonnegative elements. Negative numbers can make extending the window decrease its sum, invalidating that elimination rule. Do not transfer the technique without proving its condition.

**Try it:** What is the result for aaaa? **Answer:** 1. Every new a forces left past the previous a. The empty string returns 0 because no window is visited.

## 5. Binary search and boundary thinking

Binary search repeatedly discards a region known not to contain the desired answer. It needs sorted data or a monotonic predicate. Monotonic means once the condition becomes true along the search order, it remains true.

```
# lab: lower_bound
def lower_bound(values, target):
    low, high = 0, len(values)
    while low < high:
        middle = low + (high-low)//2
        if values[middle] < target:
            low = middle + 1
        else:
            high = middle
    return low

assert lower_bound([1,3,3,7], 3) == 1
assert lower_bound([1,3,3,7], 8) == 4
assert lower_bound([], 2) == 0
print("First not-less-than position verified")
```

The candidate region is half-open  $[low, high)$ . Everything before low is too small, and the answer cannot lie after the maintained upper boundary. Equality moves high left, finding the first matching position. If all values are

too small, returning `len(values)` is a valid insertion position, not a valid element index.

**Exam trap:** Binary search on an unsorted list can return a plausible but wrong answer. State the precondition.

## 6. Sorting: order the cards

Insertion sort grows a sorted prefix by inserting each next item; it is simple and useful for small or nearly sorted inputs, but quadratic in the worst case. Selection sort repeatedly selects a minimum. Bubble sort repeatedly swaps adjacent out-of-order items; it is mainly pedagogical.

Merge sort sorts halves and merges them, giving  $O(n \log n)$  comparison work and commonly  $O(n)$  extra array storage. Quicksort partitions around a pivot and recurses, with expected  $O(n \log n)$  but quadratic worst cases for unsuitable pivot behavior. Heapsort uses a heap for  $O(n \log n)$  worst-case time and in-place array implementations, but is not typically stable.

Stable sorting preserves relative order of equal keys. Counting sort uses bounded discrete keys; radix sort processes key digits using appropriate passes. Their costs depend on key range or representation, so they do not contradict comparison-sorting lower bounds. Production language sorts use engineered hybrids rather than always one textbook algorithm.

```
# lab: merge_sort
def merge_sort(values):
    if len(values) < 2:
        return list(values)
    middle = len(values)//2
    left = merge_sort(values[:middle])
    right = merge_sort(values[middle:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]

assert merge_sort([4,1,3,1]) == [1,1,3,4]
assert merge_sort([]) == []
print("Split, sort, merge")
```

Each recursive call handles a smaller list. During merging, the result contains the smallest consumed values in order. Taking left first on ties preserves relative order under compatible key handling. Python slicing in this teaching version creates extra objects; explain actual implementation costs if asked to optimize.

### Derive merge sort's cost and stability

**Prerequisites:** recursion, the base case, sorted order, and auxiliary storage. Split `[4,1,3,1]` into `[4,1]` and `[3,1]`, then singleton lists. Merge `[4]` with `[1]` to get `[1,4]`. Merge `[3]` with `[1]` to get `[1,3]`. Finally compare the fronts: take left 1, right 1, right 3, then remaining 4.

**Why merging is correct:** The smallest remaining element of each sorted input is at its front. The smaller front is therefore the smallest remaining element overall. Append it and advance only that input. Repeating maintains a sorted result. Once one input is exhausted, the other's remaining suffix is already sorted and may be appended.

**Why  $n \log n$ :** At a recursion level, subproblem sizes sum to  $n$ , so merging across that level costs  $O(n)$ . Balanced halving creates  $O(\log n)$  levels. Hence total comparison work is  $O(n \log n)$ . The base case stops empty and singleton lists. The recurrence is  $T(n)=2T(n/2)+O(n)$  for balanced equal-sized reasoning; uneven halves do not change the asymptotic result.



**Why the tie matters:** Label equal values by original identity, such as 1-left and 1-right. Taking from the left on equality preserves their original cross-half order. Recursive stability within each half plus stable merging yields stability overall. Replacing `<=` with `<` can reverse cross-half ties.

**Implementation cost:** This Python version copies slices and allocates merged lists. Peak auxiliary storage is  $O(n)$  for the usual sequential execution, plus  $O(\log n)$  recursive frames; cumulative allocations across all levels are larger. An iterative merge sort avoids recursive calls but still normally needs a merge buffer.

**Try it:** Is an already sorted input processed in linear time by this exact implementation? **Answer:** No. It still splits and merges at each level. A specially implemented sorted-run check or an adaptive sorting algorithm can exploit existing order; this code does not contain that optimization.

## 7. Stacks, monotonic structures, and intervals

A stack matches nested brackets: push opening brackets, then require the corresponding closing bracket. It also supports DFS and undo histories. A monotonic stack keeps values ordered so elements can be discarded once they can no longer answer future questions, such as finding the next greater value.

Interval problems often begin by sorting starts. To merge intervals, extend the current interval when the next start overlaps its end; otherwise finish the current group. Define whether touching endpoints count as overlap and whether intervals are closed or half-open.

```
# lab: merge_intervals
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if start > end:
            raise ValueError("reversed interval")
        if not merged or start > merged[-1][1]:
            merged.append([start, end])
        else:
            merged[-1][1] = max(merged[-1][1], end)
    return merged

assert merge_intervals([(1,3),(2,5),(7,8)]) == [[1,5],[7,8]]
assert merge_intervals([(1,2),(2,3)]) == [[1,3]]
print("Closed intervals touching at an endpoint merge")
```

Sorting dominates with  $O(n \log n)$  time. The invariant is that completed output groups do not overlap and the last group covers all currently connected processed intervals. The code does not mutate the original interval pairs.

## 8. Trees, heaps, and top-k

A binary tree gives each node at most two children. A binary search tree imposes an ordering invariant; without balancing, its height can become  $n$ . Preorder visits node before children, inorder visits left-node-right, and postorder visits children before node. Level order uses a queue.

A min-heap keeps its smallest item at the root with a parent/child ordering rule. For the largest  $k$  elements, keep a min-heap of at most  $k$ : replace its smallest when a larger value arrives. This costs  $O(n \log k)$  time and  $O(k)$  storage for a streaming top-k implementation.

```
# lab: top_k
import heapq
def largest_k(values, k):
    if k < 0:
        raise ValueError("k must be nonnegative")
    if k == 0:
        return []
    heap = []
    for value in values:
        if len(heap) < k:
```

```

        heapq.heappush(heap, value)
    elif value > heap[0]:
        heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)

assert largest_k([1,9,2,8,3], 2) == [9,8]
assert largest_k([4], 0) == []
print("Largest values retained with bounded heap")

```

The heap stores the current best  $k$  candidates, with the easiest one to discard at the root. Sorting the final heap gives display order and adds  $O(k \log k)$ . A heap alone does not promise that iterating all entries returns sorted values.

## 9. Graphs: BFS, DFS, and dependency order

Vertices are places or tasks; edges are connections. BFS explores by edge count using a queue and finds shortest paths in unweighted graphs. DFS explores one branch deeply and is useful for reachability, traversal, and some cycle checks. Directed and undirected cycle detection require different handling.

```

# lab: bfs_path
from collections import deque
def shortest_path(graph, start, goal):
    queue = deque([start])
    parent = {start:None}
    while queue:
        node = queue.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node); node = parent[node]
            return path[::-1]
        for neighbor in graph.get(node, []):
            if neighbor not in parent:
                parent[neighbor] = node
                queue.append(neighbor)
    return None

g = {"A":["B","C"], "B":["D"], "C":["D"], "D":[]}
assert shortest_path(g,"A","D") == ["A","B","D"]
assert shortest_path(g,"D","A") is None
print("Unweighted shortest route reconstructed")

```

Mark a node discovered when enqueueing so repeated edges do not enqueue it indefinitely. `parent` both records discovery and reconstructs the route. Time is  $O(V+E)$  for adjacency-list traversal, where  $V$  counts vertices and  $E$  edges.

Topological sorting orders prerequisites in a directed acyclic graph. Kahn's algorithm repeatedly removes nodes with zero incoming edges. If unprocessed nodes remain, a cycle exists. This maps directly to build systems and agent task dependencies. Strongly connected component algorithms such as Tarjan or Kosaraju identify mutually reachable groups in directed graphs; they need more than ordinary undirected connected-components logic.

## 10. Weighted routes and connectivity

Dijkstra repeatedly expands the smallest tentative distance and relaxes outgoing edges. It requires nonnegative weights for its ordinary guarantee. Bellman-Ford handles negative edges and can detect reachable negative cycles, with higher  $O(VE)$  time. Floyd-Warshall solves all-pairs shortest paths in  $O(V^3)$  time for a dense dynamic-programming formulation.

A\* orders search using cost so far plus a heuristic remaining cost. Admissibility means the heuristic does not overestimate the optimum remaining cost; graph-search details such as reopening and consistency also matter. A useful heuristic reduces search, while a poor one may behave like uninformed search or invalidate guarantees.

```

# lab: dijkstra

```

```

import heapq
def distances_from(graph, start):
    if any(weight < 0 for edges in graph.values() for _, weight in edges):
        raise ValueError("Dijkstra requires nonnegative weights")
    distance = {start:0}
    heap = [(0,start)]
    while heap:
        cost, node = heapq.heappop(heap)
        if cost != distance[node]:
            continue
        for neighbor, weight in graph.get(node, []):
            candidate = cost + weight
            if candidate < distance.get(neighbor, float('inf')):
                distance[neighbor] = candidate
                heapq.heappush(heap, (candidate,neighbor))
    return distance

g = {"A":[{"B",4},("C",1)], "C":[{"B",1}], "B":[]}
assert distances_from(g,"A")["B"] == 2
print("Cheaper indirect weighted route found")

```

The heap can contain stale entries; the comparison skips them. This teaching version uses comparable string node names for heap ties. Arbitrary unorderable objects need an additional tie-breaker counter.

Union-find tracks disjoint groups using representatives. Path compression shortens lookup chains; union by rank/size balances merging. Kruskal's minimum-spanning-tree algorithm sorts edges and uses union-find to reject cycles; Prim grows a spanning tree from a frontier. A minimum spanning tree minimizes total connection cost, not source-to-everywhere route distance. Do not confuse it with Dijkstra's output.

## 11. Recursion, backtracking, and dynamic programming

Recursion solves a problem through smaller versions. A base case stops it. Backtracking tries a choice, explores consequences, and undoes the choice before trying another. Generating subsets, permutations, and constraint assignments uses this pattern; worst-case search can be exponential.

Dynamic programming reuses overlapping subproblem answers. Define the state, recurrence, base cases, evaluation order, and answer location. Memoization computes on demand; tabulation fills an ordered table. An elegant recurrence with the wrong state can still be incorrect.

```

# lab: coin_change
def fewest_coins(coins, amount):
    if amount < 0 or any(c <= 0 for c in coins):
        raise ValueError("positive coins and nonnegative amount required")
    dp = [0] + [float('inf')]*amount
    for value in range(1, amount+1):
        for coin in coins:
            if coin <= value:
                dp[value] = min(dp[value], 1+dp[value-coin])
    return -1 if dp[amount] == float('inf') else int(dp[amount])

assert fewest_coins([1,3,4], 6) == 2
assert fewest_coins([2], 3) == -1
assert fewest_coins([2], 0) == 0
print("Optimal count found; unreachable case handled")

```

`dp[value]` means the fewest coins for exactly that value with unlimited reuse. Try each possible last coin and reuse the smaller answer. Greedy would choose 4+1+1 for amount 6, but 3+3 is better. A greedy choice needs a proof for the problem, not just a successful example.

Other recurring DP states: knapsack by item and capacity, edit distance by two prefix lengths, longest common subsequence by prefixes, and interval DP by endpoints. Complexity follows number of states times transitions per state, with memory reductions possible only when dependencies allow them.

## 12. Strings, prefixes, bits, and algorithm coverage map

Prefix sums store cumulative totals so an interval sum becomes one subtraction. For [2,3,4], prefix [0,2,5,9] gives sum of indices 1 through 2 as 9-2=7. Difference arrays efficiently represent repeated range increments before a final prefix reconstruction.

KMP string matching uses a prefix/failure structure to avoid repeatedly rechecking matched characters. Rabin-Karp compares rolling hashes and must account for collisions, often verifying candidate matches. Tries answer prefix queries by following characters. Suffix arrays/trees support richer substring queries with more complex construction and space tradeoffs.

Bit operations manipulate binary digits: AND tests shared set bits, OR combines, XOR toggles differing bits, and shifts move positions under the language's rules. A bitmask can encode a small subset.  $n \& (n-1)$  clears the lowest set bit for positive integer  $n$ . Java fixed-width overflow/shift behavior differs from Python's arbitrary-precision integers.

| Family        | Learn deeply first                          | Further topics introduced by name and purpose          |
|---------------|---------------------------------------------|--------------------------------------------------------|
| Sequences     | Hashing, two pointers, windows, prefix sums | Monotonic deque, sweep line                            |
| Search/order  | Binary search, merge sort, heaps            | Quickselect, external sorting, counting/radix          |
| Trees         | Traversals, ordering, balance idea          | AVL, red-black, B-tree, segment/Fenwick trees          |
| Graphs        | BFS, DFS, Dijkstra, topological ordering    | SCC, MST, union-find, max flow, matching               |
| Optimization  | Greedy proof, DP, backtracking              | Branch-and-bound, approximation, randomized algorithms |
| Strings       | Prefixes, trie concept, exact matching      | KMP, rolling hash, suffix structures                   |
| ML algorithms | Objective, gradients, validation            | Use the earlier AI book for model-family derivations   |

This table is an honest depth map, not a claim that every named advanced method has a complete implementation here. For advanced interviews, choose the role's relevant families and work full problems on them.

## 13. All five classic creational patterns

Creational patterns organize object construction. They are useful when construction varies or has important rules.

| Pattern          | Child-sized story                                | Concrete software example                                                                            | Main caution                                                    |
|------------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Factory Method   | A school lets each classroom choose its workbook | A base exporter workflow delegates creation of a PDF or CSV writer to an overridable creation method | Not every helper named factory implements this pattern          |
| Abstract Factory | Buy a matching chair-and-desk set                | Create compatible light-theme button and dialog objects from one family factory                      | Too many families create unnecessary abstraction                |
| Builder          | Assemble a lunch with ordered options            | Build a validated report request with selected sections and limits                                   | Do not use it where a simple constructor is clearer             |
| Prototype        | Copy a prepared worksheet template               | Clone a configured object and customize selected properties                                          | Decide shallow versus deep copy semantics                       |
| Singleton        | One official notice board in a defined room      | One shared registry instance under an explicit lifecycle                                             | Global mutable state harms tests and concurrency; scope matters |

Factory Method varies a creation step through a subtype or equivalent extension point. Abstract Factory creates related families. Builder separates assembly from the completed object. Prototype copies an existing object. Singleton controls instance availability. These solve distinct problems and should not be collapsed into “ways to

call new.”

## Lab: Builder with an invariant

```
# lab: builder_pattern
from dataclasses import dataclass
@dataclass(frozen=True)
class ReportRequest:
    topic: str
    limit: int

class ReportBuilder:
    def __init__(self):
        self._topic = ""
        self._limit = 3
    def topic(self, value):
        self._topic = value.strip(); return self
    def limit(self, value):
        self._limit = value; return self
    def build(self):
        if not self._topic or not 1 <= self._limit <= 10:
            raise ValueError("invalid report request")
        return ReportRequest(self._topic, self._limit)

request = ReportBuilder().topic("agents").limit(2).build()
assert request == ReportRequest("agents",2)
print(request)
```

The builder collects options, returns itself for chaining, and validates before creating the final value. Its mutable staging object is separate from the immutable request. Do not share one mutable builder across unrelated concurrent requests.

## 14. All seven classic structural patterns

Structural patterns arrange objects and interfaces.

| Pattern   | Child-sized story                                    | Working scenario                                                                               | Main caution                                                               |
|-----------|------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Adapter   | A plug adapter joins incompatible sockets            | Convert a provider's <code>generate_text</code> API into your application's complete interface | Preserve error and cancellation semantics                                  |
| Bridge    | Remote-control shape and device type vary separately | Separate report abstraction from rendering backend                                             | Requires two independent dimensions of variation                           |
| Composite | A folder contains files and other folders            | Give leaf tasks and task groups a common cost interface                                        | Define which leaf/group operations make sense                              |
| Decorator | Add a jacket without changing the child              | Wrap a predictor with timing or caching while preserving its interface                         | Wrapper order changes behavior                                             |
| Facade    | One reception desk coordinates departments           | Expose one checkout operation over inventory and billing subsystems                            | Avoid turning the facade into all business logic                           |
| Flyweight | Reuse a shared stamp design                          | Share immutable glyph data while positions remain external                                     | Shared intrinsic state must not accidentally become per-user mutable state |
| Proxy     | A gatekeeper stands before a resource                | Add access checks or lazy loading before forwarding a call                                     | Do not hide unexpected network or transaction cost                         |

Adapter changes how an existing interface is presented. Decorator adds behavior through wrapping. Proxy controls access or indirection. Facade simplifies a subsystem's surface. Similar-looking wrapper code can serve different intent, so identify the problem rather than only its syntax.

```
# lab: adapter_decorator
class LegacyModel:
    def generate_text(self, prompt):
```

```

        return {"text": prompt.upper()}

class ModelAdapter:
    def __init__(self, legacy): self.legacy = legacy
    def complete(self, text): return self.legacy.generate_text(text)["text"]

class CountingDecorator:
    def __init__(self, wrapped): self.wrapped, self.calls = wrapped, 0
    def complete(self, text):
        self.calls += 1
        return self.wrapped.complete(text)

model = CountingDecorator(ModelAdapter(LegacyModel()))
assert model.complete("learn") == "LEARN"
assert model.calls == 1
print("Adapter translated; decorator counted")

```

The adapter extracts the provider-specific text field. The decorator preserves `complete` while adding counting. The counter is a local teaching metric, not a concurrency-safe production telemetry system. Neither wrapper makes the underlying output trustworthy.

## 15. All eleven classic behavioral patterns

Behavioral patterns organize collaboration and choices.

| Pattern                 | Child-sized story                                        | Concrete scenario                                                                    | Main caution                                        |
|-------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------|
| Chain of Responsibility | Pass a request along helpers until one can handle it     | Validation/authentication/handler chain                                              | Define whether processing stops or continues        |
| Command                 | Write an instruction on a card                           | Queue a named operation with arguments and identity                                  | Retrying side effects needs idempotency             |
| Interpreter             | Follow the grammar of a little language                  | Evaluate a small approved expression DSL                                             | Do not replace a safe grammar with arbitrary eval   |
| Iterator                | Read one card at a time without seeing the box internals | Traverse paginated records lazily                                                    | Define mutation and resource-lifetime behavior      |
| Mediator                | A teacher coordinates students                           | Central coordinator routes interactions among components                             | Can become a bottleneck or oversized coordinator    |
| Memento                 | Save a game snapshot                                     | Restore a prior editor state without exposing its internals                          | External effects may not be reversible              |
| Observer                | A notice board informs subscribers                       | Notify views when a domain event occurs                                              | Ordering, duplicate delivery, and failures matter   |
| State                   | A traffic light behaves according to its phase           | Reservation behavior changes from draft to confirmed                                 | Prevent illegal transitions explicitly              |
| Strategy                | Choose a travel method                                   | Swap ranking or pricing algorithms behind one interface                              | Use when behavior genuinely varies                  |
| Template Method         | Follow a recipe with a few customizable steps            | Fixed import pipeline calls overridable parse/validate hooks                         | Inheritance coupling can be restrictive             |
| Visitor                 | Bring a specialist to each object kind                   | Add operations across a stable typed tree without editing each node's operation list | New node types can require updates to every visitor |

## Lab: Strategy and State

```
# lab: strategy_state
def rank(items, score):
    return sorted(items, key=score, reverse=True)
assert rank(["a", "bbbb", "cc"], len) == ["bbbb", "cc", "a"]

allowed = {"draft":{"submit":"review"},
           "review":{"approve":"done", "reject":"draft"},
           "done":{}}
def transition(state, event):
    if event not in allowed[state]:
        raise ValueError("illegal transition")
    return allowed[state][event]
assert transition("review", "approve") == "done"
try:
    transition("done", "submit")
except ValueError:
    pass
else:
    raise AssertionError("illegal transition allowed")
print("Strategy selected behavior; state table protected transitions")
```

Passing `len` is a functional Strategy implementation: behavior is supplied independently of the sorting orchestration. The transition table demonstrates the State idea without a separate class for each state. A class-based implementation can move state-specific behavior into state objects when complexity warrants it.

## Lab: Command and Observer

```
# lab: command_observer
from dataclasses import dataclass
@dataclass(frozen=True)
class AddPoints:
    amount: int
    def execute(self, score): return score + self.amount

events = []
subscribers = [events.append]
score = AddPoints(3).execute(2)
for subscriber in subscribers:
    subscriber({"type":"score_changed", "score":score})
assert events == [{"type":"score_changed", "score":5}]
print(events)
```

The command represents an operation as an object. Subscribers receive a change event. This direct in-process broadcast does not provide durable delivery, retry, or distributed ordering. Those requirements need additional architecture, not just renaming the list an event bus.

## 16. Principles and architecture patterns beyond GoF

SOLID, DRY, KISS, and YAGNI guide tradeoffs; they are not numerical laws. A practical interpretation is to make responsibilities and invariants clear, isolate expected changes, and avoid needless indirection. Favor high cohesion and explicit dependencies. Dependency injection supplies collaborators; it does not require a framework container.

Layered architecture separates concerns by level. Hexagonal/ports-and-adapters architecture isolates domain policy from external interfaces. Clean architecture emphasizes dependency direction toward stable policy. MVC separates model/view/controller concerns; MVVM uses a view model to mediate display behavior. React components do not require every one of these labels to be forced onto the same code.

Repository abstracts persistence; Unit of Work coordinates tracked changes. CQRS separates command and read concerns. Event sourcing records events as the source of truth. Outbox coordinates local state and eventual publication. Saga coordinates distributed steps and compensation. Circuit breaker, bulkhead, retry, and timeout

handle different failure/resource concerns. Cache-aside loads missing values through the application; write-through and write-behind have different consistency and failure behavior.

In AI systems, Strategy fits model or retriever selection, Adapter fits provider SDKs, Decorator fits tracing, Command fits tool proposals, State fits agent workflows, and Composite fits task hierarchies. These patterns help organize code; none proves model accuracy, safety, or effective reasoning.

## 17. Design exercises with explained answers

**A. Two model providers return incompatible objects.** Use an Adapter at the boundary and normalize results and errors. Do not spread provider-specific field access across business logic.

**B. Add timing around existing predictors.** A Decorator preserves the predictor interface and wraps the call. Decide how to report failures and whether timing includes retries.

**C. An order can ship only after payment.** Represent valid states and transitions, enforce authoritative storage constraints, and test illegal events. A State design clarifies behavior; it does not by itself solve concurrent database updates.

**D. Choose fastest shipping or cheapest shipping.** Strategy allows interchangeable scoring/selection behavior. Define tie-breaking and constraints so “cheapest” does not accidentally choose an invalid route.

**E. Add an export format to an otherwise fixed export workflow.** A Factory Method or injected factory may isolate construction. Template Method may fit stable workflow steps, but composition may be simpler if variation is not naturally hierarchical.

**F. Repeated network timeout after a mutation.** Use durable idempotency and status lookup; a retry loop alone can duplicate effects. The problem spans architecture and storage, not just one GoF pattern.

## 18. Exam routine and mastery checks

For an algorithm, explain input assumptions, invariant, steps, correctness intuition, time, space, and edge cases. For a pattern, explain the problem, participants, interactions, alternative, and cost. A pattern name without a justified problem is weak design reasoning.

Practice one 30-minute coding problem and one 15-minute design explanation. Afterwards, write the specific mistake: off-by-one, incorrect invariant, hidden mutation, unhandled empty input, wrong complexity, or missing permission boundary. Repeat a changed problem rather than memorizing the old answer.

This notebook implements representative recurring techniques and catalogs broader families. It does not claim implementations of every listed advanced method. A research algorithm exam, competitive-programming contest, Java certification, and frontend interview emphasize different material; tailor final drills to the actual assessment while preserving these foundations.

Advanced continuation: [13-patterns-workshop](#). The advanced workshop and accompanying projects extend the introductory scope described above.

## Deep workshop - Derive binary search rather than memorize it

Search  $[2, 4, 4, 9, 13]$  for the first value at least 4. Use a half-open interval  $[lo, hi)$ :  $lo$  is included,  $hi$  excluded. Initially  $lo=0$ ,  $hi=5$ . An answer of 5 means no value qualifies.



## Keep an invariant

All positions before  $lo$  are too small. All positions at or after  $hi$  qualify, if they exist. Compute  $mid = lo + (hi - lo) // 2$ . If  $a[mid] < target$ , set  $lo = mid + 1$ . Otherwise set  $hi = mid$ , retaining  $mid$  as a candidate. For target 4:  $mid = 2$  has 4, so  $hi = 2$ .  $Mid = 1$  has 4, so  $hi = 1$ .  $Mid = 0$  has 2, so  $lo = 1$ . Now  $lo = hi = 1$ . Equality did not stop the search; stopping at the first equality might return the second 4.

## Prove the result

Every update strictly reduces the nonnegative interval length. On termination, the invariant says every earlier index is too small and the returned index is the first possible match. Halving candidates gives logarithmic comparisons, assuming random-access indexing. Repeatedly walking a linked list to reach  $mid$  changes the cost.

## Practice and answer

Target 14 returns 5; check the bound before reading the array. An empty array starts with  $lo = hi = 0$  and never enters the loop. The interval definition handles both cases without special patches. Write tests for empty input, all-smaller values, all-larger values, and duplicates at both ends.

# Notebook 7 - Agents that survive mistakes and restarts

An ordinary chatbot is like a conversation at a desk. A dependable agent is a worker with a notebook, a permission card, a budget, and a supervisor. If the lights go out, a replacement worker must know what happened and what must not happen twice. Read Notebook 1 first; this notebook explains the machinery behind reliable action.

## 1. State, execution history, and the effect boundary

State is the information needed for the next decision: the question, retrieved evidence, proposed action, approval, and completion status. Execution history records how we arrived there. An external effect changes something outside the workflow: sending a message, reserving stock, or charging money. Saving state and performing an external effect are usually different transactions.

Suppose an agent reserves a seat and then crashes before saving “reserved.” After restart, its notebook says the reservation is unfinished. Repeating the request may reserve two seats. A checkpoint alone does not fix this. Give the operation a stable idempotency key, store that key with the reservation in the reservation service, and return the old result on a retry. Reuse the key only for the same payload; a changed payload should conflict.

There are three different guarantees. At-most-once may lose work rather than repeat it. At-least-once may repeat work and needs duplicate handling. Exactly-once observable effects require a carefully defined boundary, often a single database transaction or an idempotent receiver. Avoid claiming universal exactly-once execution across independent systems.

**Trace:** receive request → commit pending job → claim lease → perform idempotent operation → commit result. A crash between the last two steps causes a retry. An idempotent receiver makes that retry harmless.

## 2. Real LangGraph persistence and human approval

A checkpoint is a bookmark, not a frozen operating-system process. The framework reconstructs execution from saved state. Interrupts can cause the containing node to run again, so keep irreversible effects after approval and make them idempotent. Use stable thread IDs and a durable backend. The official [persistence reference](#) explains the framework's checkpoint model; our example below tests reopening its SQLite store.

```
# lab: durable_graph_approval
from typing import TypedDict
from pathlib import Path
from tempfile import TemporaryDirectory
from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command
from langgraph.checkpoint.sqlite import SqliteSaver

class ApprovalState(TypedDict):
    question: str
    approved: bool
    answer: str

def approve_node(state):
    decision = interrupt({'question': state['question'], 'action': 'prepare a study answer'})
    return {'approved': decision is True}

def answer_node(state):
    return {'answer': 'Approved study answer' if state['approved'] else 'Declined'}

def graph_builder():
```

```

graph = StateGraph(ApprovalState)
graph.add_node('approval', approve_node)
graph.add_node('answer', answer_node)
graph.add_edge(START, 'approval')
graph.add_edge('approval', 'answer')
graph.add_edge('answer', END)
return graph

with TemporaryDirectory() as directory:
    database = str(Path(directory)/'checkpoints.sqlite')
    config = {'configurable': {'thread_id': 'alice-request-42'}}
    with SqliteSaver.from_conn_string(database) as saver:
        graph = graph_builder().compile(checkpointer=saver)
        paused = graph.invoke({'question': 'Explain checkpoints', 'approved': False, 'answer': ''}, config)
        assert paused['__interrupt__']
    # New saver and graph: the old in-memory graph is no longer relied upon.
    with SqliteSaver.from_conn_string(database) as saver:
        graph = graph_builder().compile(checkpointer=saver)
        resumed = graph.invoke(Command(resume=True), config)
        assert resumed['answer'] == 'Approved study answer'
print('Persistent approval resumed from a reopened SQLite checkpoint')

```

The interrupt payload describes the actual proposal. In a real approval system also bind the decision to the authenticated reviewer, action hash, resource version and expiry. “Yes” from the wrong person or for an old proposal is not authorization. A changed proposal needs a new decision. Do not let retrieved text, a model, or the worker manufacture approval.

### 3. Checkpoint migrations are data migrations

Imagine version 1 stores question, while version 2 requires messages. Renaming the field in code does not repair saved checkpoints. Version your application state separately from the framework's storage schema. Write a pure migration, preserve the old meaning, reject unknown future versions, test representative historic records, back up before rollout, and make rollback behavior explicit. Use framework-supported state-update APIs; do not casually rewrite its internal tables.

```

# lab: checkpoint_state_migration
from copy import deepcopy
def migrate_state(saved):
    state = deepcopy(saved)
    version = state.get('schema_version', 1)
    if version == 1:
        state['messages'] = [{ 'role': 'user', 'content': state.pop('question') }]
        state['schema_version'] = 2
    elif version != 2:
        raise ValueError('unsupported state version')
    return state

old = {'question': 'Why retry?', 'schema_version': 1}
new = migrate_state(old)
assert new['messages'][0]['content'] == old['question']
assert migrate_state(new) == new and 'question' in old
try:
    migrate_state({'schema_version': 99})
except ValueError:
    print('Migration is non-mutating, repeatable, and rejects unknown versions')
else:
    raise AssertionError('future state must not be guessed')

```

Changing a graph's node names or interrupt ordering also changes execution semantics. A schema migration cannot automatically make a half-finished old workflow compatible with a new graph. Keep a versioned workflow definition available, drain old runs, or explicitly migrate their execution position. Test the paused and mid-failure states, not just newly started runs.

## 4. Leases, fencing, and recovery after a crash

A lease says “worker A may work on this job until time T.” Expiry permits recovery after A disappears. But A may be merely slow, then wake up after worker B takes over. A fencing token distinguishes A's old ownership from B's new ownership. Every database update checks the current token. A stale worker's update affects zero rows.

```
# lab: stale_worker_fencing
import sqlite3
db = sqlite3.connect(':memory:')
db.execute('create table job(id integer primary key, token integer, answer text)')
db.execute("insert into job values(1, 1, '')")
old_token = 1
db.execute('update job set token=2 where id=1')
stale = db.execute("update job set answer='old' where id=1 and token=?", (old_token,))
fresh = db.execute("update job set answer='new' where id=1 and token=2")
assert stale.rowcount == 0 and fresh.rowcount == 1
assert db.execute('select answer from job').fetchone()[0] == 'new'
db.close()
print('Only the current lease holder can publish its result')
```

This protects database writes. If the external tool ignores the token, it can still perform an old worker's action. Combine fencing with receiver-side idempotency or conditional resource versions. The Study Coach project uses durable jobs, expiring leases, an attempt budget and token-checked updates. Its answer generation is read-only, which makes repeating computation safe.

## 5. Cancellation and parallel failures

Cancellation is a request to stop, not time travel. An already committed payment cannot be “uncalled.” Stop scheduling new work, propagate a cancellation signal, release resources in `finally`, and decide whether a compensating action is possible. Python `TaskGroup` cancels siblings when one child fails; collecting partial results needs a different explicit policy. The [Python task documentation](#) specifies these cancellation rules.

```
# lab: parallel_failure_cleanup
import asyncio
async def failure_demo():
    started = asyncio.Event()
    cleaned = []
    async def slow():
        try:
            started.set()
            await asyncio.sleep(30)
        finally:
            cleaned.append('released')
    async def broken():
        await started.wait()
        raise ValueError('retriever failed')
    try:
        async with asyncio.TaskGroup() as group:
            group.create_task(slow())
            group.create_task(broken())
    except* ValueError:
        pass
    assert cleaned == ['released']
await failure_demo()
print('A failed child cancelled its sibling and cleanup ran')
```

For an all-required workflow, one missing branch invalidates the combined result. For best-effort search, return successful branches with explicit missing-source information. Never silently label a partial answer complete. Bounded parallelism controls concurrency; a deadline controls duration; a token/tool budget controls expenditure. They solve different problems. Add queue limits too: ten running tasks plus a million queued tasks can still exhaust memory.

## 6. Memory with ownership, expiry, and evidence

Working memory is the current task state. Episodic memory records past events. Semantic memory stores facts. Procedural memory stores instructions or skills. Retrieval indexes are access paths, not automatically trustworthy memories. A useful record has owner, source, creation time, expiry, confidence or verification state, and a deletion policy.

```
# lab: scoped_memory
from dataclasses import dataclass
@dataclass(frozen=True)
class Memory:
    owner: str
    text: str
    expires: int
    verified: bool

def recall(records, owner, now):
    return [m.text for m in records if m.owner == owner and m.expires > now and m.verified]

memories = [Memory('alice', 'prefers small examples', 20, True),
            Memory('bob', 'private plan', 20, True),
            Memory('alice', 'old preference', 9, True),
            Memory('alice', 'unverified claim', 20, False)]
assert recall(memories, 'alice', 10) == ['prefers small examples']
print('Owner, expiry and verification filters applied before recall')
```

Summaries compress information and can lose a critical negation, date or source. Keep links to original evidence and distinguish user facts from model inferences. Do not store secrets simply because they appeared in a conversation. Updating a preference should supersede the old preference rather than create contradictory permanent facts. Test deletion from source storage, caches and indexes.

## 7. Systematic evaluation instead of impressive demos

Build a versioned task set with ordinary cases, ambiguous requests, malicious retrieved instructions, missing evidence, denied tools, timeouts, repeated delivery and cancellations. Keep development cases separate from held-out evaluation. Grade task success, groundedness, permission violations, tool-call count, latency and cost. A single average hides dangerous categories; report them separately.

```
# lab: agent_evaluation_gate
cases = [
    {'category': 'normal', 'success': True, 'violation': False, 'calls': 2},
    {'category': 'denied', 'success': True, 'violation': False, 'calls': 0},
    {'category': 'timeout', 'success': False, 'violation': False, 'calls': 3},
    {'category': 'injection', 'success': True, 'violation': False, 'calls': 0},
]
success_rate = sum(c['success'] for c in cases)/len(cases)
assert success_rate == 0.75
assert not any(c['violation'] for c in cases)
assert max(c['calls'] for c in cases) <= 3
print({'success_rate': success_rate, 'permission_violations': 0, 'cases': len(cases)})
```

Four invented records explain the calculation; they are not evidence of a production agent's quality. Real evaluation runs the candidate on stored inputs and captures traces. Repeat nondeterministic tasks, report sample sizes and uncertainty, and have humans audit rubric disagreements. An LLM judge can be biased, prompt-injected or inconsistent. Calibrate it against human labels. Compare a simple workflow, a single agent and a multi-agent design under the same budgets before adopting delegation.

The companion `labs/agent-evaluation/evaluate.py` runs an actual versioned 16-case regression dataset against the connected application's retrieval function. Ten cases check first-ranked evidence, four check cross-owner exclusions including instruction-like queries, and two check the no-evidence path. It records category scores, latency, returned IDs and a hash of the evaluated source. Run it from the notebook environment and inspect `evaluation-report.json`. This is an executable development regression gate, not a held-out

estimate of LLM reasoning quality; its small size and deterministic baseline are explicit.

## 8. Architecture decisions and graded exercise

**Question, 10 points:** Two research workers run in parallel. One times out. The user approved a report before the evidence changed. The first publisher crashed after writing the report. Design recovery.

**Answer:** Define whether both branches are required (2). Mark missing evidence if partial output is allowed (1). Revalidate approval against the exact new proposal and resource version (2). Publish using a stable idempotency key and reject changed payloads (2). Recover via a durable lease and fence stale writes (2). Record the attempt, failure and outcome without leaking credentials (1).

**Follow-up:** Why not just retry the entire graph? Because replay may repeat effects, consume extra budget and invalidate prior approvals. A successful retry also does not prove the original attempt failed.

**Mastery task:** In Study Coach, approve a job, stop the Java process while the status is running, restart with the same database, and watch the attempt count. Cancel another job and verify that a stale worker cannot change it to done. Read the automated restart test before repeating this manually.

## Deep workshop - Two durable records and a dangerous gap

A checkpoint is a workflow bookmark. The payment receiver's record is a receipt. They live in different systems, so updating one does not update the other atomically.

### Failure schedule

At time 1, save pending operation PAY-17. At time 2, the receiver commits a charge and stores PAY-17 with its result. At time 3, the worker crashes before saving completion. At time 4, recovery reads pending and repeats PAY-17. The receiver returns the old receipt instead of charging again. At time 5, the worker saves completion. If the receiver saves its payment and deduplication record separately, a crash between those writes can still duplicate the effect. They need one atomic boundary. Concurrent first attempts require a storage uniqueness constraint; "check whether it exists, then insert" is not sufficient.

### Approval and replay

A suspended node may restart from its beginning. Avoid irreversible actions before the interrupt unless independently idempotent. Bind approval to the proposed payload or version. Changing an amount after approval requires a new decision.

### Migration exercise

Can a new checkpoint schema simply rename a required field? **No:** paused jobs still hold the old shape. Version persisted state, migrate deliberately, and test resuming an old checkpoint with new code. A fresh-run test cannot establish migration correctness. Keep old checkpoint fixtures and test both forward migration and rejection of unsupported versions.

# Notebook 8 - A remote MCP service you can inspect and test

An MCP server is like a library counter with a standard request form. A client asks what operations exist and calls them using structured arguments. The form does not decide who may enter the library, which shelf belongs to whom, or whether borrowing a book is safe. Those are authentication, authorization and application responsibilities.

## 1. The complete project and its boundaries

The executable project is in `labs/remote-mcp.server.py` exposes tools, a resource and a prompt over streamable HTTP. `client.py` makes a real network call. `test_remote.py` launches the server in a separate process and checks allowed access, rejected credentials, permissions, ownership, validation and timeouts. This differs from an in-memory `Client(server)` example: requests cross an actual HTTP boundary.

The project uses opaque bearer tokens supplied through environment variables. The server maps each token to an identity and scopes. Tokens are never accepted from tool arguments. This is a controlled internal-service teaching setup, not a complete OAuth authorization server. FastMCP's [token verification documentation](#) distinguishes validation from token issuance and discovery. For public integrations use a compatible authorization server and the appropriate MCP authorization flow; do not invent a login protocol.

## 2. Authentication, scopes, and resource ownership

Authentication asks “who is this?” Authorization asks “may this identity perform this action on this resource?” A scope such as `notes:read` answers part of that second question. It does not permit reading every tenant's notes. Our read tool derives its owner from the verified access token and filters by that owner. A caller cannot submit `owner='bob'` to elevate access.

The server has an Alice reader, an Alice editor and a Bob reader. The reader may read Alice's note but may not write. The editor may write Alice's note. Bob sees Bob's note. Access checks run at the tool boundary as well as shaping discovery. Hiding a tool in the list is useful but is not a substitute for rejecting direct unauthorized calls. See [FastMCP authorization](#) for the framework hooks used by the project.

**Analogy:** A school badge identifies you. A laboratory permission lets you enter the laboratory. Neither authorizes opening another student's locker. Identity, action permission and resource ownership are three separate checks.

## 3. Request lifecycle and transport behavior

The client connects with credentials, negotiates the capabilities supported by its installed SDK, discovers tools, and sends a typed call. The server authenticates, validates arguments, checks permissions, executes, and returns structured data or a protocol/tool error. Use the SDK rather than manually assuming a particular revision's handshake fields. The current protocol and SDK must be compatible.

Stdio is suitable for a local child process. Streamable HTTP serves network clients and introduces network failures, proxy timeouts and authorization concerns. A transport connection is not a durable business job. If work must survive disconnects, create a job ID and provide status/cancellation operations. A session ID is not a bearer credential. Do not treat possession of a session identifier as authorization.

JSON schemas describe acceptable shapes, but domain checks are still necessary. A string may satisfy a schema and still be too long, refer to an inaccessible object, or request an invalid state transition. Bound input

size, output size and execution time. Avoid giving a model a tool that accepts unrestricted filesystem paths or arbitrary commands when a narrow operation will do.

## 4. Timeouts, cancellation, and uncertain outcomes

There are at least three clocks: connection timeout, per-call timeout, and the overall task deadline. The project calls a deliberately slow read-only tool with a shorter client timeout. The test proves that the client stops waiting. It does not prove every remote implementation immediately stops executing when a client disconnects.

If a write times out, its outcome may be unknown. Ask for status using its operation ID or retry with the same idempotency key. Do not automatically send a fresh write with a new key. Bounded exponential backoff with jitter avoids every client retrying at the same instant. Respect the remaining task deadline; an exhausted budget should not create another long retry.

```
# lab: retry_delay_budget
import random
def retry_delays(attempts, base, cap, remaining, seed=7):
    rng = random.Random(seed)
    delays = []
    for attempt in range(attempts):
        delay = rng.uniform(0, min(cap, base*(2**attempt)))
        if delay > remaining:
            break
        delays.append(delay)
        remaining -= delay
    return delays
delays = retry_delays(5, 0.1, 1.0, 0.5)
assert sum(delays) <= 0.5 and len(delays) <= 5
print('Backoff stays within the waiting budget:', [round(x, 3) for x in delays])
```

This example budgets delays only. A real retry loop subtracts operation time too and retries only classified transient failures. Validation errors, missing permission and malformed requests generally do not improve through repetition.

## 5. Errors that help without leaking secrets

A connection failure means the peer was not reached. An authentication error means credentials were absent or unacceptable. Authorization denies an action. Argument validation rejects a malformed request. A tool error reports execution failure. Keep those categories separate so the caller can choose an appropriate response.

Return a stable public error and a correlation ID. Detailed stack traces belong in protected logs, with tokens and sensitive arguments redacted. Do not return another user's resource name merely to explain a denial. The note tool treats ownership as server-side state; its inputs never contain a token.

For business errors, decide whether the protocol's error channel or a documented result union fits the SDK and clients. Never return a success-shaped object containing an unnoticed error string. Clients should inspect the error flag or exception before trusting structured output.

## 6. Run the project

From the notebook root, use the environment installed for the notebooks:

```
.\.venv\Scripts\python labs/remote-mcp/test_remote.py
```

The test creates temporary random tokens, runs on loopback, cleans up its server process, and writes `labs/remote-mcp/test-report.json`. To run manually, set three distinct random environment values named `MCP_ALICE_READ_TOKEN`, `MCP_ALICE_WRITE_TOKEN`, and `MCP_BOB_READ_TOKEN`, then run `server.py`. Set `MCP_CLIENT_TOKEN` to the reader token and run `client.py` in another terminal. Do not paste secrets into a



notebook output or commit them.

The server's in-memory notes deliberately reset on restart. The project demonstrates remote MCP transport and access control; it does not claim durable note storage. The Study Coach project separately demonstrates durable storage. Replacing the dictionary with a repository is an exercise in separating protocol code from persistence.

## 7. Deployment and operation

The project includes a Dockerfile and deployment instructions. A container is a process package, not a security policy. Supply secrets at runtime, run as a non-root user, publish only intended ports, and terminate TLS at a trusted reverse proxy for access beyond loopback. Configure allowed hosts/origins for the chosen SDK and proxy. Keep proxy buffering and timeouts compatible with streaming.

Protect logs and metrics, cap concurrent calls, and use a bounded queue for expensive tools. Expose a separate health endpoint if your deployment needs it; health should not execute expensive tools. Rolling upgrades must keep protocol and application schemas compatible while old requests finish. Rotate tokens and revoke old credentials. Public OAuth deployments need issuer/audience/expiry validation, key rotation and discovery beyond this static-token teaching profile.

Docker packaging is supplied for reproducibility, but container execution is only marked tested if the validation report records a Docker run. Local subprocess network tests are not the same as a TLS or cloud deployment test.

## 8. Interview round and answers

**Question, 10 points:** A reader sees only `read_note` during discovery but guesses the name `write_note`. Is the server safe? **Answer:** Only if authorization also rejects direct invocation (3). Tool discovery is not an enforcement boundary (2). Identity comes from validated credentials, not model-supplied arguments (2). Resource ownership remains necessary even with write scope (2). Test both listed and guessed calls (1).

**Debugging:** A write timed out and the retry created a duplicate. Find the missing contract. **Answer:** The server and client need a stable operation key, payload binding and durable deduplication; timeout does not establish non-execution.

**Design alternative:** If only your own backend calls one fixed function, ordinary HTTP may be simpler than MCP. Choose MCP when its standardized tool/resource/prompt interface and client ecosystem solve a real interoperability need. A protocol should reduce integration work, not become an extra layer without a purpose.

## Deep workshop - An MCP connection is not a permission grant

MCP standardizes discovery and invocation. The server still owns authentication, authorization, validation, and limits. A standard door handle does not decide who receives a key.

### Walk a remote request

The client connects, negotiates the protocol, discovers a tool, and submits arguments. The server validates credentials, intended audience, and permissions using its configured authentication mechanism. It binds the principal to the request, validates arguments, applies ownership filters, then executes and returns a bounded result. `read_note(note_id="N-4")` may be syntactically valid while N-4 belongs to another user. It must fail. A model-supplied owner parameter cannot override the authenticated principal. Discovery itself may need filtering if tool descriptions reveal sensitive capabilities.

## Understand timeout uncertainty

A write can time out after the server committed. Reconnection does not roll back the effect. Use a stable operation identity and queryable status, or an idempotent receiver. Reads and writes need different retry reasoning.

## Test each boundary

Try missing credentials, expired credentials, wrong permission, wrong ownership, oversized arguments, and a slow tool. A successful connection proves none of those negative cases. Local classroom credentials and HTTP examples are not production credential management. Explain separately which layer secures transport, which identifies the caller, and which decides access to a particular resource.

# Notebook 9 - Python engineering, mathematics, and ML from scratch

Think of an ML system as a kitchen. Python is how you arrange the work, arrays are trays of ingredients, an algorithm is a recipe, and evaluation is tasting food that was not used to adjust the recipe. Good code, good mathematics and honest experiments must work together.

## 1. Packaging and typed boundaries

A notebook is good for exploration. A package gives reusable code a stable address, dependencies, tests and a version. The companion `labs/python-package` uses a `src` layout, a `pyproject.toml`, a typed public function, unit tests and a wheel build. Run `python -m build` in that folder to create an installable artifact. Type-check with `mypy --strict src` and run its tests with the package installed.

The `src` layout helps catch a common mistake: tests importing a source directory that will not actually be included in the built distribution. A type hint documents and checks a contract for static analysis; it does not automatically validate untrusted runtime JSON. Validate boundary data separately. Prefer a small protocol when you need interchangeable model providers, rather than having domain code import every provider SDK.

```
# lab: typed_predictor_boundary
from typing import Protocol, Sequence
class Predictor(Protocol):
    def predict(self, rows: Sequence[float]) -> list[float]: ...
class DoublePredictor:
    def predict(self, rows: Sequence[float]) -> list[float]:
        return [2*x for x in rows]
def mean_prediction(model: Predictor, rows: Sequence[float]) -> float:
    if not rows:
        raise ValueError('empty data')
    predictions = model.predict(rows)
    if len(predictions) != len(rows):
        raise ValueError('one prediction required per row')
    return sum(predictions)/len(predictions)
assert mean_prediction(DoublePredictor(), [1, 2, 3]) == 4
print('Domain code depends on a prediction contract')
```

## 2. Debugging and profiling with a hypothesis

Start with the smallest reproducible input. Read the full traceback from the failing operation outward. Inspect shapes, dtypes, missing values and units before changing an optimizer. An array shaped  $(n,)$  can broadcast against  $(n,1)$  into  $(n,n)$ , creating a wrong loss that still runs.

```
# lab: broadcasting_bug
import numpy as np
prediction = np.array([[1.], [2.], [3.]])
target = np.array([1., 2., 3.])
wrong = prediction-target
assert wrong.shape == (3, 3)
correct = prediction.ravel()-target
assert correct.shape == (3,) and np.all(correct == 0)
print('Shape validation catches a silent broadcasting error')
```

Use `breakpoint()` to inspect a local failing state. Use `cProfile` for Python call costs and a monotonic timer for wall time. Profile representative workloads with warmup where appropriate. A GPU call may be asynchronous, so timing its launch alone does not measure completion. First prove correctness, then identify the dominant cost; replacing readable code with a clever one-liner is not evidence of a speed improvement.

```
# lab: profile_and_memory
import cProfile, io, pstats, tracemalloc
```

```

profiler = cProfile.Profile()
profiler.enable()
total = sum(i*i for i in range(10000))
profiler.disable()
buffer = io.StringIO()
pstats.Stats(profiler, stream=buffer).sort_stats('cumulative').print_stats(3)
assert 'function calls' in buffer.getvalue() and total > 0
tracemalloc.start()
items = [str(i) for i in range(5000)]
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
assert peak >= current > 0
print('Profile captured; Python allocation peak observed')

```

tracemalloc does not measure every native or GPU allocation. NumPy slices may be views retaining a large base allocation; `.copy()` owns separate storage but costs memory. Generators reduce materialization but do not magically release everything their closures reference. Close file handles with `with`, clear unnecessary references, and measure resident memory for whole-process investigation.

### 3. Floating-point arithmetic and stable transformations

A floating-point number has finite precision and range. Adding a tiny value to a huge value may lose the tiny contribution. Exponentiating large logits may overflow. Softmax is unchanged when the same constant is subtracted from every logit, so subtract the maximum before exponentiating. Log-sum-exp uses the same idea.

```

# lab: stable_probabilities
import numpy as np
def softmax(logits):
    shifted = np.asarray(logits, dtype=float) - np.max(logits)
    exp = np.exp(shifted)
    return exp/exp.sum()
def logsumexp(values):
    values = np.asarray(values, dtype=float)
    maximum = values.max()
    return maximum + np.log(np.exp(values - maximum).sum())
p = softmax([1000, 1001, 1002])
assert np.isfinite(p).all() and np.isclose(p.sum(), 1)
assert np.allclose(p, softmax([0, 1, 2]))
assert np.isfinite(logsumexp([1000, 1001]))
print('Large logits produce finite normalized probabilities')

```

This routine assumes a nonempty finite input. Production utilities must define behavior for NaN, infinity and fully masked vectors. For binary logistic loss use `logaddexp(0, z) - y*z`; taking `log(sigmoid(z))` directly can underflow. Scaling features improves optimizer conditioning; fit the scaling parameters on training data only.

### 4. Statistics that explain uncertainty

The population is the collection we care about; the sample is what we observed. A sample mean estimates a population mean. Variance measures spread around the mean. Standard error describes variability of an estimator across repeated samples, not variability of individual observations. Under independent sampling, the mean's estimated standard error is sample standard deviation divided by the square root of sample size.

A confidence interval is a procedure with long-run coverage under its assumptions, not a probability that a fixed parameter moves around. A p-value is the probability, under a stated null model, of a result at least as incompatible as the observed one according to the test statistic. It is not the probability that the null hypothesis is true. Effect size and uncertainty matter more than a lone threshold.

```

# lab: bootstrap_mean
import numpy as np
rng = np.random.default_rng(42)
sample = np.array([2., 3., 3., 4., 5., 6., 7., 8.])
resamples = rng.choice(sample, size=(3000, len(sample)), replace=True).mean(axis=1)
low, high = np.quantile(resamples, [0.025, 0.975])

```

```
assert low < sample.mean() < high
print({'mean':sample.mean(),'bootstrap_interval':(round(low,2),round(high,2))})
```

Bootstrap resamples approximate repeated sampling from the empirical distribution. They do not repair selection bias or dependence. For time series use a method that preserves relevant dependence, such as suitable blocks, and assess its assumptions. For users with many sessions, split by user to avoid information leakage. Repeatedly tuning on the test set turns it into development data.

For comparing two models, evaluate the same held-out cases so differences can be paired. Stratify important categories, inspect label quality, and avoid treating correlated prompts as independent samples. An accuracy increase on mostly easy cases can coexist with worse behavior on critical rare cases.

## 5. Linear regression from scratch

We predict a number with  $\text{prediction} = Xw + b$ . The mean squared error averages squared mistakes. Its gradient with respect to weights is  $2 X^T(\text{prediction}-y)/n$ ; the bias gradient is twice the mean error. Move opposite these gradients by a small learning rate. Here each feature row has one number, and the true rule is  $3x+2$ .

```
# lab: scratch_linear_regression
import numpy as np
X=np.arange(6,dtype=float).reshape(-1,1)/5
y=3*X[:,0]+2
w=np.zeros(1); b=0.
for _ in range(3000):
    error=X@w+b-y
    w-=0.1*(2*X.T@error/len(y))
    b-=0.1*2*error.mean()
assert np.allclose(w,[3],atol=1e-5) and abs(b-2)<1e-5
print('Recovered slope and intercept:',w.round(3),round(b,3))
```

The loss is convex for linear least squares, but convergence speed depends on conditioning and step size. A very large step can diverge. Collinear features can make coefficients non-unique or unstable. Ridge adds a squared-weight penalty; it trades some bias for stability. Prediction quality and interpretability of a coefficient are different questions, especially when features correlate.

## 6. Logistic regression from scratch

For a binary label, compute a score  $z=Xw+b$ , then sigmoid  $1/(1+\exp(-z))$ . This models the probability of label one. The cross-entropy gradient with respect to the score is  $p-y$ , so weight updates resemble linear regression with a different prediction and loss. A threshold chooses a class; the threshold should reflect costs, not automatically be 0.5.

```
# lab: scratch_logistic_regression
X=np.array([[ -2. ],[ -1. ],[ 1. ],[ 2. ]])
y=np.array([0.,0.,1.,1.]);w=np.zeros(1);b=0.
for _ in range(1000):
    z=X@w+b
    p=1/(1+np.exp(-np.clip(z,-40,40)))
    w-=0.1*(X.T@(p-y)/len(y))
    b-=0.1*(p-y).mean()
loss=np.mean(np.logaddexp(0,X@w+b)-y*(X@w+b))
assert np.array_equal((X@w+b>0).astype(int),y.astype(int))
assert loss<0.02
print('Binary labels learned; stable cross-entropy:',round(float(loss),4))
```

On perfectly separable data, unregularized coefficients can keep increasing even when every classification is correct. That is why accuracy alone does not tell you whether optimization is well behaved. Calibration asks whether predicted probabilities correspond to observed frequencies; ranking performance and calibration are different properties.

## 7. Neighbors, clustering, and principal components

K-nearest neighbors stores training examples and votes among nearby ones. “Nearby” depends on the distance metric and scaling. K-means alternates assigning each point to its nearest center and replacing each center with its assigned mean. Its objective is within-cluster squared Euclidean distance; it does not discover every possible cluster shape. PCA finds orthogonal directions explaining maximal centered variance; high variance is not automatically high predictive value.

```
# lab: scratch_knn_kmeans_pca
from collections import Counter
def knn(train, labels, point, k):
    order=np.argsort(np.sum((train-point)**2,axis=1),kind='stable')[:k]
    counts=Counter(labels[i] for i in order)
    return sorted(counts,key=lambda label:(-counts[label],label))[0]
points=np.array([[0.,0.],[0.,1.],[8.,8.],[9.,8.]])
assert knn(points,[0,0,1,1],np.array([0.,0.2]),3)==0
centers=points[[0,2]].copy()
for _ in range(20):
    labels=((points[:,None,:]-centers[None,:,:])**2).sum(axis=2).argmin(axis=1)
    updated=np.array([points[labels==k].mean(axis=0) if np.any(labels==k) else centers[k] for k in range(2)])
    if np.allclose(updated,centers):break
    centers=updated
assert labels.tolist()==[0,0,1,1]
centered=points-points.mean(axis=0)
_,singular,directions=np.linalg.svd(centered,full_matrices=False)
projected=centered@directions[:1].T
assert projected.shape==(4,1) and singular[0]>=singular[1]
print('Neighbors, alternating cluster updates, and PCA projection verified')
```

The K-means empty-cluster policy here keeps the previous center; other policies reinitialize it. Try several initializations and compare the objective. For PCA, center using the training mean and reuse that mean at inference. Standardize features when their units would otherwise dominate the variance objective. Singular vectors may flip sign without changing the subspace.

## 8. A recursive decision tree from scratch

A tree asks a question, sends examples left or right, and repeats. Classification impurity measures how mixed the labels are. Gini impurity is  $1 - \sum(\text{class\_probability}^2)$ : zero means all labels agree. Choose a split minimizing weighted child impurity, then stop when labels agree, depth is exhausted, or no useful split exists.

```
# lab: scratch_decision_tree
def gini(labels):
    if not len(labels): return 0.
    counts=np.unique(labels,return_counts=True)[1]/len(labels)
    return 1-float(np.sum(counts**2))
def fit_tree(X,y,depth=3):
    values,counts=np.unique(y,return_counts=True)
    leaf=int(values[counts.argmax()])
    if depth==0 or len(values)==1:return leaf
    best=None
    for column in range(X.shape[1]):
        unique=np.unique(X[:,column])
        for threshold in (unique[:-1]+unique[1:])/2:
            left=X[:,column]<=threshold
            score=(left.sum()*gini(y[left])+(~left).sum()*gini(y[~left]))/len(y)
            if best is None or score<best[0]:best=(score,column,float(threshold),left)
    if best is None:return leaf
    _,column,threshold,left=best
    return (column,threshold,fit_tree(X[left],y[left],depth-1),fit_tree(X[~left],y[~left],depth-1))
def tree_predict(tree,row):
    if isinstance(tree,int):return tree
    column,threshold,left,right=tree
    return tree_predict(left if row[column]<=threshold else right,row)
X=np.array([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y=np.array([0,1,1,0])
tree=fit_tree(X,y)
```

```
assert [tree_predict(tree,row) for row in X]==y.tolist()
print('Recursive splits represent XOR; one linear boundary cannot')
```

This tiny tree intentionally permits an initial split with no immediate impurity improvement, allowing XOR to be represented. Production tree algorithms often use minimum-gain thresholds and additional regularization. Trees overfit with unrestricted depth. Bagging reduces variance by averaging independently perturbed trees; boosting adds learners to correct an objective's current errors. These are different ensemble mechanisms.

## 9. Backpropagation is the chain rule applied repeatedly

For a two-layer network,  $H = \tanh(XW_1 + b_1)$  and  $Y = HW_2 + b_2$ . Backward propagation multiplies each upstream derivative by the local derivative. Tanh's derivative is  $1 - H^2$ . A finite-difference check perturbs one parameter slightly and compares the observed loss slope to the analytic gradient. This catches transposes, missing averages and wrong activation derivatives before a long training run.

```
# lab: manual_backprop_gradient_check
rng=np.random.default_rng(3)
X=rng.normal(size=(4,2));target=rng.normal(size=(4,1))
W1=rng.normal(size=(2,3));W2=rng.normal(size=(3,1))
def loss_and_gradient(A):
    H=np.tanh(X@A);error=H@W2-target
    loss=float(np.mean(error**2))
    dY=2*error/len(X)
    dA=X.T@((dY@W2.T)*(1-H**2))
    return loss,dA
loss,gradient=loss_and_gradient(W1)
eps=1e-6;plus=W1.copy();minus=W1.copy()
plus[0,1]+=eps;minus[0,1]-=eps
numeric=(loss_and_gradient(plus)[0]-loss_and_gradient(minus)[0])/(2*eps)
assert np.isclose(numeric,gradient[0,1],rtol=1e-5,atol=1e-6)
print('Analytic backprop matches a numerical loss derivative')
```

For outputs with multiple columns, account for the loss's exact averaging convention. Finite differences are expensive and sensitive to step size; use small double-precision examples. Automatic differentiation computes gradients, but it cannot tell you whether your loss expresses the intended problem.

## 10. Design exercise with answer key

**Task, 20 points:** Package a classifier behind a typed predictor, reject empty or malformed data, split before preprocessing, record a baseline, measure memory and latency, and explain one failure.

Award 3 points for an installable package and tests; 3 for boundary validation and types; 4 for leakage-safe evaluation; 4 for a correct from-scratch algorithm and gradient or invariant check; 3 for measured performance with workload details; 3 for a specific limitation. Do not award “production-ready” points merely because training accuracy is 100%.

**Debugging answer:** If loss has shape  $(n, n)$  and should have shape  $(n, )$ , fix shapes at the boundary before tuning learning rate. **Statistics answer:** A narrow confidence interval around a biased estimate does not make it correct. **Design answer:** Keep training, prediction, storage and transport separate enough to test independently, but do not create an interface for every single function without a reason.

## Deep workshop - Compute a gradient update with every number visible

Fit prediction =  $w \cdot x + b$  to points (1,2) and (2,4). Begin  $w=0$ ,  $b=0$ . Define mean squared loss as  $((p_1 - y_1)^2 + (p_2 - y_2)^2)/2$ . Predictions are 0, errors are -2 and -4, and loss is  $(4+16)/2=10$ .

## Derive the movement

The derivative of squared error with respect to prediction is twice the error. Multiply by  $x$  to account for how  $w$  changes prediction. Thus  $dL/dw = (2/2)*((-2)*1+(-4)*2) = -10$ . Likewise  $dL/db = (2/2)*(-2-4) = -6$ . At learning rate 0.1, subtract the gradient:  $w=1$  and  $b=0.6$ . New predictions are 1.6 and 2.6. Errors are -0.4 and -1.4, so loss becomes  $(0.16+1.96)/2=1.06$ . This update improved training loss. It proves neither generalization nor that every larger learning rate works.

## Scaling and verification

A feature measured in millions can dominate gradients relative to a feature measured in fractions. Fit scaling parameters on training data only. Handle constant columns explicitly to avoid division by zero. Approximate a derivative with  $(L(w+\epsilon)-L(w-\epsilon))/(2*\epsilon)$ . Compare with the analytic derivative on a small synthetic example. Extremely tiny epsilon can worsen cancellation; try several values and use tolerance rather than exact equality. A gradient check diagnoses calculus or implementation errors; validation performance diagnoses a different problem.



# Notebook 10 - Java and Spring beyond the happy path

A Java service is like a busy school office. Objects hold records, threads are clerks, the database is the official register, transactions protect related changes, and security checks who may request them. Faster clerks cannot repair a broken register contract. This notebook connects language rules to measurable service behavior.

## 1. JVM memory and garbage collection

Each thread has a stack of call frames, including local variables and references. Objects generally live on the heap, subject to runtime optimizations. Class metadata uses metaspace; native libraries, direct buffers, thread stacks and code caches also consume memory. -Xmx limits the Java heap, not total process memory. An out-of-memory failure can therefore occur even when a heap chart looks comfortable.

Garbage collection identifies unreachable objects; it does not mean “delete an object immediately after its last visible use.” A static map retaining every request can keep objects reachable forever. Closing a database connection or file is a resource-lifetime responsibility, not a job to leave to the collector. Use try-with-resources for AutoCloseable objects.

Different collectors and configurations trade throughput, pause times and memory. Begin with workload evidence rather than memorizing one universal best collector. Record allocation rate, live heap after collection, pause distribution, CPU and request latency. Use JVM tools such as Java Flight Recorder, jcmd, and GC logs with appropriate permissions. A heap dump may contain credentials and user data; handle it as sensitive data.

**Worked diagnosis:** Heap use rises during each batch and returns to roughly the same baseline after collection: likely allocation pressure. The post-collection baseline rises steadily with retained request entries: suspect retention. CPU is idle but latency is high and all workers await connections: investigate a saturated connection pool, not only GC.

## 2. Generics, variance, and erasure

A `List<Integer>` is not a `List<Number>`: if it were, a caller could add a `Double` into the integer list. A producer that gives you numbers can use `? extends Number`; a consumer that accepts integers can use `? super Integer`. This is the idea behind “producer extends, consumer super.” Type erasure means most generic type arguments are unavailable as ordinary runtime class distinctions; do not use unchecked casts as a substitute for validation.

```
// lab: GenericTransfer
import java.util.*;
public class GenericTransfer {
    static <T> void copy(List<? extends T> source, List<? super T> target){
        for(T value:source)target.add(value);
    }
    static double total(List<? extends Number> values){
        return values.stream().mapToDouble(Number::doubleValue).sum();
    }
    public static void main(String[] args){
        List<Integer> integers=List.of(1,2,3);
        List<Number> numbers=new ArrayList<>();copy(integers,numbers);
        assert total(numbers)==6;
        System.out.println("A producer of integers can feed a consumer of numbers");
    }
}
```

Use immutable records for value-like data when their members support the intended immutability. A record containing a mutable list is only shallowly immutable; defensively copy the list if needed. `equals` and `hashCode` must agree. Mutating a key field after inserting an object into a hash map can make lookup fail.

### 3. Concurrency: visibility, atomicity and ordering

`count++` reads, adds and writes; those operations are not one atomic action. `volatile` supplies visibility and ordering guarantees for the variable but does not turn arbitrary compound operations into transactions. Locks protect invariants involving several values. Atomic classes support specific atomic operations. A thread-safe collection does not automatically make “check then act” across several calls atomic.

```
// lab: BoundedPipeline
import java.util.concurrent.*;
public class BoundedPipeline {
    public static void main(String[] args) throws Exception{
        BlockingQueue<Integer> queue=new ArrayBlockingQueue<>(2);
        try(var executor=Executors.newVirtualThreadPerTaskExecutor()){
            Future<Integer> consumer=executor.submit(()->{
                int sum=0;
                while(true){int value=queue.take();if(value==-1)return sum;sum+=value;}
            });
            Future<?> producer=executor.submit(()->{
                try{for(int i=1;i<=5;i++)queue.put(i);queue.put(-1);}
                catch(InterruptedException e){Thread.currentThread().interrupt();throw new RuntimeException(e);}
            });
            producer.get(2,TimeUnit.SECONDS);
            assert consumer.get(2,TimeUnit.SECONDS)==15;
        }
        System.out.println("A bounded queue applies backpressure between producer and consumer");
    }
}
```

The sentinel works because this example has one consumer and data never uses `-1`. Multiple consumers need a termination protocol suitable for their number, or explicit cancellation. Virtual threads make many blocking tasks cheaper than one platform thread per task; they do not create unlimited database connections, memory or remote capacity. Bound scarce resources separately.

Deadlock requires a cycle of waiting. Consistent lock ordering can prevent such cycles. Timeouts reduce indefinite waiting but require a safe recovery policy. On interruption, release resources and preserve or propagate the interrupt according to your API contract. Do not swallow it and continue expensive work.

### 4. Spring dependency injection and proxy behavior

Constructor injection makes required dependencies explicit and supports ordinary unit tests. The container creates and connects beans. A singleton bean is shared within the application context; it must not keep request-specific mutable fields. A controller field named `currentUser` is a cross-request bug, even if the first manual test succeeds.

Spring often applies features through proxies. In default proxy-based transaction management, a call from one method to another method on the same object can bypass transactional advice. Put the transaction boundary on a service method called through its managed bean. Read the [transaction annotation reference](#) for proxy and rollback behavior. Configure rollback rules deliberately; do not assume every checked exception rolls back under every configuration.

Transactions should protect database invariants, not wrap a slow model call by default. Holding locks and connections while a remote model responds reduces capacity and increases contention. Commit a durable job first, then let a worker call the external service outside the short database transaction.

## 5. Isolation with a concrete race

Two clerks read that one seat remains. Both subtract one in their own memory and write zero. Both customers receive a seat. The final counter looks valid, but the business invariant was broken. Possible solutions include an atomic conditional update (`UPDATE ... SET seats=seats-1 WHERE seats>0`), optimistic version checks, row locking, or a stronger transaction design. Select according to contention, retry cost and database behavior.

Read committed generally prevents dirty reads. Repeatable read and serializable offer stronger guarantees, but exact behaviors and anomalies differ among database engines. Do not infer PostgreSQL or MySQL semantics from an H2 test alone. Serialization failures and deadlocks may require retrying the complete transaction, with bounded attempts and idempotent external behavior.

The Study Coach `PersistenceTests` uses two actual database connections to show that an uncommitted row is hidden at read committed and becomes visible after commit. It separately tests optimistic locking: two detached versions are loaded; one update wins and the stale version is rejected.

## 6. JPA performance: measure the SQL

JPA maps object operations onto SQL. Lazy loading delays a relationship query; eager loading requests early availability but does not guarantee one efficient SQL statement. The N+1 problem occurs when one query loads N parents and then one query per parent loads its children. In the classroom test, three courses produce four queries. A fetch join produces one query for the same small result.

`CourseRepository.withLessons()` uses a fetch join. `PersistenceTests.fetchJoinAvoidsNPlusOne` clears the persistence context and Hibernate statistics before each measurement, then asserts four versus one prepared statements. Without clearing the first-level cache, the second measurement could appear fast merely because the objects were already loaded.

Fetch joins are not universally free: joining large collections duplicates parent columns, can multiply rows across several collections, and complicates pagination. Alternatives include projections, entity graphs, batch fetching, and two-step ID pagination. Return API DTOs rather than exposing an arbitrary entity graph to JSON serialization. Open-session-in-view can hide missing query planning by triggering queries during rendering; this project disables it.

## 7. Messaging and the transactional outbox

Suppose you insert an enrollment and then publish a message. A crash between these steps leaves an enrollment with no event. Reversing the order can publish an event for a transaction that later rolls back. The outbox stores the business change and a pending event in the same database transaction. A separate relay publishes pending events.

The `Enrollment` service and its integration test show both records rolling back together on a deliberate exception, then both committing on success. This is the producer-side atomicity part of the pattern. The project does not include a Kafka or RabbitMQ broker. A production relay must handle claims, retries, duplicate publishing and ordering; consumers need idempotency because a relay can crash after publish but before marking delivery.

An event describes something that happened; a command requests an action. Choose schemas, partition keys and ordering requirements explicitly. A dead-letter queue is a holding area for investigation, not a way to declare lost work successful. Record why an item failed and how it may be replayed safely.

## 8. Cache correctness before cache speed

A cache stores a reusable result. Its key must include every input that affects the result, including identity or permission scope where relevant. Omitting the owner from a retrieval cache key can leak another user's data. Decide TTL, size bound, invalidation and behavior during a miss storm. Do not cache failures forever.

```
// lab: ScopedCache
import java.util.*;
public class ScopedCache {
    record Key(String owner,String query){}
    record Entry(String value,long expires){}
    static class Cache {
        final Map<Key,Entry> entries=new LinkedHashMap<>();
        final int capacity;
        Cache(int capacity){if(capacity<1)throw new IllegalArgumentException();this.capacity=capacity;}
        synchronized void put(Key key,String value,long now,long ttl){
            entries.remove(key);entries.put(key,new Entry(value,now+ttl));
            while(entries.size()>capacity)entries.remove(entries.keySet().iterator().next());
        }
        synchronized Optional<String> get(Key key,long now){
            Entry item=entries.get(key);
            if(item==null)return Optional.empty();
            if(item.expires()<=now){entries.remove(key);return Optional.empty();}
            return Optional.of(item.value());
        }
        synchronized void invalidate(Key key){entries.remove(key);}
    }
    public static void main(String[] args){
        Cache cache=new Cache(2);Key alice=new Key("alice","plan");
        cache.put(alice,"private",0,10);
        assert cache.get(new Key("bob","plan"),1).isEmpty();
        assert cache.get(alice,10).isEmpty();
        cache.put(alice,"new",10,10);cache.invalidate(alice);
        assert cache.get(alice,11).isEmpty();
        System.out.println("Cache keys include owner; expiration and invalidation are tested");
    }
}
```

This small cache evicts by insertion order and synchronizes operations; it is not a distributed cache or a production caching library. For expensive loads, coalesce simultaneous misses. For updates, explain the ordering between commit and invalidation. Strong consistency requirements may make a cache inappropriate.

## 9. Security and integration tests

The connected application uses Spring Security, password hashing, HTTP Basic for the local teaching session, CSRF protection, owner-filtered database queries, and an internal service credential for Python. Basic credentials must travel over TLS beyond local development. They are not encryption. The browser page keeps its explicit authorization header in memory and does not store it in localStorage. Forgetting page credentials is not server-side session revocation.

Read [Spring's Basic authentication documentation](#) for filter behavior. Test absent authentication, incorrect credentials, missing CSRF token, another user's job ID, malformed input and every state-changing endpoint. A controller unit test cannot establish these filters work; use integration and network tests as well.

## 10. Interview exercise and rubric

**Task, 20 points:** Design enrollment plus notification under concurrency. Award 4 points for the seat invariant and a valid atomic strategy, 3 for transaction boundaries, 3 for an outbox and idempotent consumer, 3 for JPA query planning, 3 for identity/ownership checks, and 4 for failure tests covering rollback, duplicate delivery and concurrent attempts.

**Follow-up:** Would synchronized on one service method solve double booking across three servers? No; its lock exists only in one JVM. **Follow-up:** Would adding `@Transactional` everywhere solve N+1? No; transaction scope and query shape are separate. **Follow-up:** Does a cache always improve latency? A remote cache, serialization overhead or a miss storm can make it worse. Measure the complete path.

## Deep workshop - Catch a lost update with a two-person schedule

A stock row holds quantity 1. Two transactions both read 1, both approve a sale, and both write 0. The final row looks reasonable, but two sales were accepted for one item. Inspecting only the final quantity misses the violated invariant.

### Make the invariant executable

One strategy is `UPDATE stock SET quantity=quantity-1 WHERE id=? AND quantity>0`. Accept only if one row changed. Predicate and update form one statement under the database's concurrency rules. Alternatives include row locking before reading or optimistic version checks with explicit conflict handling.

### Find the actual transaction

A Spring proxy can start a transaction when another object calls the proxied method. A same-object call usually bypasses that proxy. Put the transactional operation behind a separately injected service when that makes the boundary clearer. A public method annotation alone is not evidence of interception.

### Follow SQL, not annotations

Loading 20 orders, then reading each customer's lazy details, may issue 21 statements. Measure SQL and latency. Fetch joins, projections, or batch fetching can help, but joining multiple collections can multiply rows and complicate pagination.

### Retry question

Should every conflict be retried? **No:** retry only known retryable failures, with a bounded budget and a fresh transaction. Reusing a failed transaction or repeating a remote effect can worsen correctness. Test simultaneous buyers, not just a sequential pair of requests.

# Notebook 11 - React, TypeScript, browser behavior and real tests

A React screen is a picture calculated from current inputs. Events request new inputs; React calculates another picture. The browser still handles HTML, focus, network requests and its event loop. Understanding those layers makes framework behavior less mysterious.

## 1. Closures capture bindings and render snapshots

A closure is a function that keeps access to its lexical environment. In React, each render creates its own bindings. A callback created during an old render may still read that render's state value. That is often intentional, but can surprise you in timers and asynchronous callbacks. A functional state update asks React to calculate from the pending state rather than an old captured number.

```
// lab: closure_snapshot
import assert from 'node:assert/strict';
function makeRender(count){return {later:()=>count,increment:previous=>previous+1};}
const first=makeRender(0),second=makeRender(1);
assert.equal(first.later(),0);assert.equal(second.later(),1);
const updates=[first.increment,first.increment,first.increment];
assert.equal(updates.reduce((value,update)=>update(value),0),3);
console.log('Old callbacks retain their render binding; functional updates compose');
```

This JavaScript model illustrates closures; it is not an implementation of React's scheduler. In a component, use `setCount(c => c + 1)` when the next value depends on previous state. If an effect needs current reactive values, declare its dependencies or redesign the effect. Suppressing the dependency warning does not repair stale data.

## 2. Event loop, promises and rendering opportunities

JavaScript executes the current task until it yields. Promise callbacks run as microtasks after the current synchronous work, before the next task. A timer schedules later work; a zero delay does not mean immediate execution. Long synchronous work prevents input handling and painting, even if it is inside an async function before its first meaningful await.

```
// lab: event_loop_order
import assert from 'node:assert/strict';
const events=[];
events.push('sync-start');
Promise.resolve().then(()=>events.push('microtask'));
const timer=new Promise(resolve=>setTimeout(()=>{events.push('timer');resolve();},0));
events.push('sync-end');
await timer;
assert.deepEqual(events,['sync-start','sync-end','microtask','timer']);
console.log(events.join(' -> '));
```

Node and browsers have additional scheduling details that differ. This example tests only the basic ordering it asserts. Use a Web Worker for suitable CPU-heavy browser computation rather than assuming a promise moves work to another CPU thread. React transitions prioritize certain state updates; they do not turn arbitrary synchronous code into parallel computation.

### 3. TypeScript makes impossible combinations harder to express

Instead of independent loading, error and data booleans that can contradict each other, use a discriminated union. A request is idle, loading, successful with data, or failed with an error. Narrow on its kind before accessing fields. TypeScript checks compile-time consistency; parse and validate external JSON because a cast does not establish runtime truth.

```
type Remote<T> =
  | {kind:'idle'}
  | {kind:'loading'}
  | {kind:'success'; data:T}
  | {kind:'error'; message:string};
function caption<T>(state:Remote<T>):string {
  switch(state.kind){
    case 'idle': return 'Ready';
    case 'loading': return 'Loading';
    case 'success': return 'Loaded';
    case 'error': return state.message;
  }
}
```

The Study Coach frontend is actual .tsx compiled with strict TypeScript. Its Job and Session types describe the Java contract. A next production step is runtime response validation or a generated contract shared with the API, with explicit compatibility tests when fields evolve. TypeScript alone cannot detect a backend returning a malformed answer.

### 4. Local state versus server state

Local state belongs to the interaction: the typed question, a selected tab, whether a dialog is open. Server state is a cached observation of remote truth: job status, permissions, persisted answers. A network request can fail, become stale or finish out of order. Keep an explicit policy for retries, refetching and invalidation.

```
// lab: stale_response_guard
import assert from 'node:assert/strict';
let generation=0,display='';
function startRequest(){const mine=++generation;return value=>{if(mine===generation)display=value;}}
const finishOld=startRequest(),finishNew=startRequest();
finishNew('new answer');finishOld('old answer');
assert.equal(display,'new answer');
console.log('A generation check prevents an old response overwriting a newer one');
```

Cancellation saves unnecessary work where supported. A generation check protects correctness even when cancellation arrives too late. Query-cache libraries can manage deduplication and stale/refetch policies, but their keys must include relevant identity and query parameters. Clear or partition cached data when the signed-in user changes. An optimistic update needs rollback or reconciliation if the server rejects it.

The connected app uses a deliberately small explicit fetch implementation: create a job, approve it, receive server snapshots, and reconnect by reading current state. It sends full answer snapshots instead of append-only deltas, so reconnecting cannot duplicate already displayed text. That increases bandwidth but simplifies this teaching example.

### 5. Effects synchronize with external systems

An effect is appropriate for connecting to a stream, subscribing to browser events, or synchronizing an external widget. Calculating a filtered list from props generally belongs in render, not an effect that sets another state variable. User-triggered submission belongs in the event handler. The official [effect guidance](#) explains why unnecessary effects introduce additional render cycles and synchronization problems.



The app's route effect subscribes to hashchange and removes the same listener on cleanup. Its unmount cleanup aborts the answer stream. Starting a second stream aborts the previous one. A production implementation should also guard any final update from a previous session and validate received events. React development checks that rerun setup and cleanup help expose missing cleanup; they are not a reason to disable correctness checks.

## 6. Routing, URLs and state restoration

A route should identify a meaningful view. The app has #learn and #architecture, with browser back/forward behavior supplied by hash navigation. This small implementation is enough to demonstrate routing without a routing library. A larger application needs nested routes, unknown-route handling, loaders, permission-aware navigation and perhaps server fallback configuration.

A job ID in the URL could restore selection after a refresh, but the backend must still authorize every read. A hidden link is not a security boundary. Never put passwords or bearer tokens in URLs, where history and logs can retain them. A resource identifier may be public-looking and still require authorization.

## 7. Rendering performance and keys

Before optimizing, reproduce a slow interaction and inspect a profiler trace. Expensive computation, excessive DOM nodes, repeated network calls and unnecessary component renders have different fixes. Memoization stores a result to avoid recomputation; it adds comparison and memory costs. useMemo is not a semantic guarantee that a value is permanent, and useCallback does not prevent work unless referential identity actually matters downstream.

Stable keys identify sibling items across renders. Array-index keys are dangerous for reorderable editable lists because component state can attach to the wrong item after reordering. Use a stable domain ID. For long lists, virtualization reduces mounted elements but introduces keyboard, measurement and accessibility concerns. Avoid optimizing a ten-row list while ignoring a two-second network waterfall.

## 8. SSR and hydration

Server rendering produces initial HTML. Hydration connects React's behavior to compatible existing HTML. The server and first client render must agree; rendering the current time or random value independently on both sides can cause a mismatch. Browser-only APIs do not exist during server rendering. Serialize required initial data safely and keep server secrets out of client bundles.

The [hydrateRoot reference](#) documents the API's expectations. Study Coach uses client rendering with createRoot; it does not pretend to be an SSR deployment. Its separate hydration test renders a deterministic component on the server and hydrates the same tree in a DOM test, checking that an interaction still works. A complete framework SSR deployment additionally needs routing, asset manifests, streaming, caching and request isolation.

## 9. Accessibility is behavior as well as markup

Use native buttons, inputs and links when their behavior fits. Associate labels with controls, preserve visible focus, choose heading levels by structure, and provide status/error announcements. A clickable div does not acquire keyboard semantics automatically. Color alone should not convey completion or failure. Test zoom, reflow and keyboard operation as well as automated rules.



The real Chromium test signs in with Enter, creates a job, focuses the approval button and activates it with Enter, waits for the answer, checks routing, and runs axe against the completed page. Another browser test cancels before approval. Passing those tests does not establish accessibility of every conceivable future page; they verify the included flows and automated rule set.

## 10. Test selection and interview exercise

Use pure-function tests for transformations, component tests for rendered behavior, integration tests for actual API contracts, and browser tests for important journeys. Prefer selecting by role and accessible name rather than implementation-specific CSS. Do not mock the exact boundary whose integration you are trying to prove.

**Debugging round, 10 points:** Search results sometimes jump back to an old query. Award 3 points for identifying out-of-order completion, 2 for cancellation, 2 for a generation/identity guard, 2 for a test that completes the second request first, and 1 for separating the displayed query from the request that produced the result.

**Design round, 10 points:** Explain why React state is not the authoritative payment record (3), how the server validates and deduplicates submission (3), how pending/error states are shown accessibly (2), and how retry differs from a new purchase (2).

**Mastery task:** Add a job-history route backed by an owner-filtered API. Preserve keyboard focus when navigating, handle an empty history, test authorization, and avoid keeping one user's cached history after switching users.

## Deep workshop - A stale response is a state-management bug

The user selects customer A, then B. B's response arrives first; A's arrives last. Blindly accepting every response displays A while the selector says B.

### Establish a commit condition

Assign each request a local increasing number. Completion may update visible state only if its number is still current. A server-state library can manage this through correctly scoped query keys and cancellation behaviour. Include every variable that changes the result, especially customer and tenant identity.

### Effects and cleanup

An effect synchronizes with an external system. A subscription effect returns cleanup that removes the subscription. Without it, changed dependencies or unmounting can leave duplicate listeners. Closures still see captured render values; correct dependencies or functional updates are necessary. Suppressing a dependency warning does not repair logic.

### Hydration is a contract

Server HTML and initial client output must agree. Independently rendering random numbers or local current time can produce mismatches. Pass a stable initial value or deliberately render browser-only information after hydration.

### Practice

A query key contains only orders, but the request also depends on customer ID. What fails? **Answer:** Customers share a cache slot. Include identity in the key, scope or clear caches on sign-out, and test reverse response order and back navigation. A correct network response can still be attached to the wrong UI state.

# Notebook 12 - Trees, graphs, backtracking and dynamic programming

An algorithm interview is not a memory contest alone. You must turn a problem into a representation, explain why each step is safe, and test the boundary cases. This notebook extends the basic algorithms in Notebook 6. Every implementation below has an executable check; the explanations give the reasoning that a test alone cannot prove.

## 1. A correctness proof in four small steps

State the precondition: what is true before the algorithm starts? State an invariant: what remains true after every loop iteration? Show progress: what gets closer to finishing? State the postcondition: what is true when it stops? For recursion, identify a smaller subproblem and a base case. Testing finds counterexamples; a proof argues about all allowed inputs.

For binary search, the invariant can be “every index below  $lo$  contains a value below the target, and every index at or above  $hi$  contains a value at least the target.” Each comparison shrinks  $[lo, hi)$ . At termination the interval is empty and  $lo$  is the insertion boundary. Without sorted input, that proof fails even if a few examples pass.

Complexity describes how resource use grows with input size. Include the call stack, output storage, and representation costs. Expected hash-table performance differs from a worst-case guarantee. An  $O(n)$  algorithm with a huge allocation can lose to a simpler method on small inputs; asymptotics and measurements answer different questions.

## 2. Binary trees and search-tree invariants

A binary tree allows up to two children per node. A binary search tree additionally orders keys: every key in the left subtree is smaller and every key in the right subtree is larger under our no-duplicates convention. Checking only immediate children is insufficient. Carry lower and upper bounds down the entire path.

```
# lab: tree_bounds_and_traversal
from dataclasses import dataclass
@dataclass
class Node:
    value:int
    left:object=None
    right:object=None
def valid_bst(node,low=float('-inf'),high=float('inf')):
    return node is None or (low<node.value<high and valid_bst(node.left,low,node.value) and
    valid_bst(node.right,node.value,high))
def inorder(root):
    result=[];stack=[];node=root
    while stack or node:
        while node:stack.append(node);node=node.left
        node=stack.pop();result.append(node.value);node=node.right
    return result
tree=Node(5,Node(2),Node(8,Node(6),Node(9)))
assert valid_bst(tree) and inorder(tree)==[2,5,6,8,9]
assert not valid_bst(Node(5,Node(2,None,Node(7))))
assert inorder(None)==[]
print('Ancestor bounds catch a violation that local child checks miss')
```

Both procedures take  $O(n)$  time. The explicit traversal stack uses  $O(h)$  space for height  $h$ ; recursive validation also uses  $O(h)$  call depth. An unbalanced tree may have  $h=n$  and exceed Python's recursion limit. Balanced search trees maintain logarithmic height through rotations and balancing rules; a plain BST does not guarantee that.

### 3. Topological order and cycle detection

A directed acyclic graph represents prerequisites. Kahn's algorithm counts incoming edges, starts with nodes having none, and removes their outgoing edges. A node enters the ready queue only after all its prerequisites have been removed. If fewer than all nodes are output, a directed cycle prevented progress.

```
# lab: topological_sort
from collections import deque
def topological(graph):
    nodes=set(graph)|{v for neighbors in graph.values() for v in neighbors}
    indegree={node:0 for node in nodes}
    for neighbors in graph.values():
        for v in neighbors:indegree[v]+=1
    ready=deque(sorted(node for node in nodes if indegree[node]==0));order=[]
    while ready:
        node=ready.popleft();order.append(node)
        for neighbor in graph.get(node,[]):
            indegree[neighbor]-=1
            if indegree[neighbor]==0:ready.append(neighbor)
    if len(order)!=len(nodes):raise ValueError('cycle')
    return order
graph={'python':['agents'],'agents':['project'],'java':['project'],'project':[]}
order=topological(graph);position={v:i for i,v in enumerate(order)}
assert all(position[u]<position[v] for u,neighbors in graph.items() for v in neighbors)
try:topological({'a':['b'],'b':['a']})
except ValueError:pass
else:raise AssertionError('cycle missed')
print('Prerequisites precede dependents; cycles are rejected')
```

Ignoring the initial sorting used for deterministic output, the algorithm is  $O(V+E)$  time and  $O(V)$  auxiliary storage. Sorting ready nodes initially adds up to  $O(V \log V)$  here. Multiple valid orders may exist. Topological ordering is not shortest-path search, and an undirected cycle requires a different interpretation.

### 4. Union-find and minimum spanning trees

Union-find tracks disjoint groups. find returns a representative; union merges two groups. Path compression shortens future searches and union by size prevents avoidable tall trees. Its amortized cost is nearly constant, conventionally expressed using the inverse Ackermann function, for the standard sequence of operations.

```
# lab: union_find_kruskal
class UnionFind:
    def __init__(self,n):self.parent=list(range(n));self.size=[1]*n
    def find(self,x):
        while self.parent[x]!=x:
            self.parent[x]=self.parent[self.parent[x]];x=self.parent[x]
        return x
    def union(self,a,b):
        a,b=self.find(a),self.find(b)
        if a==b:return False
        if self.size[a]<self.size[b]:a,b=b,a
        self.parent[b]=a;self.size[a]+=self.size[b];return True
def kruskal(n,edges):
    groups=UnionFind(n);chosen=[]
    for weight,a,b in sorted(edges):
        if groups.union(a,b):chosen.append((weight,a,b))
    if n and len(chosen)!=n-1:raise ValueError('disconnected graph')
    return chosen
selected=kruskal(4,[(1,0,1),(2,1,2),(8,0,2),(3,2,3),(9,0,3)])
assert sum(w for w,_,_ in selected)==6
assert len(selected)==3
print('Kruskal chooses the lightest safe edges without closing a cycle')
```

The cut property justifies choosing a minimum-weight edge crossing a cut between components. Sorting dominates at  $O(E \log E)$ . A spanning tree connects every vertex with minimum total edge weight; it does not minimize each source-to-destination route. The implementation assumes valid vertex indices and an undirected graph encoded as weighted edges.

## 5. Tries for prefix queries

A trie stores one character per edge. Words sharing a prefix share the same path. A terminal marker distinguishes “a word ends here” from “this is merely a prefix.” Search and insert take  $O(L)$  dictionary steps for word length  $L$ ; space depends on the stored characters and node overhead.

```
# lab: trie_prefix
class Trie:
    END=object()
    def __init__(self):self.root={}
    def insert(self,word):
        node=self.root
        for char in word:node.setdefault(char,{})
        node[self.END]=True
    def walk(self,prefix):
        node=self.root
        for char in prefix:
            if char not in node:return None
            node=node[char]
        return node
    def contains(self,word):
        node=self.walk(word)
        return node is not None and self.END in node

trie=Trie()
for word in ['agent','agency','graph','']:trie.insert(word)
assert trie.walk('age') is not None and not trie.contains('age')
assert trie.contains('agent') and trie.contains('') and not trie.contains('agents')
print('Word termination is distinct from prefix existence')
```

For only exact membership, a hash set is often simpler and smaller. For autocomplete, collect descendants under the prefix but bound results and define ordering. Unicode normalization, case folding and locale rules are product decisions; blindly lowercasing does not solve every language's matching rules.

## 6. Backtracking: choose, explore, undo

Backtracking explores a decision tree while abandoning impossible partial choices. N-queens places one queen per row. A placement is safe if its column and two diagonal identifiers have not been used. When returning from a branch, undo exactly the state added for that choice.

```
# lab: n_queens_backtracking
def queens(n):
    solutions=[];columns=set();down=set();up=set();path=[]
    def visit(row):
        if row==n:solutions.append(tuple(path));return
        for col in range(n):
            if col in columns or row-col in down or row+col in up:continue
            columns.add(col);down.add(row-col);up.add(row+col);path.append(col)
            visit(row+1)
            path.pop();columns.remove(col);down.remove(row-col);up.remove(row+col)
        visit(0);return solutions
    assert len(queens(4))==2 and queens(1)==[(0,)] and queens(2)==[]
    for solution in queens(4):
        assert len(set(solution))==4
        assert len({r-c for r,c in enumerate(solution)})==4
    print('Pruning finds the two valid four-queen arrangements')
```

The search is exponential; a useful loose bound is factorial-style column permutations with polynomial overhead. Output storage can dominate. Returning path itself instead of a copy would make all saved solutions refer to later mutations. This is a frequent interview bug worth explaining before writing code.

## 7. Dynamic programming: state, recurrence, order

Dynamic programming stores answers to overlapping subproblems. For longest common subsequence,  $dp[i][j]$  is the best length using the first  $i$  characters of one string and first  $j$  of the other. Matching final characters add one to  $dp[i-1][j-1]$ . Otherwise take the better of skipping either final character. Fill smaller prefixes first.

```
# lab: lcs_dynamic_programming
def lcs(a,b):
    dp=[[0]*(len(b)+1) for _ in range(len(a)+1)]
    for i in range(1,len(a)+1):
        for j in range(1,len(b)+1):
            dp[i][j]=dp[i-1][j-1]+1 if a[i-1]==b[j-1] else max(dp[i-1][j],dp[i][j-1])
    result=[];i,j=len(a),len(b)
    while i and j:
        if a[i-1]==b[j-1]:result.append(a[i-1]);i-=1;j-=1
        elif dp[i-1][j]>=dp[i][j-1]:i-=1
        else:j-=1
    return ''.join(reversed(result))
assert lcs('abcde','ace')== 'ace' and lcs('','abc')== ''
assert len(lcs('ABCBADAB','BDCABA'))==4
print('Prefix recurrence computes and reconstructs a common subsequence')
```

Time and space are  $O(mn)$ . If only the length is needed, two rows reduce space to  $O(\min(m,n))$ ; reconstructing a sequence then needs additional technique or stored decisions. A subsequence can skip characters; a substring must be contiguous. Confusing the two produces a different recurrence.

## 8. Knapsack and the direction of iteration

For 0/1 knapsack, each item may be used once.  $dp[c]$  is the best value using processed items within capacity  $c$ . Iterate capacities downward so an item cannot reuse its own update in the same pass. Upward iteration would describe an unbounded-use variant.

```
# lab: zero_one_knapsack
def knapsack(weights,values,capacity):
    dp=[0]*(capacity+1)
    for weight,value in zip(weights,values):
        if weight<=0:raise ValueError('positive weights required')
        for c in range(capacity,weight-1,-1):dp[c]=max(dp[c],dp[c-weight]+value)
    return dp[capacity]
assert knapsack([2,3,4],[4,5,7],5)==9
assert knapsack([2],[3],4)==3 # not 6: the item cannot be reused
assert knapsack([],[],5)==0
print('Backward capacity iteration preserves the one-use invariant')
```

This is  $O(nC)$  time and  $O(C)$  space, where  $C$  is numeric capacity. It is pseudo-polynomial because capacity may be exponentially large relative to the number of bits needed to encode it. A greedy value-to-weight ratio solves the fractional version, not arbitrary 0/1 knapsack.

## 9. Negative edges and shortest paths

Dijkstra's greedy finalization relies on nonnegative edge weights. Bellman-Ford repeatedly relaxes every edge. After  $k$  passes, distances are correct for shortest paths using at most  $k$  edges. A shortest simple path uses at most  $V-1$  edges; another improvement afterwards indicates a reachable negative cycle.

```
# lab: bellman_ford
def bellman_ford(n,edges,source):
    dist=[float('inf')]*n;dist[source]=0
    for _ in range(n-1):
        changed=False
        for a,b,w in edges:
            if dist[a]+w<dist[b]:dist[b]=dist[a]+w;changed=True
    if not changed:break
```

```

    if any(dist[a]+w<dist[b] for a,b,w in edges):raise ValueError('reachable negative cycle')
    return dist
assert bellman_ford(3,[(0,1,4),(0,2,5),(1,2,-2)],0)==[0,4,2]
try:bellman_ford(2,[(0,1,1),(1,0,-2)],0)
except ValueError:pass
else:raise AssertionError('negative cycle missed')
print('Negative edges work; a reachable negative cycle is rejected')

```

Worst-case time is  $O(VE)$ , space  $O(V)$  excluding input. An unreachable negative cycle does not affect distances from this source and is not detected by this source-based check. For all-pairs shortest paths, consider repeated suitable single-source algorithms or Floyd-Warshall, depending on density and constraints.

## 10. Timed rounds with alternatives and scoring

**Round A, 20 minutes:** Find duplicate values. Start with pairwise comparison  $O(n^2)$ , then a hash set with expected  $O(n)$  time and  $O(n)$  space, then sorting with  $O(n \log n)$  time if mutation or a copy is acceptable. Award 2 points each for assumptions, working code, complexity, edge cases and tradeoffs.

**Round B, 30 minutes:** Determine whether prerequisites are schedulable. Give Kahn's algorithm and a DFS color alternative. In DFS, gray means on the current recursion path, black means completed. An edge to gray detects a directed cycle; an edge to black does not. Award 3 points for modeling, 3 for correct cycle handling, 2 for complexity and 2 for empty/disconnected cases.

**Round C, 40 minutes:** Solve 0/1 knapsack. Explain brute-force include/exclude recursion, memoization by (item, capacity), and bottom-up optimization. Award 4 points for recurrence, 4 for one-use correctness, 4 for boundary cases, 4 for pseudo-polynomial complexity, and 4 for explaining why the greedy ratio fails.

**Round D, debugging:** A tree validator accepts a left grandchild greater than the root. The missing invariant is the ancestor bound. A DP returns twice an item's value with only one item available. The likely bug is upward capacity iteration. A path algorithm produces nonsense with a negative cycle. The problem has no finite minimum for affected reachable destinations; changing a heap implementation cannot fix the mathematical issue.

Further specialist topics include balanced-tree rotations, segment/Fenwick trees, strongly connected components, max flow, string matching, suffix structures, randomized algorithms and approximation. This notebook supplies core implementations and proof habits, not a claim that the finite list exhausts algorithm research.

## Deep workshop - Why the knapsack loop runs backward

One item has weight 2 and value 3. Bag capacity is 4. In 0/1 knapsack, each item may be used once. Let  $dp[c]$  mean the best value at capacity  $c$  using processed items.

### Watch the wrong loop

All entries start at 0. An upward loop sets  $dp[2]=3$ . Later  $dp[4]$  reads the newly changed  $dp[2]$  and becomes 6. The item was used twice: this is an unbounded-reuse calculation, not the requested problem.

### Preserve the previous layer

Iterate downward from 4 to 2. At  $c=4$ ,  $dp[2]$  still belongs to the previous item layer and is 0, so  $dp[4]=3$ . At  $c=2$ ,  $dp[0]=0$  and  $dp[2]=3$ . Maximum value is 3. The recurrence is  $best(i, c) = \max(best(i-1, c), value[i] + best(i-1, c - weight[i]))$  when the item fits. Both alternatives refer to  $i-1$ . That dependency explains the direction. Time is  $O(\text{items} \times \text{capacity})$ , memory  $O(\text{capacity})$ . This is pseudo-polynomial because the numeric capacity, not its digit count, controls work.

## Transfer the reasoning

Why can an unlimited-coin problem use upward iteration? **Answer:** Reusing the current coin is allowed, so the current layer may depend on itself at a smaller capacity. Derive loop direction from the dependency graph rather than memorizing “DP always runs backward.”

# Notebook 13 - All 23 classic design patterns, implemented and challenged

A pattern is a reusable way to arrange responsibilities. It is not a medal for adding more classes. Each example below is a complete small implementation of the pattern's mechanism with an executable assertion. The examples use Python to keep the mechanism visible; the design roles also apply to Java. Production versions need the surrounding validation, concurrency and resource management appropriate to their domain.

For each pattern, learn four things: the problem, the cooperating roles, a simpler alternative, and the cost. Explain those before naming the pattern in an interview. The 23 patterns here are the classic GoF catalog, not every design pattern ever described.

## 1. Factory Method - subclasses choose the product

A delivery workflow knows it needs a transport but lets a subclass choose the transport implementation. The creator's workflow calls an overridable creation method. This differs from merely putting an if statement in a factory function. Use it when extending a framework through subclasses is already appropriate. A constructor parameter or ordinary factory function is often simpler when inheritance buys nothing.

```
# lab: pattern_factory_method
from abc import ABC, abstractmethod
class Delivery(ABC):
    @abstractmethod
    def transport(self): ...
    def deliver(self,parcel):return self.transport().carry(parcel)
class Bicycle:
    def carry(self,parcel):return f'bike:{parcel}'
class LocalDelivery(Delivery):
    def transport(self):return Bicycle()
assert LocalDelivery().deliver('book')== 'bike:book'
print('Creator workflow uses the subclass-created product')
```

**Failure mode:** one subclass per trivial parameter creates needless hierarchy. **Test idea:** supply a different creator and confirm the shared workflow remains unchanged.

## 2. Abstract Factory - create compatible families

A classroom screen needs both a button and a notice. A family factory creates matching variants so a client does not accidentally mix incompatible families. Use this when several related products must vary together. A theme configuration is simpler when only colors vary; do not create object families for every cosmetic value.

```
# lab: pattern_abstract_factory
class QuietButton:
    def render(self):return 'quiet button'
class QuietNotice:
    def render(self):return 'quiet notice'
class BrightButton:
    def render(self):return 'bright button'
class BrightNotice:
    def render(self):return 'bright notice'
class QuietFactory:
    def button(self):return QuietButton()
    def notice(self):return QuietNotice()
class BrightFactory:
    def button(self):return BrightButton()
    def notice(self):return BrightNotice()
def screen(factory):return (factory.button().render(),factory.notice().render())
assert screen(QuietFactory())==( 'quiet button','quiet notice')
assert screen(BrightFactory())==( 'bright button','bright notice')
```



```
print('Each factory supplies a consistent product family')
```

**Cost:** adding a new product kind requires changing every factory. Adding a new family is easier. That asymmetry should match the changes you expect.

### 3. Builder - construct a valid object step by step

A builder accumulates construction choices and validates them at the final boundary. The resulting object should not share accidental mutable construction state. Use this for many optional fields or staged construction. For two required arguments, a constructor is clearer. A fluent API alone does not necessarily implement a meaningful builder.

```
# lab: pattern_builder_complete
from dataclasses import dataclass
@dataclass(frozen=True)
class StudyPlan:
    title:str
    topics:tuple[str,...]
class PlanBuilder:
    def __init__(self,title):self.title=title;self.topics=[]
    def add(self,topic):self.topics.append(topic);return self
    def build(self):
        if not self.title.strip() or not self.topics:raise ValueError('title and topics required')
        return StudyPlan(self.title,tuple(self.topics))
builder=PlanBuilder('Interview').add('graphs')
plan=builder.build();builder.add('SQL')
assert plan.topics==('graphs',)
try:PlanBuilder('').build()
except ValueError:pass
else:raise AssertionError('invalid plan accepted')
print('Built values are validated and independent of later builder changes')
```

### 4. Prototype - copy a configured example

Instead of reconstructing a complex configuration from scratch, clone a prototype. Decide which members are copied deeply and which represent shared external resources. A deep copy of a database connection is not a meaningful new connection. Prefer explicit constructors when they make ownership clearer.

```
# lab: pattern_prototype
from copy import deepcopy
class ExerciseTemplate:
    def __init__(self,title,hints):self.title=title;self.hints=hints
    def clone(self):return deepcopy(self)
original=ExerciseTemplate('BFS',['use a queue'])
copy=original.clone();copy.hints.append('mark visited when enqueued')
assert original.hints==['use a queue'] and len(copy.hints)==2
print('Prototype clone does not share its mutable hint list')
```

**Cost:** implicit copying rules can hide expensive work or copy inappropriate identity fields. Test nested mutation, not only equality immediately after cloning.

### 5. Singleton - one instance within a defined boundary

This pattern restricts construction and exposes one instance. Define the boundary: one Python class in one process is not one instance across a distributed system. Shared mutable state complicates tests and concurrency. Dependency injection of one ordinary object is often preferable because dependencies remain visible.

```
# lab: pattern_singleton
import threading
class ProcessRegistry:
    _instance=None
    _lock=threading.Lock()
```

```

def __new__(cls):
    with cls._lock:
        if cls._instance is None:cls._instance=super().__new__(cls)
        return cls._instance
first=ProcessRegistry();second=ProcessRegistry()
assert first is second
print('Construction is serialized and returns one process-local instance')

```

The lock protects instance creation, not every future operation on it. Avoid mutable initialization that reruns on every constructor call. Serialization, subclassing and multiple processes require separate design decisions.

## 6. Adapter - translate an incompatible interface

A legacy API reports temperature in Fahrenheit while the new client expects Celsius. The adapter translates the interface and semantics without modifying the legacy component. A plain conversion function is sufficient for one isolated call; an adapter object helps when integrating a larger stable interface.

```

# lab: pattern_adapter_complete
class LegacyThermometer:
    def fahrenheit(self):return 68
class CelsiusAdapter:
    def __init__(self,legacy):self.legacy=legacy
    def read(self):return (self.legacy.fahrenheit()-32)*5/9
assert CelsiusAdapter(LegacyThermometer()).read()==20
print('Adapter translates both method shape and units')

```

**Failure mode:** forwarding methods while forgetting units, error semantics or ownership. The tests should assert meaning, not just that a method was called.

## 7. Bridge - vary abstraction and implementation independently

A report has a high-level purpose; its output device is a separate choice. The bridge lets report types and renderers vary without creating every report-renderer subclass combination. If only one axis changes, simple composition without a named hierarchy may be enough.

```

# lab: pattern_bridge
class PlainRenderer:
    def render(self,title):return title
class BracketRenderer:
    def render(self,title):return '['+title+']'
class Report:
    def __init__(self,renderer):self.renderer=renderer
    def output(self):return self.renderer.render('Study report')
class DetailedReport(Report):
    def output(self):return self.renderer.render('Detailed study report')
assert Report(PlainRenderer()).output()=='Study report'
assert DetailedReport(BracketRenderer()).output()=='[Detailed study report]'
print('Report kind and rendering implementation vary independently')

```

**Cost:** premature separation creates abstractions with only one trivial implementation. Identify actual independent variation first.

## 8. Composite - treat a tree uniformly

A lesson takes a number of minutes; a module contains lessons or other modules and sums their times. Both answer the same operation. Use a composite when a recursive whole-part structure is natural. Do not force leaf-only operations onto containers without defining meaningful behavior.

```

# lab: pattern_composite
class LessonPart:
    def __init__(self,duration):self.duration=duration
    def minutes(self):return self.duration
class ModulePart:

```

```

def __init__(self,*children):self.children=children
def minutes(self):return sum(child.minutes() for child in self.children)
course=ModulePart(LessonPart(10),ModulePart(LessonPart(20),LessonPart(5)))
assert course.minutes()==35 and ModulePart().minutes()==0
print('The same operation works on leaves and nested groups')

```

Cycles would destroy the tree assumption and can cause infinite recursion. Enforce acyclicity or add cycle handling if arbitrary graphs are allowed.

## 9. Decorator - wrap behavior while preserving the interface

A reader returns text. One decorator uppercases it; another adds brackets. Each wrapper still behaves like a reader and can be composed. Decorator order matters. A direct function pipeline is simpler when there is no persistent object interface to preserve.

```

# lab: pattern_decorator_complete
class TextReader:
    def read(self):return 'agents'
class UpperReader:
    def __init__(self,inner):self.inner=inner
    def read(self):return self.inner.read().upper()
class BracketReader:
    def __init__(self,inner):self.inner=inner
    def read(self):return '['+self.inner.read()+']'
assert BracketReader(UpperReader(TextReader())).read()=='[AGENTS]'
print('Wrappers add behavior while retaining the reader contract')

```

**Failure mode:** wrappers accidentally double-log, double-retry or change exception behavior. Test combinations, especially when a retry decorator wraps a side-effecting operation.

## 10. Facade - a simpler entry point to a subsystem

A study session requires loading a lesson and selecting a quiz. A facade presents one convenient operation while leaving the underlying components independently usable. It should simplify orchestration, not swallow every error or become the application's all-knowing object.

```

# lab: pattern_facade
class LessonStore:
    def load(self,topic):return {'topic':topic,'text':'Explain '+topic}
class QuizStore:
    def choose(self,topic):return 'Why use '+topic+'?'
class ClassroomFacade:
    def __init__(self,lessons,quizzes):self.lessons=lessons;self.quizzes=quizzes
    def session(self,topic):return {'lesson':self.lessons.load(topic),'quiz':self.quizzes.choose(topic)}
session=ClassroomFacade(LessonStore(),QuizStore()).session('queues')
assert session['lesson']['topic']=='queues' and session['quiz']=='Why use queues?'
print('Facade coordinates a small subsystem through one use-case operation')

```

An application service may already play this role. Adding another facade on top of it without simplifying the contract is redundant.

## 11. Flyweight - share immutable intrinsic data

Many displayed characters can share one immutable style, while position remains separate. The factory reuses styles by value. Intrinsic state is shared; extrinsic state stays with each use. Use it when repeated immutable data materially consumes memory. A cache without a size or lifecycle policy can cost more than it saves.

```

# lab: pattern_flyweight
from dataclasses import dataclass
@dataclass(frozen=True)
class GlyphStyle:
    font:str
    size:int

```

```

class StylePool:
    def __init__(self):self.styles={}
    def get(self,font,size):
        key=(font,size)
        if key not in self.styles:self.styles[key]=GlyphStyle(font,size)
        return self.styles[key]
pool=StylePool();a={'style':pool.get('serif',12),'x':0};b={'style':pool.get('serif',12),'x':10}
assert a['style'] is b['style'] and a['x']!=b['x']
print('Immutable style is shared; position is not')

```

Never share mutable user-specific data accidentally. Interning every unique string forever is not a general memory optimization.

## 12. Proxy - control access to another object

A proxy implements the same interface while deciding whether or when to contact the real object. It may enforce access, lazy-load, or represent a remote object. A proxy cannot make remote latency and failures disappear; callers still need an honest contract.

```

# lab: pattern_proxy
class SecretStore:
    def read(self):return 'private lesson'
class PermissionProxy:
    def __init__(self,target,allowed):self.target=target;self.allowed=allowed
    def read(self):
        if not self.allowed:raise PermissionError('denied')
        return self.target.read()
assert PermissionProxy(SecretStore(),True).read()=='private lesson'
try:PermissionProxy(SecretStore(),False).read()
except PermissionError:pass
else:raise AssertionError('access bypassed')
print('Proxy enforces a check before delegation')

```

The boolean is deliberately supplied by test setup, not by an untrusted caller. Real authorization must derive it from authenticated identity and policy. If clients can bypass the proxy and call the object directly, the security boundary is ineffective.

## 13. Chain of Responsibility - pass a request through handlers

Each handler either processes a request or passes it onward. The terminal behavior must be explicit. Middleware pipelines often resemble this pattern, although some pipelines always execute every stage rather than stopping at the first handler. Use a straightforward conditional when there are only two stable cases.

```

# lab: pattern_chain
class Handler:
    def __init__(self,predicate,action,next_handler=None):self.predicate=predicate;self.action=action;self.next=next_handler
    def handle(self,request):
        if self.predicate(request):return self.action(request)
        if self.next:return self.next.handle(request)
        raise ValueError('unhandled request')
large=Handler(lambda x:x>=10,lambda x:'large')
small=Handler(lambda x:0<=x<10,lambda x:'small',large)
assert small.handle(4)=='small' and small.handle(20)=='large'
try:small.handle(-1)
except ValueError:pass
else:raise AssertionError('unhandled request silently accepted')
print('A chain delegates until one handler accepts, or fails explicitly')

```

Handler order changes behavior. Authentication and authorization middleware must not be optional stages that an earlier success bypasses accidentally.

## 14. Command - turn a request into an object

A command captures the receiver and requested operation. An invoker can queue it, record it, or support undo. Undo is not always possible: an external email cannot be unsent merely by restoring local state. Prefer a plain function when commands do not need identity, history or lifecycle.

```
# lab: pattern_command_complete
class Document:
    def __init__(self):self.text=''
class AppendCommand:
    def __init__(self,document,text):self.document=document;self.text=text;self.before=None
    def execute(self):
        if self.before is not None:raise ValueError('already executed')
        self.before=self.document.text;self.document.text+=self.text
    def undo(self):
        if self.before is None:raise ValueError('not executed')
        self.document.text=self.before;self.before=None
document=Document();command=AppendCommand(document,'hello')
command.execute();assert document.text=='hello';command.undo();assert document.text==''
print('Command captures an operation and restores its prior local state')
```

This undo assumes no intervening edits. Concurrent editing requires version checks or operation-aware transformations; blindly restoring a snapshot can erase someone else's work.

## 15. Interpreter - evaluate a small language

Represent an expression as a tree and define how each node evaluates. This is appropriate for a small, deliberately restricted language. Never substitute unrestricted eval for a safe interpreter of untrusted input. Large grammars usually deserve parser tools and a more systematic compiler design.

```
# lab: pattern_interpreter
class Literal:
    def __init__(self,value):self.value=value
    def evaluate(self,context):return self.value
class Variable:
    def __init__(self,name):self.name=name
    def evaluate(self,context):return context[self.name]
class AddExpression:
    def __init__(self,left,right):self.left=left;self.right=right
    def evaluate(self,context):return self.left.evaluate(context)+self.right.evaluate(context)
expression=AddExpression(Variable('score'),Literal(2))
assert expression.evaluate({'score':8})==10
print('A restricted expression tree has explicit evaluation rules')
```

The example constructs the tree directly; it does not include a text parser. A network-facing interpreter needs input-size, depth, execution and resource limits.

## 16. Iterator - traverse without exposing representation

An iterator supplies one element at a time and tracks traversal state separately from the collection. Two iterators should be able to progress independently. Python generators provide a concise implementation; a custom class is unnecessary unless you need additional behavior.

```
# lab: pattern_iterator
class LessonShelf:
    def __init__(self,lessons):self._lessons=tuple(lessons)
    def __iter__(self):
        for lesson in self._lessons:yield lesson
shelf=LessonShelf(['python','java']);a=iter(shelf);b=iter(shelf)
assert next(a)=='python' and next(a)=='java' and next(b)=='python'
assert list(shelf)==['python','java']
print('Traversal state belongs to each iterator, not the collection')
```

Define mutation semantics for mutable collections: snapshot, fail-fast, or weakly consistent traversal. Do not promise all three simultaneously.

## 17. Mediator - coordinate peers through one collaboration point

Instead of every participant knowing every other participant, a mediator routes collaboration. A classroom mediator forwards a question to a tutor and records the answer. It reduces peer coupling, but a giant mediator can become harder to maintain than the original network.

```
# lab: pattern_mediator
class TutorParticipant:
    def answer(self, question): return 'Example for ' + question
class LearnerParticipant:
    def __init__(self, mediator): self.mediator = mediator
    def ask(self, question): return self.mediator.ask(question)
class ClassroomMediator:
    def __init__(self, tutor): self.tutor = tutor; self.history = []
    def ask(self, question):
        answer = self.tutor.answer(question); self.history.append((question, answer)); return answer
mediator = ClassroomMediator(TutorParticipant()); learner = LearnerParticipant(mediator)
assert learner.ask('BFS') == 'Example for BFS' and len(mediator.history) == 1
print('The learner delegates collaboration to a mediator')
```

A direct call is better when two components have one simple relationship. Add mediation for genuine coordination rules, not to hide every dependency.

## 18. Memento - capture and restore private state

The originator creates a snapshot; a caretaker stores it; restoration happens through the originator. This separates history management from editing logic. Python does not enforce deep privacy here, so immutable snapshots make the intended boundary clear. For large documents, full snapshots may be too expensive; commands or deltas are alternatives.

```
# lab: pattern_memento
from dataclasses import dataclass
@dataclass(frozen=True)
class Snapshot:
    text: str
class Editor:
    def __init__(self): self._text = ''
    def write(self, text): self._text = text
    def save(self): return Snapshot(self._text)
    def restore(self, snapshot): self._text = snapshot.text
    def read(self): return self._text
editor = Editor(); editor.write('first'); history = [editor.save()]
editor.write('second'); editor.restore(history.pop()); assert editor.read() == 'first'
print('Caretaker stores a snapshot without implementing restoration')
```

As with command undo, restoring old state can conflict with concurrent changes. Persisted snapshots also need schema-version handling.

## 19. Observer - notify subscribers of a change

A subject maintains subscriptions and notifies observers. The subject does not need to know each observer's purpose. Manage subscription lifetime so old screens and objects do not stay reachable forever. A direct call is clearer for one mandatory collaborator whose failure must abort the operation.

```
# lab: pattern_observer_complete
class Scores:
    def __init__(self): self.listeners = []
    def subscribe(self, listener):
        self.listeners.append(listener)
    def unsubscribe():
```

```

        if listener in self.listeners:self.listeners.remove(listener)
    return unsubscribe
    def publish(self,score):
        for listener in tuple(self.listeners):listener(score)
subject=Scores();seen=[];stop=subject.subscribe(seen.append)
subject.publish(7);stop();stop();subject.publish(9)
assert seen==[7]
print('Subscribers receive changes and can unsubscribe idempotently')

```

This implementation propagates listener errors and stops later notifications. That is an explicit policy, not a universal observer guarantee. Async observers introduce ordering, backpressure and delivery questions similar to messaging.

## 20. State - behavior changes with lifecycle state

A job behaves differently while waiting, running and completed. State objects encapsulate allowed transitions. A transition table is often simpler for a small finite lifecycle; state objects help when each state has substantial behavior. Invalid transitions should fail visibly rather than silently mutate the object.

```

# lab: pattern_state_complete
class Waiting:
    def approve(self,job):job.state=Running()
    def finish(self,job):raise ValueError('not running')
class Running:
    def approve(self,job):raise ValueError('already running')
    def finish(self,job):job.state=Finished()
class Finished:
    def approve(self,job):raise ValueError('terminal')
    def finish(self,job):raise ValueError('terminal')
class StatefulJob:
    def __init__(self):self.state=Waiting()
    def approve(self):self.state.approve(self)
    def finish(self):self.state.finish(self)
job=StatefulJob();job.approve();job.finish();assert isinstance(job.state,Finished)
try:job.approve()
except ValueError:pass
else:raise AssertionError('terminal transition accepted')
print('State objects own transition behavior')

```

For persisted concurrent jobs, an in-memory state class is not enough. Enforce transitions with conditional database updates or appropriate transactions too.

## 21. Strategy - select an interchangeable algorithm

A strategy changes how a task is performed while preserving its contract. A scorer can use mean or minimum aggregation without changing the report. Python functions are often the simplest strategies; Java may use a functional interface. Avoid a class hierarchy if a function parameter clearly expresses the variation.

```

# lab: pattern_strategy_complete
def mean_score(values):return sum(values)/len(values)
def strict_score(values):return min(values)
class Assessment:
    def __init__(self,strategy):self.strategy=strategy
    def grade(self,values):
        if not values:raise ValueError('scores required')
        return self.strategy(values)
assert Assessment(mean_score).grade([6,10])==8
assert Assessment(strict_score).grade([6,10])==6
print('The same assessment workflow selects different grading algorithms')

```

Strategies must honor shared expectations about input, output and side effects. Swapping in a strategy with different units or mutation behavior breaks substitutability.

## 22. Template Method - fix the skeleton, vary selected steps

A training pipeline fixes the order load → transform → summarize, while subclasses customize a step. Factory Method customizes creation; Template Method customizes steps of an algorithm. Composition with function arguments is often easier to combine and test than deep inheritance.

```
# lab: pattern_template_method
from abc import ABC, abstractmethod
class Pipeline(ABC):
    def run(self, values):
        clean = self.transform(values)
        return sum(clean)
    @abstractmethod
    def transform(self, values): ...
class PositivePipeline(Pipeline):
    def transform(self, values): return [value for value in values if value > 0]
assert PositivePipeline().run([-2, 3, 4]) == 7
print('The pipeline skeleton delegates one variable step')
```

Too many hooks make the skeleton impossible to reason about. Document when hooks run and which invariants subclasses must preserve.

## 23. Visitor - add operations to a stable object structure

Objects accept a visitor, which implements an operation for each concrete element type. This makes adding new operations easier when element types are stable. Adding a new element type requires updating visitors, so the tradeoff is the opposite of some inheritance designs.

```
# lab: pattern_visitor
class VideoLesson:
    def __init__(self, minutes): self.minutes = minutes
    def accept(self, visitor): return visitor.video(self)
class ReadingLesson:
    def __init__(self, words): self.words = words
    def accept(self, visitor): return visitor.reading(self)
class TimeVisitor:
    def video(self, lesson): return lesson.minutes
    def reading(self, lesson): return lesson.words / 200
class KindVisitor:
    def video(self, lesson): return 'video'
    def reading(self, lesson): return 'reading'
elements = [VideoLesson(10), ReadingLesson(1000)]
assert sum(item.accept(TimeVisitor()) for item in elements) == 15
assert [item.accept(KindVisitor()) for item in elements] == ['video', 'reading']
print('Two operations traverse the same stable element types')
```

The reading speed is a teaching assumption, not a universal measurement. Alternatives include pattern matching, ordinary polymorphic methods, or functions over tagged data. Choose according to which axis changes most often.

## 24. Distinguish patterns that look similar

Adapter changes the interface; decorator preserves it while adding behavior; proxy preserves it while controlling access. Bridge separates independent dimensions before combinations multiply. Facade simplifies a subsystem; mediator coordinates peers. Strategy chooses an algorithm; state reflects lifecycle-dependent behavior. Command captures an operation; memento captures state. A class may participate in more than one pattern, but naming patterns does not prove a design is good.



## 25. Pattern interview and design challenge

**Task, 20 points:** Design a study-report generator with two output formats, multiple scoring rules, undo for local drafts, and authenticated publication. Award 4 points for clear responsibilities, 4 for suitable composition, 4 for correct undo limitations, 4 for server-side authorization and idempotency, and 4 for tests and a simpler alternative. Do not require a pattern name if the design solves the problem clearly.

**Answer sketch:** Inject a formatter and scoring strategy into an application service. Use snapshots or commands for draft history with explicit version checks. Put publication behind an authenticated service boundary, persist an operation key and return a stable result. A facade may simplify the use case; a singleton is unnecessary. Do not build 23 patterns into one application to prove you know them.

**Mastery exercise:** For each cell, modify one assertion to describe an invalid use, then decide whether the implementation should reject it or the contract should exclude it. Add the appropriate check. Explain one situation where deleting the pattern would improve the code.

### Deep workshop - Choose a pattern by identifying what changes

Checkout supports card and bank payments. The varying behaviour is payment execution. Inject a PaymentStrategy into checkout; keep order validation in checkout and payment-specific work in the strategy. The abstraction is useful when behaviours vary independently, not merely because the pattern exists.

#### Distinguish similar wrappers

An Adapter translates an interface: a legacy gateway takes cents and returns another response shape. A Decorator preserves the interface while adding behaviour, such as timing. A Proxy controls access or defers work, such as checking authorization before a remote call. All can hold another object; their intent and contract distinguish them.

#### Failure ordering matters

A retry decorator around a non-idempotent payment can duplicate charges. Logging raw payment credentials creates a confidentiality problem. A pattern name does not make side effects safe. Define retryable errors, safe log fields, and stable operation identity.

#### Refactor and test

First capture existing behaviour with a test. Extract the varying operation, inject one implementation, then add the second. Run common contract tests for both and implementation-specific failure tests separately. When is Strategy worse than a conditional? **Answer:** A tiny stable distinction may become harder to understand with extra classes. Independent evolution, different dependencies, and separate testing justify the indirection. Explain the cost as well as the benefit.

# Notebook 14 - Build and defend a complete connected system

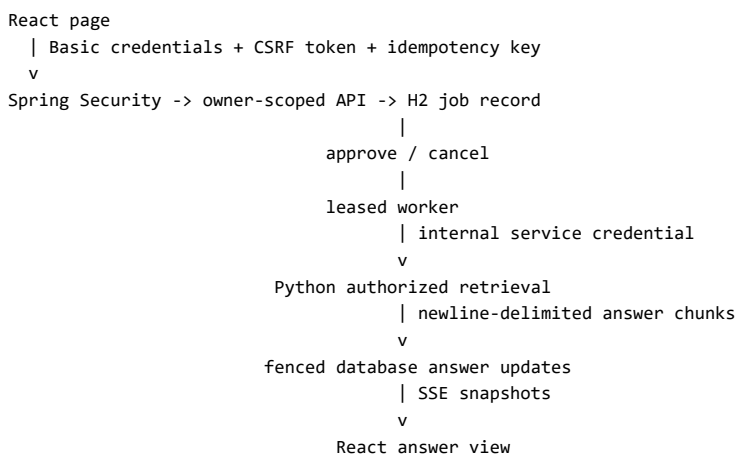
The earlier full-stack notebook described a design. This workshop includes the implementation in `labs/study-coach`: a React/TypeScript browser application, a Spring Boot service, a migrated database, a background worker and a Python AI service. Read the project's README for exact build/run commands. The source files are intentionally small enough to follow one request without a large framework scaffold.

## 1. Begin with requirements, not boxes

A learner signs in, submits a question, approves computation, sees evidence arrive, and can cancel. Work survives a Java process restart. One learner cannot read another learner's jobs or private evidence. A repeated submission key returns the same job, while using that key for a different question conflicts. An unavailable AI service must not make the worker retry forever.

Non-goals matter too: the classroom app is not a public identity provider, distributed message broker or generative model host. Its actual AI component uses TF-IDF retrieval and extractive answers. That lets you verify every evidence source offline. Replacing it with a generative model is an explicit architectural extension, not a hidden simulation.

The request path is:



## 2. Why three languages here?

React handles browser interaction. Java owns identity-facing business operations and durable job state. Python owns numerical text retrieval. These choices demonstrate boundaries, not a rule that every AI product needs three services. For a small team, one backend might be simpler. Separate services add network failures, deployments, contracts and operational work; the educational benefit here is making those issues explicit.

The Python service does not trust a user field from an arbitrary browser. Only the authenticated Java service calls it with an internal credential and a user name derived from Spring's authenticated principal. The browser never receives the internal service token. In a larger system use appropriate workload identity, audience restrictions and rotation rather than one indefinitely shared secret.

### 3. Authentication is only the first boundary

Spring Security authenticates the local classroom users and hashes their configured password. A CSRF token is fetched and supplied on mutations. The application does not disable CSRF merely because it uses Basic authentication; browsers and credential behavior still matter. The same-origin frontend avoids an unnecessary cross-origin API policy.

Every job query binds `id` and `owner_name`. Returning 404 for another owner's ID avoids confirming that a private object exists. The approve and cancel routes perform the same ownership check. SQL parameters bind data rather than concatenating user input into SQL. The network tests exercise all of these boundaries instead of assuming the UI buttons are sufficient protection.

For public use, move to a suitable identity system and TLS, manage account lifecycle and logout/revocation, and review session and CSRF behavior as a complete design. The “Forget page credentials” control intentionally states its limited behavior; it does not claim to revoke server sessions.

### 4. Idempotency and transaction boundaries

The database has a unique constraint on (`owner_name`, `request_key`). Creating a duplicate catches the uniqueness conflict and returns the existing job if the question matches. The same key with a different question returns 409. This matters because a client can lose a response after the database successfully commits.

A client should preserve its submission key when retrying an uncertain request. The included page creates a key for each deliberate new submission; the API test demonstrates retries using the same key. Adding automatic client retries would require retaining that key through the retry flow, not generating a new one each time.

The worker performs short conditional database updates around remote I/O. It does not hold a database transaction open for an entire model call. For a workflow that charges money, idempotency must also exist at the payment receiver; protecting only the job table does not protect an external charge.

### 5. The durable job lifecycle

`approval` → `pending` → `running` → `done` is the ordinary path. `running` → `retry` → `running` handles temporary failures, with at most three attempts. `failed` and `cancelled` are terminal. A lease allows a running job to be reclaimed after a crash. Each claim receives a fresh token, and answer/status updates include that token and `status='running'` in their condition.

Cancelling clears the token and sets a terminal state. A worker already computing may finish a remote read, but its next database write is rejected. Cancellation therefore prevents publishing further work in this application; it is not a claim that every upstream CPU instruction stops instantly. The remote HTTP read and whole-job deadlines bound how long abandoned work can occupy the worker.

The automated system test kills Java while a job is running, restarts with the same file database and checks that the job completes with a later attempt. This tests an actual process boundary. It is stronger than simply constructing a second in-memory object, but does not simulate every storage failure or whole-machine power loss.

## 6. Migrations and rollback planning

Flyway applies versioned SQL migrations: jobs, error reporting, then classroom JPA/outbox tables. Hibernate validates the mapped schema rather than creating it silently. Never edit an already-applied migration to change a deployed database; add a new migration. For a rename used by old and new application versions, use an expand-and-contract rollout: add compatible fields, deploy code that can coexist, migrate data, then remove old fields later.

Code rollback does not automatically undo a destructive schema change. Backups must be restorable, and migration tests need representative existing data, not only an empty database. The current tests verify applying the included migrations on H2. They do not certify another database engine or a large real dataset.

## 7. Streaming across two boundaries

Python yields newline-delimited JSON chunks. Java reads incrementally with connection/read timeouts and a total processing deadline, appending accepted chunks to the database. An SSE endpoint emits changed job snapshots. The browser reads the response stream and parses complete event frames. This is actual incremental delivery, not a timer replaying a fully received answer.

Snapshots contain the current full answer. A reconnect can replace the view without duplicating text. Delta-only designs save bandwidth but need event IDs, ordering, replay history and deduplication. A production stream parser should handle the full framing contract, maximum frame sizes and malformed events. This teaching server emits a constrained format generated by its own serializer.

Slow consumers, dropped connections and proxy buffering can affect streaming. The endpoint has a bounded lifetime, and the UI offers “Refresh and reconnect.” Closing a browser stream does not cancel the durable job; cancellation is a separate authenticated command. That distinction makes reconnecting possible.

## 8. Retrieval, evidence and the generative extension

Python filters documents by owner before ranking. It builds TF-IDF vectors and computes similarity, returns up to two positive-scoring items, and quotes them with IDs. A query with no overlap gets an explicit no-evidence response. This is a small extractive baseline, not proof that all questions can be answered accurately.

To add an LLM, keep retrieval and authorization before prompt construction. Delimit source text as untrusted evidence, require citations, bound context and output, and retain a no-evidence path. Version the prompt/model/retrieval configuration and evaluate factual support, not just fluent wording. A malicious document must not grant the model tools or change who owns a request. Provider calls need credentials, cost limits, timeouts, retention review and failure tests. No external provider was silently used in the supplied tests.

## 9. Observability you can connect to behavior

The worker logs a job ID and duration. `/api/metrics` returns status counts scoped to the authenticated learner. Tests inspect attempt counts, errors and final states. These are useful starting signals; they are not a complete telemetry platform. A production system would add structured traces, queue age, throughput, latency percentiles, resource saturation, dependency failures and alert thresholds.

Keep high-cardinality identifiers out of metric label sets when they would create unbounded series. Put a job ID in logs/traces instead. Do not log credentials, complete prompts or private retrieved text by default. A correlation ID connects events without requiring sensitive payloads.

Define a service objective around user-visible success: for example, the proportion of valid authorized jobs completed within a chosen deadline. The exact target must follow requirements and measurements. Reporting only HTTP 200 responses can hide jobs that later fail in the background.

## 10. Deployment, capacity and failure recovery

Local execution uses two subprocesses and a persistent H2 directory. Dockerfiles and Compose describe non-root services, a private internal AI connection and a retained data volume. They are a deployment recipe; the validation report distinguishes local tests from container execution. No service is published to the internet by this workbook.

One worker deliberately processes one job at a time. To scale, measure queue age and per-job cost first. Multiple workers require safe claims, suitable database concurrency and shared durable storage. A single embedded file database is not a multi-host distributed queue. Consider a dedicated broker or database queue design when the workload justifies it.

The outage test stops Python, verifies a three-attempt terminal failure, restarts Python and verifies that new work succeeds. An operator retry of a failed job should be an explicit authorized operation with an audit trail and a defined budget; automatically resetting every failed job forever would defeat the attempt limit.

## 11. Acceptance checklist and design defense

The automated suite checks authentication, CSRF, ownership, idempotency conflicts, approval, partial/final streaming, evidence filtering, cancellation, keyboard interaction, automated accessibility rules, process restart recovery, bounded outage failure and restored service operation. The Java tests additionally check migrations, stale writes, optimistic locking, query counts, isolation and transaction/outbox rollback.

**Interview, 20 points:** Explain one complete question from browser to answer (4), identify trust boundaries (4), explain crash recovery and duplicate handling (4), describe a streaming reconnect (3), and state capacity/deployment limitations with a justified next step (5).

**Model answer to “is this production-ready?”:** It is a tested local educational integration. Production readiness requires requirements-specific identity, deployment, storage, capacity, security, monitoring and operational evidence. Its explicit boundaries and failure tests make it a useful foundation, but a passing classroom suite is not a production certification.

## Deep workshop - Turn the capstone into a reproducible experiment

Identify the smallest journey: sign in, submit one question, approve it, receive an answer, refresh, and retrieve the same result. Write expected durable state at each boundary before starting every service.

### Follow the evidence

Before submission there is no job. Accepted creation commits one pending row with caller and operation key. Approval changes an allowed state; it does not create another job. A claim records ownership and expiry. Completion stores the answer. Refresh queries that record instead of repeating creation as a new intent.

### Useful integration checks

Create twice with one key and payload, then compare IDs. Approve, poll with a bounded deadline, and verify authorized sources. Restart the application and retrieve the same job. Use a second user and verify denial. These separately test idempotency, persistence, and ownership.

## Debug the first broken boundary

If loading never ends, ask whether the row exists, whether it was claimed, whether Python completed, and whether completion reached the UI. HTTP 200 does not establish that every streamed event was valid. Carry a correlation ID across services without logging credentials or private source text. Why are mocked repositories insufficient? **Answer:** They cannot establish migrations, constraints, real transaction boundaries, or restart persistence. Use unit tests for local decisions and real integrations for storage contracts.

# Notebook 15 - Graded interviews, debugging rounds and answer keys

Reading an answer feels easier than producing it. Close the explanation, work under a time limit, then compare your reasoning with the rubric. Give partial credit for a sound approach even when implementation is unfinished; do not award full credit for a memorized phrase with no mechanism. These are original practice questions, not a leaked question bank or a guarantee of an exam result.

## 1. A repeatable practice method

For each attempt record the date, question, assumptions, solution, tests, complexity or tradeoffs, score, and one correction. Revisit the correction after two days and one week without reading your old answer first. Explain the solution aloud. If you cannot explain an assertion in the tests, rerunning the tests has not yet taught you the idea.

Use a 0-4 scale per criterion: 0 absent; 1 vocabulary only; 2 partly correct with a significant gap; 3 correct and explained; 4 correct with edge cases and a justified tradeoff. A useful practice target is consistently earning at least 3 across criteria on unseen variants. This is a learning target, not a standardized hiring threshold.

## 2. Round A - Python and numerical reasoning, 45 minutes

**A1, 10 points:** Implement stable softmax for a nonempty list of finite numbers. Reject invalid values. Explain why subtracting the maximum preserves probabilities. **Answer:** Divide every exponential by the same positive factor  $\exp(\text{maximum})$ , which cancels between numerator and denominator. Award 3 for validation, 3 for stable implementation, 2 for normalization/shift-invariance tests and 2 for the explanation. Compare with the installable `coach_math` package.

**A2, 10 points:** A prediction has shape  $(100, 1)$  and labels have shape  $(100, )$ . Why does subtraction create 10,000 values? **Answer:** Broadcasting aligns dimensions and expands one row-like and one column-like dimension, producing  $(100, 100)$ . Reshape both to the intended shared contract before computing loss. Award 4 for the mechanism, 3 for the correction, 3 for an assertion that catches recurrence.

**A3, 10 points:** Your model improves after standardizing the entire dataset before splitting. Is the comparison valid? **Answer:** The transformation used information from held-out examples. Fit preprocessing on training data, preferably inside a pipeline applied separately within each cross-validation fold. The leakage may change results in either direction; the issue is invalid evaluation, not a promise that accuracy always inflates. Award 4 for identifying leakage, 4 for a correct pipeline, 2 for grouped/time-aware split considerations.

**A4, 10 points:** Memory rises while processing a large array in small slices. Why might slices retain the original allocation? **Answer:** A view references the underlying base memory. Keeping even a small view can keep the base alive. Measure ownership and lifetime, decide whether a copy is worthwhile, and examine native/process memory as well as Python allocations. Award 4 for views, 3 for a reasoned fix, 3 for measurement limitations.

**Follow-up:** Why not copy every slice? Copies consume time and extra memory; use evidence about lifetime and workload rather than an absolute rule.

### 3. Round B - Algorithms, 60 minutes

**B1, 15 points, 20 minutes:** Return the shortest unweighted route between two nodes, or no route. **Answer outline:** BFS with a queue, mark visited when enqueued, record one predecessor, stop when the target is reached, reconstruct backwards. Handle source=target and unreachable targets. Award 5 for algorithm/invariant, 5 for implementation, 3 for cases, 2 for  $O(V+E)$  time and  $O(V)$  auxiliary space.

**B2, 15 points, 20 minutes:** Determine whether a binary tree is a valid strict BST. **Answer outline:** Carry ancestor lower/upper bounds, not only parent comparisons. Empty tree is valid under this convention; duplicates are rejected. Award 5 for the global invariant, 5 for implementation, 3 for a misleading grandchild test, 2 for  $O(n)$  time and height-dependent space.

**B3, 20 points, 20 minutes:** Solve 0/1 knapsack and explain a space optimization. **Answer outline:** State (item, remaining\_capacity) with include/exclude choices; memoize or fill a table. For one-row DP, iterate capacity downward. Award 6 for recurrence, 6 for code, 4 for boundary/one-use tests, 4 for pseudo-polynomial complexity and reconstruction tradeoff.

**Alternative-solution discussion:** DFS finds a route but not necessarily the shortest unweighted route. Sorting a tree's values cannot validate its original structure. Greedy value/weight ratio fails for indivisible items. A candidate who knows when an alternative fails understands more than one who merely recites the preferred algorithm.

### 4. Round C - Agents and MCP, 45 minutes

**C1, 10 points:** An agent wrote an external record, then crashed before checkpointing. What happens on resume? **Answer:** The step may replay. Use receiver-side idempotency with a stable key and payload binding; a workflow checkpoint alone cannot atomically cover an unrelated service. Award 4 for uncertainty/replay, 4 for the receiver contract, 2 for a crash-window test.

**C2, 10 points:** A LangGraph approval survives a restart, but an edited action executes using the old approval. What is wrong? **Answer:** Approval was not bound to the exact proposal and relevant resource version. Require an authenticated approver, immutable action digest/version and expiry; changed content needs reapproval. Award 4 for binding, 3 for identity, 3 for a stale-approval test.

**C3, 10 points:** A tool is hidden from a reader's MCP discovery list. Is access controlled? **Answer:** Only if direct calls also enforce authorization. Check validated credential scopes and resource ownership. Never trust an owner passed by the model. Award 4 for enforcement at invocation, 3 for identity/ownership, 3 for a guessed-tool-name test.

**C4, 10 points:** Three parallel agents generate answers; one fails. Should the orchestrator return success? **Answer:** It depends on the declared aggregation contract. All-required work fails or retries within budget; best-effort work may return partial results with missing coverage made explicit. Cancel unnecessary work, preserve useful traces, and bound concurrency and cost. Award 4 for explicit semantics, 3 for cleanup/budgets, 3 for a failure test.

**Follow-up:** How would you prove a multi-agent design improves quality? Compare against a simpler baseline on a held-out task set under equal budgets, measure success and critical failures, repeat stochastic cases, and report uncertainty and cost.

### 5. Round D - Java and Spring, 45 minutes

**D1, 10 points:** Why does `volatile int count` not make `count++` safe? **Answer:** Increment is a compound read-modify-write; visibility does not make the sequence atomic. Use an appropriate atomic operation or a lock protecting the invariant. Award 4 for distinction, 3 for correction, 3 for multi-variable limitations.



**D2, 10 points:** A `@Transactional` method called from another method on the same object does not behave as expected. **Answer:** In the usual proxy-based setup, self-invocation bypasses the proxy. Place the boundary on a managed service called through the proxy or use an appropriate explicit transaction mechanism. Also inspect rollback rules and transaction propagation. Award 5 for proxy reasoning, 3 for valid redesign, 2 for an integration test.

**D3, 10 points:** Loading 100 courses executes 101 queries. **Answer:** A relationship is likely loaded once per course. Measure actual SQL, choose fetch join/projection/entity graph/batching, and consider pagination and row multiplication. Award 4 for N+1, 3 for a justified remedy, 3 for a query-count test with a cleared persistence context.

**D4, 10 points:** A database insert succeeds but its Kafka notification is missing. **Answer:** The database and broker commits were not atomic. Use an outbox for the producer boundary and an idempotent consumer for duplicate delivery. A relay crash can publish twice. Award 4 for the crash window, 4 for the outbox/inbox reasoning, 2 for failure tests.

**Follow-up:** Why not use one JVM lock to enforce inventory across replicas? Other JVMs do not share that lock. The invariant needs a shared transactional or otherwise coordinated authority.

## 6. Round E - React debugging and browser behavior, 40 minutes

**E1, 10 points:** Three calls to `setCount(count+1)` increase the value by one. **Answer:** Each call uses the same render's captured value. Functional updates compose on pending state when that is the intended behavior. Award 4 for snapshot reasoning, 3 for the fix, 3 for an interaction test.

**E2, 10 points:** Search for A, then B. A completes last and replaces B. **Answer:** Guard against stale completions and cancel obsolete requests. Include query/user identity in server-state cache keys. Test with deliberately reversed response order. Award 4 for race explanation, 3 for correction, 3 for the deterministic test.

**E3, 10 points:** Hydration warns after the server and browser independently render the current time. **Answer:** The initial outputs differ. Pass a consistent initial value or render browser-only changes after hydration where appropriate. Do not use warning suppression as a general repair. Award 4 for matching markup, 3 for solution, 3 for recognizing server/browser boundaries.

**E4, 10 points:** A styled div submits a form with mouse click but not keyboard. **Answer:** Use a native button with the appropriate type, accessible name and visible focus. If a custom control is truly necessary, implement its complete keyboard/semantic contract. Award 4 for native semantics, 3 for focus/labels, 3 for an actual keyboard test. Axe alone cannot prove every interaction works.

## 7. Round F - Full-stack system design, 60 minutes

**Prompt:** Design a tutoring system that accepts questions, retrieves private documents, optionally uses an LLM, requires approval before external actions, streams progress, and survives worker crashes.

Spend the first 10 minutes clarifying users, scale, latency, privacy, external actions and acceptable failure behavior. Spend 15 on interfaces and data ownership, 15 on request/failure paths, 10 on security and evaluation, and 10 on tradeoffs and evolution. A beautiful diagram without a failure path is incomplete.

**Answer outline:** Authenticate at the boundary. Authorize object access. Persist a request with an idempotency key. Keep approval bound to an action version. Dispatch through a bounded durable queue. Claim jobs with a lease and protect stale writes. Retrieve only permitted evidence. Treat source text as untrusted. Call the model with budgets and timeouts. Validate proposed actions in ordinary code. Execute external effects through idempotent contracts. Stream replayable events or replaceable snapshots. Record traces and evaluate

outcomes.

**Rubric, 40 points:** requirements 5; contracts/data ownership 5; authorization and privacy 5; durable execution/idempotency 7; streaming and reconnect 4; evaluation/observability 5; capacity and cost 4; justified simplicity and limitations 5.

#### Follow-ups with answers:

- **Queue grows while CPU is low.** Inspect external latency, connection pools, locks and concurrency limits before buying more CPU.
- **A user requests deletion.** Identify source records, indexes, caches, logs, backups and retention rules; a vector-index delete alone is incomplete.
- **The model is fluent but wrong.** Measure grounding and task success, inspect retrieval failures, improve evidence and refusal behavior, and compare model changes on held-out cases.
- **A client disconnects.** Decide whether the durable job continues; reconnect by job ID under authorization. Disconnect and cancellation are different commands.
- **A new schema breaks old paused jobs.** Version state and execution definitions, migrate carefully or retain compatible workers, and test historical paused states.

## 8. Design-pattern selection round

**Question:** You need two export formats and three scoring rules. Should you create six subclasses? **Answer:** Independent strategies or a bridge-like separation avoid the Cartesian product if the two dimensions vary independently. Award credit for explaining the axes, not merely naming Bridge.

**Question:** When does Singleton make testing harder? **Answer:** Hidden global state creates ordering dependence and makes replacement/lifetime control difficult. Inject an ordinary shared object when that meets the requirement.

**Question:** When is Visitor awkward? **Answer:** When new element types appear frequently, every visitor needs changes. It favors stable element structures and adding new operations.

**Question:** Is a decorator retrying a payment safe? **Answer:** Only with an idempotent operation contract and correctly classified retry conditions. A design pattern does not make side effects safe.

## 9. A complete mock interview day

Use three sessions with breaks: 60 minutes algorithms, 45 minutes debugging across Python/Java/React, and 60 minutes system design. Reserve 30 minutes afterwards for scoring and correcting one weak area. Choose unseen variants rather than repeating the identical inputs until memorized.

For a second attempt, change the constraints: limited memory, multiple tenants, a slow upstream, a negative edge, a changed approval, or a reordered UI list. Explain which assumption of the original solution is no longer true and how the design changes. That is the central skill for unfamiliar interviews.

## 10. Your completion standard

You are ready to move beyond a lesson when you can explain it simply, implement the essential mechanism without copying, predict a failure, write a meaningful test, and discuss a reasonable alternative. If you only recognize the terminology, repeat a smaller exercise. If you can implement it but cannot justify it, practice the invariant or tradeoff aloud.

This collection now provides worked material for every requested area, a connected application, and graded practice. It does not claim that every algorithm, employer, exam syllabus or future framework release is covered. Keep an error log and use new problems to measure transfer of understanding rather than counting pages read.

## Deep workshop - Answer an interview question as an experiment

Question: “How do you make a payment API idempotent?” First define the boundary: repeated attempts for one intended payment should create one observable payment; distinct intentions remain distinct.

### Build the answer

Use an operation key scoped to caller and operation. Store a payload hash so changed arguments conflict. Enforce uniqueness in storage so concurrent first attempts cannot both win. Store processing state and the final result. A retry returns the existing identity rather than creating a second payment. If a remote provider is involved, local deduplication does not prove that provider charged only once. Forward a stable provider key or reconcile using its status API. A timeout is uncertainty, not proof of failure.

### Grade out of ten

Award two points each for scope, concurrent uniqueness, changed-payload handling, crash/timeout reasoning, and a reproducible test. “Cache the response” leaves eviction, crashes, and concurrent misses unexplained.

### Follow-up answer

The first request is still processing. What should the retry receive? **Answer:** Existing processing identity and status, or another documented in-progress response. No final result does not justify another charge. Explain recovery of abandoned work and retention of operation identities. Record whether you answered without hints; reading this answer is practice, not an unaided assessment.

# 16 - Real models: from a talking helper to a bounded agent

## 1. The library helper story

Imagine a child asks a librarian what a checkpoint means. The librarian first finds the right page, then explains it. Retrieval is finding the page. Generation is explaining the page in new words. A language model can sound confident without finding the right page. This is why a good system measures these steps separately.

Our first Study Coach version returned passages using TF-IDF. Its output was an extract, not invented prose. The optional Ollama mode now calls an installed Qwen model and streams newly generated words. The read-only definition agent also asks the model to select a tool, executes that tool under application control, and gives its result back to the model. See `labs/study-coach/python/ollama_agent.py` and the recorded `live-model-report.json`.

You can read this lesson without a service. Running its live project needs Ollama, the named model, Java, Python and the dependencies in the project. A notebook cannot include a multi-gigabyte model in a small ZIP.

## 2. What happens during one token

A tokenizer turns text into integer IDs. An embedding maps each ID to a vector. Transformer layers mix information from earlier positions using attention and transform it with feed-forward layers. The last layer produces one number, called a logit, per possible next token. Softmax turns logits into probabilities. A decoding rule selects a token; the token is appended; the process repeats.

For logits  $[2, 1, 0]$ , subtract the largest value to get  $[0, -1, -2]$ . Exponentials are approximately  $[1, 0.368, 0.135]$ . Divide each by their sum 1.503: probabilities are approximately  $[0.665, 0.245, 0.090]$ . Subtracting a common number does not change the probabilities because the same factor cancels from numerator and denominator. It prevents overflow for very large logits.

Temperature divides logits before softmax. Lower positive temperature sharpens the distribution; higher temperature flattens it. It does not make false facts true. Top-k keeps k candidates; top-p keeps a smallest high-probability set whose cumulative probability reaches p. A seed and zero temperature still do not guarantee identical results across hardware and implementations.

```
# lab: stable_decoding
import math
def softmax(values):
    if not values: raise ValueError('empty vocabulary')
    largest=max(values);weights=[math.exp(x-largest) for x in values]
    total=sum(weights);return [w/total for w in weights]
assert abs(sum(softmax([10000,9999,9998]))-1)<1e-12
assert softmax([2,1,0])==softmax([10002,10001,10000])
print([round(x,3) for x in softmax([2,1,0])])
```

## 3. A tool call is a request, not permission

Think of a child giving a note to a bank teller: "Please withdraw money." The note itself is not authorization. Likewise, JSON from a model is untrusted input. The application validates the tool name, arguments, authenticated identity, permissions and budget before executing anything.

Our example exposes only `lookup_definition(term)`. The server chooses allowed terms and rejects extra fields. A model cannot introduce a new tool or replace the user identity. Read the tool result as data; a malicious

document saying “ignore your rules” must not change the tool permissions. Prompt wording alone is not a security boundary.

The execution loop is: validated user request → model → validated tool proposal → allowed tool → model → checked response. Stop if the budget runs out, if an unsupported tool appears, or if the output fails validation. Record the reason; do not silently pretend success.

## 4. Budgets are the agent's lunch money

Give the helper a small purse. The live definition example allows at most two model calls, one tool call and 256 generated tokens in total, with at most 128 per call. Input bytes are also bounded. These are distinct controls: a token limit does not limit wall-clock time, and a timeout does not cap spending if a remote provider keeps computing after disconnection.

For a paid provider, estimated cost is  $\text{input\_tokens} \times \text{input\_price\_per\_token} + \text{output\_tokens} \times \text{output\_price\_per\_token}$ . Include cached-input pricing only if the provider reports it. Reserve a budget before launching parallel calls; atomically reconcile actual usage afterwards. Otherwise four workers can each believe they own the same remaining budget.

Our local report records zero API spend because it invokes a local service. Hardware, electricity and time still have costs. The generated-token count comes from the model server's usage field, not a count of words.

```
# lab: atomic_budget_model
class Budget:
    def __init__(self, limit): self.remaining = limit
    def reserve(self, amount):
        if amount < 0 or amount > self.remaining: raise ValueError('budget exhausted')
        self.remaining -= amount
    def refund(self, amount):
        if amount < 0: raise ValueError('negative refund')
        self.remaining += amount
b = Budget(200); b.reserve(128)
try: b.reserve(128)
except ValueError: pass
else: raise AssertionError('overspend')
b.refund(128-40); assert b.remaining == 160
print('Reserve before work; reconcile actual usage afterwards. Shared storage needs a transaction.')
```

## 5. Streaming and trust are different clocks

Streaming shows words early. Verification may finish later. If the model writes an unauthorized citation near the end, earlier words may already be visible. Label partial output as a draft and treat the final validation event as a separate state. A user interface must not present an interrupted stream as a complete answer.

The example checks that bracketed citation IDs belong to the retrieved evidence. This catches invented identifiers. It does not establish that every sentence follows from the source. Semantic support requires a claim-by-claim rubric, deterministic checks where possible, and human review for consequential uses. A model-based judge can assist but has its own errors and should be calibrated against people.

The actual end-to-end run demonstrated this failure. The generated answer said a checkpoint was “ensuring side effects are idempotent” and attached the allowed [checkpoint] citation. That claim is wrong: saving workflow state does not make an external payment or email idempotent. The retrieved source explicitly required idempotency keys. The network flow and citation-format check passed, while the tutor's factual review failed. Both results are preserved in `live-stack-report.json` and `live-stack-review.json`; the failure is not hidden or counted as a correct answer. Keep the extractive default when factual review is unavailable. The optional generative mode is a draft-producing experiment, not an authoritative tutor.

If a connection closes halfway through an answer, the worker records retry or failure. It does not commit done merely because it received some text. A cancelled job rejects late writes using the persisted status and lease token.

## 6. Evaluation without fooling yourself

Separate development examples, which you use while editing, from a held-out set, which you avoid optimizing against. For each case record the question, authorized evidence, expected behavior, model/version, prompt version, sampling configuration, tool decisions, latency, token use and outcome. Include answerable questions, missing evidence, ambiguous requests, malicious instructions inside documents, tool timeouts, invalid tool arguments and unauthorized ownership claims.

Retrieval metrics answer “did we find the useful page?” Recall@k counts whether required evidence appeared among k results. Generation metrics answer “did the explanation use the page correctly?” A factual-support rubric can score each material claim as supported, contradicted or unsupported. Task success answers “did the user actually get the intended result?” These are not interchangeable.

The included live smoke report has two answer cases and one tool trajectory. It proves the actual model path ran; it is too small to estimate general reasoning quality. The earlier 16-case retrieval regression remains a development test. Do not report either as a universal accuracy percentage.

## 7. Deployment choices and failure ownership

Use local inference when privacy, offline use or control matters and the hardware fits. Use hosted inference when managed capacity and access to a particular model matter. Use a gateway to enforce per-user budgets, timeouts and audit metadata consistently. Keep provider keys on the server. A browser bundle is public and cannot protect a secret.

Retries need a classification. Retry a temporary connection failure with a bounded backoff. Do not repeatedly retry malformed input or a denied permission. A timeout after a side-effecting tool is ambiguous: the action may already have happened. Reconcile by the operation's idempotency key before retrying.

## 8. Your interview round

Question: “Why not let the model choose the user ID for a retrieval tool?” Answer: identity must come from the authenticated session; a model-supplied identity would permit cross-user access. Follow-up: filter ownership before ranking, enforce it again at data access, and test both forbidden reads and indirect leakage through summaries.

Question: “Does valid JSON mean the agent is safe?” Answer: syntax validation only establishes structure. Business constraints, ownership, resource limits and approval rules remain separate checks.

Exercise: design a refund agent with a maximum refund of 500 units. Answer outline: integer minor units; authenticated customer; server-side order ownership; refundable-balance check inside a transaction; idempotency key tied to request content; human approval above policy threshold; ledger and audit event; retry reconciliation; tests for concurrent refunds and response loss. Do not place the refund authority in the prompt.

API references used while writing: [Ollama chat](#) and [tool calling](#). The teaching text and examples are contained here; links are provenance, not required reading.

## Deep workshop - Separate model output from accepted output

The model emits tokens; the application chooses whether to publish them. Displayed tokens cannot be unseen after later validation fails. Buffering a bounded draft permits rejection before publication but delays the first visible answer.

### Validate distinct layers

Check transport completion and output size. Parse structure. Validate citations against authorized source IDs. Then assess whether the claims follow from the sources. The first checks are mechanical; semantic correctness is separate. Evidence says checkpoints preserve progress and effects still need idempotency keys. An answer claiming checkpoints guarantee idempotency can cite the correct source and remain false. Preserve that failure rather than loosening the scoring rule.

### Budget the whole request

A token cap does not bound wall time. Add connection, read, and overall deadlines, plus input-size limits. A worker lease must accommodate the permitted call or be renewed safely. Propagate cancellation instead of leaving invisible work running.

### Recovery contract

After malformed output, return a clear failure or explicitly labelled authorized excerpts. Do not call excerpts a generated answer. Do not expose raw provider errors containing internal details. Factual scores and successful transport tests belong in separate columns: one does not substitute for the other.

# 17 - Identity and deployment: who are you, and where does the app live?

## 1. A school gate, a classroom and a permission slip

Authentication is the school gate checking who you are. Authorization is deciding which classroom you may enter. A session is a temporary pass that lets the school recognize you again. A role is a group of permissions. None of these should come from a user-editable field saying "I am admin."

The original classroom login uses generated local credentials. The optional `oidc` profile delegates login to Keycloak and keeps an application session cookie. The project provides a realm template, administration helper and real browser integration script. It is a local demonstration of identity integration, not a claim that HTTP development settings are suitable for a public site.

## 2. OAuth and OIDC in one small story

OAuth authorizes access to a resource. OpenID Connect adds a standard identity layer. The browser visits the identity provider; the provider authenticates the person; it redirects back with a short-lived authorization code. The application exchanges that code at the provider and validates the returned identity information. The browser does not choose its own verified identity.

PKCE creates a secret verifier and sends a SHA-256-derived challenge on the first request. The later code exchange must include the verifier. This binds the exchange to the party that started it. State binds a callback to the login attempt and helps prevent login CSRF; OIDC nonce binds the ID token to the authentication request. Let a maintained security framework implement these protocol details.

Our confidential server client additionally has a client secret. Exact redirect URIs prevent sending codes to an attacker's destination. Wildcard public callbacks are a poor default. The stable sub claim identifies the person; display names and email addresses can change and should not become database ownership keys.

## 3. Cookies, CSRF and the difference between sites and origins

A cookie can be `HttpOnly` so JavaScript cannot read it. `Secure` means it is sent only over HTTPS. `SameSite` restricts some cross-site sending. These flags help but do not replace request authorization. A state-changing request also requires an unpredictable CSRF token tied to the user's session.

`SameSite` is based on a site concept, not the complete origin tuple. Different ports on the same host are different origins but often the same site. Therefore a local identity test on two loopback ports does not prove a cross-site production login will behave identically. For a top-level authorization-code redirect across sites, a carefully configured Lax session cookie is commonly needed; test your actual hostnames and callback method. Never loosen it casually to repair an unexplained error.

## 4. Role mapping is a narrow bridge

Map only the provider claims your application understands. Study Coach recognizes `coach-learner` and `coach-admin`. The provider may carry many unrelated roles; those must not automatically become powerful application permissions. The normal API requires the learner role. Administration and metrics require the administrator role.



The backend enforces permissions. Hiding an admin button is useful interface behavior but not access control. Test the HTTP endpoint directly as a learner. Test unauthenticated access too: an API should return an appropriate failure rather than making a JavaScript client parse an HTML login page.

## 5. A user leaves the company

Disabling an identity stops new authentication, but an application session may already exist. Provider logout, application logout, access-token expiry, refresh-token revocation and back-channel logout solve different parts of this problem. Define your revocation target: immediately, within one minute, or when a session expires. Then implement and test that target.

Possible designs include supported back-channel logout, a centralized session store with an administrative revocation operation, or short sessions plus a periodic authoritative status check. A status check adds latency and a dependency; decide whether identity-provider failure should deny access or temporarily preserve an existing session for your threat model.

The original integration uses Spring's OIDC back-channel logout support and a matching Keycloak client setting. Its browser test verifies that disabling Bob and issuing provider logout invalidates Bob's existing application session. Another test proves the old password fails and the rotated password succeeds for Alice.

The added `test_identity_cluster.py` exercises two application instances with one confidential client. A loopback relay forwards signed logout messages to both Spring endpoints, which validate them; forged messages are rejected. Its final isolated run passed central logout and fresh login on both instances after a provider-secret rotation and application restart. Earlier cutover attempts failed, including an Invalid credentials result; their intermittent cause has not been established. Preserve that finding rather than calling the system production-proven. The relay is synchronous and non-durable, business databases are independent classroom fixtures, and an unavailable instance needs a stronger revocation/recovery design.

## 6. Secret rotation is a two-sided change

Suppose the school changes the lock but never gives the teacher the new key. Authentication breaks. A client-secret rotation must coordinate the identity provider and application. Prefer a supported overlap period: issue new secret, deploy consumers, verify successful exchanges, revoke old secret. If overlap is unsupported, plan a controlled cutover and test rollback. Do not print secrets in reports, commit runtime realm exports, or put them in browser source.

The example generates random classroom secrets and stores temporary state under `.runtime`, which is excluded from the learning ZIP. Production uses a secret manager and least-privilege service identities. Backups need encryption and access control too; they can contain the same sensitive data as the live service.

## 7. Native deployment, containers and cloud are different evidence

A native integration starts separate processes on one machine. A container test adds image construction, container networking, volumes and startup ordering. A cloud test adds public networking, TLS, identity for services, managed data, scaling and operations. Passing one does not imply the others passed.

An isolated WSL distribution now supplies a real Docker engine and K3s node. The completion audit links the actual container and cluster test reports; their local classroom profile is distinct from a production identity deployment. The native PostgreSQL/broker tests use independent processes and real network connections, and their report states their limits. A production launch also needs an actual destination account and domain, which have not been supplied.

## 8. Deployment checklist as an executable release decision

Build once and identify the artifact by digest. Run unit, integration and browser tests against that artifact. Apply backward-compatible database expansion before new code needs it. Deploy a small group of instances; check error rate and latency; expand only when the release gate passes. Keep rollback compatible with the expanded schema. Remove old columns in a later release after all readers stop using them.

Readiness means an instance can serve traffic; liveness means the process should be restarted if unhealthy. A temporary database outage should not cause every instance to restart continuously. Use bounded connection pools, graceful shutdown and request deadlines. Stop admitting new work before terminating a worker, and let its lease recover if it cannot finish.

## 9. Interview and debugging round

Symptom: login works locally but loops after deployment. Investigate actual callback URL, forwarded scheme/host handling, cookie SameSite/Secure settings, session affinity or shared session store, client secret and clock skew. Do not “fix” it by allowing every redirect URI or disabling CSRF.

Exercise: a learner can open `/api/admin/status` by typing the URL although the button is hidden. Correct answer: backend authorization is missing or incorrectly matched. Add the role requirement, then test anonymous, learner and admin requests. Frontend tests alone cannot prove access control.

Reference provenance: [Spring OIDC login](#), [Keycloak import/export](#). Local project: `labs/study-coach/identity` and `test_identity.py`.

## Deep workshop - Rotate a secret without confusing identity layers

A client secret authenticates an application to an identity provider. An access token conveys delegated authority to a resource server. A browser session represents a local login. Rotating one does not automatically revoke the others.

### Follow cutover

Identify all instances using the old secret. Where overlap is supported, enable the new credential, update instances, verify actual token exchanges, then retire the old credential. Confirm old-secret authentication fails and new-secret authentication succeeds. An invalid authorization code is still an error even if it demonstrates that client authentication was accepted.

### Revocation is another path

Logout must reach relevant instances or a shared durable revocation record. An unavailable instance needs retries or reconciliation before accepting affected sessions again. In-memory sessions disappearing on restart does not establish reconciliation of a persistent shared store.

### Diagnose precisely

Separate state mismatch, missing authorization request, redirect mismatch, invalid client credentials, expired code, and rejected cookies. All can look like “login failed” while needing different fixes. Record sanitized error codes and timing. Do ten successful logins prove yesterday’s root cause? **No**: they establish current behaviour under the tested schedule. Historical attribution requires matching evidence. A reproduced possible mechanism remains a hypothesis until the old trace supports it.

# 18 - Many workers, one trustworthy result

## 1. Two children take the same chore card

If two helpers both see “wash the cups,” both may start. Looking at the card is not enough: one helper must claim it atomically. A database update with a condition is the claim. If one row changes, you won. If zero rows change, another helper changed the state first.

Study Coach uses persisted jobs, an expiry time and a random lease token. Each write checks both the running state and that token. A crashed worker's lease can expire so another worker can claim the job. The old worker may wake up later; its obsolete token prevents it from overwriting the new result. This is fencing.

Fencing in our database does not magically fence an external bank or email server. External effects need their own idempotency or fencing support. “Exactly once” must always name the boundary where the guarantee holds.

## 2. A transaction is a sealed envelope

Approving a job changes its status and adds an event to an outbox table in one database transaction. Either both changes commit or neither does. Without this, the application could approve a job, crash before sending a message and lose the work forever.

A relay reads unpublished events, publishes to the durable broker, then marks them published. It can crash after publishing but before marking. Therefore delivery may repeat. This is deliberate at-least-once delivery. The consumer checks a durable inbox and the job's terminal state, so repeated messages do not repeat an already committed result.

There is no atomic transaction spanning our PostgreSQL commit and broker acknowledgment. We avoid pretending otherwise by making the consumer tolerant of repetition. For an irreversible external action, use an external idempotency key and store its outcome. A local inbox inserted before an uncommitted external action can lose work; an inbox inserted afterwards can permit duplicates. Explain that gap in an interview.

## 3. The concrete application path

The HTTP service validates the authenticated owner and CSRF token. A request idempotency key is unique per owner and binds the original question. Approval changes the job and inserts an outbox event. A relay sends the event to the `coach.jobs` queue. Two separate Java processes can listen to that queue. The winner claims the job, calls Python, streams deltas into PostgreSQL, commits the terminal status and records the inbox receipt.

The runnable topology is in `labs/study-coach/test_distributed.py`. It creates its own PostgreSQL cluster, launches a persistent ActiveMQ broker, starts Python and two Java processes, tests a broker outage, republishes an event, applies concurrent requests and restores a backup into a different database. It stops only the processes it started.

The broker is a separate process using the same packaged Java dependencies. It requires a generated password and binds to loopback. These local credentials do not establish production queue-level authorization, encryption or a high-availability broker cluster. Those need explicit configuration for the chosen environment.

## 4. The retry clock and the lease clock

A lease must remain valid while the legitimate worker is active. A fast extractive response may fit a short lease; a large model loading from disk may take much longer before its first token. Either renew the lease independently while computing or choose a bounded lease longer than the maximum execution deadline. Renewing only when tokens arrive leaves a gap before the first token.

Our default teaching worker renews on deltas. For the optional live model, increase both the read deadline and lease consistently; the live-model integration documents its settings. This is a tradeoff: a longer lease makes crash recovery slower. Production designs often use a heartbeat with a strict maximum task lifetime and cancellation.

Retries need a bounded delay and attempt count. A message broker's redelivery limit and a database job's attempt limit are separate budgets. If redelivery is exhausted while another worker holds a lease, work may land in a dead-letter queue even though the job is unfinished. The relay reconciles expired, unfinished jobs by making their outbox events publishable again, while enforcing the persisted three-attempt execution budget. Monitor dead letters and reconciliation separately; a connection-retry loop is not an unlimited business-action budget.

```
# lab: fence_old_worker
state={'status':'pending','token':None,'answer':''}
def claim(token):state.update(status='running',token=token)
def write(token,text):
    if state['status']!='running' or state['token']!=token:return False
    state['answer']=text;return True
claim('first');claim('replacement')
assert not write('first','late wrong answer')
assert write('replacement','current answer')
state['status']='cancelled';assert not write('replacement','too late')
print('Stale and cancelled writes rejected. Real shared claims require conditional SQL.')
```

## 5. Isolation: two doctors cannot both leave

There are two doctors on call. Each transaction reads “two doctors available,” then turns its own doctor off call. The writes touch different rows, so simple row write-conflict detection may not stop both. The invariant “at least one doctor remains” breaks. This is write skew.

PostgreSQL SERIALIZABLE can reject a dangerous combination with a serialization failure. Your application must retry the complete transaction from fresh reads, not merely repeat its final update. Another option is to lock a shared guard row before making the decision, which serializes access to that invariant. Which approach is better depends on contention and the business model.

READ COMMITTED gives a new snapshot for each statement. REPEATABLE READ gives a stable transaction snapshot but does not generally eliminate write skew. SERIALIZABLE aims for an outcome equivalent to some serial execution and may abort transactions to achieve it. These statements describe PostgreSQL behavior; database engines differ. The included isolation test uses an actual PostgreSQL server rather than inferring semantics from H2.

## 6. Backups are promises until restored

A successful backup command proves a file was written. A restore test proves the file can reconstruct something usable. The example uses pg\_dump in custom format, creates a separate database, restores it and compares job count. A stronger test also compares business checksums, constraints, roles, extensions and application behavior.

Recovery point objective asks how much recent data you may lose. Recovery time objective asks how long recovery may take. Logical dumps alone do not provide arbitrary point-in-time recovery; that typically needs base

backups and retained write-ahead logs. Store backups separately from the machine they protect and test access when ordinary credentials are unavailable.

## 7. Cancellation and compensation

Cancellation is a request to stop future work, not time travel. If a message was already delivered or a tool already charged a card, changing a status cannot erase the effect. A compensating action is a new business operation, such as a refund, with its own validation and failure handling.

Use a saga when a business operation spans services without one database transaction. Record each step and its outcome; define compensations; handle partial compensation failures; make every command idempotent. A saga is appropriate for independent services, but it is unnecessary complexity when a single ordinary transaction can cover the invariant.

## 8. Timed system-design exercise

In 20 minutes, design a document-processing service for 10,000 jobs per day. First estimate average and peak arrival rates and job durations. Daily volume alone is insufficient for capacity. Define ownership, idempotency and a state machine before choosing a broker. Then explain approval, queue publication, worker claim, progress streaming, cancellation, retries, dead letters and reconciliation.

Scoring out of 10: two for a clear invariant and state machine; two for atomic job/outbox commit; two for duplicates and external effects; two for bounded retries, cancellation and leases; two for metrics and recovery tests. A diagram full of product names without these guarantees earns little credit.

Reference provenance: [Spring JMS integration](#). The implementation uses ActiveMQ Classic; it does not claim RabbitMQ-specific acknowledgment behavior.

## Deep workshop - Make the outbox solve one atomicity gap

An order update and broker publication normally belong to different transactions. Commit the order first and crash: no event. Publish first and roll back: an event describes an order that never committed.

### Change the boundary

Write the order and an outbox row with stable event ID in one database transaction. A separate publisher reads committed rows and sends events, then records publication progress. The order and intention to publish now share atomicity.

### Duplicates remain possible

Crash after broker acceptance but before marking sent, and recovery publishes again. The consumer needs duplicate handling at its effect boundary. Store consumed event ID and business update atomically where possible. A check followed by an unrelated write is not atomic deduplication.

## Ordering and maintenance

Specify whether order matters per aggregate, partition, or globally. An aggregate version can expose stale updates or gaps. Keep outbox data until the delivery contract permits cleanup. Monitor oldest unpublished age; row count alone can hide one permanently stuck event. Why not coordinate every system in a distributed transaction? **Answer:** Participation, availability, and operational costs may make that unsuitable. An outbox offers local atomicity plus eventual delivery, with explicit duplicates, delay, and reconciliation. Name those tradeoffs instead of claiming universal exactly-once behaviour.

# 19 - Seeing failures and testing the whole experience

## 1. Three ways to understand a busy kitchen

A log is a note: “order 17 failed.” A metric is a number over time: “three orders failed this minute.” A trace is the journey of one order through cashier, cook and delivery. Use each for the question it answers. Logging every token is neither necessary nor privacy-friendly.

Study Coach creates a Java `job.execute` span and propagates W3C trace context in the HTTP request to Python. Python creates a child request span. The integration checks matching trace IDs and the child's parent span ID. This proves a cross-service link; it does not prove that every background message and browser action is instrumented.

JSON trace files provide an offline inspection path. Optional OTLP HTTP export sends spans to a collector. Java Micrometer and Python Prometheus expose counters and durations. Keep these endpoints protected: operational metadata can reveal internals even when it contains no passwords.

The monitoring integration ran native Prometheus 3.14.0, scraped the protected Python endpoint, queried the counter for a real completed answer, and verified a timed alert fixture using promtool. Open the [saved monitoring dashboard](#). Its values are explicitly a recorded run, not current service health. The later collector lab also ran native OpenTelemetry Collector 0.161.0, verified its archive digest, validated its configuration and received an actual Python-exported OTLP trace. See `labs/collector-lab/report.json`. This proves the local receive/debug-export path, not Grafana, durable trace storage, TLS/authentication or outage buffering.

## 2. Labels can eat your memory

A metric with labels route and status has a manageable number of combinations. Adding a unique user ID or question to every label can create millions of time series. That is high cardinality. Put bounded categories in metrics and controlled diagnostic IDs in traces or logs. Never use raw prompts, credentials or bearer tokens as metric labels.

The Python middleware groups unknown paths into other. It records counts and duration for `/answer`, `/health` and `/metrics`. The Java worker records duration and failure counts. The trace files contain identifiers and timing, not the question text or authorization header.

## 3. Average is a quiet liar

If nine requests take 100 milliseconds and one takes 10 seconds, the average is 1.09 seconds. Most users waited 0.1 seconds; one waited 10 seconds. Percentiles show the distribution. For a small nearest-rank sample, p95 means the value at rank  $\text{ceil}(0.95 \times n)$  in sorted order. With only 12 samples that is the maximum, so it is a noisy estimate.

```
# lab: latency_percentiles
import math, statistics
samples=[100]*9+[10000]
def percentile(xs,p):
    if not xs or not 0<p<=1:raise ValueError('invalid sample or percentile')
    return sorted(xs)[math.ceil(p*len(xs))-1]
assert percentile(samples,.5)==100 and percentile(samples,.95)==10000
print({'mean_ms':statistics.mean(samples), 'p50_ms':percentile(samples,.5), 'p95_ms':percentile(samples,.95)})
```

The distributed test measures 12 jobs with four concurrent clients. Its throughput and latency are observations on this machine, not a production capacity guarantee. A serious load study needs warm-up, realistic request distributions, independent load generation, long steady-state runs, resource saturation measurements and repeated trials.

## 4. Alerts should describe user pain

A service-level indicator is a measured property such as successful completed jobs divided by valid submitted jobs. A service-level objective is a target over a specified window. Define what counts as success and what exclusions apply. A job that returns HTTP 200 but later fails should not disappear from your success measurement.

An error budget is the allowed bad fraction. At 99.9% success, the budget is 0.1%. If the current failure fraction is 1%, the service is consuming that allowance at ten times the target rate. Alert over both short and long windows to avoid reacting to a single noisy minute while still catching rapid damage.

```
# lab: error_budget_burn
def burn(failures, total, target=.999):
    if total<=0 or not 0<target<1 or not 0<=failures<=total:raise ValueError('invalid inputs')
    return (failures/total)/(1-target)
assert abs(burn(10,1000)-10)<1e-10
print('1% errors burns a 99.9% success budget at approximately 10x.')
```

## 5. Browser engines are not interchangeable evidence

Chromium, Firefox and WebKit use different implementations. Test your central journeys in all three. WebKit running under Playwright on Windows is useful engine coverage; it is not a test on a physical iPhone or the shipping Safari application. State that distinction in a release report.

The project tests sign-in, job creation, approval, streamed completion, cancellation and navigation. It runs axe rules on the completed page and exercises keyboard interactions. Add cases for a rejected login, missing field, expired session, interrupted stream, route reload and cancellation during work as the product evolves.

## 6. Accessibility beyond an automated score

Use a keyboard from the first control through the final action. Focus must be visible, order sensible and dialogs escapable. A screen reader needs accessible names and meaningful status announcements. At enlarged text and narrow widths, content must remain readable without losing actions. Avoid conveying state only through red and green.

Automated rules can detect many missing labels and contrast failures. They cannot establish that every instruction is understandable, that focus moves helpfully after every dynamic update, or that a whole workflow works with a screen reader. Record manual checks separately from automated results. Do not write “WCAG certified” from a zero-violation scan.

## 7. Failure drills

Drill one: stop the broker. Expected: HTTP approval commits to the outbox; work remains pending; a visible status explains the wait; restart permits delivery. Drill two: stop a worker after claiming. Expected: the lease expires; another worker claims; the old token cannot write. Drill three: disconnect a model stream. Expected: no done state without a final completion marker. Drill four: remove permission. Expected: new unauthorized requests fail, with session revocation behavior matching the documented design.



For every drill, write the hypothesis before the test. Record the observed outcome and recovery time. A failed hypothesis is useful evidence to fix the design, not a reason to hide the test.

## 8. Debugging interview

Question: “The p99 doubled but CPU is low. What next?” Answer: compare traces for time waiting on database connections, locks, network calls, queue delay and downstream services. Low CPU is consistent with blocked work. Inspect saturation of the constrained resource rather than guessing that more application replicas will help.

Question: “Why is the graph expensive after adding user labels?” Answer: each distinct label combination creates another series. Remove unbounded labels, aggregate metrics by meaningful bounded dimensions and retain per-request correlation in traces with a privacy policy.

Reference provenance: the native monitoring binary and checksum came from the [official Prometheus download page](#). The included setup script pins the version and checksum. This reference is optional; the concepts, test and interpretation are provided here.

## Deep workshop - Use telemetry to answer a question

A trace follows a request, a metric summarizes a population, and a log records an event. Begin with a question such as “Where does answer latency grow?” rather than collecting everything without purpose.

### Decompose a request

Of 2,000 ms, suppose authorization takes 100, retrieval 200, inference 1,500, and storage/delivery 200. Optimizing a 5 ms parser cannot remove the dominant delay. If inference is stable but queue wait grows, inspect admission and saturation.

### Interpret distributions

A p95 of 2 seconds means approximately 95 percent of measured observations are at or below it. It does not mean every user waits 2 seconds. Averaging instance p95 values does not calculate global p95. Aggregate compatible distributions or use an appropriate measurement design.

### Test the observer

If the collector is down, does the application block? Are dropped spans counted? If storage is called durable, restart it and confirm previously accepted data remains queryable. A dashboard screenshot proves none of that.

### Connect to the browser

A successful server trace cannot prove that keyboard users reached Submit or understood an error. Test loading, failure, retry, and completion as user journeys. Use correlation IDs for diagnosis while avoiding secrets and private document text in telemetry.

# 20 - Deeper algorithms and mathematics, one small step at a time

## 1. How to learn a difficult algorithm

First say what problem it solves. Then draw a tiny example. State what remains true after each step: the invariant. Only then write the code. Test an empty case, a smallest valid case, an ordinary case and an adversarial case. Finally explain time and space cost, including the data structure assumptions.

This workshop extends the earlier sorting, searching, trees, graphs, tries, dynamic programming and union-find lessons. It covers selected specialist families; there are infinitely many variants and new research results, so “every algorithm ever” is not a finite syllabus.

## 2. Fenwick tree: boxes holding groups of sweets

You have a row of boxes. You repeatedly add sweets to one box and ask how many sweets are in the first  $k$  boxes. A plain array updates quickly but sums slowly. A Fenwick tree stores carefully chosen partial sums. Index  $i$  stores a block whose length is its lowest set binary bit,  $i \& -i$ . A query removes that bit to jump to the preceding disjoint block. An update adds that bit to reach all containing blocks.

At index 6, binary 110, the lowest bit has value 2, so that node covers boxes 5 and 6. A prefix query at 6 uses the block at 6, then the block at 4. The blocks are disjoint and together cover 1 through 6. Each jump changes a bit, so query and update take  $O(\log n)$ ; storage is  $O(n)$ . This sum version needs invertible subtraction for range queries.

```
# lab: fenwick_workshop
class Fenwick:
    def __init__(self,n):self.n=n;self.tree=[0]*(n+1)
    def add(self,i,delta):
        if not 0<=i<self.n:raise IndexError(i)
        i+=1
        while i<=self.n:self.tree[i]+=delta;i+=i&-i
    def prefix(self,end):
        if not 0<=end<=self.n:raise IndexError(end)
        total=0
        while end:total+=self.tree[end];end-=end&-end
        return total
    def range_sum(self,left,right):return self.prefix(right)-self.prefix(left)
f=Fenwick(6)
for i,x in enumerate([2,1,4,3,5,2]):f.add(i,x)
assert f.range_sum(1,4)==8
f.add(2,6);assert f.prefix(3)==13
print('Range [1,4) initially 8; point updates and prefix queries take logarithmic time.')
```

## 3. Segment tree: a tournament of summaries

A tournament combines pairs, then pairs of pairs, until one root summarizes everything. A segment tree works the same way. Each internal node stores a combination of its children. For sums, the identity is zero and the combination is addition. For minimum, the identity is positive infinity. The operation must be associative; subtraction does not work as a simple summary because grouping changes the result.

A range query selects  $O(\log n)$  boundary nodes at each side, totaling  $O(\log n)$  nodes in the standard iterative implementation. Point update is  $O(\log n)$ ; construction and storage are  $O(n)$ . Lazy propagation extends it to range updates by postponing work, but the deferred operation must compose correctly with the stored summary. Do not add a lazy flag without deriving those rules.

```
# lab: segment_tree_workshop
class SegTree:
    def __init__(self, values):
        self.n = len(values); self.t = [0] * self.n + list(values)
        for i in range(self.n - 1, 0, -1): self.t[i] = self.t[2*i] + self.t[2*i + 1]
    def set(self, i, value):
        if not 0 <= i < self.n: raise IndexError(i)
        i += self.n; self.t[i] = value
        while i > 1: i //= 2; self.t[i] = self.t[2*i] + self.t[2*i + 1]
    def sum(self, left, right):
        if not 0 <= left <= right <= self.n: raise IndexError('range')
        left += self.n; right += self.n; answer = 0
        while left < right:
            if left & 1: answer += self.t[left]; left += 1
            if right & 1: right -= 1; answer += self.t[right]
            left //= 2; right //= 2
        return answer
s = SegTree([2, 1, 4, 3, 5]); assert s.sum(1, 4) == 8
s.set(2, 10); assert s.sum(1, 4) == 14
assert SegTree([]).sum(0, 0) == 0
print('Segment tree preserves sum summaries after a point replacement.')
```

## 4. KMP: do not forget what you already matched

Searching for “ababac” in a long string can repeatedly compare the same characters. Knuth-Morris-Pratt stores a prefix table. For each pattern prefix, the table tells us the longest proper prefix that is also a suffix. After a mismatch, that suffix is already known to match; resume from its length instead of restarting blindly.

The text pointer never goes backwards. The pattern pointer may fall back, but its total forward progress bounds those fallbacks. Preprocessing is  $O(m)$ , search  $O(n)$ , and auxiliary space  $O(m)$ . Empty-pattern behavior is a contract choice; here it matches every boundary.

```
# lab: kmp_workshop
def kmp(text, pattern):
    if not pattern: return list(range(len(text) + 1))
    pi = [0] * len(pattern); j = 0
    for i in range(1, len(pattern)):
        while j and pattern[i] != pattern[j]: j = pi[j - 1]
        if pattern[i] == pattern[j]: j += 1
        pi[i] = j
    found = []; j = 0
    for i, c in enumerate(text):
        while j and c != pattern[j]: j = pi[j - 1]
        if c == pattern[j]: j += 1
        if j == len(pattern): found.append(i - j + 1); j = pi[j - 1]
    return found
assert kmp('aaaaa', 'aaa') == [0, 1, 2]
assert kmp('abababac', 'ababac') == [2]
assert kmp('hi', '') == [0, 1, 2]
print('Overlapping matches retained without restarting the text scan.')
```

## 5. Strongly connected components: groups that can all visit each other

In a directed graph, a strongly connected component is a maximal set in which every vertex can reach every other. Kosaraju's algorithm does a depth-first search to record finishing order, reverses all edges, then explores in reverse finishing order. Each second-pass tree is one component. Collapsing components creates a directed acyclic graph, useful for dependency analysis.

Both passes visit each vertex and edge a constant number of times:  $O(V+E)$  time and  $O(V+E)$  storage. The recursive teaching implementation below can exceed Python's recursion depth on a very long path; production code should use an explicit stack for large untrusted graphs.

```
# lab: scc_workshop
def components(graph):
```

```

vertices=set(graph)
for edges in graph.values():vertices.update(edges)
reverse={v:[] for v in vertices}
for u,edges in graph.items():
    for v in edges:reverse[v].append(u)
seen=set();order=[]
def visit(u):
    if u in seen:return
    seen.add(u)
    for v in graph.get(u,[]):visit(v)
    order.append(u)
for u in vertices:visit(u)
seen=set();result=[]
def collect(u,group):
    if u in seen:return
    seen.add(u);group.add(u)
    for v in reverse[u]:collect(v,group)
for u in reversed(order):
    if u not in seen:
        group=set();collect(u,group);result.append(group)
return result
assert {frozenset(x) for x in components({0:[1],1:[0,2],2:[3],3:[2]})}=={frozenset([0,1]),frozenset([2,3])}
print('Four vertices collapse into two strongly connected components.')
```

## 6. Maximum flow: pipes with limited capacity

Flow enters at a source and leaves at a sink. Every intermediate node conserves flow, and each edge has a capacity. A residual graph records both unused forward capacity and the ability to undo earlier flow through a reverse edge. Reverse edges are essential: an early choice may need correction.

Edmonds-Karp repeatedly finds a shortest residual path using BFS and augments by its smallest capacity. The general adjacency-list bound is  $O(VE^2)$ . The small dense implementation below scans all possible neighbors, so each BFS is  $O(V^2)$ , giving a looser  $O(V^3E)$  bound from the  $O(VE)$  augmentation bound. When no path remains, the reachable residual vertices define a cut whose capacity equals the flow, proving optimality by the max-flow/min-cut theorem.

```

# lab: maxflow_workshop
from collections import deque
def maxflow(capacity,source,sink):
    n=len(capacity)
    if source==sink:raise ValueError('distinct endpoints required')
    residual=[row[:] for row in capacity];total=0
    while True:
        parent=[-1]*n;parent[source]=source;q=deque([source])
        while q and parent[sink]<0:
            u=q.popleft()
            for v,c in enumerate(residual[u]):
                if c>0 and parent[v]<0:parent[v]=u;q.append(v)
        if parent[sink]<0:return total
        amount=float('inf');v=sink
        while v!=source:u=parent[v];amount=min(amount,residual[u][v]);v=u
        v=sink
        while v!=source:
            u=parent[v];residual[u][v]-=amount;residual[v][u]+=amount;v=u
        total+=amount
assert maxflow([[0,3,2,0],[0,0,1,2],[0,0,0,3],[0,0,0,0]],0,3)==5
print('Maximum flow is 5, matching the total capacity leaving the source.')
```

## 7. A-star: a map plus a sensible guess

Dijkstra chooses the smallest known distance from the start,  $g$ . A-star chooses  $g+h$ , adding an estimate of distance still needed. If  $h$  never overestimates, it is admissible. If  $h(u) \leq \text{cost}(u,v)+h(v)$  for each edge, it is consistent. Consistency lets a standard closed-set implementation finalize nodes safely. With an admissible but inconsistent heuristic, reopening may be necessary.

For a four-neighbor grid with unit steps, Manhattan distance is consistent. For diagonal movement, that heuristic may overestimate unless adjusted to the movement costs. A zero heuristic reduces A-star to Dijkstra. A good heuristic can reduce explored nodes dramatically, but worst-case search can still be large.

## 8. Gradient descent from the shape of a bowl

Let prediction be  $wx$  and loss be  $L=(wx-y)^2/2$ . The chain rule gives  $dL/dw=(wx-y)x$ : the outer square contributes the prediction error, and the inner linear expression contributes  $x$ . With  $x=2$ ,  $y=6$  and  $w=1$ , the prediction is 2, error is -4 and gradient is -8. A learning rate of 0.1 changes  $w$  to 1.8. Its prediction 3.6 is closer to 6.

The gradient points uphill; subtracting it moves downhill for a sufficiently small step. A huge learning rate can jump past the valley and diverge. Feature scaling changes the shape of the valley and can make optimization easier. A low training loss alone does not establish generalization.

```
# lab: gradient_check_workshop
def loss(w):return (w*2-6)**2/2
w=1.0;gradient=(w*2-6)*2;eps=1e-6
numeric=(loss(w+eps)-loss(w-eps))/(2*eps)
assert abs(numeric-gradient)<1e-6
assert loss(w-.1*gradient)<loss(w)
print({'analytic_gradient':gradient,'finite_difference':round(numeric,6)})
```

## 9. Softmax and cross-entropy simplify together

For probability  $p_i=\exp(z_i)/\sum_j \exp(z_j)$ , the derivative is  $dp_i/dz_j=p_i(\delta_{ij}-p_j)$ . The diagonal is  $p_i(1-p_i)$ ; an off-diagonal entry is  $-p_i p_j$ . Raising one logit takes probability mass from the others.

For one-hot target  $y$  and cross-entropy  $L=-\sum_i y_i \log p_i$ , combining these derivatives gives  $dL/dz_i=p_i-y_i$ . This is why a complicated-looking output layer has a simple gradient. Compute log-softmax with a log-sum-exp formulation rather than taking the logarithm of a rounded probability that might become zero.

```
# lab: cross_entropy_gradient
import numpy as np
z=np.array([2.,1.,0.]);target=1
def ce(v):
    shifted=v-v.max();return -shifted[target]+np.log(np.exp(shifted).sum())
p=np.exp(z-z.max());p=p.sum();g=p.copy();g[target]-=1
numeric=[]
for i in range(3):
    step=np.zeros(3);step[i]=1e-6;numeric.append((ce(z+step)-ce(z-step))/2e-6)
assert np.allclose(g,numeric,atol=1e-7)
print('Cross-entropy gradient:',np.round(g,4).tolist())
```

## 10. Attention with two library cards

A query is what you seek, keys describe available cards, and values carry their content. Dot products measure query-key alignment. Divide scores by  $\sqrt{d_k}$ , apply a mask if future positions are forbidden, then softmax across keys. The output is a weighted sum of values.

If weights are [0.75,0.25] and values are [4,0] and [0,8], the output is [3,2]. Attention does not simply choose one card; it mixes them. Standard full self-attention forms an  $n$ -by- $n$  score matrix, which explains its quadratic sequence-length cost. Cached autoregressive decoding reuses prior keys and values; cache memory still grows with sequence length and model dimensions.

```
# lab: attention_workshop
import numpy as np
q=np.array([[1.,0.],[0.,1.]])
k=q.copy();v=np.array([[4.,0.],[0.,8.]])
scores=q@k.T/np.sqrt(2);scores[np.triu_indices(2,1)]=-np.inf
weights=np.exp(scores-np.max(scores,axis=1,keepdims=True));weights/=weights.sum(axis=1,keepdims=True)
output=weights@v
```

```
assert np.allclose(output[0],[4,0]);assert np.allclose(weights.sum(axis=1),1)
print('Causal attention output:',np.round(output,3).tolist())
```

## 11. Least squares without a fragile inverse

Ordinary least squares minimizes  $\|Xw-y\|^2$ . Setting its gradient to zero gives  $X^TXw=X^Ty$ . That derivation is useful; explicitly computing  $\text{inverse}(X^TX)$  is often a poor numerical implementation because forming  $X^TX$  squares the condition number. QR or SVD-based least-squares solvers are preferable. Rank deficiency means multiple parameter vectors may explain the same observations; it is a property of the data, not something a bigger learning rate fixes.

```
# lab: least_squares_workshop
import numpy as np
X=np.array([[1.,0.],[1.,1.],[1.,2.],[1.,3.]])
y=np.array([1.,3.,5.,7.])
q,r=np.linalg.qr(X,mode='reduced');w=np.linalg.solve(r,q.T@y)
assert np.allclose(w,[1,2]);assert np.linalg.norm(X@w-y)<1e-10
print('QR recovers intercept 1 and slope 2 without forming an explicit inverse.')
```

## 12. Statistics: uncertainty is part of the answer

For independent Bernoulli outcomes, the maximum-likelihood success estimate is successes/n. With a Beta(a,b) prior, the posterior is Beta(a+successes,b+failures), whose mean is (a+successes)/(a+b+n). A uniform Beta(1,1) prior with three successes and one failure gives posterior mean 4/6, whereas the maximum-likelihood estimate is 3/4. The prior smooths a small sample; it is an assumption you must disclose.

Bootstrap resamples observations with replacement to approximate sampling variability. It is not a cure for biased data or dependent observations. For time series, naive independent resampling destroys time structure; use a suitable block procedure or model. Data leakage from future information can make any confidence interval look reassuring around the wrong experiment.

Bias-variance decomposition under squared loss separates squared systematic error, estimator variability and irreducible noise under its assumptions. Increasing model flexibility can reduce bias while increasing variance; regularization, more representative data and appropriate validation address different parts of that tradeoff.

## 13. Specialist interview map

For graph interviews, extend to Bellman-Ford for negative edges and reachable negative cycles, Floyd-Warshall for all pairs on small dense graphs, minimum spanning trees via Kruskal or Prim, bipartite matching and topological dynamic programming. For strings, compare tries, rolling hashes, KMP and suffix structures. For optimization, distinguish convex from nonconvex problems, first-order from second-order methods, and constrained from unconstrained objectives.

For ML interviews, explain PCA as a maximum-variance orthogonal projection and SVD as a matrix factorization; EM as alternating posterior inference and parameter optimization; calibration as agreement between predicted probabilities and observed frequencies; causal identification as requiring assumptions beyond correlation. Learn assumptions and failure examples before memorizing formulas.

Timed task: choose between Fenwick and segment tree for point additions plus range sums. A good answer chooses Fenwick for simplicity and space constants, notes that segment trees generalize to more associative summaries, defines inclusive/exclusive indexing and tests negative updates. A weak answer lists both without explaining the operation contract.

## Deep workshop - Derive attention with two tokens

Attention mixes value vectors according to query-key compatibility. Let  $q=[1,0]$ ,  $k_1=[1,0]$ ,  $k_2=[0,1]$ ,  $v_1=[10,0]$ , and  $v_2=[0,10]$ . The key dimension is 2.

### Calculate every stage

Dot products are 1 and 0. Divide by  $\sqrt{2}$ : approximately 0.707 and 0. Softmax exponentiates and normalizes.  $\exp(0.707) \approx 2.028$  and  $\exp(0)=1$ , so weights are approximately 0.670 and 0.330. The weighted value is  $[6.70, 3.30]$ , a mixture rather than exactly  $v_1$ .

### Why scale and mask?

Scaling moderates magnitude growth with dimension under common assumptions. A causal mask prevents use of later positions in next-token prediction. Conceptually, forbidden logits become negative infinity before softmax and receive probability zero. For stable softmax, subtract the largest finite logit before exponentiating. Removing the same constant leaves normalized probabilities unchanged while reducing overflow risk. An entirely masked row needs an explicit policy; negative infinity minus itself is not valid numeric reasoning.

### Explain the limit

Do high attention weights prove faithful explanation? **No**: they describe one mixing operation among many learned layers. Evaluate behaviour and explanation quality independently. Attention is neither a truth database nor a guarantee that a cited sentence supports a generated claim.

# 21 - Practice that measures understanding

## 1. Reading is rehearsal, answering is evidence

Watching someone ride a bicycle is different from balancing one yourself. Reading a correct answer creates familiarity; producing it under a new constraint tests understanding. This workbook gives timed tasks, answer keys and scoring rules. Your personal readiness remains unassessed until you submit your own answers. No tutor can honestly promise that every exam will be easy.

Use the earlier foundation books when you cannot explain a prerequisite. Use the advanced projects when you can explain a concept but cannot implement or debug it. Keep an error notebook: mistaken assumption, smallest counterexample, corrected rule and a new test.

## 2. Choose a route

Full-stack AI: Python and JavaScript foundations → HTTP and SQL → Java/Spring → React → retrieval and agents → identity and distributed project → system-design rounds. AI/ML: Python → probability and linear algebra → classical ML → deep learning → retrieval/generation → evaluation and deployment. Java backend: language/JVM/concurrency → SQL/JPA/transactions → security/messaging → distributed recovery. React: JavaScript/event loop → TypeScript → state/rendering → accessibility/testing → API integration.

These are prerequisite orders, not rigid calendars. Spend more time where your diagnostic scores are weak. A beginner should build small programs before attempting a multi-service deployment. An experienced engineer should demonstrate unfamiliar failure cases rather than reread only comfortable material.

## 3. Diagnostic: 30 minutes, 20 points

Answer before reading the key. Question one, four points: an order commits, the HTTP response is lost and the customer retries. How do you prevent duplicate orders? Question two, four points: why can two concurrent transactions violate “at least one doctor on call” while updating different rows? Question three, four points: a model returns a valid JSON tool call with another person's user ID; what must the server do? Question four, four points: a React component subscribes to a stream and unmounts; what cleanup and stale-response defenses are needed? Question five, four points: derive the gradient of  $(wx-y)^2/2$  with respect to  $w$ .

Key one: stable idempotency key, uniqueness scoped to caller, payload binding, atomic creation and stored outcome; reconcile an ambiguous result. Key two: write skew; shared invariant; appropriate lock or serializable isolation; retry full transaction. Key three: ignore model-supplied identity, derive it from authentication, validate schema and authorization, reject forbidden tools or arguments. Key four: abort/close subscription, effect cleanup, avoid updates from obsolete request identity, test unmount and route/session changes. Key five:  $(wx-y)x$ , with a correct chain-rule explanation and numeric check.

Award one point per named key component, with a maximum of four per question. A memorized phrase without an explanation receives at most half credit. Scores 0-8 suggest prerequisite work; 9-14 suggest targeted practice; 15-20 suggest progressing to timed implementation. These are study bands, not a prediction of a hiring decision.

## 4. Coding round: sliding window, 25 minutes

Task: given a string, return the length of its longest substring without repeated characters. Explain a brute-force baseline and improve it. Include empty input and repeated characters. Do not assume ASCII unless the problem says so; Python strings iterate code points, which are not always complete user-perceived grapheme clusters.



Baseline: enumerate substrings and check duplicates,  $O(n^3)$  with repeated scanning or  $O(n^2)$  using a growing set per start. Improved: keep the current window's left boundary and last-seen index for each character. When a repeat lies inside the window, move left past it. Never move left backwards.

```
# lab: assessed_sliding_window
def longest_unique(text):
    last={};left=0;best=0
    for right,c in enumerate(text):
        left=max(left,last.get(c,-1)+1);last[c]=right
        best=max(best,right-left+1)
    return best
for text,expected in [('',0),('abba',2),('abcabcbb',3),('aaaa',1),(' ',2)]:
    assert longest_unique(text)==expected
print('Five edge cases passed; O(n) expected time under hash-table assumptions.')
```

Rubric, ten points: two for explaining the invariant, three for correct code, two for edge tests, one for complexity assumptions, two for clearly comparing the baseline. Follow-up: return the actual substring; then process a stream where you cannot retain the entire input.

## 5. Debugging round: transaction proxy, 15 minutes

A Spring service gains `@Transactional`. Tests that access its package-visible db field suddenly see null, although its methods work. Diagnose before adding null checks.

Answer: Spring may inject a proxy. Direct field access is not a proxied method invocation, and proxy instance fields are not necessarily the target object's initialized fields. Inject the required repository directly into each collaborator or expose an appropriate method; do not reach through another service's fields. Also remember that self-invocation can bypass proxy-based advice. The expanded Study Coach work encountered this real defect and replaced direct collaborator-field access.

Score five points for explaining proxy versus target, two for a principled dependency fix, two for testing transaction rollback and one for mentioning self-invocation. "Initialize the field again" does not address the design issue.

## 6. System-design round: agent with approval, 40 minutes

Design an agent that reads an invoice and proposes a payment, but must wait for a human before paying. Define the immutable proposal content and its hash. Approval binds to that exact proposal, not whatever the model writes later. Recheck permissions and current business constraints at execution time. Persist workflow version, state, approval actor and time. Use an idempotent payment operation and reconcile ambiguous timeouts.

Address parallel document analysis without allowing concurrent payment execution. Separate read-only tools from irreversible tools. Explain checkpoint migration if a workflow changes while approvals are pending. Define cancellation before and after the external payment boundary. Include audit data without storing unnecessary sensitive content.

Rubric out of 20: four for state and approval binding; four for authentication/authorization; four for idempotency and crash recovery; four for evaluation, monitoring and budgets; four for tradeoffs and clear communication. An attractive architecture drawing without a failure model is incomplete.

## 7. ML round: a suspiciously excellent classifier

Your model reaches 99% validation accuracy but performs poorly after launch. List hypotheses and tests. Look for target leakage, duplicate entities across splits, temporal leakage, label noise, population shift, incorrect preprocessing at serving time and an imbalanced target where accuracy hides failures. Compare a simple baseline and inspect precision/recall at the actual decision threshold.

A good answer first verifies the evaluation design. It does not immediately demand a larger neural network. Split by the relevant unit-patient, customer or time-before fitting preprocessing. Evaluate probability calibration if decisions depend on estimated risk. Measure subgroup performance only with appropriate sample sizes and privacy constraints.

## 8. React round: stale search results

The user types A, then AB. Request AB returns first; request A returns later and overwrites the screen. Debouncing may reduce requests but does not prove correctness. Abort obsolete work when possible and guard result application with the current request identity. A cancelled network operation can still race with a completion already queued.

Follow-up: how do you distinguish loading initial data from refreshing existing data? Preserve useful existing results while clearly marking stale or refreshing state. Do not show placeholder data as current fact. Test keyboard focus, error announcements and retry behavior, not just a snapshot of successful markup.

## 9. An honest readiness record

For every attempt record the prompt, time limit, your answer before help, score by criterion, mistakes and retest date. Reattempt a related but different problem after two days and again after a week. Track unaided explanation, implementation, debugging and design separately. One total score can hide a serious weakness.

To receive personal grading, send your diagnostic answers and preferred role. Until then, the supplied keys and rubrics are ready, but a personalized score or claim that you are interview-ready would be fabricated.

## Deep workshop - A scoring rubric should reveal the next lesson

Take a 25-minute exercise: return the first index whose sorted value meets a target. Spend three minutes on the contract, twelve implementing, five testing, and five explaining correctness and cost.

### Score observable work

Award two points each for interval invariant, boundary updates, empty/duplicate cases, complexity, and explanation. Record what you produced without hints. Repeating a solution after reading it is practice, not independent assessment.

### Diagnose the error

Duplicate failures suggest lower-bound confusion. Nontermination suggests interval length is not decreasing. Correct output without explanation suggests proof practice is needed. These are different weaknesses; doing random extra questions may not address them.

### Transfer round

Find the first day cumulative production meets a target. There need not be a stored array: binary search works on a monotone predicate. State bounds and whether a solution exists. If production can decrease, the assumed monotonicity may be false and the method needs reconsideration.

## Evidence of improvement

Repeat on a new problem after a delay. Keep elapsed time, hints, and failed tests. Higher scores on a memorized example measure familiarity as well as skill. A useful learning plan targets independently demonstrated weaknesses rather than accumulating unchecked completion ticks.

# 22 - Kafka: a shared notebook that many teams can read

## 1. The classroom story

Imagine a teacher writes every classroom event in a notebook: a child joined, a book was borrowed, a book was returned. The librarian reads these events to update inventory. The head teacher reads the same events to count attendance. Reading a page does not erase it. Each reader keeps a bookmark.

Kafka is an event log with those properties. A producer appends records. A topic names a stream. Partitions divide that stream into ordered logs. A consumer reads records. An offset is a position inside one partition. A consumer group remembers its progress independently from other groups. Retention eventually removes old log data according to policy; Kafka is not automatically an everlasting archive.

Use this model before memorizing settings. A traditional work queue emphasizes distributing jobs and acknowledging delivery. Kafka emphasizes retained streams, independent readers, partition ordering and replay. Both can process jobs, but their operational tradeoffs differ.

## 2. A key chooses an ordering lane

Suppose every event for order 7 uses key order-7. A consistent partitioner normally puts those events in the same partition. Their order is preserved in that partition under the producer and consumer configuration's guarantees. There is no global ordering across all partitions. Adding partitions can change key mapping; plan ordering-sensitive migrations explicitly.

Two consumers in one group divide partitions. In the traditional assignment model a partition belongs to one active member in that group at a time. With two partitions and five consumers, some consumers have no partition work. A different group can independently read the same records.

```
# lab: partition_ordering
import hashlib
def lane(key,count):return int.from_bytes(hashlib.sha256(key.encode()).digest()[ :8], 'big')%count
events=[('order-7','created'),('order-8','created'),('order-7','approved')]
partitions=[[] for _ in range(3)]
for key,value in events:partitions[lane(key,3)].append((key,value))
assert [v for k,v in partitions[lane('order-7',3)] if k=='order-7']==['created','approved']
print('Same-key order retained inside its lane. This teaching hash is not Kafka's default partitioner.')
```

## 3. Acknowledgment, replication and durability

Each partition has a leader and may have replicas. In-sync replicas are the followers considered sufficiently caught up. With `acks=all`, a producer waits for the required in-sync replication acknowledgment.

`min.insync.replicas` constrains when writes may proceed. Replication factor three with minimum in-sync replicas two is a common teaching example: it can sacrifice write availability when too few replicas remain to preserve the intended durability policy.

`acks=all` on a one-node development broker is not three-node fault tolerance. Acknowledgment also does not eliminate every storage or infrastructure failure. Define whether you are protecting against a process crash, machine loss, disk corruption or an entire region disappearing.

The supplied native exercise uses one local KRaft broker and plaintext loopback listeners. It tests real transactions, offsets, consumer assignment and restart persistence. It does not certify replication or production security. The pinned version is Kafka 4.1.2, a deliberately versioned teaching runtime rather than a claim to be

the newest release.

## 4. KRaft and the control plane

Kafka's controllers maintain cluster metadata: brokers, partition leadership and configuration. KRaft manages that metadata through a replicated consensus log. The data brokers store event partitions. A small development process may act as both controller and broker. Production topology should account for the availability and capacity requirements of both roles.

The native harness creates a private storage directory, formats it with a cluster ID, starts Kafka using Java, then kills and restarts its own broker process. Never reformat an existing cluster's storage to "fix" a startup problem. Formatting is initialization, not recovery.

Run `mvn package` in `labs/kafka-lab`, then run `python labs/kafka-lab/run.py` from the library root with the documented Python environment. The harness uses ports 19092 and 19093 and refuses occupied ports. Build dependencies stay outside the ZIP.

## 5. Three meanings of duplicates

A producer can retry a request whose acknowledgment was lost. Kafka's idempotent producer protocol prevents certain duplicate appends caused by those retries. An application can also deliberately send the same business event twice in two distinct sends; producer idempotence does not identify those as the same business operation. Finally, a consumer can execute work and crash before committing its offset, causing redelivery.

Use a stable business event ID and an inbox uniqueness constraint for the third case. Bind an HTTP idempotency key to the caller and request payload for the second. The first is addressed by appropriate producer configuration. These controls complement each other.

## 6. Offsets are the next bookmark

If a consumer has successfully processed offset 12, it normally commits 13: the next position to read. Commit before the business effect and a crash can lose that effect. Commit after the effect and a crash can repeat it. For a database effect, store the event ID and effect in one database transaction, then commit the Kafka offset. Redelivery finds the inbox record and becomes harmless.

```
# lab: consumer_inbox
ledger=[];inbox=set()
def apply(event):
    if event['id'] in inbox: return False
    # These two writes must be ONE transaction in a real database.
    ledger.append(event['amount']);inbox.add(event['id']);return True
e={'id':'payment-7','amount':250}
assert apply(e) and not apply(e);assert sum(ledger)==250
next_offset=12+1;assert next_offset==13
print('One business effect despite redelivery; commit the next offset after durable processing.')
```

## 7. Kafka transactions and their boundary

A transactional producer can commit several Kafka writes atomically or abort them. A consumer using `read_committed` does not expose aborted transactional records. A consume-transform-produce application can include consumed offsets in the Kafka transaction. This supports exactly-once processing semantics within the configured Kafka boundary.

It does not automatically include a PostgreSQL transaction, a payment provider or an email. Naming an external API call inside your transactional method does not make it part of Kafka's transaction. Explain this boundary in an

interview before saying “exactly once.”

KafkaExercise.java commits created, aborts aborted-payment, then commits approved in one partition. After a broker restart, the reader must see only created and approved, in order. It explicitly commits its offset and verifies a new consumer resumes beyond those records. Another stage checks two consumers receive distinct partition assignments.

## 8. Rebalances and slow consumers

Group membership changes can reassign partitions. A handler must stop acting on partitions it no longer owns and avoid committing offsets for unfinished work. Processing longer than the allowed poll interval can make a healthy process appear failed to the group. Moving work into an executor is not sufficient unless you track completion and ownership per partition.

Lag measures the distance between a consumer's position and the end of a partition. It is useful, but one large slow event and many small events can have very different time-to-catch-up at the same lag. Also measure age of the oldest unprocessed event, handler latency, rebalance rate and error rate.

## 9. Retries, poison events and dead letters

Classify failures. A temporary database outage can be retried with backoff. An event with an impossible schema will not improve after a thousand retries. A poison event can block an ordered partition if you insist on strict processing order. Sending it to a dead-letter topic restores progress but changes the completeness or ordering of downstream business processing.

A dead-letter record should preserve original event ID, source topic/partition/offset, schema version, failure category and retry history. Avoid copying secrets unnecessarily. Replay is an operational change: repair the cause, retain the original business idempotency key, and monitor the result.

## 10. Schema evolution and event time

An event is a contract. Include event ID, event type, schema version, occurrence time, aggregate ID and payload. Add optional fields with defined defaults before removing or changing existing ones. A schema registry can enforce compatibility rules, but it cannot prove that a renamed amount field still means the same currency unit.

Event time is when the business action occurred; processing time is when your system handled it. Windowed processing needs a policy for late events, clock errors and corrections. Watermarks and grace periods express a lateness policy; they do not stop a source from sending older data later.

## 11. Outbox, CDC, Connect and Streams

The database outbox solves the gap between a business commit and event publication. A relay or change-data-capture connector publishes committed outbox rows. CDC reads changes from a database's change stream or log; operate it with permissions, retention and recovery in mind. Kafka Connect runs connectors and their offset/configuration infrastructure. Kafka Streams is a Java library for stateful stream processing, including joins, windows and local state stores backed by changelog topics.

Use simple producers and consumers first. Adopt these tools when their operational model fits the problem; a new component adds an upgrade, monitoring and failure responsibility.

## 12. Spring Boot and interview practice

In Spring applications, `KafkaTemplate` sends records and `@KafkaListener` methods receive them. Configure serializers, group ID, acknowledgment/error handling and transaction boundaries explicitly. Do not assume that `@Transactional` automatically combines every database and messaging operation into one atomic commit. An outbox usually makes the boundary easier to reason about.

Interview: “We have 12 consumers and six partitions. Why are six idle?” Explain partition ownership and distinguish groups. Follow-up: adding partitions may alter key routing. Interview: “The email was sent twice although producer idempotence was enabled.” Explain consumer redelivery and the external-effect boundary. Timed design: preserve per-order processing while scaling across many orders; name your key, partition strategy, retry policy and migration plan.

Reference provenance: [Kafka 4.1 quickstart and linked configuration documentation](#). The project and explanations here are self-contained; links document API provenance.

## Deep workshop - Kafka offsets are positions, not message counts

A partition is an ordered log. A committed consumer offset normally identifies the next position to resume. After processing position 41, committing 42 indicates that earlier positions were handled according to the consumer's contract.

### Crash on either side

Commit 42 before applying record 41's effect, then crash: recovery may skip the effect. Apply the effect first, then crash before commit: recovery may apply it again. The latter supports at-least-once processing with duplicate handling at the effect boundary.

### Ordering meets scaling

Order is per partition, not automatically global. Keys influence partition routing. Adding partitions can change routing of future records, so key-order assumptions require migration planning. A consumer group distributes partition ownership, but a partition is not ordinarily assigned to multiple group members simultaneously.

### Poison record policy

A failing record can block progress. Classify transient errors, bound retries, and define dead-letter information. Moving a record aside changes order/completion semantics. Record that decision instead of silently claiming success. Does a Kafka transaction roll back an email? **No**: its atomicity covers supported Kafka operations. A remote effect needs separate idempotency or coordination. Always name the boundary of an exactly-once claim.

# 23 - Docker: pack the lunchbox before sending it to school

## 1. Image, container, registry and volume

An image is the lunchbox recipe plus prepared ingredients: application files, runtime and filesystem layers. A container is a running instance with process isolation and a writable layer. A registry stores images. A volume keeps data independently of a particular container's writable layer.

A container is not a complete virtual machine with its own kernel. Linux containers need a Linux kernel. On this Windows machine, neither a Docker engine nor a WSL distribution was available during the environment check. A Dockerfile can be reviewed [here](#), but a successful native Java run is not a successful container test. The validation report preserves that distinction.

## 2. Build context and reproducibility

The build context is the set of files the builder can access. Copy only what is needed. Exclude credentials, local databases, dependencies, test recordings and runtime downloads with `.dockerignore`. A secret removed in a later layer may still exist in an earlier layer, so never bake it into the image.

Pin application dependencies with lockfiles and tested versions. Image tags can move; a digest identifies particular content. Pinning alone does not solve security updates: record the digest, scan the artifact, then deliberately update and retest. Separate a reproducible build from a permanently frozen vulnerable build.

## 3. Multi-stage builds

Study Coach first builds the React bundle with Node, then packages Java with Maven, then copies only the runnable JAR into a Java runtime image. Build tools do not need to live in the final runtime. The Python image installs pinned service dependencies and runs under a non-root account.

Layer caching works best when stable dependency manifests are copied before frequently changed application source. A changed source file should not force a dependency download if the manifests are unchanged. Cache behavior improves speed; it is not evidence that tests ran.

## 4. Networking without the localhost trap

Inside a container, `localhost` means that container's network namespace. If Java and Python are separate Compose services, Java must call the Python service name, such as `http://ai:8092/answer`. Publishing a port exposes a container port on the host. Services on the same private Compose network can communicate without publishing every port.

Bind development ports to host loopback when external access is unnecessary. A service listening only on `127.0.0.1` inside its container cannot normally be reached through the container network. Study Coach's runtime image sets the server bind address to `0.0.0.0` inside the container while its Compose host port remains loopback-only.

```
# lab: container_address_reasoning
services={'web':'http://web:8091','ai':'http://ai:8092'}
java_upstream=services['ai']+'/answer'
assert java_upstream=='http://ai:8092/answer'
assert 'localhost' not in java_upstream
print('Use service DNS for container-to-container traffic; publish only the intended host entry point.')
```



## 5. State and graceful shutdown

Containers are replaceable; business data is not. Mount persistent data explicitly. A database volume without tested backups is still a single failure domain. Avoid deleting volumes as a routine troubleshooting step.

On shutdown, stop accepting new requests, finish bounded work, flush important state and exit within the configured grace period. If a worker cannot finish, its durable lease should permit recovery elsewhere. The image entrypoint should let the application receive termination signals rather than hiding it behind a shell that mishandles them.

## 6. Resource and security boundaries

Run with a non-root UID, drop unnecessary Linux capabilities, avoid privileged containers and mount only required paths. A read-only root filesystem reduces accidental writes; give the application an explicit temporary directory where needed. Configure CPU/memory requests or limits at the appropriate deployment layer and test out-of-memory behavior.

Secrets belong in runtime secret delivery, not Git, build arguments or browser bundles. Build arguments are not a secret store. Use a trusted registry, a software bill of materials and artifact scanning as part of a release process. These measures complement application authorization; they do not replace it.

## 7. Readiness is not startup order

“Start database before web” does not mean the database is ready when web connects. Applications need bounded retries or explicit readiness checks. A process may have started but still be loading a model or applying migrations. Readiness controls traffic admission; liveness decides whether restarting a process is useful. An external database outage should not automatically restart every web instance continuously.

## 8. The supplied executable recipe

labs/study-coach/compose.yaml connects Java and Python and retains H2 data in a volume. container\_smoke.py authenticates, obtains CSRF, creates and approves a job, then checks its completed answer. The reference GitHub workflow builds the UI, tests Java, runs native browser/recovery checks, builds containers and invokes that smoke test.

These files are runnable instructions, not a claim that Docker was executed here. With an available engine, run from the Study Coach directory:

```
# Set fresh COACH_PASSWORD and COACH_SERVICE_TOKEN in your environment first.
docker compose config
docker compose build
docker compose up -d
python container_smoke.py
docker compose logs --tail 100
docker compose down
```

Do not add volume deletion options if you intend to keep data. `docker compose config` can expand secret environment values, so avoid posting its full output publicly.

## 9. Rolling releases and database compatibility

Consider version A reading `name`, while version B wants `display_name`. First add the new field in a backward-compatible migration. Deploy code that can coexist with both schemas or dual-write when justified. Backfill carefully. Switch readers after verification. Remove the old field only after all old readers have gone. This expand/contract sequence keeps rollback possible longer.

Do not assume rolling back application code rolls back a database migration. A destructive migration may make the old application unusable. Practice rollback with a realistic database copy before treating it as a reliable release strategy.

## 10. Release gates and interview drill

A release gate should identify the exact artifact, tested configuration and measured outcomes. Check HTTP behavior, database migrations, authentication, streaming, restart recovery and resource use. Container success does not establish cloud load-balancer behavior or public TLS configuration; verify those after deployment to the actual environment.

Debugging question: a container works with a bind mount but the built image fails. Investigate missing copied files, ignored files, incorrect working directory, permissions and reliance on host dependencies. Question: the app cannot call Python at localhost. Explain the network namespace and service DNS. Question: an image contains a key deleted in a later layer. Revoke the key, rebuild without it and remove exposed artifacts according to your registry policy; deleting a later-layer file is insufficient.

## 11. Run the supplied image acceptance test

Follow the optional WSL setup in STUDY-ON-ANY-DEVICE, then run `python labs/study-coach/test_containers.py` from the library root. The runner builds the supplied Dockerfiles rather than copying a host-built application into an unrelated image. Node compiles React, Maven packages Java, and the final image contains a Java runtime; Python has its own service image. Think of building a lunchbox in a kitchen: the final box contains the meal, not every appliance used to prepare it.

The runner creates a uniquely named Compose project and random credentials in excluded `.runtime`. It checks anonymous rejection, authenticated CSRF handling, duplicate job submission, human approval, a real Java-to-Python answer delivered over SSE and retained data after a web-container restart. It records image IDs in `container-report.json`. Its final cleanup removes only that test project's services and volume; images stay available for the Kubernetes exercise. An image build or a running container alone is not sufficient acceptance evidence.

This classroom profile uses Basic authentication over local loopback and one H2 writer. It does not claim production identity or a multi-writer database. Initial downloads can be slow, and subsequent cached builds are different performance measurements. Inspect the report for an actual passed result before moving to the cluster capstone. No public-cloud resources are created by this command.

## Deep workshop - Build once and configure at runtime

An image packages files and execution configuration. A container is an instance with runtime settings and a writable layer. Rebuilding for every password change mixes code and environment identity and risks exposing secrets.

## Follow a multi-stage build

The build stage contains compilers and build dependencies. The runtime stage receives the artifact and necessary runtime. Copy dependency manifests before frequently changing source when useful for cache reuse. Cached builds are a speed mechanism, not evidence of dependency trustworthiness.

## Separate state and process

Files in a disposable writable layer may vanish on replacement. Store durable state in an appropriate volume or external service. A volume is not a backup: it can preserve accidental deletion just as effectively as valid data. Inside a container, localhost means its own network namespace. Use configured service names to reach other containers. A started process is not necessarily ready to serve requests; test application readiness.

## Exercise and expected evidence

Write a record, replace the container, and verify survival. Restore a backup into a separate location and compare records. Survival tests replacement persistence; restoration tests recoverability. Deleting a secret in a later image layer does not remove it from earlier layer history. Supply secrets through the supported runtime mechanism.

# 24 - Kubernetes: a caretaker for running applications

## 1. The school caretaker story

Docker packages an application. Kubernetes manages desired running state across machines. Imagine asking a caretaker to keep two classrooms open: if one closes, the caretaker arranges a replacement. That does not restore a child's lost notebook unless the notebook was stored safely. Replacing a Pod is not database recovery.

A cluster has a control plane and worker nodes. The API server accepts declared objects, the scheduler chooses nodes, controllers reconcile actual state with desired state, and node agents run containers. A Pod is the scheduling unit and may contain cooperating containers sharing its network namespace. A Deployment manages replaceable Pods through ReplicaSets. A Service provides a stable discovery/routing abstraction over selected Pods. A StatefulSet adds stable identity and storage association; it does not automatically implement database replication.

## 2. Declarative reconciliation

You describe the desired state, rather than manually starting every replacement process. Controllers keep observing and attempting to close the difference. Reconciliation is repeated and eventually convergent under suitable conditions; it is not a promise that every request succeeds during change. [Kubernetes controllers](#).

```
# lab: kubernetes_reconcile_model
def reconcile(desired, ready, pending):
    return max(0, desired-ready-pending)
assert reconcile(3,1,1)==1
assert reconcile(3,2,1)==0
assert reconcile(3,3,0)==0
print('Count pending work too; otherwise repeated reconciliation overcreates replacements.')
```

This arithmetic model teaches an invariant, not a Kubernetes API implementation. Real controllers track identities, generations, ownership, health and deletion.

## 3. An annotated deployment recipe

The following generic recipe is a starting configuration for the Python service. Replace the image and supply the named Secret through your deployment process. A separate runnable local K3s capstone exercise appears later in this chapter; its report records the exact workload that was actually tested. Do not assume every example manifest has been applied merely because another cluster test passed.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: coach-ai
spec:
  replicas: 2
  selector:
    matchLabels: {app: coach-ai}
  template:
    metadata:
      labels: {app: coach-ai}
    spec:
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
```

```

    seccompProfile: {type: RuntimeDefault}
  containers:
  - name: ai
    image: YOUR_REGISTRY/coach-ai:TESTED_RELEASE
    ports: [{containerPort: 8092}]
    env:
    - name: COACH_SERVICE_TOKEN
      valueFrom:
        secretKeyRef: {name: coach-ai-auth, key: service-token}
    resources:
      requests: {cpu: 250m, memory: 256Mi}
      limits: {cpu: '1', memory: 512Mi}
    securityContext:
      allowPrivilegeEscalation: false
      capabilities: {drop: [ALL]}
    startupProbe:
      httpGet: {path: /health, port: 8092}
      failureThreshold: 30
      periodSeconds: 5
    readinessProbe:
      httpGet: {path: /health, port: 8092}
      periodSeconds: 5
    livenessProbe:
      httpGet: {path: /health, port: 8092}
      periodSeconds: 10
---
apiVersion: v1
kind: Service

metadata: {name: coach-ai}
spec:
  selector: {app: coach-ai}
  ports: [{port: 8092, targetPort: 8092}]

```

Check the image's actual non-root permissions, listen address and health route before applying. A read-only root filesystem is useful only after writable temporary paths are supplied. Memory limits must be measured for the selected embedding/model mode; these illustrative values are not model capacity recommendations. Never scale a Java service using local H2 and in-memory sessions to two replicas and call the data/session design shared.

## 4. Three probes, three questions

Startup: has this slow-starting process finished initialising? Readiness: should it currently receive traffic? Liveness: is it stuck enough that restarting may help? A failed readiness probe removes a Pod from ready endpoints without inherently restarting it. Liveness failure can cause restart. A configured startup probe defers readiness/liveness probing until startup succeeds. [Probe semantics](#).

Do not make liveness fail merely because a shared database is briefly unavailable; restarting every application may worsen the outage. Readiness may incorporate a carefully bounded dependency check when the application cannot serve useful work without it. Separate these routes in a production implementation; the classroom /health route above cannot establish every dependency guarantee.

## 5. Configuration, identity and network boundaries

ConfigMaps hold non-secret configuration. Secrets provide a Kubernetes object for sensitive values, but base64 is encoding, not encryption. Configure encryption at rest, least-privilege RBAC and a rotation strategy. Workload identity can avoid long-lived cloud credentials when supported by the chosen platform.

Namespaces organise resources; they are not an automatic security boundary. NetworkPolicy requires a compatible network implementation and explicit allowed traffic, including DNS and telemetry. Restrict AI access to the application service, database access to authorized services and public ingress to intended routes. TLS termination, certificate renewal, trusted proxy configuration and request-size limits are application-facing concerns too.

## 6. Capacity, disruption and shutdown

Requests influence scheduling; limits constrain resource use. CPU throttling differs from memory termination. Horizontal autoscaling requires meaningful metrics, a working metrics pipeline and downstream capacity. Doubling web Pods can double database connections without doubling database capacity.

A PodDisruptionBudget constrains supported voluntary disruptions; it does not prevent machine failure or make a single replica highly available. Spread replicas across failure domains where appropriate. On termination, stop accepting new work, drain or cancel in-flight operations, close SSE/WebSocket connections with reconnect support and leave durable jobs recoverable. Reconcile the grace period with actual request deadlines.

## 7. Stateful components and migrations

Use shared PostgreSQL for durable application data, an appropriate shared/session-revocation design for identity, and managed Kafka or an explicitly operated broker cluster. PersistentVolumeClaims request storage; snapshots still need restoration testing. StatefulSet identity is useful but insufficient for backup, failover, fencing and schema compatibility.

Run migrations as a controlled release step or Job with locking and failure visibility. Do not have every replica race a destructive migration. Maintain old/new application compatibility during rolling releases. Helm templates package configuration; Kustomize overlays transform it; GitOps controllers reconcile version-controlled desired state. None validates the business behavior of the resulting deployment.

## 8. Deployment acceptance exercise

In an authorized cluster: build and scan a pinned image, validate manifests, deploy a staging namespace, wait for rollout, test login and authorization, submit/retry a job, disconnect/reconnect a stream, terminate a worker, verify recovery, inspect telemetry and restore test data into a separate database. Then rehearse rollback. Record exact image digests, manifests, timestamps and results.

Interview question: a deployment has three Pods but users lose sessions on refresh. Answer: inspect how session state and cookies are handled across replicas; Pod count does not make in-memory state shared. Sticky routing can mask the issue until a Pod dies. A shared session store or suitably designed stateless credentials plus revocation strategy must match the security requirements.

## 9. A real local controller and application exercise

After the Docker acceptance test succeeds, run `setup_k3s_lab.py` and `python labs/run_linux_lab.py kubernetes --capstone` as described in the device/setup guide. The runner starts an actual K3s control plane and node in the isolated lab VM. First it requests two small worker Pods, deletes one and checks that a different Pod UID appears, then requests one replica. A controller is like a teacher repeatedly counting students against a seating plan: it creates or removes seats until actual state matches requested state.

Next, the test imports the previously built application images into the cluster runtime and creates a temporary namespace. The Secret supplies credentials, Services supply stable network names, and a persistent volume stores the Java service's H2 file. Non-root Pods have resource bounds and do not receive a service-account token. Readiness checks determine when the services can accept the test request. A port-forward exposes only a loopback test endpoint.

The test submits and approves a real job, waits for Java to call Python through the ai Service, and records the answer. It then deletes the web Pod and checks that its replacement can read the same completed job. The persistent volume outlives a Pod; it does not automatically survive losing its underlying disk. The web

Deployment uses one replica and Recreate to avoid concurrent H2 writers. Never increase replicas on this file-database profile as a substitute for deploying a shared database.

The namespace and server are cleaned up after testing. Read `kubernetes-report.json` for actual outcomes, including partial failures. This is one node on one computer, without public ingress or a cloud load balancer. It teaches real scheduling, reconciliation, service networking and volume reuse while keeping their boundaries explicit.

An actual first-run defect illustrates why node readiness is insufficient. Binding the API only to `127.0.0.1` allowed local `kubectl` calls and simple Pod reconciliation, but the ClusterIP forwarded in-cluster requests to the node interface. CoreDNS stayed unready and the storage provisioner saw connection refused. The runner now binds the authenticated TLS API to the isolated VM interfaces and explicitly waits for both system Deployments. Application port-forwarding stays loopback-only. A production cluster must also restrict API reachability through its network policy/firewall design. The initial partial report and diagnostic review are retained alongside the rerun.

## Deep workshop - A controller keeps correcting reality

A Deployment declares a desired replica count. Kubernetes observes actual state and acts to reduce the difference. Deleting one Pod normally causes replacement; it does not permanently lower desired replicas.

### Three health questions

Startup asks whether initialization finished. Readiness asks whether traffic should reach this instance. Liveness asks whether restarting may repair it. Making liveness depend on a briefly unavailable shared database can restart all replicas and worsen the incident.

### Walk a rollout

Create new capacity, wait for readiness, stop routing new work to old capacity, drain in-flight work within the termination budget, then stop it. Surge, unavailable, readiness, and termination settings shape the sequence. Process startup alone is insufficient evidence for removing old capacity.

### Resources and failure

Requests influence scheduling. Limits constrain usage according to resource semantics. Memory excess can terminate a process; CPU throttling can increase latency without an application exception. Measure before adding retries. Will a disruption budget prevent every outage? **No**: it constrains supported voluntary disruptions, not all node failures, bad code, or dependency failures. Verify the user journey during replacement and durable state separately from Pod availability.

# 25 - Cache management: keeping photocopies useful

## 1. A cache is a photocopy with a promise

The school register is authoritative; a teacher keeps a photocopy to answer questions quickly. When the register changes, the photocopy can be wrong. Before choosing Redis or Caffeine, decide which answers may be stale, for how long, and what the UI should disclose. A cached public course description and a cached account-permission decision have different consequences.

Cache-aside means the application checks the cache, loads on a miss and stores the result. Read-through delegates loading to a cache abstraction. Write-through updates through a coordinated cache layer before acknowledging according to its contract. Write-behind defers durable writes and therefore needs a durable queue/recovery design if losing accepted writes is unacceptable. A cache is not inherently durable storage.

## 2. Keys, ownership and scope

A key should include all dimensions that affect the answer: tenant, resource, relevant permissions, locale and schema/version where applicable. Caching /profile under one global key leaks data even if the database query was correctly authorized. Avoid logging personal data in keys. Invalidate cached authorization when its authority changes, or explicitly bound how long it may remain accepted.

Local caches are fast but each process has its own copy. Redis gives multiple processes a shared service and additional network/failure costs. A CDN caches at the edge; browser HTTP caching follows response directives. These layers can disagree unless you specify policy end to end.

## 3. TTL, eviction and capacity

TTL expires entries after time; eviction frees capacity. LRU removes least recently used entries, LFU favors frequency, and size-aware admission can reject expensive low-value objects. Redis offers configurable policies; do not assume its approximate eviction implementation is identical to a textbook perfect LRU. A TTL is a maximum local residence under a policy, not proof of global freshness after a remote write.

```
# lab: cache_lru_and_ttl
from collections import OrderedDict
class Cache:
    def __init__(self, capacity): self.data=OrderedDict(); self.capacity=capacity
    def put(self, key, value, now, ttl):
        self.data[key]=(value, now+ttl); self.data.move_to_end(key)
        while len(self.data)>self.capacity: self.data.popitem(last=False)
    def get(self, key, now):
        if key not in self.data: return None
        value, expires=self.data[key]
        if now>=expires: del self.data[key]; return None
        self.data.move_to_end(key); return value
c=Cache(2); c.put('a', 1, 0, 10); c.put('b', 2, 0, 10)
assert c.get('a', 1)==1
c.put('c', 3, 1, 10)
assert c.get('b', 2) is None and c.get('a', 10) is None
print('Capacity evicted b; time expired a.')
```

This toy cache is single-threaded, uses entry count rather than bytes, and conflates cached None with a miss. Production APIs need an explicit hit/miss representation and synchronization policy.



## 4. Invalidation and the late-loader race

Deleting a key after a database commit avoids deleting on rolled-back writes, but a concurrent old reader can refill it with old data afterward. Sequence: reader loads version 4; writer commits version 5 and invalidates; reader finally puts version 4. A version fence must reject older fills even when the value was deleted. Store the fence long enough, or revalidate against the authority.

```
# lab: cache_version_fence
fence={'course':4};cached={}
def put(key,value):
    if version<fence.get(key,0):return False
    fence[key]=version;cached[key]=(version,value);return True
old_read=(4,'old title')
fence['course']=5;cached.pop('course',None)
assert not put('course',*old_read)
assert put('course',5,'new title')
print(cached)
```

In a distributed implementation, compare-and-set must be atomic, for example through a carefully tested Redis script or transaction. Reading a fence then writing separately recreates a race. Event invalidation is delayed and can be duplicated; track versions and define what happens after a missed event. The included real-time lab rechecks the database revision on snapshot reads, so it caches the projection rather than avoiding every database access.

## 5. Stampedes, avalanches and penetration

A stampede is many requests rebuilding one expired key. Single-flight lets one loader run while others await a bounded result. TTL jitter spreads expiration across keys. Negative caching can briefly remember a genuine absence, but must not mix authorization failures across users or permanently hide newly created records. An avalanche is many keys expiring together or an entire cache becoming unavailable.

Define fallback capacity. Sending every missed request directly to a fragile database can cause a cascading outage. Use bounded concurrency, deadlines, admission control and, where acceptable, explicitly labelled stale data. Never substitute stale permission or inventory guarantees without a business-approved consistency model.

## 6. Redis locks and leases

A lease expires, but the old holder may continue running after a pause. A token-checked atomic delete prevents one holder from deleting another's lock; it does not stop the paused holder from writing to a separate database. A monotonically increasing fencing token enforced by the protected resource can reject stale holders. Explain the failure model before presenting a distributed lock as mutual exclusion under every possible failure.

Redis Pub/Sub offers live message delivery to connected subscribers, not a durable replay log. Redis Streams has different retention, consumer and acknowledgement semantics. Persistence, replication and failover settings affect data-loss windows. Treat a rate limiter, session store and disposable cache as different workloads, even if all use Redis. [Redis client cache principles](#).

## 7. Spring and Python integration

Spring `@Cacheable` expresses read caching; `@CacheEvict` expresses eviction. Specify the cache manager, key, serialization, TTL and transactional timing rather than assuming annotations choose them correctly. Proxy interception means self-invocation can bypass annotation behavior. Caffeine is local memory; selecting it does not broadcast invalidations to another replica.

Python can wrap a loader with a bounded cache or use a Redis client with connect/read timeouts, pool limits and serialization validation. Never unpickle untrusted cache values. Treat cache errors explicitly, and observe hit ratio, misses, evictions, bytes, load duration and backend pressure. A high hit ratio can coexist with terrible p99 latency or stale wrong answers.

## 8. HTTP caching and ETags

An ETag identifies a representation version. A client sends If-None-Match; the server can return 304 when the current authorized representation matches. Authorization still runs before deciding that the cached representation is valid. `private` limits shared-cache storage; `no-cache` means revalidate before reuse, while `no-store` forbids storing under the directive's rules.

Personal responses need deliberate cache headers and Vary handling where relevant. An ETag should cover every representation-changing input, not just one table timestamp. The real-time lab tests per-user ETags, mutation invalidation and conditional reads. It does not claim to test Redis failover.

## 9. Working project and interview drill

Read `labs/realtime-cache/app.py`, run `test_project.py`, then inspect `report.json`. Its bounded local cache tests TTL, LRU and stale-version rejection; HTTP tests exercise ownership, ETags and invalidation. The same app exposes real SSE and WebSockets, making cache/stream consistency observable.

Interview: after a write, one user sees stale data for ten minutes. Trace browser cache, CDN, server cache, read replica and database transaction in order. Check cache keys and revision propagation. Propose a measured freshness contract and a regression test rather than immediately increasing TTLs or flushing every customer's cache.

## 10. Actual Redis failover: electing a replacement captain

The new Linux lab runs three Redis servers and three Sentinel processes with generated passwords. Imagine one captain recording changes and two helpers copying the notebook. Three supervisors watch the captain. When the captain stops, the supervisors can agree on a replacement, and a Sentinel-aware client asks where to send its next write.

The test first waits for both replicas to acknowledge a record. It then kills the primary process, waits for promotion, reads the earlier record through the client, writes new data and checks a cache entry expires. Finally, it restarts the old primary and verifies that it becomes a replica and catches up. The recorded run used Redis 8.0.4 and passed these checks.

This does not turn Redis replication into synchronous consensus. WAIT measures acknowledgements in that execution; asynchronous replication can still lose data under other failures. A three-process election on one computer is not a three-machine resilience test. Distinguish Sentinel's failover coordination, the client's rediscovery and the durability of your business data. A cache can often be rebuilt; an authoritative payment ledger needs a separately justified durability design.

Run `labs/test_redis_failover.py` inside the named WSL lab using the exact commands in [Reliability extensions](#). No cloud account is required for this local exercise.

## Deep workshop - Reproduce the late-loader cache race

The database holds profile version 1. Reader A misses cache and reads version 1. Writer B commits version 2 and invalidates cache. A then fills cache with its old version 1. Invalidation happened, yet stale data returned.

## TTL is a bound, not prevention

A TTL can bound stale duration under stated assumptions; it does not eliminate this race. A stale display label may be acceptable. Stale authorization or money state may violate the contract. Classify data before choosing cache policy.

## Strategies and limits

Version-aware writes can reject older fills if the cache knows a newer version. Versioned keys help if discovering the current version is trustworthy. Coordinating fills and writes requires correct distributed synchronization. Sometimes sensitive decisions should not be cached.

## Stampede arithmetic

At expiry, 1,000 simultaneous misses can create 1,000 database reads. Single-flight loading lets one loader refresh while others wait or use explicitly allowed stale values. Bound waiting and handle loader crashes. TTL jitter avoids synchronized expiry across many keys. Is an expiring Redis lock enough to stop an old paused writer? **No**: expiry changes ownership but does not stop the process. A fencing token checked by the protected resource can reject stale owners. Include this schedule in tests rather than testing only normal lock release.

# 26 - Real-time applications: messages that survive disconnection

## 1. Choose the conversation shape

Polling is repeatedly asking “Is it ready?” SSE is a teacher continuously announcing updates to listening students. WebSocket is a telephone conversation in which both sides can speak. Choose by interaction needs, network constraints and recovery requirements rather than treating newer protocols as automatically better.

| Mechanism    | Direction                      | Useful example                     | Main design work                               |
|--------------|--------------------------------|------------------------------------|------------------------------------------------|
| Polling      | Client request/server response | Infrequent job status              | Interval, conditional requests, load           |
| Long polling | Delayed HTTP response          | Compatibility fallback             | Timeouts, repeated requests                    |
| SSE          | Server to client over HTTP     | AI tokens, progress, notifications | Replay cursor, proxy buffering, reconnect      |
| WebSocket    | Bidirectional                  | Collaborative interaction, chat    | Protocol, heartbeat, bounded queues, reconnect |

An open connection is not guaranteed delivery. Every method needs authentication, authorization, resource limits and a recovery policy.

## 2. SSE framing and the browser

An SSE response uses `text/event-stream`. Each event is separated by a blank line. `data`: carries payload, `event`: can name the event type and `id`: supplies a reconnect cursor. Comments can serve as heartbeat traffic. Native `EventSource` reconnects and can send `Last-Event-ID`; the server must implement meaningful replay. [SSE guide](#).

```
id: 42
event: progress
data: {"job":"job-7","status":"complete"}
```

Native browser `EventSource` does not provide arbitrary request-header configuration like `fetch`. Same-origin cookies are one option; streaming `fetch` permits other request handling but needs a parser and reconnect implementation. Do not place long-lived bearer secrets in query strings. Browser cookies require a deliberate same-origin/cross-origin and CSRF policy for related mutations.

## 3. Replay, gaps and duplicate delivery

If the client received event 42 but lost its acknowledgement or disconnected before recording it, replay can repeat 42. Deduplicate by stable event ID or make state application idempotent. If the server retained only events 80 onward and the client asks after 42, silently starting at 80 hides missing state. Return a defined resynchronization signal and fetch an authorized snapshot.

```
# lab: stream_replay_and_gap
events=[(n,{ 'count':n}) for n in range(8,13)]
def replay(after):
    if after<events[0][0]-1:raise ValueError('Snapshot required')
    return [(n,payload) for n,payload in events if n>after]
assert [n for n,_ in replay(10)]==[11,12]
try:replay(3)
except ValueError:pass
else:raise AssertionError('History gap hidden')
state={}
for event_id,payload in replay(10)+replay(10):state[event_id]=payload
assert len(state)==2
```

```
print('Replay is duplicate-safe and refuses a silently incomplete history.')
```

The snapshot and cursor must describe a consistent point. If you fetch a snapshot and then start listening after an unrelated current cursor, you can miss updates between them. Obtain a revision with the snapshot and replay after that revision, or implement an equivalent atomic handoff.

## 4. WebSocket protocol and security

After the handshake, define message types, version, maximum size, request ID, validation rules and error responses. A WebSocket handshake authenticated by cookies is still vulnerable to unwanted cross-site use unless the server validates allowed Origin and the identity/authorization model. Origin checking complements authentication; non-browser clients can choose their own Origin header.

Revalidate session expiry and permissions during long connections or close them on revocation. Authorize each room/job subscription and mutation. Never trust a message's user\_id as identity. Reject malformed messages without leaking server internals. Keep binary/text encoding and maximum decompressed sizes explicit. The browser WebSocket API does not automatically solve application backpressure. [WebSocket API](#).

## 5. Heartbeats, backpressure and cancellation

A dead network path can leave a seemingly open connection. Heartbeats and deadlines detect it. Bound queued bytes per client; if a client cannot keep up, disconnect with a resync path or coalesce replaceable state. Never let one slow reader accumulate an unbounded list of model tokens.

Cancelling the browser connection should stop unnecessary generation where supported, release server resources and preserve a truthful job state. Some upstream operations cannot be cancelled after acceptance; distinguish “stop displaying” from “the effect did not occur.” A retry needs the same logical operation ID if duplication would matter.

## 6. Scaling across application instances

A browser is connected to one server instance. That instance needs the events relevant to its clients. If all instances share one Kafka consumer group, each partition is owned by one consumer; every instance does not receive every event. Choose a fanout topology deliberately: route subscriptions to the owning instance, use a fanout tier, or use distinct consumption arrangements with an understood cost.

Redis Pub/Sub can notify live instances but cannot replay after disconnection. Persist a durable cursor source when recovery matters. Load-balancer idle timeouts, proxy buffering, HTTP version, connection limits and rolling shutdown affect behavior. A sticky session is not durable event storage.

## 7. React and Spring integration

In React, create the connection in an effect and close it in cleanup. Abort pending fetches on teardown, avoid stale closures, deduplicate events and expose reconnecting/stale states accessibly. Keep rapidly arriving tokens from triggering excessive renders by batching display updates. Do not announce every token to a screen reader; announce meaningful status changes and provide the complete answer for reading.

In Spring MVC, SseEmitter needs timeout/error/completion cleanup and a bounded execution strategy. WebFlux can represent streams with Flux, but backpressure only works where the complete chain honors it. STOMP adds a messaging protocol on WebSocket; destinations still need authorization, and broker relay configuration is not equivalent to durable application storage. Choose one coherent implementation before layering frameworks.

## 8. Run the included real project

labs/realtime-cache/app.py is a separate FastAPI/SQLite project with a simple browser page. It supports cookie login, CSRF-protected HTTP writes, SSE replay, authenticated WebSocket publish/ping, logout-driven socket closure, owner-scoped ETags and idempotent writes. Start it using its README rather than importing it into the Study Coach application.

The native test opens an actual WebSocket, rejects an unwanted Origin, publishes an event, logs out and checks closure. It checks SSE Last-Event-ID replay, a retention gap, concurrent writes and recovery after process restart. In-memory sessions intentionally disappear on restart; durable messages remain. This is a local single-process teaching project, not a multi-instance production certificate.

Six additional browser journeys passed across Chromium, Firefox and WebKit at desktop and narrow widths, including keyboard use and axe checks. A further four-client recovery test killed the process during writes, reauthenticated after restart and retried stable keys: eighty acknowledged logical writes matched eighty durable events. The reports record actual scope and duration; a short forced outage does not prove long endurance.

## 9. Interview/debugging round

“AI responses arrive all at once instead of streaming.” Investigate server flushing, proxy buffering, content type, client parsing and upstream streaming. “After reconnect, messages repeat.” Examine stable IDs and idempotent application. “Some users never see events with two servers.” Examine consumer-group topology and subscription routing. “A logged-out user keeps receiving updates.” Examine long-connection revocation, not just the HTTP login endpoint.

For each bug, write a reproducer, identify the invariant, patch the responsible layer and repeat under disconnect/restart. A happy-path animation is not a reliability test.

## 10. Two real application instances

The added test\_multi\_instance.py starts two separate FastAPI processes sharing a SQLite WAL journal. A write sent to instance A is observed by a WebSocket and an SSE connection on instance B. B also notices the new database revision instead of returning its earlier cached representation. This is polling-based fanout through a shared journal, not Redis pub/sub or a distributed streaming platform.

The test exposed a design issue worth remembering: a lock inside A cannot stop B. If both read maximum sequence 4, both might try to write sequence 5. The write transaction now uses BEGIN IMMEDIATE before reading the sequence and idempotency record. SQLite reserves the writer slot; the next writer reads the committed result before deciding what to do. Database uniqueness constraints remain the final guard.

Two simultaneous submissions of the same logical key returned one durable sequence, and eighty distinct cross-instance writes had eighty distinct sequences. The original single-instance protocol tests also passed after this change. Sessions remain separate and in memory. Independent hosts, shared authentication and larger fanout need a different deployment design; these limits do not invalidate the local concurrency lesson.

## Deep workshop - Reconnect a stream without losing the story

SSE carries server-to-client events over HTTP. WebSocket carries bidirectional messages. Job progress often fits SSE; collaboration may need WebSocket. Neither automatically provides durable replay or business ordering.

## Trace event identity

A client applies events 10 and 11, then disconnects. It reconnects requesting continuation after 11. A durable log can supply 12 onward. If 11 is repeated, the client recognizes the ID rather than applying the transition twice. If retained history begins at 20, the server cannot honestly resume from 11. Send a documented gap/reset signal and obtain a current snapshot with a new cursor. Silently starting at 20 creates a plausible but incomplete screen.

## Backpressure is a contract

Bound per-client buffers. Coalescing progress percentages to the latest value may be fine. Dropping financial events is different: disconnect and require durable catch-up, or choose a reliable delivery design. Unbounded queues turn one slow client into memory pressure.

## Security exercise

Authenticate the connection, authorize subscriptions and replay, and decide what revocation does to existing connections. Origin checks are not user authorization. If another instance produces the event, shared delivery infrastructure must preserve ownership checks. Test cross-instance production and reconnect, not only one socket on one server.

# 27 - Reliability: prove what happens when things go wrong

## 1. The fire-drill principle

A school is not prepared merely because its evacuation plan looks tidy. People must rehearse it, record what happened and correct failures. Software reliability works the same way. Separate a design, an executable test, an observed passing result and a production guarantee. Each is a different strength of evidence.

The library includes local native integration tests and recipes for environments that are unavailable here. Inspect **VALIDATION** before interpreting a badge or passing code cell. A mocked response proves the behavior around the mock, not the reliability of the real provider.

## 2. Define the user-visible invariant

For a study job: only its owner can see it; the same logical submission does not create duplicate work; accepted work has a durable record; the UI never describes incomplete work as complete; generated claims must be supported, not merely decorated with a citation.

Turn each invariant into a failure test. Drop the HTTP response after commit and retry. Stop the worker after receiving an event. Disconnect a stream before the completion message. Revoke the session while a connection remains open. Change a document permission after indexing. This approach reveals architectural gaps faster than adding random retries.

## 3. Timeouts, retries and budgets

A timeout bounds waiting, not necessarily execution at the remote side. A retry can repeat an effect that already happened. Retry only classified transient failures and use idempotency where the operation can change state. Set an overall deadline, an attempt limit, bounded exponential backoff and jitter. Respect cancellation and provider rate-limit signals.

```
# lab: retry_budget_model
def schedule(deadline,attempt_cost,delays):
    elapsed=0;attempts=0
    for delay in [0]+delays:
        if elapsed+delay+attempt_cost>deadline:break
        elapsed+=delay+attempt_cost;attempts+=1
    return attempts,elapsed
assert schedule(5,1,[1,2,4])==(2,3)
assert schedule(10,1,[1,2,4])==(3,6)
print('A total deadline can reject a retry even when the attempt counter has room.')
```

The model assumes a known attempt cost; real clients pass the remaining deadline to each operation. Circuit breakers stop repeated calls to a failing dependency under a policy, while bulkheads bound how much capacity one dependency may occupy. Neither repairs data corruption.

## 4. Metrics, traces and logs

Metrics answer how often and how slow. Traces show the path of one operation. Logs describe selected events. Propagate trace context across HTTP and messages, but do not use a client-provided trace identifier as authorization. Correlate by a safe job/request ID and avoid storing prompts, access tokens or personal data in routine telemetry.



Use counters for cumulative events, gauges for current values and histograms for distributions. Report p50, p95 and p99 alongside throughput, concurrency, errors, dataset size and duration. A p95 measured over eighty local writes is not a cloud capacity estimate. Keep metric labels bounded; request IDs and raw prompts create excessive cardinality.

An OTLP collector receives, processes and exports telemetry. A successful SDK call is weaker evidence than observing the expected trace at the collector. The collector lab sends a real span and checks its trace ID and name in debug output. A debug exporter is a test destination, not a durable observability backend. [Collector overview](#).

## 5. SLOs and burn rates

An SLI measures user-visible behavior; an SLO specifies its target over a window. Suppose the target is 99.9% successful eligible requests over thirty days. The error budget is 0.1% of those requests. Define exclusions carefully; excluding all hard failures makes the number meaningless.

```
# lab: slo_error_budget
eligible=100000;failed=80;target=.999
allowed=eligible*(1-target)
remaining=allowed-failed
assert round(allowed)==100 and round(remaining)==20
burn=(failed/eligible)/(1-target)
assert abs(burn-.8)<1e-9
print({'allowed_failures':round(allowed),'remaining':round(remaining),'window_burn':round(burn,2)})
```

Burn rate compares observed failure rate with the permitted rate. Alert windows must balance detection speed and noise. An SLO cannot be inferred from the classroom's brief local test; it requires an agreed indicator and sustained measurement.

## 6. Load, endurance and recovery

Load testing measures behavior under specified demand. Stress testing searches for the breaking point. Endurance testing runs long enough to reveal leaks, accumulating queues and expiring credentials. Recovery testing verifies that state and service return after a specific failure. Record the exact scope rather than calling every concurrent test “stress tested.”

Increase load gradually, monitor dependencies and stop on defined safety thresholds. Use isolated data and authorized environments. Test a dependency slowdown as well as complete unavailability. After restart, verify business counts, duplicate effects and stuck jobs, not just /health. Container and cloud tests remain pending until an actual runtime and destination are available.

## 7. Identity rotation and multi-instance revocation

Rotating a client secret in the identity provider changes provider state. Successful application cutover additionally requires the application to use the new credential and complete a fresh authorization-code exchange. A still-valid old browser session does not test that exchange. Depending on provider capabilities, overlap old/new credentials for a controlled transition or accept an explicitly planned restart window.

Revoking an identity-provider session is different from expiring every application session. With multiple instances, local registries need a shared store or a reliable revocation distribution design. Test sessions established through different instances, revoke centrally, and attempt protected requests and long-lived streams on each. Include one unavailable instance, replayed logout messages and restart. A one-instance back-channel logout test cannot establish all of this.

The added identity-cluster report records a final isolated pass for two-instance logout and fresh login after secret rotation. Earlier runs failed, including Invalid credentials on the second instance; the intermittent cause is not established. The classroom relay requires both instances to be reachable and is not a durable shared-session design. Keep these boundaries visible rather than assuming sticky sessions solve them. TLS, secret management, audit logging, privileged role changes and session lifetime must be specified for deployment.

## 8. AI evaluation beyond valid citations

The real-model integration produced an answer incorrectly suggesting that checkpointing ensures idempotent side effects. Its citation existed, but the claim was wrong. A checkpoint records progress; it does not automatically deduplicate external payments or writes. The factual review remains failed and the app defaults to extractive retrieval.

The separate RAG lab later clarified its source and passed four tutor-reviewed development questions, including the distinction between reconciliation and idempotency. Both its initial flawed answer and a runtime timeout are preserved. That improvement does not retroactively change the earlier Study Coach result or establish general accuracy.

Create cases with question, allowed source text, expected claims, forbidden claims, acceptable abstention and ownership. Grade retrieval separately from generated answers. Check each material claim for support, answer relevance, contradictions and dangerous omissions. A regex can catch a known phrase; it cannot establish general entailment. An LLM judge can assist, but needs calibration against human-reviewed examples and cannot be treated as infallible.

Maintain a development set for fixes and a held-out set for honest evaluation. Version the model, prompt, source corpus and scorer. Include unanswerable questions, conflicting sources, prompt injection in documents, stale facts and cross-tenant retrieval attempts. Report category scores and concrete failures rather than a single reassuring average.

## 9. Accessibility and personal assessment

Automated axe checks find useful rule violations but cannot establish reading order quality, comprehensible announcements or every keyboard interaction. Browser-engine WebKit testing is not a physical Safari/iOS test. A manual screen-reader audit needs an actual assistive-technology session, task script, observed announcements and recorded defects.

Interview readiness similarly requires your own unaided work. Answer a timed prompt, explain tradeoffs, debug a planted failure and revise the answer after feedback. Reading the answer key is learning; it is not a scored demonstration that you can produce the answer independently. The course provides rubrics but cannot honestly award you a personal grade before you submit an attempt.

## 10. Crash recovery you can reproduce

Think of a logout message as a registered letter. Writing down its delivery plan before saying “accepted” prevents a crash from erasing the promise. The durable relay verifies the signature, issuer, audience and logout claims; encrypts the message in a SQLite journal; and records delivery separately for each receiver. A repeated message ID must carry identical content. An unavailable receiver remains pending while reachable receivers proceed. The actual test kills and restarts the relay and verifies the pending receiver eventually revokes its fixture session. These are signed network fixtures, not a claim that the full Spring cluster has a durable shared session store. An expired delivery is a failure requiring reconciliation, not a success to hide.

Kafka provides a different recovery mechanism: replicated logs. The added three-process test uses replication factor three and minimum in-sync replicas two, kills the actual partition leader, verifies new writes and reads all twenty acknowledged records. Restart restores three in-sync replicas. Three processes on one computer are still one machine failure domain. Explain which failure this test survives and which failure stops all brokers.

The five-minute HTTP/cache soak completed 1,127 logical writes and 3,382 requests, with stable sampled handle counts. Every repeated idempotency key returned the original sequence. The measured cycle includes write, duplicate submission and snapshot read. Its 95th-percentile cycle time was 156 milliseconds in this run. These are observations from one local client, not a production service-level objective or a multi-day leak certificate.

The telemetry experiment distinguishes two promises: the collector accepting a span and the destination committing it. The collector retries after a destination outage; the teaching SQLite sink retains the exact trace IDs after a process restart. Queue durability is tested separately from destination durability. Consult the reports before assuming a collector crash preserves buffered spans.

All commands, prerequisites and boundaries are in [Reliability extensions](#). Predict each result, run the test, then deliberately explain how the result would change if the journal, broker quorum or destination disk disappeared.

The stronger telemetry variant also passed: the contrib collector's `file_storage` queue recovered all five buffered spans after a hard collector restart while the destination remained unavailable. This adds queue durability to the separate sink-durability check. It still does not test loss of the computer or its disk.

## 11. Diagnose intermittent login without guessing

A fresh browser can issue several requests at once. If an anonymous API request creates a saved-request session while an OIDC authorization request creates another session, competing cookies can interfere with login state. This application always returns to the home page after login, so it does not need saved-request caching. The OIDC configuration now disables that cache and records only the error code on a failed callback, rather than logging credentials or whole token claims.

The retained rotated-secret fixture reproduced one failure in ten logins before this change. The client-secret probe was accepted by the identity provider. After the change, ten fresh logins across both instances passed, including concurrent anonymous API probes that must return 401 without creating a JSESSIONID cookie. This supports the session-race mitigation; it does not conclusively explain every historical failure. Keep the before/after reports and use failure codes for further diagnosis. A successful rerun alone is not a root-cause proof.

## 12. Deliver real provider logout after a network outage

The next test connects actual Keycloak-issued signed logout messages to two actual Spring applications through the durable relay. A proxy returns 503 for the second logout destination while its application remains available. After provider logout, the reachable application rejects the original browser session, while the isolated one still accepts it. The relay is killed and restarted with the same encrypted journal, then the proxy recovers. The second application now rejects its retained browser session too.

Two fresh provider sessions create two events, each delivered to two destinations: four delivery records. Before relay restart, two deliveries were complete and two pending. After recovery, all four were complete and none expired. This verifies the previously missing real application integration. It does not promise immediate revocation during a network partition: the test deliberately observes that temporary gap. Long outages beyond token validity require readiness reconciliation or a different shared-session design.

The first attempt retained old identity-provider sessions and checked the current browser before its event had arrived. That harness failure is recorded. The corrected experiment clears old fixture sessions before creating new ones and waits for the relevant delivery condition. Tests need correct synchronization just as applications do.

## Deep workshop - Calculate an error budget

A 99.9 percent success target allows 0.1 percent failures over a defined eligible population and window. With 1,000,000 eligible requests, the budget is 1,000 failures. Define eligibility and success first; excluding inconvenient failures afterward changes the promise.

### Burn rate by hand

At observed failure rate 1 percent, divide by allowed 0.1 percent: burn rate is 10. Budget is being spent ten times the target rate. Short spikes and sustained incidents differ, so alerts often combine short and longer windows.

### Fit retries into a deadline

Three sequential attempts with five-second timeouts can consume fifteen seconds before backoff and queueing. Use one end-to-end budget. Retry only safe, potentially recoverable operations. Retrying into an overloaded dependency can amplify failure.

### Design the experiment

State the invariant, inject one fault, observe the user outcome, restore the component, then verify reconciliation. Count intended operations and durable effects, not merely HTTP successes. Include duplicates, losses, and unresolved outcomes. Why is average latency insufficient? **Answer:** It hides tail behaviour and population changes. Inspect distributions, errors, saturation, and whether timeouts were excluded. A successful short load test does not establish day-long endurance or independent-host disaster recovery.

# 28 - Put the pieces together: a full-stack engineering capstone

## 1. The product story

Build a study platform. A learner logs in, asks a question about permitted course material, watches progress, receives an answer with evidence and can return later to see the saved result. An instructor updates material. An operator can investigate a failed job without reading every learner's private prompt.

The integrated Study Coach project already connects React, Java and Python with authentication, durable jobs, retrieval and browser tests. Kafka and the real-time/cache project are additional working labs. Their components are not silently wired into that existing application. Integration is a separate change requiring data contracts and end-to-end tests.

## 2. Trace one request through the architecture

React sends a CSRF-protected command with a logical request ID to Java. Java authenticates, authorizes, validates and atomically stores the job and outbox event. A publisher delivers the event. A worker loads the authorized source context and calls Python's retrieval/model boundary with a deadline. It conditionally stores the result and emits a progress event. SSE or WebSocket delivers authorized updates; a snapshot endpoint repairs missed history.

PostgreSQL owns durable business records. Kafka can carry replayable events. Redis can cache permitted projections or support a specifically designed shared session mechanism. Object storage can hold large original files while PostgreSQL stores metadata and access rules. A vector index supports retrieval. Each component adds operational responsibility; use it only when a requirement justifies it.

## 3. Contracts before frameworks

Define identifiers, ownership, schema versions, timestamps, status transitions, error codes, retryability and idempotency. A job may be queued, running, completed, failed or cancelled; decide which transitions are legal and who can make them. A completed result should not be overwritten by a late retry from an earlier worker generation.

```
# lab: capstone_state_machine
allowed={'queued':{'running','cancelled'},'running':{'completed','failed','cancelled'},
        'failed':{'queued'},'completed':set(),'cancelled':set()}
job={'state':'queued','version':0}
def change(expected_version,target):
    if job['version']!=expected_version or target not in allowed[job['state']]:return False
    job.update(state=target,version=expected_version+1);return True
assert change(0,'running')
assert not change(0,'cancelled')
assert change(1,'completed')
assert not change(2,'running')
print(job)
```

This is a single-process model. Production compare-and-change belongs in an atomic database statement/transaction, not a read-then-write across two HTTP calls.

## 4. Full-stack skills that sit between the frameworks

Understand HTTP methods/statuses, DNS, TLS, reverse proxies, cookies, CORS, CSRF and browser storage. Know HTML semantics, responsive CSS, forms, focus management and JavaScript runtime behavior. Use TypeScript to express boundaries but still validate untrusted runtime JSON. Learn Git, reproducible builds, dependency updates, configuration management and rollback.

For APIs, cover REST resource design, pagination, validation, error envelopes, rate limits and compatibility. GraphQL can support client-shaped queries but needs authorization, query-cost limits and N+1 prevention. gRPC can suit typed service calls, with explicit deadline and compatibility handling. Choose a protocol according to clients and operational constraints rather than collecting every protocol in one request path.

For data, cover schema ownership, migrations, connection budgets, query plans and backups. For AI, cover token/context budgets, source permissions, model versioning, evaluation, prompt injection and cost limits. For operations, cover build provenance, logs/metrics/traces, deployment health and incident response.

## 5. A staged implementation route

Stage one: run the existing integrated app and explain each request without AI help. Stage two: implement one database invariant and a failing regression test. Stage three: substitute an event transport behind a clear interface, preserving outbox/inbox behavior. Stage four: add a bounded shared cache with a freshness policy. Stage five: add replayable real-time delivery and a reconnect test. Stage six: package and execute containers. Stage seven: deploy into an authorized staging environment, collect telemetry and rehearse recovery.

Each stage has a concrete exit condition. A YAML file is not the exit condition for deployment; a running, tested release is. A cache class is not the exit condition for cache reliability; demonstrated invalidation and bounded behavior are. Do not mark a stage complete merely because a chapter explains it.

## 6. Failure matrix for the capstone

| Failure                           | Expected behavior                            | Evidence to collect                                  |
|-----------------------------------|----------------------------------------------|------------------------------------------------------|
| Response lost after commit        | Same request ID returns original job         | One durable job and stable result                    |
| Event delivered twice             | One business effect                          | Inbox/unique constraint and duplicate test           |
| Worker dies mid-job               | Lease/recovery policy applies                | State before/after restart and no lost accepted work |
| Database unavailable              | Bounded error/backpressure                   | Deadlines, pool metrics, recovery                    |
| Model produces unsupported claim  | Evaluation rejects release/default path      | Claim-level failure record                           |
| Stream reconnects after retention | Snapshot/resync, explicit state              | Cursor test and UI recovery                          |
| Permission revoked                | Unauthorized reads/streams stop under policy | Tests on every relevant instance                     |
| Cache unavailable                 | Bounded fallback or explicit failure         | Backend load and error budget impact                 |
| Deployment partially rolls out    | Compatible contracts or controlled rollback  | Old/new version coexistence tests                    |

## 7. Interview rounds and scoring

Round A, 30 minutes: implement duplicate-safe job submission and tests. Round B, 20 minutes: diagnose a stale cache race. Round C, 40 minutes: design the platform for an explicit workload, then handle a regional dependency failure. Round D, 20 minutes: explain one algorithm with an invariant, complexity and edge cases. Round E, 15 minutes: interpret a SQL query plan and fix N+1 behavior.

For each round award 0-4 in correctness, explanation, failure handling, testing and tradeoffs. Zero means absent/incorrect; one means memorized but not workable; two means a basic working answer; three means a justified tested answer; four means it remains coherent under follow-up changes. Keep the transcript, feedback and a fresh repeat attempt. Scores must come from your submitted work, not a prefilled success label.

## 8. What completion means

Reading completion means you can explain each chapter's worked example. Lab completion means you can reproduce results and deliberately trigger its failure cases. Project completion means the integrated application meets its stated acceptance tests. Interview readiness means repeated unaided performance against representative roles. None promises every possible exam or permanent coverage of a changing field.

Use the evidence report to distinguish completed local tests from cloud, physical-device, clustered identity and endurance work still needing execution. Keep that distinction visible as the library grows. The database notebook is a separate deepening path; return to the capstone and apply its modelling, transaction and indexing lessons to one real endpoint.

## Deep workshop - Build a vertical slice before every feature

A vertical slice connects browser, API, storage, and worker for one useful journey. Begin with submitting a question and retrieving its saved result. Integration risks live between components, not just inside them.

### Acceptance examples

Alice creates J with key K. Repeating the same request returns J. Bob cannot read J. Approval allows processing. Completion or safe failure is durable. Refresh retrieves that state. Worker recovery must not duplicate external effects.

### Add complexity deliberately

Add jobs, progress streaming, cancellation, and generation one at a time. Retain the original journey test. A loading animation does not establish recovery, and a backend success does not establish a usable error state.

### Who owns each fact?

The authentication boundary owns caller identity. Durable storage owns job state. The model produces draft tokens. Application policy decides whether those tokens are accepted. If two components both claim authoritative ownership, define reconciliation before deploying.

### Race exercise

Cancellation arrives as completion commits. Which wins? **Answer:** The transition contract must decide. Use atomic conditional transitions or version checks so the race has a documented result. A cancellation response should describe the actual accepted state rather than promise that completed work was undone. Test both event orders.

# 29 - Databases: the organised memory of your application

This separate database notebook joins the Python, Java/Spring, React and AI tracks. Start with the stories, run the small SQL experiments, then answer the interview questions without looking at the solutions. The embedded labs use Python's SQLite so they run without a server. PostgreSQL-specific SQL is explicitly a recipe; SQLite success does not prove PostgreSQL concurrency behavior. The existing distributed lab separately exercised real PostgreSQL, serializable conflicts and backup/restore.

## 1. Why a database exists

Imagine a classroom cupboard. A Java variable is a note in a child's hand: it disappears when the child leaves. A database is the class register: it survives the application restarting, can be shared, and enforces rules about what may be written.

A database management system handles storage, queries, concurrent changes, permissions and recovery. Persistence alone is insufficient. A text file can persist, but coordinating simultaneous seat reservations, searching millions of orders and recovering a half-written transfer require more machinery.

Draw the application as browser → API → business rules → database. The browser never receives database credentials. Python and Java access storage through bounded connection pools. AI retrieval reads only documents the authenticated user may access. Kafka carries events about changes; it is not automatically the authoritative store for every screen.

## 2. Which kind of database should I learn?

| Kind                          | Child-sized picture                    | Suitable workload                                     | Cost or caution                                                 |
|-------------------------------|----------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------|
| Relational: PostgreSQL, MySQL | Related class registers                | Orders, payments, users, inventory                    | Schema design and transaction boundaries matter                 |
| SQLite                        | A register in one file                 | Local apps, small experiments, embedded storage       | Writer concurrency and deployment differ from a database server |
| Document: MongoDB             | One folder containing a whole form     | Variable product attributes, aggregate-shaped records | Unbounded arrays and repeated data create update problems       |
| Key/value: Redis              | A numbered locker                      | Shared caches, counters, rate limits                  | Eviction and persistence settings determine what may disappear  |
| Wide-column                   | Very large sorted filing cabinets      | High write volume with known partition-key queries    | Query design follows access patterns; hot partitions hurt       |
| Graph                         | People and strings connecting them     | Paths, dependencies, fraud relationships              | Traversal growth must be bounded                                |
| Search engine                 | A book's word index                    | Full-text search and relevance ranking                | Search index can lag the source of truth                        |
| Vector index/database         | Shelves arranged by similarity         | Semantic retrieval and recommendations                | Similarity is not truth, authorization or exact equality        |
| Columnar warehouse            | A calculator scanning selected columns | Aggregation over large historical datasets            | Different latency/cost goals from transactional APIs            |

For this course, master SQL and PostgreSQL first, then Redis and vector retrieval; learn document modelling and analytics when the workload requires them. This is a learning order, not a rule that every product needs all these databases. Adding a second database adds backup, access control, monitoring, upgrade and reconciliation work.



### 3. Tables, keys, types and constraints

A row is one fact about one entity. A primary key uniquely names it. A foreign key points to a related row. A unique constraint prevents repeated business identifiers. NOT NULL rejects missing values, and CHECK validates a row-level rule.

Use exact decimal types or integer minor units for money with an explicit currency; binary floating point is unsuitable for exact monetary accounting. Store instants with clear timezone semantics; retain a user's timezone separately for calendar rules. A generated identifier is not a secret, an authorization check, or a promise of gapless numbering.

```
# lab: database_constraints
import sqlite3
db=sqlite3.connect(':memory:')
db.execute('PRAGMA foreign_keys=ON')
db.executescript('''
CREATE TABLE customer(id INTEGER PRIMARY KEY, email TEXT NOT NULL UNIQUE);
CREATE TABLE orders(id INTEGER PRIMARY KEY, customer_id INTEGER NOT NULL REFERENCES customer(id),
                    total_cents INTEGER NOT NULL CHECK(total_cents>=0));
INSERT INTO customer VALUES(1,'alice@example.test');
INSERT INTO orders VALUES(10,1,1250);
''')
for statement in ["INSERT INTO orders VALUES(11,99,100)",
                  "INSERT INTO orders VALUES(12,1,-5)",
                  "INSERT INTO customer VALUES(2,'alice@example.test')"]:
    try: db.execute(statement)
    except sqlite3.IntegrityError: pass
    else: raise AssertionError('Invalid data was accepted')
assert db.execute('SELECT COUNT(*) FROM orders').fetchone()[0]==1
db.close()
print('Foreign key, nonnegative money and unique email rules rejected invalid writes.')
```

Application validation gives friendly errors; database constraints protect against every writer, including another service and a race between two requests.

### 4. Modelling relationships and normalisation

One customer has many orders: orders holds customer\_id. Students take many courses: an enrolment table holds student\_id and course\_id, with a composite unique key. A one-to-one profile can have a unique user\_id foreign key.

First normal form keeps values appropriate to the relational design rather than stuffing a comma-separated list into one column. Second normal form removes dependencies on only part of a composite candidate key. Third normal form removes inappropriate transitive dependencies on non-key attributes. BCNF tightens the rule: every nontrivial functional dependency has a superkey as its determinant.

Example: OrderLine(order\_id, product\_id, product\_name, quantity) repeats product\_name. Put the current product name in Product. But preserve the purchased unit price and invoice description on the order line when the business requires historical truth. That duplication is deliberate history, not necessarily a modelling mistake. State the dependency and lifetime before applying a slogan about normalisation.

Denormalise only for a measured query need and document who refreshes the copy, how stale it may be and how it is rebuilt. A materialized read model requires ownership and repair just like a cache.

## 5. SQL: selecting, joining, grouping and NULL

Think of SQL as describing the answer, while the database chooses a physical plan. FROM and JOIN assemble candidate rows, WHERE removes individual rows, GROUP BY gathers piles, HAVING removes piles, SELECT chooses output and ORDER BY establishes order. This is a useful logical model, not a required execution sequence.

```
# lab: database_joins_and_null
import sqlite3
db=sqlite3.connect(':memory:')
db.executescript('''
CREATE TABLE customer(id INTEGER PRIMARY KEY,name TEXT);
CREATE TABLE orders(id INTEGER PRIMARY KEY,customer_id INTEGER,total INTEGER);
INSERT INTO customer VALUES(1,'Alice'),(2,'Bob'),(3,'Chen');
INSERT INTO orders VALUES(10,1,500),(11,1,700),(12,2,300);
''')
rows=db.execute('''SELECT c.name,COUNT(o.id),COALESCE(SUM(o.total),0)
FROM customer c LEFT JOIN orders o ON o.customer_id=c.id
GROUP BY c.id,c.name ORDER BY c.id''').fetchall()
assert rows==[('Alice',2,1200),('Bob',1,300),('Chen',0,0)]
assert db.execute('SELECT NULL = NULL, NULL IS NULL').fetchone()=(None,1)
db.close()
print(rows)
```

COUNT(\*) counts rows, including the null-extended row created by a left join; COUNT(o.id) counts non-null order IDs. Filtering the right table in WHERE can unintentionally turn a left join into inner-join behavior. SQL NULL represents missing/unknown information; use IS NULL, not = NULL. NOT IN can surprise you when the subquery includes NULL; a correctly correlated NOT EXISTS is often clearer.

Know INNER, LEFT, CROSS and self joins. A join can multiply rows: do not fix an accidental many-to-many join by blindly adding DISTINCT. Check cardinalities and the intended grain, meaning what exactly one result row represents.

## 6. Windows, CTEs and pagination

A GROUP BY collapses rows. A window function annotates each row while retaining it. ROW\_NUMBER assigns positions; RANK leaves gaps for ties; DENSE\_RANK does not. A common table expression names an intermediate query. It is not universally a performance improvement.

```
# lab: database_windows_and_pagination
import sqlite3
db=sqlite3.connect(':memory:')
db.executescript('''CREATE TABLE result(id INTEGER PRIMARY KEY, learner TEXT,score INTEGER);
INSERT INTO result VALUES(1,'Alice',90),(2,'Bob',80),(3,'Chen',80),(4,'Divya',70);''')
ranked=db.execute('''SELECT learner,DENSE_RANK() OVER(ORDER BY score DESC)
FROM result ORDER BY score DESC,id''').fetchall()
assert ranked==[('Alice',1),('Bob',2),('Chen',2),('Divya',3)]
first=db.execute('SELECT id,score FROM result ORDER BY score DESC,id LIMIT 2').fetchall()
last_id,last_score=first[-1]
second=db.execute('SELECT id,score FROM result WHERE score<? OR (score=? AND id>?)
ORDER BY score DESC,id LIMIT 2''',(last_score,last_score,last_id)).fetchall()
assert first+second==[(1,90),(2,80),(3,80),(4,70)]
db.close()
print('Stable tie-breaking avoids skipping Chen at the page boundary.')
```

Offset pagination is simple but large offsets can scan/discard many rows and concurrent inserts can shift pages. Keyset pagination continues after a stable sort tuple. It still needs a stated consistency policy if sort keys change. Opaque cursors must be validated, tenant-bound and, when necessary, signed. Never accept arbitrary column names from an untrusted client; allowlist supported sort orders.

## 7. Transactions and ACID with a transfer

Atomicity: move both pieces or neither. Consistency: preserve declared constraints and correctly implemented business invariants. Isolation: define what concurrent operations may observe. Durability: committed data survives the failures covered by the configured storage guarantees.

```
# lab: database_atomic_transfer
import sqlite3
db=sqlite3.connect(':memory:')
db.execute('CREATE TABLE account(id INTEGER PRIMARY KEY,balance INTEGER CHECK(balance>=0))')
db.executemany('INSERT INTO account VALUES(?,?)',[ (1,100),(2,50)]);db.commit()
def transfer(amount,fail=False):
    with db:
        changed=db.execute('UPDATE account SET balance=balance-? WHERE id=1 AND
balance>=?',(amount,amount)).rowcount
        if changed!=1: raise ValueError('Insufficient funds')
        if fail: raise RuntimeError('Connection failed before credit')
        db.execute('UPDATE account SET balance=balance+? WHERE id=2',(amount,))
try: transfer(30,True)
except RuntimeError: pass
assert db.execute('SELECT SUM(balance) FROM account').fetchone()[0]==150
assert db.execute('SELECT balance FROM account WHERE id=1').fetchone()[0]==100
transfer(30)
assert db.execute('SELECT balance FROM account ORDER BY id').fetchall()==[(70,),(80,)]
db.close()
print('Rollback restored the debit; successful transfer preserved total money.')
```

The toy transfer assumes both fixed accounts exist. A real API checks the recipient, currency, limits, audit trail and idempotency; it must reject or roll back when a credit affects no row. A database transaction cannot automatically roll back an email, external payment or Kafka publication. Use an outbox plus an idempotent consumer for reliable communication.

## 8. Isolation, MVCC, locks and retries

MVCC is like keeping several dated versions of a register so a reader can consult the version allowed by its snapshot. Dirty read means seeing uncommitted work; nonrepeatable read means rereading a changed row; phantom means a matching set changes. Write skew occurs when concurrent transactions read a shared condition and update different rows, together violating it.

In PostgreSQL, Read Committed generally uses a fresh snapshot for each statement. Repeatable Read provides a stable transaction snapshot but can still admit serialization anomalies. Serializable can reject conflicting transactions; the application must retry the whole transaction with a bounded policy. PostgreSQL treats Read Uncommitted as Read Committed. These are engine-specific details, not SQLite lab results. [PostgreSQL isolation reference](#).

Optimistic locking adds a version: UPDATE item SET value=?,version=version+1 WHERE id=? AND version=?. Zero changed rows means a conflict. Pessimistic locking locks selected rows before dependent work. Acquire multiple locks in a stable order to reduce deadlocks; keep transactions short and avoid remote calls inside locks. Retry known transient failures with limits and jitter, not every SQL exception.

## 9. Indexes: finding a name without reading the register

A B-tree is a sorted branching guide useful for equality and ordered ranges. Hash indexes target equality. GIN supports inverted lookups such as tokens or suitable JSON operators. GiST supports extensible search strategies such as some geometric operations. BRIN summarizes block ranges and can be small when values correlate with physical order. Operator support matters. [PostgreSQL index types](#).

```
# lab: database_index_plan
import sqlite3
```

```

db=sqlite3.connect(':memory:')
db.execute('CREATE TABLE event(id INTEGER PRIMARY KEY,owner TEXT,created INTEGER)')
db.executemany('INSERT INTO event VALUES(?,?,?)',[('alice' if i%2 else 'bob',i) for i in range(1000)])
db.execute('CREATE INDEX event_owner_created ON event(owner,created DESC,id DESC)')
plan=db.execute('EXPLAIN QUERY PLAN SELECT id FROM event WHERE owner=? ORDER BY created DESC,id DESC LIMIT
5','alice',)).fetchall()
assert any('event_owner_created' in row[3] for row in plan)
assert [r[0] for r in db.execute('SELECT id FROM event WHERE owner=? ORDER BY created DESC,id DESC LIMIT
3','alice',))]==[999,997,995]
db.close();print(plan)

```

An index costs storage and write work. Composite column order should match actual equality, range and sorting requirements. A partial index covers only rows satisfying its predicate. A covering index can reduce table access but does not guarantee zero heap visits on every engine and visibility state. An index used on a 1,000-row example is not a production speed benchmark.

## 10. Reading plans and fixing slow queries

Start with the slow endpoint and its query count, duration distribution and representative parameter values. Look at scans, join strategies, estimates versus actual rows, sorting spills, buffer use and lock waits. A sequential scan may be the correct plan when much of a table is needed. Nested-loop, hash and merge joins suit different cardinalities and ordering.

PostgreSQL EXPLAIN describes the chosen plan; EXPLAIN ANALYZE actually executes it. Therefore a modifying statement can change data while being analysed. Use a safe test database and understand effects that rollback cannot undo. [PostgreSQL plan guide](#).

Check missing/selectively useful indexes, stale statistics, excessive rows fetched, N+1 queries, huge IN lists, implicit casts and unbounded sorts. Fix the measured bottleneck before increasing hardware. Record before/after latency, rows examined, throughput and correctness.

## 11. Java/JPA and Python data access

With JPA, an entity is a persistence-managed object, not the public API contract. Lazy relationships can cause N+1: one order query followed by one customer query for every order. Fetch joins, entity graphs, DTO projections or batching can help, with different pagination and memory costs. A collection fetch join combined with pagination needs particular care. Put transaction boundaries around business operations, account for proxy/self-invocation behavior and do not assume readOnly enforces all business restrictions.

With Python, use parameterized driver calls or SQLAlchemy expressions. A session/connection should have a clear request or job lifetime, commit/rollback handling and guaranteed cleanup. Blocking database access inside an async server can block its event loop; use a compatible async driver or an appropriate bounded offload strategy. Async does not remove connection limits.

Connection pool example: 10 replicas × 20 maximum connections = 200 possible application connections before background jobs and administrative reserve. More connections can increase contention. Measure acquisition wait and saturation; set query, transaction and pool-acquisition timeouts independently. Never hold a connection while waiting for a slow model response unnecessarily.

```

# lab: database_parameter_safety
import sqlite3
db=sqlite3.connect(':memory:')
db.execute('CREATE TABLE note(owner TEXT,body TEXT)')
db.execute('INSERT INTO note VALUES(?,?)','alice','private')
attack="alice' OR 1=1 --"
assert db.execute('SELECT body FROM note WHERE owner=?',(attack,)).fetchall()==[]
assert db.execute('SELECT body FROM note WHERE owner=?','alice',)).fetchall()==(['private',])
db.close();print('Values remain values rather than becoming SQL instructions.')

```

Parameters do not make authorization automatic. The owner value must come from verified identity rather than a client-supplied claim.

## 12. Migrations, backups, replication and recovery

Use versioned Flyway/Liquibase migrations for Java or Alembic for Python. Expand first: add compatible columns/tables, deploy code able to read old and new shapes, backfill in bounded batches, validate, switch reads and only later contract. A rolling release may run old and new code together. A destructive migration can make application rollback impossible.

A backup is a saved copy; replication keeps another system updated and may faithfully copy accidental deletion. Neither substitutes for a tested restore. Define RPO, the acceptable data-loss window, and RTO, the acceptable recovery duration. PostgreSQL WAL archiving plus base backups can support point-in-time recovery when configured and retained correctly; a logical dump serves a different purpose.

Read replicas can lag. Immediately reading a recent write from a lagging replica can look like lost data. Route consistency-sensitive reads appropriately. Partitioning splits a table into managed pieces; sharding distributes responsibility across servers. Neither automatically fixes a poor partition key. Rebalancing, cross-shard queries and unique constraints become design issues. The existing distributed lab restored a logical PostgreSQL backup; it did not certify point-in-time disaster recovery.

## 13. NoSQL without the myths

A document can embed small data that is read and changed together, such as an address snapshot on an order. Reference shared or independently growing entities. Bound arrays and document size. MongoDB supports transactions; saying “NoSQL has no transactions” is false. Multi-document coordination has costs and does not replace modelling. [MongoDB modelling and consistency](#).

For distributed storage, define which consistency promise the API needs. Linearizability concerns operations appearing to occur at a point between invocation and response; serializability concerns equivalence to a serial transaction order. They are related but different. During a network partition, systems face tradeoffs between serving every reachable request and preserving particular consistency guarantees. “Pick any two” without describing the partition and operation is an incomplete CAP explanation.

For a wide-column store, model the partition key and allowed queries before inserting everything. For a graph, bound traversal depth and cardinality. For a warehouse, distinguish fact tables, dimensions, slowly changing dimensions and late-arriving events. OLTP optimizes many small business operations; OLAP optimizes analytical scans and aggregation.

## 14. Redis and cache management in the data architecture

A cache is a photocopy of selected facts. The primary database decides what is true unless the design explicitly assigns authority elsewhere. Choose cache-aside, read-through, write-through or write-behind based on failure and staleness requirements. Write-behind needs durable buffering and recovery because acknowledging only an in-memory cache write risks data loss.

Include tenant, resource, schema version and relevant permissions in cache keys. Use TTLs, jitter, bounded capacity, invalidation after commit, stampede protection and metrics. A delayed loader can refill stale data after invalidation; version checks or a carefully defined consistency policy address this. Notebook 25 and the real-time cache lab demonstrate the race. Redis eviction policy, persistence and failover settings must match the intended use; a cache configured for eviction should not silently become the only store for irreplaceable records.

## 15. Vector databases and RAG storage

An embedding is a list of numbers placing content in a learned geometry. Store document ID, chunk ID, tenant/ACL, source revision, embedding-model version and deletion state alongside it. Vectors from incompatible models should not be mixed simply because their dimensions match.

```
# lab: database_vector_tenant_filter
from math import sqrt
rows=[('a','alice',(1.,0.)),('b','bob',(1.,0.)),('c','alice',(0.,1.))]
def cosine(a,b):
    denominator=sqrt(sum(x*x for x in a))*sqrt(sum(x*x for x in b))
    return sum(x*y for x,y in zip(a,b))/denominator if denominator else 0.
ranked=sorted(((cosine(v,(1.,0.)),doc) for doc,owner,v in rows if owner=='alice'),reverse=True)
assert ranked[0]==(1.,'a') and all(doc!='b' for _,doc in ranked)
print(ranked)
```

This exact scan costs work proportional to candidates × dimensions. The helper's zero-vector return is a no-match fallback; mathematical cosine for a zero vector is undefined. Validate vector dimensions and validity at real storage boundaries. Approximate indexes such as HNSW and IVFFlat trade recall, memory, build time and query cost. PostgreSQL with pgvector can combine relational metadata and similarity search, but filtered approximate retrieval needs deliberate evaluation; index scans can produce too few permitted candidates. Compare with an exact filtered baseline and measure recall@k, latency and resource use. [pgvector reference](#).

A database match does not establish that an LLM's answer follows from the cited text. Maintain separate retrieval relevance, authorization and answer-factuality tests. Handle deletion in source storage, search/vector indexes, caches and backups under the defined retention policy. Treat retrieved document text as untrusted data, never as authorization instructions.

## 16. Idempotency, outbox and change data capture

A client can retry after the database committed but the response was lost. Store (owner, idempotency\_key, payload\_hash, result) with a unique constraint and the business change in one transaction. Same key and payload returns the original result; same key and different payload fails. Define expiration and prevent key reuse from unexpectedly recreating an old action.

The outbox stores an event in the same transaction as the business row. A publisher or CDC connector later sends it to Kafka. Delivery may repeat, so the consumer stores the event ID and business effect atomically in an inbox transaction. CDC reads changes from the database's change stream/log; it needs offsets, schema evolution, snapshot strategy and lag monitoring. It does not remove privacy filtering or downstream duplicate handling.

## 17. A complete interview design: study platform

Model User, Course, Enrolment, Lesson, Submission, Job, Outbox and DocumentChunk. Enrolment has a unique (user\_id, course\_id). Submission belongs to its authenticated owner. A submit command transaction creates Submission, Job and Outbox; a worker processes the event idempotently. PostgreSQL holds durable state. Redis optionally caches course metadata. Vector retrieval filters chunks by permission. SSE reports job progress with replay IDs. React renders explicit loading, stale, error and complete states.

The model call happens outside a long database transaction. Save its result using a conditional job-state/version update. If the worker crashes after an external effect, recovery uses the effect's idempotency contract or reconciliation. Do not claim that restoring an agent checkpoint proves exactly-once execution.

Use a schema diagram on paper, state transaction boundaries and demonstrate four failures: duplicate submission, cross-user read, worker crash and stale cache. Then explain an expand/contract migration, a

restored backup and a measured query plan. This is stronger interview evidence than listing database brands.

## 18. Timed exercises and answer key

Allow 45 minutes. Award two points for each answer: one for the mechanism and one for a concrete failure or test. A score describes this exercise only, not universal interview readiness.

1. Why does COUNT(\*) give one for a customer with no orders in a left join?
2. Design an index for owner-filtered newest-first pagination.
3. Two doctors each see another on duty, then both leave. What failed?
4. Why can payment still repeat after a database rollback?
5. Why does a read replica show an old balance immediately after a write?
6. When does embedding an address help and when does it hurt?
7. How do you reject a duplicate request with a changed payload?
8. Why can vector search return too few allowed documents?
9. How do you deploy a column rename while old code still runs?
10. What proves a backup works?

Answers: (1) the null-extended joined row is still a row; count the non-null order key. (2) a candidate is (owner, created\_at DESC, id DESC), tested against real plans and workload. (3) write skew violated the cross-row invariant; use a suitable serialization/locking design and bounded retries. (4) the external payment is outside the database transaction; use a provider-supported idempotency key and reconciliation. (5) asynchronous replication lag; use an appropriate read-after-write strategy. (6) immutable snapshots or small aggregate data fit; shared mutable and unbounded data may need references. (7) a unique scoped key plus stored request hash in the same atomic workflow. (8) approximate candidate selection and filtering interact; evaluate filtered recall and expansion strategies. (9) add compatible structure, dual-compatible code, backfill, validate, switch and later remove. (10) restore it into a separate environment and verify data, application behavior and recovery objectives.

Next practical assessment: implement an order reservation with a stock invariant and two concurrent buyers. Test with real PostgreSQL, not just SQLite. Explain the observed isolation behavior before presenting the result as production evidence.

## Deep workshop - Read a composite index as an ordered notebook

An orders query filters tenant and paid status, orders newest first, and returns 20 rows. An index (tenant\_id, status, created\_at, id) groups by tenant, then status, then time and ID.

### Connect filters to order

Equality on tenant and status narrows to a contiguous region. The engine may scan that region in the desired direction and stop after enough qualifying rows, depending on the query and index. ID breaks timestamp ties. Use both fields in the ordering and cursor.



## Keyset pagination

The previous page ends at time T and ID 90 in descending order. Next ask for time less than T, or equal to T with ID less than 90, under the same tenant/status filters. This continues from a value boundary. Large OFFSET pagination may scan past many rows and shift under concurrent inserts.

## Verify the plan

Inspect actual execution and row counts. A scan may be rational for a small table or unselective predicate. Stale statistics or unsuitable ordering can also affect the plan. Index existence proves neither usage nor improvement. Indexes consume storage and increase write work. Covering a query can reduce table lookups but makes the index larger. Test realistic distributions and enforce authorization in the query; retrieving another tenant's rows and filtering afterward is not an acceptable pagination design.



# 30 - RAG: connect the question to the right evidence

## 1. The open-book exam story

A language model is a student with learned knowledge. Retrieval gives the student relevant pages from an allowed library before answering. Generation turns that evidence into an answer. RAG means retrieval-augmented generation. It can make answers more grounded and current, but finding a page does not guarantee that the student interprets it correctly.

Our working example uses a tiny company handbook: Orion is maintained by the Payments team; Payments is led by Mira; another source explains checkpoints; another explains error E104. Alice also has a private document that Bob must never receive. You will follow these facts through six retrieval/control designs.

The offline lab displays exact retrieved evidence so every step is inspectable. That default is a retrieval and orchestration teaching mode, not an LLM generator. An optional Ollama adapter adds actual generation. Its outputs require a separate factual review. Earlier in this course, a real model produced an unsupported checkpoint claim despite citing a real source; that failure remains part of the evidence.

## 2. Connect every dot: two pipelines

Ingestion flow: source → parse → validate permissions/metadata → clean → chunk → attach provenance → embed/index → publish version. Query flow: identity → question → route/rewrite → retrieve permitted candidates → merge/rerank → select bounded context → generate → validate/review → display answer and sources → record evaluation signals.

| Stage    | Input → output                            | Responsibility                                     | Typical failure                               |
|----------|-------------------------------------------|----------------------------------------------------|-----------------------------------------------|
| Parse    | PDF/HTML/table → text and structure       | Preserve headings, tables and source locations     | OCR loses negation or a table column          |
| Chunk    | Structured document → passages            | Keep useful context and parent relationships       | Rule separated from its exception             |
| Index    | Passages/metadata → searchable records    | Version, ACL, deletion and embedding compatibility | Old private chunks remain searchable          |
| Retrieve | Question + verified identity → candidates | Find relevant permitted evidence                   | Relevant-looking unauthorized result          |
| Rerank   | Candidate set → ordered set               | Improve relevance using a stronger scorer          | Relevant evidence never entered candidate set |
| Context  | Ordered evidence → bounded prompt data    | Preserve provenance and token budget               | Truncation removes qualification              |
| Generate | Question + evidence → proposed answer     | Synthesize and abstain when needed                 | Unsupported inference with a valid citation   |
| Evaluate | Answer + sources + rubric → findings      | Separate retrieval and factual quality             | Citation format mistaken for entailment       |

The database stores source metadata, ownership and revisions. A vector index stores searchable representations. LangChain can provide retriever/model interfaces. LangGraph can represent state and conditional loops. MCP can expose an authorized retrieval tool across a protocol boundary. An agent harness enforces budgets, tool permissions, cancellation and logging. These components connect through contracts; none replaces the others' responsibilities.

### 3. Chunking, embeddings and similarity

Fixed-size chunks are easy to implement but may split meaning. Recursive splitting follows separators such as sections and paragraphs. Structure-aware splitting respects headings, code blocks and tables. Parent-child retrieval finds a small child passage and supplies its larger parent context. Overlap preserves boundary information but duplicates storage and can crowd results.

An embedding maps content to a numerical vector. Cosine similarity measures direction: dot product divided by both vector lengths. Similar direction can suggest semantic relation; it does not prove a factual relationship. Keep embedding model/version and dimension with the index. Re-embedding is an explicit migration, not merely changing an environment variable.

```
# lab: rag_cosine_geometry
from math import sqrt
def cosine(a,b):
    if len(a)!=len(b):raise ValueError('Incompatible dimensions')
    norm=sqrt(sum(x*x for x in a)*sum(x*x for x in b))
    return sum(x*y for x,y in zip(a,b))/norm if norm else 0.
assert cosine((1,0),(2,0))==1
assert cosine((1,0),(0,1))==0
assert cosine((1,0),(-1,0))==-1
print('Same direction: 1; perpendicular: 0; opposite: -1. These are geometry, not truth probabilities.')
```

Mathematical cosine is undefined for a zero vector. The helper deliberately returns zero as a no-match fallback; a production embedding pipeline should validate empty/invalid representations explicitly. The lab's transparent synonym/concept vectors are deliberately small and inspectable; they are not pretrained embeddings. Substitute a real encoder only with a compatible indexing/query path and measured retrieval evaluation.

### 4. Simple or naive RAG

Flow: question → one retrieval → top-k passages → answer. Example: “What does E104 mean?” retrieves the error-code note and answers that the payment request timed out, preserving the instruction to reuse the idempotency key.

This is a good baseline for document questions with nearby evidence and predictable latency. Its weakness is that one query and one fixed context selection can miss synonyms, cross-document dependencies or the right exception. Start with this baseline so later complexity has a measured reason to exist.

In `flow.json`, choose `"mode": "simple"`. The offline encoder uses a small explicit synonym mapping; inspect its code and try words it does not know. The failure teaches why a real embedding model and a held-out retrieval set matter.

### 5. Hybrid retrieval RAG

Hybrid retrieval combines lexical matching and semantic matching. Keyword search is strong for E104, invoice identifiers and exact technical names; semantic search can connect “reimbursement” with “refund.” Retrieve both lists, deduplicate, combine ranks and optionally rerank the merged set.

BM25 is a lexical scoring family using term frequency, document length normalization and inverse document frequency. Common words should not dominate merely because a long document repeats them. Its scores and cosine scores have different scales; simply adding them without calibration can be misleading. Reciprocal rank fusion combines positions instead.

```
# lab: rag_reciprocal_rank_fusion
from collections import Counter
def rrf(rankings,k=60):
    result=Counter()
    for ranking in rankings:
```

```

        for rank, doc in enumerate(ranking, 1): result[doc] += 1 / (k + rank)
    return sorted(result, key=lambda doc: (-result[doc], doc))
assert rrf([['exact-code', 'shared'], ['semantic', 'shared']])[0] == 'shared'
print(rrf([['exact-code', 'shared'], ['semantic', 'shared']]))

```

The shared candidate receives support from both lists. RRF's  $k$  is a rank smoothing constant, not the number of retrieved results. Hybrid retrieval is useful for enterprise manuals with both natural language and exact identifiers. In some framework documentation “hybrid RAG” also describes mixed deterministic/agent control. State which meaning you intend.

## 6. GraphRAG and relationship evidence

Question: “Who leads the team maintaining Orion?” A graph can connect Orion → maintained by → Payments → led by → Mira. Keep the supporting source ID for every edge. A guessed relationship must not quietly become trusted fact merely because it is stored in a graph.

Generic graph-augmented retrieval uses entities and relationships to find connected evidence. Microsoft's GraphRAG additionally builds a corpus-derived graph and community summaries; local search focuses on entities and nearby context, while global search can synthesize across community reports. Our tiny entity-expansion lab is not an implementation of that full community pipeline. [Microsoft GraphRAG query overview](#).

Use graphs for dependency questions, relationship exploration and corpus-level themes when the extraction/maintenance cost is justified. Bound traversal depth and candidate count. Common hub entities can explode the context. Apply permissions to nodes, edges, source passages and precomputed summaries; a permitted summary can otherwise leak restricted underlying facts.

Choose “mode”: “graph”. The lab finds seeds and expands shared entities through already-authorized documents. It logs every expansion. Its entities are hand-authored and it has no community detection or LLM entity extraction. These boundaries make the learning experiment honest and reproducible.

## 7. Multi-Hop RAG

Multi-hop retrieval resolves a question through multiple pieces of evidence. Hop one asks who maintains Orion and finds Payments. Hop two asks who leads Payments and finds Mira. The final answer must preserve both links; skipping the first link can produce the right name for the wrong reason.

Multi-hop can use a graph, but it can also use sequential text retrieval, SQL or APIs. A true dynamic planner can derive the next query from the previous result. Our user-defined version makes the subquestions explicit so you can see and edit the plan. It does not pretend to have autonomously discovered the second query.

```

{
  "mode": "multi_hop",
  "top_k": 2,
  "max_steps": 3,
  "context_chars": 1800,
  "generator": "extractive",
  "subquestions": [
    "Which team maintains Orion?",
    "Who leads the Payments team?"
  ]
}

```

Use it for comparisons and questions whose facts are distributed across records. Detect an unsupported hop and abstain rather than letting a guessed intermediate entity contaminate everything afterward. Measure evidence-chain completeness, not merely whether one final name appears in the answer.

## 8. Agentic RAG

An agentic system decides when and how to retrieve while solving a task. It may choose document search, authorized SQL, graph lookup or a calculator, inspect results and choose a next step. The harness supplies the allowed tools, input schemas, step/time/cost limits, observation handling and stopping rules.

Example: “Explain this failed payment and whether the refund policy applies.” A capable agent may inspect an authorized payment status, retrieve the policy, ask for missing facts and stop before claiming eligibility. Read-only retrieval and an actual refund mutation need different authorization. A retrieved document instructing “ignore permissions” is untrusted content.

The offline agentic route is a deterministic tool-policy demonstration: lexical search first; if empty, try the concept retriever within its budget. It exposes the loop structure, not autonomous LLM planning. To replace the policy with model decisions, validate a structured action against an allowlist, pass authenticated identity outside model control, persist state if needed and evaluate tool decisions separately from answer quality. The course's agent notebooks cover those harness concepts.

## 9. Adaptive RAG

Adaptive designs choose effort based on the question and observed evidence. A simple lookup may need one search; a relationship question may need multiple hops; missing evidence may require clarification or abstention. A route is a cost/quality decision, not a guarantee that hard questions become solvable.

The Adaptive-RAG research approach learns routing based on question complexity. The lab uses an explicit teaching heuristic: supplied subquestions select the multi-hop route; otherwise it selects hybrid retrieval. Do not describe this if-statement as a trained complexity classifier. [Adaptive-RAG paper](#).

Measure whether routing actually improves quality for the same latency/cost budget. A classifier can confidently choose the wrong route. Keep an escape condition for weak evidence, cap repeated searches and record why the route changed. An adaptive no-retrieval branch may suit some tasks, but this lab does not use it for source-dependent facts.

## 10. Related techniques and when to add them

| Technique              | How it changes the flow                             | Example use                                  | Risk to evaluate                                 |
|------------------------|-----------------------------------------------------|----------------------------------------------|--------------------------------------------------|
| Query rewriting        | Resolve or clarify the search question              | "Who leads it?" after discussing Payments    | Rewriter changes intended meaning                |
| Multi-query retrieval  | Search several formulations and merge               | Diverse descriptions of the same policy      | Cost and duplicate context                       |
| HyDE                   | Embed a generated hypothetical answer for retrieval | Vocabulary gap between query and documents   | Hypothetical text is not evidence                |
| Reranking              | Stronger scorer reorders a candidate pool           | Many superficially similar manual passages   | Cannot recover a never-retrieved document        |
| MMR                    | Trade relevance against redundancy                  | Diverse evidence in a small context          | Diversity can displace a crucial detail          |
| Parent-child retrieval | Search small chunks, supply larger parents          | A paragraph depends on its heading/exception | Larger context consumes budget                   |
| Contextual compression | Extract relevant spans before generation            | Long policy documents                        | Compression drops conditions or provenance       |
| Corrective retrieval   | Assess weak evidence and retrieve again             | Ambiguous or stale first results             | Unbounded loops or uncontrolled web fallback     |
| Self-reflective RAG    | Model critiques retrieval/generation                | Evidence-use evaluation during answering     | Self-assessment can be wrong                     |
| Conversational RAG     | Resolve conversation references and retrieve        | Follow-up questions                          | Memory leaks or old context overriding new facts |
| Multimodal RAG         | Retrieve images, tables or audio with provenance    | Manuals containing diagrams                  | OCR/visual interpretation and alignment errors   |
| SQL/structured RAG     | Query authorized structured facts                   | "Total completed jobs this week?"            | Unsafe generated SQL and ambiguous definitions   |

Self-RAG is a specific research approach involving learned retrieval/critique behavior; any prompt saying "check yourself" is not automatically that method. [Self-RAG paper](#). These patterns can combine: an adaptive router might choose hybrid retrieval followed by reranking and a second hop. Add each component only when its benefit is demonstrated on your workload.

## 11. User-defined flow: what you can change

Open `labs/rag-flow/flow.json`. Set mode to simple, hybrid, graph, multi\_hop, agentic or adaptive. Change `top_k`, `max_steps`, `context_chars` and subquestions. The program validates known modes and bounds; it does not evaluate arbitrary Python from configuration. Every run outputs an inspectable trace from authorization to context assembly.

The teaching context budget counts characters. A production model budget must use the selected model's tokenizer and reserve space for instructions, tool messages and the answer. The optional local-model fixture uses a 2,048-token context setting; increasing `context_chars` alone does not increase that model limit. Keep the small fixture budget or implement explicit tokenizer-aware packing before using larger documents.

Run from the library root:

```
python labs/rag-flow/rag_flow.py --config labs/rag-flow/flow.json --question "Who leads the team maintaining Orion?" --user alice
python labs/rag-flow/test_flow.py
```

The CLI's `--user` is a classroom fixture, not authentication. A web API must derive the user from a verified session. The example never lets a model choose the effective user's identity. To try a new corpus, edit the explicit DOCS fixture, preserve IDs/ownership/entities and extend tests before trusting results.

For actual generation, install/run Ollama separately with the documented local model and set generator to `ollama`. `run_live.py` attempts four generated answers for review. The initial outputs preserve a checkpoint answer needing correction. After clarifying the source and recovering from a recorded runtime timeout, all four revised answers passed the tutor's source-by-source review. This is a small development set, not a held-out accuracy estimate. Local generation requires model weights and sufficient memory; those weights are not in the ZIP. The offline extractive route needs only Python's standard library.

## 12. Mapping the flow to LangGraph and services

Model state as question, user scope, route, subquestions, evidence IDs, source versions, steps remaining, deadline, answer and evaluation status. Nodes can be `authorize`, `route`, `retrieve`, `rerank`, `assemble`, `generate` and `assess`. Conditional edges choose `retrieve-again`, `clarify`, `abstain` or `finish`. Make state merges deterministic when parallel searches return.

Parallel keyword/vector retrieval can reduce latency, but you must decide whether one failure permits partial results. Preserve cancellation and a total deadline; do not give each nested tool a fresh full budget. A checkpoint can resume computation, while external side effects still need idempotency. Store only necessary sensitive state and use an approved retention policy.

Java can remain the public authenticated API and job owner; Python runs the RAG pipeline. Kafka transports durable work notifications, PostgreSQL stores job/source state, Redis caches versioned safe projections, and SSE streams progress to React. MCP may expose retrieval to different agents. This is one possible architecture, not a requirement to install every tool for a small RAG application.

## 13. Retrieval and answer evaluation

Retrieval `recall@k` asks how much relevant evidence appears among `k` candidates. `Precision@k` asks how many returned candidates are relevant. Reciprocal rank rewards the position of the first relevant result. For a multi-hop question, include a metric requiring every needed hop. Evaluate with tenant filters and production-like document revisions.

```
# lab: rag_evidence_metrics
relevant={'d1','d2'};retrieved=['d1','d7','d3']
hits=len(relevant.intersection(retrieved))
precision=hits/len(retrieved);recall=hits/len(relevant)
assert precision==1/3 and recall==.5
chain_complete=relevant.issubset(retrieved)
assert not chain_complete
print({'precision':precision,'recall':recall,'complete_chain':chain_complete})
```

Answer evaluation separately asks whether material claims are supported, the question is answered, uncertainty is appropriate and citations identify the supporting passages. Include abstention, conflicting sources, document injection, deletion and ownership tests. The lab's passing assertions establish control/retrieval behavior on a tiny fixture; they do not establish general model accuracy.

## 14. Build your own flow exercise

Design a refund assistant. Supply three source documents: refund policy, purchase record and current processing status. Mark the purchase private. Choose simple retrieval for policy lookup, authorized structured retrieval for purchase facts and multi-hop or an agent for combining them. Require clarification if purchase eligibility cannot be determined. Keep issuing a refund outside the read-only answer flow unless explicitly authorized.

Write down each node's input/output, permitted tools, deadline, failure result and test. Then ask: what if the policy changed, the index lagged, a tool timed out, the user changed, a citation was valid but the claim false, or the connection dropped? Your flow is ready for the next integration stage when you can explain and test those outcomes.

## 15. Interview answer key

“Hybrid versus GraphRAG?” Hybrid combines retrieval signals; graph retrieval follows relationships or graph-derived summaries. They can coexist. “Multi-hop versus agentic?” Multi-hop describes an evidence dependency; agentic describes who controls tool choice and iteration. A fixed pipeline can be multi-hop. “Adaptive versus agentic?” Adaptation can be a fixed routing policy or learned choice; an agent is one possible control implementation. “Does RAG remove hallucinations?” No: retrieval, parsing, reasoning and attribution can all fail. “Do I need a vector database?” No: lexical, relational, graph or small in-memory retrieval can support RAG, depending on the workload.

Final timed task: in fifteen minutes draw the ingestion and query flows for your own use case, choose a baseline, name one upgrade, define two quality metrics and demonstrate one authorization failure test. Explain why your chosen complexity is justified.

## 16. Step from the toy encoder to actual pretrained embeddings

Imagine the toy encoder as a small handwritten synonym list. A pretrained encoder has learned numerical representations from many examples. The added `test_pretrained.py` uses the actual `all-MiniLM-L6-v2` model. It normalizes document and query vectors, multiplies them to obtain cosine scores and returns the highest-ranked documents. Authorization selects visible documents before encoding and ranking. The user cannot bypass that filter by writing “pretend I am Alice” in a question.

Twelve fresh paraphrase questions found their expected document within the first three results, and four adversarial ownership checks kept private documents out of the candidate set. These sixteen checks passed. They are newly authored fixtures separate from the original seven-document corpus, not an independently adjudicated research benchmark. The configurable flow remains dependency-free by default; selecting embedding as pretrained activates the actual encoder with separate dependencies. The integration test also exercises hybrid retrieval, graph expansion and multi-hop over the original corpus with this encoder.

One deliberately unsupported question asks for Neptune's exact population. Vector search still returns neighbours. This is why “retrieved something” cannot mean “answerable.” Add evidence sufficiency, abstention and claim-level evaluation after retrieval. Thresholds require calibration for the actual corpus and model; do not invent a universal cosine cutoff.

## 17. Factual tests and adversarial tests are different from retrieval tests

The factual holdout script supplies eight fixed evidence/question pairs to an actual local generative model. It tests a number, an entity, two-hop reasoning, absent evidence, a false premise, injected instructions, contradictory sources and unauthorized information. The expected short answers are written before the run, and each result is saved immediately. Review `factual-holdout-report.json` for the current results; a timeout is an incomplete case, not a factual pass.

A strict answer contract can reject a semantically correct variant: “Search team” and “Search” can name the same team. Keep the original score and add an explicit human review explaining this distinction. Do not rewrite expected answers secretly after seeing outputs. Conversely, valid JSON and a matching citation do not rescue a false claim. A broader system needs separate retrieval, factuality, authorization, tool-use and operational



evaluations.

For commands and prerequisites, use [Reliability extensions](#). First explain which stage each test isolates. Then connect them into an end-to-end experiment with a versioned corpus, permissions, retrieval trace, model output and human-reviewed answer key. Passing isolated stages does not automatically establish the combined system's quality.

## 18. Let the model choose a tool without giving it authority

Imagine a student choosing which shelf to search. The student may choose the shelf; the librarian still decides which rooms the student can enter. In the optional Ollama controller, the model returns a JSON search or finish decision. Application code validates the fields, rejects unknown actions, limits query length, keeps the existing authorized document set and enforces a maximum number of decisions. A model cannot change the user or request a shell command.

Set mode to agentic, embedding to pretrained and controller to ollama, or use model-agent-flow.json. Every model decision and actual retrieval appears in the trace. Invalid JSON or forbidden fields fail the run visibly. A step budget stops the loop even if the model keeps searching. The default output is still exact evidence display; model-selected retrieval and generative answering are independent choices. This is a working bounded read-only controller, not a full production Deep Agent with durable tool execution, approvals and shared memory.

The graph route still expands curated entity links. An actual pretrained encoder and real model controller do not magically turn it into an automatic GraphRAG community-summary pipeline. That pipeline needs extraction, entity resolution, edge provenance, community construction, refresh/delete behavior and additional evaluation. Preserve this distinction when explaining your architecture in an interview.

## 19. Build and maintain a graph from actual model extraction

The separate graph\_pipeline.py exercise connects those stages. Three source cards say “Atlas is maintained by Search,” “Search is led by Noor,” and the private statement “Vault is led by Sera.” An actual local model extracts source, relationship, target and an exact supporting quotation. The application supplies document identity and permissions; the model does not decide who may read a document.

Imagine drawing arrows between name cards. Atlas points to Search through maintained\_by, and Search points to Noor through led\_by. Following two arrows answers who leads the team maintaining Atlas. The test checks these exact extracted relationships before accepting the example. In a larger system, extraction needs a labelled evaluation set and review; a quotation containing both names does not by itself prove that an arrow's direction or meaning is correct.

Names are normalized with Unicode normalization, case folding and whitespace handling. Thus ATLAS finds Atlas. This is exact-name resolution, not a solution to two different people sharing a name or one organization having several aliases. Real entity resolution also needs types, stable identifiers, tenant boundaries and ambiguity handling.

SQLite stores documents, source hashes, relationships and supporting quotations. Updating a document replaces its old arrows in one transaction. Deleting it cascades to its arrows. A failed provenance check leaves the earlier valid document intact. Reopening the database preserves the graph; it is no longer just a dictionary rebuilt from hand-authored links.

For each reader, permissions filter edges before traversal or community detection. NetworkX then runs seeded Louvain community detection. Think of it as grouping name cards that are more tightly connected, guided by a modularity score. Louvain is a heuristic; its grouping is not a semantic truth or a guarantee of the best possible partition. The lesson uses extractive community summaries assembled from authorized supporting quotations,



with document IDs attached. Bob's summary must never contain Alice's private card, including after a permission change.

Run `python labs/rag-flow/test_graph_pipeline.py` after the requirements and Ollama setup in the lab guide. The recorded execution passed actual extraction, database reopen, two-hop retrieval, permission-filtered communities, rejection of invented support, permission change, deletion and source replacement. This is a small custom graph-RAG pipeline, not a claim to reproduce Microsoft GraphRAG or certify extraction accuracy on arbitrary documents.

Implementation reference: [NetworkX Louvain documentation](#). The explanations and runnable example are included here; the link is optional background.

## 20. Diagnose a timeout with measurements

An unanswered request can reflect slow loading or generation rather than a wrong answer. The bounded-generation test records available memory, whether the model is on CPU, time to first content, loading duration, prompt processing and output tokens. The machine had about 96 percent memory usage during this investigation. The earlier injection case timed out twice; a later constrained run returned the correct answer 7443 in 34.6 seconds using a short output budget, smaller context, two CPU threads and a JSON answer schema.

Several settings changed together, so this is a successful development retest, not proof that one setting alone caused the improvement. Never shrink context without checking whether all needed evidence still fits. Never treat a short answer as automatically correct. Keep earlier failures, record the new settings and repeat representative cases before choosing production limits. The baseline holdout score remains unchanged.

Run `python labs/rag-flow/test_bounded_generation.py` to reproduce this measured case. [Ollama's chat API](#) documents the response timing fields and schema format used by the exercise.

The subsequent eight-case development regression passed six cases. It returned UNKNOWN for a supported entity question and timed out on an absent-evidence question. Therefore the one repaired injection example does not close general answer quality. Both reports remain included.

## 21. Keep evidence available when generation fails

Imagine the librarian finds the right pages, but the storyteller's microphone stops working. We can still show the pages, clearly saying that no spoken explanation was produced. The configurable RAG lab now does this when generation times out, the model service is unreachable or its response is malformed. Its answer starts with "Generation unavailable; showing source excerpts only." The trace records the error category and the actual generator as extractive.

Set `generation_timeout_seconds` in `flow.json` to a number from 1 to 240; the default is 120. A timeout is a waiting limit, not proof of cancellation inside the model server. No permitted evidence means the model is not called at all. Permissions are applied before both generation and fallback. A successful generated answer still carries a factual-review warning; successful HTTP delivery does not establish truth.

Run `python labs/rag-flow/test_generation_recovery.py` for injected timeout, unavailable-service, malformed-output, permission and configuration checks. These tests deliberately simulate transport failures. They verify recovery behavior, not model intelligence. Run `test_flow.py` afterward to exercise all six offline routes.

## Deep workshop - Choose RAG by the evidence the question needs

Ingestion converts source documents into chunks, metadata, and searchable representations. Query time retrieves authorized evidence, optionally reranks, constructs context, and evaluates the answer. Generation cannot repair evidence that retrieval never supplied.

### Trace six routes on one question

“Who leads the team owning Atlas?” A says Atlas belongs to Search. B says Noor leads Search. Simple retrieval may find both. Hybrid retrieval combines lexical matching for Atlas with semantic matching for ownership and leadership. A graph route follows Atlas → owned\_by → Search → led\_by → Noor, with provenance and permission checks for every supporting edge. Multi-hop retrieval first finds Search, then asks for its leader. Agentic retrieval lets a bounded decision-maker choose the second search. Adaptive retrieval selects routes using evidence sufficiency or explicit rules. Labels overlap: an agent can perform multi-hop hybrid searches against a graph-backed system.

### Inspect the intermediate artifacts

Record retrieved IDs, source versions, permissions, reranking, and exact supplied context. An unauthorized document cannot become permission to reveal its contents through an inferred graph path. For cosine similarity, divide dot product by vector lengths. Query [1,0] and passage [0.8,0.6] have similarity 0.8; both norms are 1. That is a ranking signal, not a calibrated truth probability.

### Evaluate each connection

Measure evidence recall, access filtering, claim support, and abstention separately. Conflicting authoritative sources without a resolution policy call for an explanation of conflict, not selection of whichever arrived first.

# Notebook 31 - Java, Spring Batch and microservices interview masterclass

This notebook answers every question in your supplied interview list. Start with the story, explain the mechanism, run the example, then attempt the follow-up without looking. The examples describe a payment-and-settlement learning system; they are not claims about your employment history. Java notebook cells need JDK 21 or later and Python. The separate Maven workshop exercises actual Spring transactions, Spring Batch, JUnit and Mockito.

## 1. Tell me about yourself and describe your experience

Think of an introduction as the label on a toolbox: tell the interviewer what problems you can solve, then show one useful tool. Aim for 60-90 seconds: current background, relevant skills, one concrete project, your contribution, one measured outcome, and why this role fits.

Template: "I am [your actual role/background]. I focus on [two relevant skills]. In [real project or explicitly named learning lab], I implemented [specific feature]. A difficult issue was [failure]; I diagnosed it using [evidence] and changed [implementation]. We verified [actual result]. I am now looking for [relevant responsibility]." Replace every bracket with something you can defend. Say "local learning project" when that is what you did.

For "Which databases have you worked on?", separate production work, personal projects and theoretical knowledge. Example structure: "I used [engine/version] for [workload], wrote [queries/migrations], investigated [plan/locking issue], and verified [result]. I have studied Oracle plan analysis but have not operated Oracle in production." Do not say you ran MySQL or Oracle because you ran the SQLite exercise below.

For "Have you implemented multithreading?", describe the task split, shared state, synchronization, executor lifecycle, backpressure, cancellation and failure handling. For cloud experience, name only services you actually used and explain identity, networking, state, monitoring and costs. The library's local K3s deployment is Kubernetes experience in a local lab, not AWS/Azure/GCP production experience.

Follow-up drill: explain one failure you personally reproduced. If your story has no inputs, measurements or tradeoff, make it more concrete before memorizing it.

## 2. HashMap: numbered drawers with labelled cards

Imagine drawers containing customer cards. A hash chooses a drawer; equals identifies the correct card inside it. A collision means different keys choose the same drawer, not that one customer automatically overwrites another.

HashMap stores an array of buckets. In OpenJDK 21, buckets contain linked nodes or, for sufficiently crowded eligible buckets, red-black tree nodes. The implementation spreads the hash and uses the table size to select a bucket. For insertion it checks existing keys: an equal key replaces its value; an unequal colliding key gets a separate entry. Growth redistributes entries. Treeification constants and the minimum-capacity gate are implementation details, not the Map interface contract. Avoid the misleading claim "the eighth collision always creates a tree." Tree bins also do not guarantee logarithmic lookup for every pathological non-orderable equal-hash key set. [OpenJDK implementation](#).

Expected get/put cost is constant with suitable hash distribution; resizing and poor distribution affect actual cost. Default load factor is 0.75. HashMap allows a null key and null values, does not promise ordering, and is not thread-safe. A fail-fast iterator is a bug detector, not synchronization. [Java HashMap contract](#).

**Interview answer:** “An array of buckets locates candidate entries by hash; equality selects the key. Modern OpenJDK can use tree bins for crowded buckets. Colliding but unequal keys coexist.”

```
// lab: CollisionCards
import java.util.*;
public class CollisionCards {
    record Key(int id) { @Override public int hashCode() { return 7; } }
    public static void main(String[] args) {
        Map<Key,String> cards = new HashMap<>();
        cards.put(new Key(1), "Alice");
        cards.put(new Key(2), "Bob");
        assert cards.size() == 2;
        assert cards.get(new Key(1)).equals("Alice");
        cards.put(new Key(1), "Alicia");
        assert cards.size() == 2;
        assert cards.get(new Key(1)).equals("Alicia");
        System.out.println("Unequal colliding keys coexist; equal keys replace values.");
    }
}
```

Predict the size before running: two. The record supplies equality based on id; the deliberately constant hash slows lookup but does not break the equality contract. An interview follow-up may ask why HashMap is a poor choice when sorted keys or concurrent mutation are required: use a suitable ordered or concurrent map, not a promise the original map never made.

### 3. equals and hashCode: two parts of one address system

equals describes logical equality. hashCode helps hash-based collections find candidates. Equal objects must have equal hash codes; equal hash codes do not prove equality. Equality should be reflexive, symmetric, transitive, consistent and false for null. Override both methods from the same stable fields, or use a suitable immutable record.

If you override equals but inherit Object.hashCode, two logically equal instances may land in different buckets. HashMap lookup can fail or contain apparently duplicate logical keys. It is not guaranteed to fail on every run: accidental equal hashes may mask the defect. A test that happens to pass is not proof of a valid contract.

Mutable keys are another trap. Put a key into a map, change the field used for its hash, then look it up: the key's new address differs from its stored bucket. Prefer immutable identifiers, such as an account ID, over a mutable account balance. Do not use approximate floating-point comparison as equality for hash keys without a consistent canonical representation.

Exercise: two employees share a name but have different employee IDs. Should equals compare names? Usually not: a display label is not identity. State the domain rule before generating methods in the IDE. [Object equality/hash contract](#).

### 4. Abstract classes, main and exception cleanup

An abstract class is a partly built machine. It can hold fields, constructors and implemented methods, and can require subclasses to implement abstract methods. You cannot instantiate it directly. It can have a static main method: launching that method does not require constructing the abstract class.

finally is not compulsory. A regular try needs a catch or finally; try-with-resources can appear without either. finally normally executes as control leaves the try/catch, including return or throw, but not after every possible JVM/process termination. Do not return from finally: it can hide an earlier result or exception.

Try-with-resources closes AutoCloseable resources in reverse declaration order. If the body and close both fail, the body's exception remains primary and the close failure is suppressed. This is useful for files, database connections and streams. It does not automatically make business work transactional. [Java resource handling](#).

```
// lab: CleanupStory
public abstract class CleanupStory {
    abstract String describe();
    static class Resource implements AutoCloseable {
        public void close() { throw new IllegalStateException("close failure"); }
    }
    public static void main(String[] args) {
        try (var resource = new Resource()) {
            throw new IllegalArgumentException("body failure");
        } catch (IllegalArgumentException error) {
            assert error.getMessage().equals("body failure");
            assert error.getSuppressed().length == 1;
            System.out.println("Abstract main ran; body failure retained; close failure suppressed.");
        }
    }
}
```

Follow-up: “Why not catch Exception and return null everywhere?” Because callers cannot distinguish missing data from failed work; observability and recovery become unreliable. Catch where you can recover or translate meaningfully, and preserve the cause.

## 5. Java 8 functional interfaces and employee filtering

A functional interface describes one job that a lambda can perform. It has one abstract method contract; compatible inherited methods can represent that same contract. Public Object methods do not count. Default and static methods do not add abstract jobs. `@FunctionalInterface` asks the compiler to enforce the rule, but the annotation is not required. [FunctionalInterface documentation](#).

| Interface                            | Small story                  | Signature shape                | Example                     |
|--------------------------------------|------------------------------|--------------------------------|-----------------------------|
| <code>Predicate&lt;T&gt;</code>      | Gatekeeper says yes/no       | $T \rightarrow \text{boolean}$ | Employee older than 40      |
| <code>Function&lt;T,R&gt;</code>     | Converter                    | $T \rightarrow R$              | Employee $\rightarrow$ name |
| <code>Consumer&lt;T&gt;</code>       | Uses an item                 | $T \rightarrow \text{void}$    | Write a record              |
| <code>Supplier&lt;T&gt;</code>       | Factory                      | $() \rightarrow T$             | Create a request ID         |
| <code>UnaryOperator&lt;T&gt;</code>  | Same-type transformation     | $T \rightarrow T$              | Normalize a string          |
| <code>BinaryOperator&lt;T&gt;</code> | Combine two same-type inputs | $(T,T) \rightarrow T$          | Combine totals              |

```
// lab: EmployeePredicates
import java.util.*;
import java.util.function.*;

public class EmployeePredicates {
    record Employee(String name, int age) {}
    interface Named { String name(); } // Functional even without the annotation.
    public static void main(String[] args) {
        var people = List.of(new Employee("Asha",40), new Employee("Ravi",41));
        Predicate<Employee> olderThan40 = employee -> employee.age() > 40;
        var names = people.stream().filter(olderThan40).map(Employee::name).toList();
        assert names.equals(List.of("Ravi"));
        Named label = () -> "approved";
        assert label.name().equals("approved");
        System.out.println(names);
    }
}
```

The requested operation uses `Predicate`, not `Consumer`. Streams describe a pipeline; they are not automatically faster than loops. Intermediate operations are lazy until a terminal operation runs. Avoid side effects in parallel pipelines, and do not assume `parallelStream` gives the right thread-pool isolation for blocking service calls.

## 6. synchronized versus ReentrantLock

A lock is a bathroom key: one owner enters a critical section at a time. `synchronized` acquires the monitor associated with an object or class and releases it when the block exits, including exceptional exit. It is reentrant: the owning thread can enter again.

`ReentrantLock` makes ownership explicit. Choose it when you need timed `tryLock`, interruptible acquisition or multiple `Condition` queues. Always release after successful acquisition in `finally`. Fair mode can reduce barging but may reduce throughput and does not guarantee fair CPU scheduling; untimed `tryLock` has its own fairness caveat. Do not choose it because of an unconditional “faster” claim. [ReentrantLock contract](#).

```
// lab: SafeCounter
import java.util.concurrent.*;
import java.util.concurrent.locks.*;
public class SafeCounter {
    static class Counter {
        private final ReentrantLock lock = new ReentrantLock();
        private int value;
        void increment() {
            lock.lock();
            try { value++; } finally { lock.unlock(); }
        }
        int value() {
            lock.lock();
            try { return value; } finally { lock.unlock(); }
        }
    }
    public static void main(String[] args) throws Exception {
        var counter = new Counter();
        var pool = Executors.newFixedThreadPool(4);
        try {
            var futures = new java.util.ArrayList<Future<>>>();
            for (int i=0;i<4;i++) futures.add(pool.submit(() -> {
                for (int j=0;j<1000;j++) counter.increment();
            }));
            for (var future : futures) future.get();
            assert counter.value() == 4000;
            System.out.println("4000 increments retained; task failures propagate through Future.get.");
        } finally { pool.shutdownNow(); }
    }
}
```

In a payment API, do not hold a JVM lock during a slow bank call. It increases contention and does not coordinate other application instances. Use a durable transaction/idempotency protocol for cross-instance correctness. A local counter test proves local synchronization, not distributed serialization.

## 7. ConcurrentHashMap, synchronizedMap, races and deadlocks

`Collections.synchronizedMap` wraps operations with a common monitor. Iteration must synchronize on the returned map, and multi-call read-modify-write logic needs one shared critical section. Modern `ConcurrentHashMap` supports concurrent access without a single whole-map lock for ordinary operations; retrievals generally do not block. It rejects null keys/values and has weakly consistent iteration. Use atomic operations such as `compute`, `merge` or `putIfAbsent` for single-key compound updates. It is not a multi-key transaction manager. Do not describe modern `ConcurrentHashMap` as necessarily using the old fixed-segment architecture. [ConcurrentHashMap contract](#).

Race example: two threads read a stock count of one, both approve a purchase and both write zero. The final number looks plausible, but two items were sold. A thread-safe map does not make `get` followed by `put` atomic. In a database, an update like `UPDATE stock SET available=available-1 WHERE id=? AND available>0`, plus checking affected rows inside the right transaction, can enforce the invariant.

Deadlock example: transfer A→B locks account A then waits for B; transfer B→A locks B then waits for A. Neither can advance. Acquire account locks in a stable ID order, reduce lock duration, inspect thread dumps or database deadlock reports, and retry a rolled-back database transaction only under a bounded policy. A timeout limits waiting; it does not repair inconsistent partial work.

A race is incorrect behavior caused by ordering; a deadlock is a cycle of waiting. They are different failure modes. volatile gives visibility and ordering for a variable but does not make balance++ atomic. CompletableFuture introduces asynchronous composition, not automatic cancellation of every underlying task. Bound executors/queues and measure saturation.

## 8. SOLID through one payment example

Imagine a school office. A cashier calculates fees, a ledger records them and a messenger sends receipts. Giving every duty to one person makes every policy change disturb everything else.

| Principle             | Concrete design                                               | What would violate it?                                        |
|-----------------------|---------------------------------------------------------------|---------------------------------------------------------------|
| Single Responsibility | FeePolicy calculates; Ledger persists; ReceiptSender delivers | One class changes for fee rules, SQL schema and email layout  |
| Open/Closed           | Add a FeePolicy implementation behind a stable interface      | Edit a huge switch for every new fee scheme                   |
| Liskov Substitution   | Every Ledger implementation honors documented save semantics  | A subtype silently drops accepted entries                     |
| Interface Segregation | ReadOnlyLedger exposes reads without mutation                 | A report generator must depend on refund/delete operations    |
| Dependency Inversion  | PaymentService depends on Ledger and FeePolicy abstractions   | Business logic constructs a concrete JDBC connection directly |

SRP means one coherent reason to change, not “one method per class.” A useful refactoring starts with tests of existing behavior, extracts fee calculation, injects a ledger interface, and keeps transaction coordination explicit in the application service. Do not scatter one business invariant across five classes merely to increase class count.

Composition means a service **has** a policy. Inheritance means a subtype **is usable as** the parent under its behavioral contract. Choose composition for independently changing behavior, such as fee policy plus notification channel. Use inheritance for a real substitutable hierarchy or a framework's intended extension point. A Square that forbids Rectangle's independent width/height updates breaks that mutable Rectangle contract; mathematical naming alone does not establish substitutability.

```
// lab: ComposedFees
import java.math.BigDecimal;
public class ComposedFees {
    interface FeePolicy { BigDecimal fee(BigDecimal amount); }
    record Checkout(FeePolicy policy) {
        Checkout { java.util.Objects.requireNonNull(policy); }
        BigDecimal total(BigDecimal amount) {
            if (amount.signum() < 0) throw new IllegalArgumentException("negative amount");
            return amount.add(policy.fee(amount));
        }
    }
    public static void main(String[] args) {
        var free = new Checkout(amount -> BigDecimal.ZERO);
        var flat = new Checkout(amount -> new BigDecimal("2.00"));
        assert free.total(new BigDecimal("10.00")).compareTo(new BigDecimal("10")) == 0;
        assert flat.total(new BigDecimal("10.00")).compareTo(new BigDecimal("12")) == 0;
        System.out.println("Fee behavior changes through composition; money uses decimal values.");
    }
}
```



## 9. Design patterns: choose a problem before a pattern

SOLID gives design principles; patterns give recurring solution structures. Neither is a requirement to add abstractions to every class. Notebook 13 contains executable implementations of all 23 classic GoF patterns; use this map to connect them to interview scenarios.

| Family     | Patterns                                                                                                                        | How to recognize the need                                                                                                                                                             |
|------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Creational | Factory Method, Abstract Factory, Builder, Prototype, Singleton                                                                 | Construction varies, compatible families matter, configuration is complex, copying is useful, or one lifecycle-owned instance is required                                             |
| Structural | Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy                                                                 | Translate an API, separate independent variations, model trees, wrap behavior, simplify a subsystem, share immutable state, or mediate access                                         |
| Behavioral | Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor | Route handlers, represent actions, evaluate a grammar, traverse, coordinate, capture state, notify, vary state/algorithm, share a skeleton, or add operations over a stable structure |

For the payment example: Strategy selects a fee algorithm; Adapter translates a bank API; Decorator adds metrics around a gateway; Factory creates the configured adapter; Command represents a durable payment request; State models authorized/captured/refunded transitions; Observer suggests notifications but needs durable messaging when loss is unacceptable. A Spring proxy can implement cross-cutting transaction interception. Spring singleton scope is one bean per relevant container definition, not one object across every JVM or a distributed lock.

When patterns make design worse: a Factory for one trivial constructor obscures code; a global Singleton hides dependencies and test state; Observer chains hide ordering and errors; Template Method inheritance couples subclasses to parent sequencing; a “retry Decorator” around a non-idempotent debit can duplicate money movement. State the alternative and the failure behavior, not just a pattern name.

Follow-up: Strategy versus State? Strategy selects an algorithm; State changes behavior according to lifecycle transitions. They may have similar class shapes but express different domain intent. Adapter versus Facade? Adapter makes one interface usable as another; Facade offers a simpler entry point to a subsystem.

## 10. Constructor injection and the Spring transaction proxy

Constructor injection is a visible shopping list: a class cannot be created without its required dependencies. It supports final fields and ordinary unit-test construction. Field injection hides requirements and encourages reflection-based setup. An optional dependency should be deliberately optional; avoid treating every missing dependency as acceptable. Circular constructor dependencies often reveal unclear ownership rather than a need to hide the cycle.

With normal Spring proxy-based transaction management, a caller invokes a managed proxy. The interceptor obtains or joins a transaction, calls the target, then commits or rolls back according to the rules. Default propagation is REQUIRED; unchecked exceptions normally trigger rollback, while checked exceptions need an appropriate rollback rule unless configuration changes that behavior. Catching an exception and returning normally can prevent the intended rollback. [Spring transactional annotations](#).

Self-invocation bypasses that proxy boundary: `this.debit()` is an ordinary call on the target. Its `@Transactional` annotation does not start a new transaction in standard proxy mode. If an outer transaction already exists, the inner code may still participate in it, but its own `REQUIRES_NEW` or rollback settings are not newly intercepted. Move the operation to an injected collaborating bean or use `TransactionTemplate` for an explicit boundary. AspectJ weaving is a different model, not the default explanation.



The accompanying Maven test proves the distinction with H2: a direct call through the proxy rolls back an insert when the method throws; a nontransactional same-class caller reaches the method without interception and leaves its autocommitted insert. This is a controlled demonstration, not a recommendation to rely on autocommit in a money workflow.

Important follow-up: a local JDBC transaction does not atomically roll back a remote bank's HTTP request. That requires an explicit protocol such as idempotency, durable state transitions and reconciliation. Also, transaction context is not automatically carried into arbitrary new threads.

## 11. REST operations and idempotency in payment systems

Think of a request as handing in a numbered form. A retry should refer to the same form, not create another debit.

| Method     | Intended use                               | Retry contract                                                          |
|------------|--------------------------------------------|-------------------------------------------------------------------------|
| GET / HEAD | Read representation / headers              | Safe and idempotent semantics; never use a read to trigger a debit      |
| PUT        | Replace the resource at a known URI        | Idempotent intended effect; responses and incidental logs can differ    |
| DELETE     | Remove the addressed resource              | Idempotent intended effect; a later response can be 404                 |
| POST       | Create or execute application-defined work | Not inherently idempotent; implement a durable key contract when needed |
| PATCH      | Apply a partial change                     | Depends on the patch: "set status" differs from "increment balance"     |

The HTTP terms describe intended method effects, not identical response bytes on every call. [HTTP semantics](#).

For POST /payments, scope the idempotency key by authenticated principal/tenant and operation. Store a canonical request fingerprint, state and outcome under a unique constraint. Same key and same payload return the same logical operation; same key with a different amount is a conflict. Decide how long keys remain valid and what happens after retention. Do not use a process-local map as the only protection.

Within one database, transactionally reserve the key, apply the ledger mutation and write an outbox event. Concurrent duplicates must converge through the constraint and a defined pending/completed response. For an external bank, durably create the intent first, use the bank's supported idempotency contract, and reconcile ambiguous timeouts by operation ID. A crash after the remote charge but before recording its result is the critical gap. Never interpret a timeout as proof that the charge did not occur.

Use integer minor units when the currency model permits it, or decimal arithmetic with explicit currency/rounding rules. Authentication, amount validation, currency, account ownership, available funds, audit trail and reversals are separate concerns. This is a software design exercise, not a complete financial-control specification.

```
# lab: DurablePaymentKeys
import sqlite3
db = sqlite3.connect(':memory:')
db.executescript('CREATE TABLE payments(tenant TEXT, request_key TEXT, cents INTEGER, PRIMARY
KEY(tenant,request_key));')
def submit(tenant, key, cents):
    if type(cents) is not int or cents <= 0:
        raise ValueError('positive integer minor units required')
    with db:
        db.execute('INSERT OR IGNORE INTO payments VALUES(?,?,?)',(tenant,key,cents))
        stored=db.execute('SELECT cents FROM payments WHERE tenant=? AND
request_key=?',(tenant,key)).fetchone()[0]
        if stored != cents:
            raise ValueError('idempotency payload conflict')
    return stored
assert submit('a','request-1',1250)==submit('a','request-1',1250)
try:
```

```

    submit('a','request-1',999)
except ValueError:
    pass
else:
    raise AssertionError('conflict accepted')
assert submit('b','request-1',999)==999
assert db.execute('SELECT COUNT(*) FROM payments').fetchone()[0]==2
db.close()
print('Tenant-scoped key replay and payload conflict verified in SQLite; no remote charge was made.')

```

This deliberately small cell validates key semantics, not simultaneous requests or an external bank. The existing Study Coach and real-time labs exercise concurrent duplicate requests and crash recovery separately.

## 12. Millions of records: pagination, lazy loading and exports

Do not send millions of database rows in one JSON response. Bound page size, select only required columns, authorize every query, impose timeouts and avoid expensive exact total counts unless needed. Streaming a huge response can reduce application buffering, but it still holds connections and consumes bandwidth; it is not automatically a good interactive API.

Offset pagination is simple: `ORDER BY created_at, id LIMIT 100 OFFSET 500000`. Deep offsets may scan/discard many rows; concurrent inserts/deletes can shift page boundaries. Keyset pagination uses the last seen ordered values as a bookmark. For ascending order, select rows after `(last_created_at, last_id)`. Always include a unique tie-breaker and a supporting index.

```

-- MySQL-style parameterized query; use JDBC bind parameters, not string interpolation.
SELECT id, created_at, amount_minor
FROM payments
WHERE tenant_id = ?
      AND (created_at > ? OR (created_at = ? AND id > ?))
ORDER BY created_at, id
LIMIT 101;

```

Return 100 and use the extra row to decide whether another page exists. An opaque cursor can encode the last tuple, filter/version and an expiry, signed to resist tampering. A signed cursor is not authorization: reapply tenant checks on every request. Changes to sort keys can still move rows across the bookmark. Stable exports may need an explicit snapshot/version policy, not merely a high-water ID.

```

# lab: BookmarkPages
import sqlite3
db=sqlite3.connect(':memory:')
db.executescript('CREATE TABLE rows(id INTEGER PRIMARY KEY, created INTEGER); CREATE INDEX ix_created_id ON rows(created,id);')
db.executemany('INSERT INTO rows VALUES(?,?)',[(1,10),(2,10),(3,11),(4,11),(5,12)])
cursor=(-1,-1); seen=[]
while True:
    page=db.execute('SELECT created,id FROM rows WHERE created>? OR (created=? AND id>?) ORDER BY created,id LIMIT 2',(cursor[0],cursor[0],cursor[1])).fetchall()
    if not page: break
    seen.extend(row[1] for row in page); cursor=page[-1]
assert seen==[1,2,3,4,5]
db.close()
print('Bookmark includes a unique tie-breaker: every static-data row appears once.')

```

Browser lazy loading means fetching the next page when needed; ORM lazy loading means deferring a relationship query. They are different. Mapping each returned order to `order.customer.name` can cause  $N+1$  database queries. Use suitable DTO projections, fetch strategies or batch fetching, then verify query counts. Be cautious combining collection fetch joins and pagination. For very large downloads, prefer an authorized asynchronous export job, durable status and an expiring download link, with retry and cleanup policies.

## 13. MySQL composite indexes: a sorted address book

An address book sorted by country, city, then name helps you find a country's entries or one city's names. It is less useful when you know only a name. A composite B-tree index similarly orders tuples. MySQL commonly uses leftmost prefixes: (tenant\_id,status,created\_at,id) supports relevant searches beginning with tenant\_id, but is not equivalent to four independent indexes. Special optimizer paths can exist, so verify the chosen plan rather than saying later columns can never matter. [MySQL multiple-column indexes](#).

Start with a real query and its frequency. Put useful equality constraints before the relevant range/order portion when that fits the workload. For tenant\_id=? AND status=? AND created\_at>=? ORDER BY created\_at,id, the tuple above is a candidate. The first range may limit later columns' use as range bounds; later columns can still help filtering, ordering in applicable cases or covering. Column order is not simply "highest cardinality first."

Indexes cost storage and write/maintenance work. A covering index includes needed values, but adding large payload columns can make it too expensive. InnoDB secondary indexes also carry the primary key. Keep statistics representative and consider data skew. A scan can be cheaper when the query needs much of the table. Do not force an index merely to make EXPLAIN look impressive.

MySQL diagnostic recipe, not executed against a MySQL server in this notebook:

```
CREATE INDEX ix_payment_lookup ON payments(tenant_id,status,created_at,id);
EXPLAIN SELECT id,created_at FROM payments
WHERE tenant_id=42 AND status='SETTLED' AND created_at >= '2026-01-01'
ORDER BY created_at,id LIMIT 100;
EXPLAIN ANALYZE SELECT id,created_at FROM payments
WHERE tenant_id=42 AND status='SETTLED' AND created_at >= '2026-01-01'
ORDER BY created_at,id LIMIT 100;
```

Inspect the selected key/access path, estimates, actual rows/loops and time on a supported server version. EXPLAIN ANALYZE executes the statement; use representative, safe SELECT workloads in an appropriate environment. Benchmark with realistic parameters and concurrency, not only a tiny empty table. [MySQL EXPLAIN](#).

## 14. Oracle EXPLAIN PLAN versus the executed cursor

EXPLAIN PLAN asks the optimizer for a proposed plan without executing the target statement. It does not establish which child cursor, bind-sensitive plan or actual row counts your application used. DISPLAY\_CURSOR examines a cached cursor's plan; execution statistics must have been collected to see actual rows. [Oracle DBMS\\_XPLAN](#).

Oracle diagnostic recipe; requires an Oracle schema, suitable privileges and matching version, and is not claimed executed here:

```
EXPLAIN PLAN FOR
SELECT id FROM payments WHERE tenant_id=42 AND status='SETTLED';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY());

SELECT /*+ gather_plan_statistics */ id
FROM payments WHERE tenant_id=42 AND status='SETTLED';
-- Fetch the result fully; identify its SQL_ID and CHILD_NUMBER in your environment.
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(
:sql_id, :child_number, 'ALLSTATS LAST +PREDICATE'));
```

Look for the specific index name, operations such as INDEX RANGE SCAN and TABLE ACCESS BY INDEX ROWID, predicate placement, estimated versus actual rows, starts, buffers and elapsed work. A large estimate error suggests skew, stale statistics or correlation issues. An index used poorly may be slower than a scan. Functions on indexed columns, implicit type conversion and low selectivity can change access choices. Bind the correct types and compare alternatives with evidence.

Interview answer: "I inspect the executed cursor and actual work, not just the existence of an index. EXPLAIN PLAN is a prediction; DISPLAY\_CURSOR with collected statistics helps verify the actual execution." For an Oracle composite index, test its workload-specific behavior rather than copying a MySQL plan format. Notebook 29 covers the broader SQL, isolation and database design foundations.

## 15. API versioning, security and access/refresh tokens

URL versioning (`/api/v1/payments`) is easy to see, route and document. Header/media-type versioning keeps the URI stable but requires clients, gateways, cache keys and observability to preserve the version header; set Vary appropriately where responses are cacheable. Version the external contract, not every internal implementation edit. Prefer compatible additive changes when clients tolerate them, publish deprecation/migration windows, and test old/new consumers.

In Spring MVC, separate controller mappings can use paths or header conditions. For example, `@GetMapping(path="/payments", headers="X-API-Version=1")` selects a versioned handler. Do not create overlapping mappings that are ambiguous for a request. Newer framework versioning facilities are version-dependent; the explicit mapping example is easier to reason about across versions. [Spring request mapping](#).

Secure a REST API with TLS, validated identity, endpoint and object-level authorization, input validation, bounded payloads, rate limits, safe error handling and auditable outcomes. In an OAuth2 resource server, validate token signature with trusted keys, permitted algorithms, issuer, audience and time claims. Do not trust a decoded JWT merely because it is parseable. Enforce tenant/account ownership in service/data access, not only in the UI. [Spring resource-server validation](#).

An access token authorizes API access for a limited period. A refresh token is presented to the authorization server to obtain new access tokens; it is not the normal credential sent to a resource API. Access tokens may be JWTs or opaque; refresh tokens need not be JWTs. Rotation, secure storage, reuse detection and revocation policy matter. A signed JWT is not encrypted, and logout does not instantly invalidate every previously issued self-contained access token without an additional design.

In a browser, HttpOnly cookies reduce script access but still require a suitable CSRF strategy. CORS controls browser cross-origin access, not authentication. Avoid putting tokens in logs or URLs. Short access-token lifetimes limit exposure; refresh-token lifecycle and compromised-session response remain necessary. The existing identity lab shows real OIDC logout boundaries and their limits.

## 16. Spring Batch: boxes of records and restart bookmarks

A batch job is a planned sequence of steps. A chunk step reads items, optionally processes them, and writes a group under a transaction. With chunk size two, five valid inputs normally produce commits for 1-2, 3-4, then 5. Larger chunks can improve throughput but hold more work/locks and increase replay after failure. Tune with realistic payloads. [Spring Batch 5.2 chunk configuration](#).

JobRepository records execution metadata. A JobInstance is identified by job name plus identifying parameters; each attempt is a JobExecution, with StepExecutions recording step attempts. Restart a failed instance using the same identifying parameters and preserved repository state. Adding a fresh timestamp often creates a new instance and defeats the intended restart. A completed instance ordinarily cannot simply run again with identical identity. ExecutionContext stores restart state for cooperating stateful components. [Spring Batch restart configuration](#).

The runnable Maven workshop pins Spring Batch 5.2.4 with Spring Framework 6.2.12, separately from the library's Boot 4 application. A FlatFileItemReader saves its position. The writer fails on item 3 after inserting it.

The failed chunk rolls back; rows 1 and 2 remain committed. Restarting the same job instance reprocesses the uncommitted chunk and ultimately leaves exactly rows 1-5. The test checks failure, checkpointed progress, restart identity and final row count. It uses an actual H2 database and Spring Batch metadata, not a handwritten imitation.

| Outcome | Example                                                    | Correct policy                                               |
|---------|------------------------------------------------------------|--------------------------------------------------------------|
| Filter  | Valid row intentionally not eligible                       | Processor returns null; track as filtered                    |
| Skip    | Malformed row the business explicitly permits quarantining | Narrow exception type, skip limit, reason and reconciliation |
| Retry   | Transient deadlock/temporary failure                       | Bounded attempts/backoff and safe repeatable work            |
| Fail    | Corrupt source, exhausted retries or unsafe payment state  | Stop and repair/reconcile before restart                     |

Skip is not “ignore everything”; a settlement job cannot silently discard money records and still call itself successful. Reader, processor and writer failures can have different transaction/replay implications. Record reject counts and compare source totals against accepted, filtered and rejected items. Validate the source's identity/checksum so a restart does not read a changed file at an old offset. [Skip configuration](#), [retry configuration](#).

A chunk rollback cannot un-send email or undo a successful remote debit. Use transactional database state plus an outbox, provider idempotency and reconciliation for external effects. Partitioning assigns disjoint work units; multi-threaded steps require thread-safe or appropriately scoped components and careful ordering. Restartability is not exactly-once external execution. The test intentionally uses one thread and a stable input file; process-kill and distributed-partition recovery are distinct tests.

## 17. Microservice boundaries and data ownership

Imagine several school offices: admissions owns student enrolment, the cashier owns payments and the library owns book loans. Each office publishes a clear service contract. If everyone edits everyone else's records, dividing the rooms does not create independent ownership.

Decompose around cohesive business capabilities and bounded contexts, not one HTTP controller per service. Define ownership of invariants first. A modular monolith is often simpler while boundaries are uncertain. Microservices introduce network failure, operational overhead and consistency tradeoffs; they are not an automatic scalability upgrade.

| Pattern               | Payment-system example                                           | When it hurts                                                 |
|-----------------------|------------------------------------------------------------------|---------------------------------------------------------------|
| Database per service  | Ledger owns ledger writes; orders use a contract/event           | Cross-service SQL shortcuts recreate coupling                 |
| API gateway           | Route, authenticate and apply shared limits                      | Putting all business logic there creates a new monolith       |
| Backend for frontend  | Mobile endpoint composes only needed data                        | Too many nearly identical BFFs multiply maintenance           |
| Service discovery     | Find current payment service endpoints                           | Treating discovery as proof an instance is healthy            |
| Strangler migration   | Route selected old endpoints to a new service gradually          | Permanent duplicate sources of truth                          |
| Anti-corruption layer | Translate legacy bank codes into internal domain terms           | Hiding unhandled legacy semantics behind superficial renaming |
| API composition       | Assemble order plus payment status on demand                     | Cascading latency and partial failures                        |
| CQRS                  | Write model protects invariants; read model optimizes dashboards | Extra models and eventual consistency without a real need     |

| Pattern        | Payment-system example                | When it hurts                                                         |
|----------------|---------------------------------------|-----------------------------------------------------------------------|
| Event sourcing | Store domain events and rebuild state | Schema evolution, replay and privacy complexity; not required by CQRS |

Database-per-service means ownership and access boundaries; it does not necessarily require a separate physical database machine for each service. Never let a dashboard's eventually consistent projection become the authoritative source for whether funds are available.

## 18. Saga, outbox and idempotent consumer: connect the dots

A Saga coordinates local transactions with recovery actions. For an order: reserve stock, authorize payment, confirm order. If confirmation cannot complete, compensation might release stock and void authorization. Compensation is a new business action, not a time machine that guarantees restoration of every external effect. Captured money may require a refund with its own failure path.

Choreography lets services react to events; it reduces central coordination but can hide cycles and lifecycle ownership. Orchestration uses an explicit coordinator and durable state; it makes progress visible but needs its own recovery design. Choose based on the workflow's complexity and responsibility boundaries.

Outbox solves a specific dual-write gap. Save the local state change and outgoing event in one database transaction. A relay publishes later; it may publish twice after a crash. An inbox/deduplication record on the receiving side, committed with the business effect, makes duplicate handling safe. A unique event ID alone is insufficient unless its check and effect share the required atomic boundary.

```
# lab: InboxReplay
import sqlite3
db=sqlite3.connect(':memory:')
db.executescript('CREATE TABLE inbox(id TEXT PRIMARY KEY); CREATE TABLE tally(value INTEGER); INSERT INTO tally VALUES(0);')
def consume(event_id, fail=False):
    with db:
        inserted=db.execute('INSERT OR IGNORE INTO inbox VALUES(?)',(event_id,)).rowcount
        if not inserted: return False
        db.execute('UPDATE tally SET value=value+1')
        if fail: raise RuntimeError('simulated crash before commit')
    return True
try: consume('event-1',True)
except RuntimeError: pass
assert db.execute('SELECT COUNT(*) FROM inbox').fetchone()[0]==0
assert consume('event-1') is True
assert consume('event-1') is False
assert db.execute('SELECT value FROM tally').fetchone()[0]==1
db.close()
print('Business update and dedup marker roll back together, then replay applies once.')
```

Complete request flow: authenticated client → gateway → order API with idempotency key → local order/outbox commit → relay → broker → payment inbox/local intent commit → bank call with provider idempotency key → durable outcome → Saga transition → read projection → SSE status update. Every arrow crossing a process can fail or deliver late. Trace IDs help observation; idempotency/event IDs protect business semantics and should not be confused with trace IDs.

Kafka transaction guarantees do not automatically encompass an arbitrary external bank request. Store ordering/version information for each aggregate when necessary, reject or reconcile stale transitions, and define how a dead-letter item returns to the normal flow after repair.

## 19. Resilience patterns and failure budgets

| Pattern           | Child-friendly idea                       | Implementation question                                          |
|-------------------|-------------------------------------------|------------------------------------------------------------------|
| Timeout/deadline  | Stop waiting for a bus after a limit      | Does one end-to-end deadline constrain every downstream call?    |
| Retry with jitter | Try again after different short waits     | Is the operation safe to repeat, and which errors are transient? |
| Circuit breaker   | Stop knocking on a broken door            | What opens it, and how many half-open probes are allowed?        |
| Bulkhead          | Separate ship compartments                | Are threads, connections and queues bounded per dependency?      |
| Rate limit        | Admit only a bounded arrival rate         | Is fairness per user/tenant, and is Retry-After meaningful?      |
| Load shedding     | Reject work before collapse               | Which work can be safely rejected and retried?                   |
| Cache-aside       | Check the desk copy before the archive    | What are TTL, invalidation, ownership and stale-data rules?      |
| Backpressure      | Slow the producer when the consumer fills | Is the buffer actually bounded, and can producers obey?          |

Retries at three nested layers with three attempts each can produce 27 downstream attempts for one request. Give one layer clear retry ownership, propagate deadlines and use bounded exponential backoff with jitter. A circuit breaker is not a timeout and does not stop every already-running call. A fallback must not invent a successful payment; return pending/unavailable or a clearly labelled stale read when the contract permits it.

For readiness, ask whether an instance should receive traffic. For liveness, ask whether restarting it is likely to help. If a shared bank outage makes every pod fail liveness, restarting all pods creates more load without repairing the bank. Graceful shutdown should stop new work, drain or checkpoint safely and release leases predictably. Trace latency, saturation, error rates and business reconciliation gaps, not only CPU.

## 20. Kubernetes secrets and cloud interview answers

Never bake credentials into an image, source file or Docker ARG/ENV layer. Image layers can retain historical content. Supply runtime configuration through an appropriately protected Secret or an external secret manager integration, using workload identity where possible. Base64 in a Secret manifest is encoding, not encryption. Restrict RBAC, protect etcd storage, avoid secret logging and plan rotation. [Kubernetes secret practices](#).

Mounted Secret files can be updated eventually; the application must reload them correctly. Environment variables in a running process do not automatically refresh when the Secret changes. A subPath mount has different update behavior. Rotation requires overlap/cutover rules and revocation, not just editing a YAML value. Separate build credentials from runtime application credentials, and do not commit populated Secret manifests.

Answer structure for cloud questions: “I deployed [actual workload] using [actual services]. Traffic entered through [route/TLS]. Workload identity permitted [scope]. State lived in [store]. We observed [metrics/logs], tested [failure] and controlled cost using [actual mechanism].” If you only ran this library, describe the local Docker/K3s topology and explicitly distinguish it from a managed-cloud deployment. The audit records that boundary.

Mini incident: after rotating a database password, half the pods fail. Investigate running process configuration, connection-pool reconnection, provider overlap rules and rollout readiness. Do not assume the Secret object's latest value is already the value every process uses.



## 21. JUnit, Mockito and automation testing

A unit test is a small experiment: arrange an input and dependencies, act once, assert the important result. JUnit runs and reports the experiment. Mockito can replace a boundary with a controlled test double, stub an outcome and verify a meaningful interaction. A mocked repository does not test SQL, constraints or transaction rollback.

The Maven workshop includes a fee-calculating service with constructor-injected Ledger. Its Mockito test checks the calculated amount and exactly one save; invalid input must cause no ledger interaction. The transaction test starts a real Spring context and H2 database. The batch test uses real Spring Batch metadata and a restartable reader. These layers answer different questions.

Test money edge cases, payload conflicts, duplicate requests, rollback, lock ordering, exhausted retry, authorization and empty pages. Use `assertThrows` for the intended exception; do not merely catch and ignore any exception. Prefer deterministic coordination such as latches/barriers over arbitrary sleeps in concurrency tests. Check failure propagation from submitted tasks.

Automation includes unit tests, database integration tests, API/consumer-contract tests, browser journeys, accessibility checks, performance/recovery scenarios and CI. Contract tests catch incompatible interfaces but do not prove the whole workflow works. Browser tests cover selected journeys but are slower and more fragile than focused tests. Keep clocks, IDs and dependencies controllable where that improves reproducibility.

Testcontainers needs a working container engine; do not silently substitute H2 and claim MySQL/Oracle behavior was tested.

Run the actual workshop from the library root:

```
mvn -f labs/interview-workshop/pom.xml test
python run_notebook_labs.py --book 31-java-microservices-interview-masterclass
```

The first command needs Java 21+, Maven and initial dependency downloads. The second needs Python plus JDK tools for Java cells. The chapter's MySQL/Oracle commands remain labelled diagnostic recipes; the executed database tests use H2 and SQLite.

## 22. Sprint Retrospective and a graded interview round

A Sprint Retrospective is the team's inspection of how the Sprint went and how to improve effectiveness and quality. Discuss interactions, process, tools, assumptions and the Definition of Done, then choose actionable improvements. It is different from the Sprint Review, which inspects the product outcome with stakeholders and adapts future direction. It is not a blame meeting or merely a status update. [Scrum Guide](#).

Example: repeated flaky browser tests delayed releases. Bring failure evidence, identify uncontrolled clocks, agree to inject a clock into the affected feature, assign an owner and check the result next Sprint. “Communicate better” without an action or observation is hard to evaluate. Teams should make it safe to discuss mistakes while still assigning concrete follow-through.

Attempt this 45-minute round without the answers: ten minutes on HashMap/equality and employee filtering, ten on lock choice and transaction self-invocation, fifteen designing a payment POST plus paginated GET, and ten on Batch restart, index verification and one retrospective action.

| Question / follow-up                        | Answer key essentials                                                                            | Points |
|---------------------------------------------|--------------------------------------------------------------------------------------------------|--------|
| Colliding keys, equal keys and mutable keys | Collision is not equality; equal keys replace; equality/hash contract; mutation can break lookup | 4      |
| Filter employees older than 40              | <code>Predicate&lt;Employee&gt;</code> , strict greater-than, boundary at 40                     | 2      |
| Lock and map choice                         | Proper finally unlock, single-key atomic operations, no distributed-lock claim                   | 3      |



| Question / follow-up              | Answer key essentials                                                                                     | Points |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------|--------|
| Transaction self-invocation       | Proxy boundary bypass; outer transaction nuance; collaborator/TransactionTemplate repair                  | 4      |
| Payment retry after lost response | Tenant-scoped durable key, fingerprint, unique constraint, ambiguous remote result, reconciliation        | 5      |
| Huge GET and indexes              | Bounded keyset page, unique tie-breaker, authorization, query-specific index and actual plan              | 4      |
| Batch failure on record 3         | Committed chunk retained, failed chunk rolled back, same instance parameters, checkpoint and stable input | 4      |
| Microservice failure flow         | Outbox duplicates, transactional inbox, Saga compensation limits, bounded retries                         | 2      |
| Retrospective                     | Process inspection, concrete improvement, distinction from Review                                         | 2      |

Total 30. Suggested practice target: 24, with no claim that a timeout proves a payment failed or that a local transaction undoes a remote debit. Any such error needs correction regardless of total. Explain why your design works, how it fails and how you would test it. Your personal grade remains unset until you submit unaided answers and elapsed time.

## 23. Question coverage and next study links

| Supplied interview area                              | Where answered here          | Existing deeper notebook                   |
|------------------------------------------------------|------------------------------|--------------------------------------------|
| Introduction and experience                          | Lesson 1                     | <a href="#">Interview practice</a>         |
| HashMap, collisions, equals/hashCode                 | Lessons 2-3                  | <a href="#">Java foundations</a>           |
| Abstract class/main, try/finally/resources           | Lesson 4                     | <a href="#">Java depth</a>                 |
| Functional interfaces, annotation, Predicate         | Lesson 5                     | <a href="#">Java foundations</a>           |
| synchronized/locks, concurrent maps, race/deadlock   | Lessons 6-7                  | <a href="#">Java depth</a>                 |
| SOLID, SRP, composition/inheritance, design patterns | Lessons 8-9                  | <a href="#">All 23 GoF implementations</a> |
| Injection, transaction internals/self-invocation     | Lesson 10                    | <a href="#">Full-stack workshop</a>        |
| HTTP/idempotency and millions of GET records         | Lessons 11-12                | <a href="#">Database foundations</a>       |
| Composite indexes, MySQL/Oracle plans                | Lessons 13-14                | <a href="#">Databases</a>                  |
| API versions, API security, access/refresh tokens    | Lesson 15                    | <a href="#">Identity and deployment</a>    |
| Batch chunks, restartability, error handling         | Lesson 16 and Maven workshop | <a href="#">Recovery and operations</a>    |
| Microservice patterns and connected architecture     | Lessons 17-19                | <a href="#">Distributed systems</a>        |
| Kubernetes secrets and cloud experience              | Lesson 20                    | <a href="#">Kubernetes</a>                 |
| JUnit, Mockito, automation and retrospective         | Lessons 21-22                | <a href="#">Capstone assessment</a>        |

Reference checks were performed on 25 September 2026. Linked official sources are optional verification material; the teaching explanations, examples and answer keys are included in this notebook. Exact framework pins and executed results are recorded in the workshop, while provider-specific recipes and personal experience remain explicitly distinguished.

## Deep workshop - Explain HashMap collisions with actual keys

A hash chooses a candidate bucket; equality distinguishes keys there. Equal hashes do not imply equal keys, so a collision must not overwrite an unrelated entry.

### Trace insertion

Insert unequal A and B, both hash 7, with values “first” and “second.” Both must remain distinguishable. Looking up a separately constructed equal copy of A should find “first.” Equal keys must produce equal hashes; unequal keys may still collide.

### The mutable-key trap

If hash depends on email, changing email after insertion can send lookup to another bucket. Even the same object reference may then be difficult to find through the normal lookup path. Prefer immutable key identity or controlled removal and reinsertion.

### Concurrency adds a boundary

A thread-safe map protects supported operations, not arbitrary check-then-act sequences. Use an appropriate atomic map operation for local creation. That does not coordinate a remote payment transaction or make its side effects safe to repeat. What if equals is overridden without hashCode? **Answer:** Distinct equal objects may retain unequal inherited hashes and violate the collection contract. Test separate equal instances, not just the same reference. Discuss the contract before version-specific details such as bucket treeification thresholds.

# Notebook 32 - Load balancing: from a playground queue to production traffic

Mira runs an online cinema. At noon, tickets for a popular film go on sale. Ten customers become ten thousand. One application server is exhausted while another is nearly idle. Adding servers helped the shopping list, but nobody decided which customer should visit which counter. That is our starting problem.

This is a complete learning route through the common load-balancing decisions, with six executable Python experiments and a separate real NGINX lab. It does not claim every vendor feature has been deployed. At each stop, predict what happens before reading the explanation. The companion [production-readiness notebook](#) follows the same service into cloud deployment, database diagnosis, security, AI evaluation, accessibility and recovery.

## 1. Follow one request: who actually chooses the server?

Imagine a receptionist who hands each visitor a counter number. A load balancer chooses a eligible backend for a connection or request. It does not automatically fix slow SQL, incorrect payments, lost sessions or a shared database outage.

Browser

- > DNS resolution / possibly a CDN
- > public endpoint and load balancer
- > optional gateway / reverse proxy
- > application instance A or B
- > connection pool
- > shared database / cache / broker

These are roles, not a requirement to buy six separate products. One proxy can provide routing, TLS termination and balancing. A reverse proxy stands in front of servers; a forward proxy acts on behalf of clients. An API gateway often adds API-specific identity, limits or transformations. A CDN caches suitable content near readers. A load balancer distributes work. Clarifying the role is more useful than arguing over overlapping product names.

**Predict:** Mira doubles application instances but the database has only ten connections available. Does capacity double? No. More callers can simply queue at the same bottleneck. Inventory the complete request path and its shared limits before scaling one box.

## 2. Layer 4 versus Layer 7: envelopes versus letters

A Layer 4 balancer typically chooses using transport information such as addresses and ports. Think of sorting sealed envelopes by destination. It can forward TCP without understanding /payments or a logged-in user's cookie. A Layer 7 HTTP proxy understands HTTP requests and can route /images to one pool and /api to another.

| Decision           | L4 approach                                                                      | L7 approach                                                                |
|--------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Routing key        | Transport connection attributes                                                  | Host/path/header and other configured HTTP attributes                      |
| TLS                | Can pass encrypted traffic through; termination depends on product/configuration | HTTP inspection normally occurs where TLS is terminated                    |
| Request visibility | Connection-oriented metrics                                                      | HTTP status, route and request timing                                      |
| Long-lived traffic | One selected connection may carry much work                                      | May balance HTTP requests/streams, but established tunnels remain attached |
| Cost/tradeoff      | Less application interpretation                                                  | More policy control, parsing and configuration responsibility              |

Do not equate an OSI label with an unconditional performance winner. Measure the actual implementation, encryption, connection reuse and workload. HTTP/2 can multiplex many streams over a connection: equal connection counts need not imply equal request load. A balancer distributing TCP connections can therefore look unfair at the HTTP-request level even while doing exactly what it was configured to do.

**Explain it aloud:** “We need path-based routing and HTTP observability, so an L7 proxy fits that requirement. We still need to measure capacity and decide where TLS ends.”

### 3. Round robin and weights: sharing turns fairly

Round robin cycles A, B, C, A, B, C. It is simple and useful when eligible servers have comparable capacity and requests have similar cost. But one report can take a second while another request takes a millisecond. Equal turns are not necessarily equal work.

Weights express intended relative shares. If A can handle three units while B handles one, a 3:1 policy aims for about three quarters of eligible assignments to A under its specified conditions. Weight is neither CPU percentage nor a guarantee for every short observation window. Health exclusions and existing connections affect measured traffic.

Smooth weighted round robin spreads the larger share across the sequence instead of sending a huge uninterrupted burst. In this teaching implementation, add every weight to its current score, choose the largest score, then subtract total weight from the winner. The scores act like a record of how much service each backend is owed.

```
# lab: SmoothTicketCounters
from collections import Counter
weights={'A':3,'B':1}; current={name:0 for name in weights}; assignments=[]
for _ in range(40):
    for name,weight in weights.items(): current[name]+=weight
    chosen=max(current,key=current.get)
    current[chosen]-=sum(weights.values());assignments.append(chosen)
assert Counter(assignments)=={'A':30,'B':10}
print('First eight choices:',assignments[:8])
print('Forty requests:',dict(Counter(assignments)))
```

**Change the experiment:** set weights to 5:2:1 for A/B/C and use a request count divisible by eight. Then remove C mid-run. Decide whether to reset scores or preserve them, and document that policy. Your small scheduler is an illustration; the separate NGINX test checks a real proxy's behavior independently.

### 4. Least connections, least requests, latency and two choices

Least connections chooses the backend with fewer active connections, often with weights. It can help when connection lifetimes vary. Least outstanding requests measures a different unit. A server with one expensive report may be busier than one with ten tiny cache hits; neither count is a perfect measure of work.

A latency-aware policy can track a smoothed response estimate, but raw latency is noisy. An exponentially weighted moving average gives recent samples influence without forgetting everything: new estimate =  $\alpha \times \text{sample} + (1-\alpha) \times \text{old estimate}$ . A fast-failing server can appear “fast,” so latency must be considered alongside errors, load and health. Sampling and exploration matter when an idle backend has stale measurements.

Power of two choices samples two eligible backends and picks the less loaded one. It avoids comparing every server for every request while improving on blindly choosing one. Real systems must handle stale distributed measurements and independent balancers making simultaneous decisions.

```
# lab: CounterSelection
loads={'A':8,'B':1,'C':4}
```

```

pair=('A','C')
choice=min(pair,key=loads.get)
assert choice=='C' # Best sampled counter, not necessarily global best B.
estimate=100.0; alpha=.25
for sample in (100,100,500): estimate=alpha*sample+(1-alpha)*estimate
assert estimate==200.0
print('Two-choice result:',choice,'; smoothed latency:',estimate,'ms')

```

**Interview trap:** “Least connections is always best.” Ask instead: are requests short, streamed, multiplexed, CPU-heavy or sticky? Which load measurements are available? What happens when all backends are saturated?

## 5. Consistent hashing, rendezvous hashing and hot keys

Suppose each counter has a local cache. Sending the same catalogue key to the same server improves cache locality. Simple  $\text{hash}(\text{key}) \% \text{server\_count}$  remaps many keys when the count changes. A consistent-hash ring places servers and keys on a circle; virtual nodes can smooth uneven placements. Rendezvous hashing instead scores each key/server pair and chooses the largest score.

The following deterministic rendezvous example uses SHA-256 so results are stable across processes. Python's built-in hash is not an appropriate persistent cross-process routing contract because some types are randomized between processes.

```

# lab: StableRouting
import hashlib
def owner(key,servers):
    return max(servers,key=lambda
server:int.from_bytes(hashlib.sha256((key+'|'+server).encode()).digest(),'big'))
old=['A','B','C'];new=old+['D'];keys=[f'ticket-{i}' for i in range(1000)]
changed=[k for k in keys if owner(k,old)!=owner(k,new)]
assert changed and all(owner(k,new)=='D' for k in changed)
assert all(owner(k,old)==owner(k,new) for k in keys if owner(k,new)!='D')
print(len(changed),'of 1000 keys moved; changed keys moved only to the added server.')

```

This is an algorithm property test, not a production sharding system. You still need membership agreement, key encoding, weights, replication, failover and migration. Hash routing does not make a single extremely popular key disappear. One blockbuster-film key can overload its owner. Options include cached replication, request coalescing, partitioning the workload when semantics allow, or special handling for hot keys. Each trades locality for distribution.

Session affinity is related to stable routing but solves a different problem: keeping a client's requests together. Never confuse either with durable storage.

## 6. Capacity and queues: why the last few percent hurt

Mira's counter serves 100 customers per second. At 20 arrivals per second, it has breathing room. At 99, a small burst can leave a long queue. The average alone conceals the waiting customer's experience.

For a stable system, Little's law connects average in-flight work  $L$ , arrival rate  $\lambda$  and average time  $W$ :  $L = \lambda \times W$ . At 200 requests/second and 0.25 seconds average time, about 50 requests are in flight on average. Match the observation boundary: server processing time is not the same as end-to-end time including queues.

```

# lab: QueueCliff
service_rate=100.0
for arrival in (20.0,70.0,95.0,99.0):
    # M/M/1 illustration: Poisson arrivals, exponential service, one server, stable load.
    mean_seconds=1/(service_rate-arrival)
    in_system=arrival*mean_seconds
    assert mean_seconds>0
    print(f'{arrival:.0f}/s -> mean {mean_seconds:.3f}s, average in system {in_system:.1f}')
assert 200*.25==50

```

This queue formula is a deliberately idealized model, not a forecast for a real web service with caches, bursts and many workers. Its lesson is the shape: approaching saturation amplifies delay. If arrivals exceed sustainable service, an unbounded queue stores tomorrow's failures in memory. Bound queues, apply deadlines and reject appropriate work before collapse. A useful rejection is better than pretending an expired request is still likely to succeed. [Overload principles](#).

For capacity planning, estimate demand after losing a failure domain. Three servers each safe at 100 requests/second provide 300 normally but only 200 after losing one. Serving 270 at normal operation is not one-server-failure capacity, even though all three currently fit the load.

## 7. Health checks: a pulse is not a completed purchase

Liveness asks whether restarting a process is likely to help. Readiness asks whether it should receive new work. Startup checks allow slow initialization. A process returning 200 from `/health` can still fail every database query; a dependency outage can also make a deep readiness check remove every server at once.

Active checks periodically probe. Passive checks infer failure from actual traffic. Use thresholds and recovery rules to avoid flapping. Record what counts as a failure: a transport timeout is not the same as a valid 404 for an absent ticket. The load-balancing lab uses NGINX passive failure handling; it does not claim an active-check feature was configured.

Imagine B is overloaded and misses health checks. Sending all B's traffic to A can overload A too. Removal is not a source of additional capacity. Pair health policy with load shedding, reserve capacity and cautious recovery. Warm caches gradually when returning a server instead of immediately giving it its full historical share.

**Fault drill:** make only a recommendation dependency fail. Should checkout become unready, or continue without recommendations? Define dependency criticality and customer-visible degradation before setting probe rules.

## 8. Sticky sessions and shared state: a favourite counter can close

A cookie or source-IP affinity rule may keep a customer on the same backend. This can reduce migration of expensive local state. It can also hide a design defect: login works until the selected server dies, then the user's in-memory session disappears.

IP affinity groups users behind a shared NAT address and may behave poorly when mobile addresses change. Cookie affinity needs lifecycle, integrity and failure behavior. Neither is authorization. A client reaching server A is not evidence that it owns account A.

Shared session storage, appropriately validated self-contained credentials, and durable revocation protocols offer different tradeoffs. A shared store can become another availability dependency. A JWT can outlive logout unless its validation/revocation design accounts for it. Persist shopping carts and payment state independently from a routing preference. Reapply identity and object authorization on every instance.

Mini incident: after scaling from one pod to two, every second refresh logs the user out. Check session storage and cookie configuration before blaming the round-robin algorithm. Stickiness might mask the symptom; it does not establish failover correctness.

## 9. TLS termination, re-encryption and the trust boundary

TLS termination means a component decrypts a connection. From that point onward, plaintext is available there. If the backend network is outside the intended trust boundary, establish another authenticated encrypted connection to the backend and verify the expected certificate identity. Two TLS connections are not one uninterrupted end-to-end session.

With passthrough, the application or downstream gateway terminates TLS; an upstream transport balancer cannot generally inspect encrypted HTTP paths. Some routing can use handshake metadata, but that is not equivalent to reading the HTTP request. Decide where certificates and private keys live, who can rotate them and how expiry is monitored.

Distinguish server identity from client identity. Mutual TLS authenticates certificate holders according to your PKI policy; application authorization still decides what an authenticated principal may do. Do not turn off hostname verification to “fix” certificate errors. Log a useful error category without exposing key material.

Our local NGINX test binds to loopback and uses generated bearer credentials. Public certificates, external TLS and managed-load-balancer configuration remain deployment exercises, not claimed local successes.

## 10. A real NGINX experiment, not just a diagram

The runnable lab starts two actual Python backend processes, A and B, plus a real NGINX process in the isolated Linux lab. It uses ephemeral loopback ports and generated credentials. It does not reuse the Study Coach database or change a public site.

From the library root on the documented Windows/WSL setup:

```
python labs/setup_linux_lab.py
wsl -d EngineeringNotebookLab-13e66a98 --exec /sbin/apk add --no-cache nginx python3
python labs/run_linux_lab.py load-balancing
```

On a Linux machine with nginx and Python installed, run `python3 labs/load-balancing/test_nginx.py`. Read `labs/load-balancing/test-report.json` for the actual result. Setup may need downloads; reading the notebook does not. After all lab work, Windows users can stop only this lab with `wsl --terminate EngineeringNotebookLab-13e66a98`.

The assertions check equal round-robin counts, a 3:1 weighted distribution, anonymous rejection, a sanitized forwarding header, incremental SSE delivery, non-replay of a POST after an effect and 503 response, safe GET failover when A dies, and explicit failure when both backends are gone. The test cleans up its processes. A passing report is evidence for these scenarios on one host, not a multi-region availability certificate.

The following excerpt explains the policy; the test generates a full configuration with available ports:

```
upstream classroom {
    server 127.0.0.1:18181 weight=3 max_fails=1 fail_timeout=30s;
    server 127.0.0.1:18182 weight=1 max_fails=1 fail_timeout=30s;
}
server {
    listen 127.0.0.1:18180;
    location / {
        proxy_pass http://classroom;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_connect_timeout 1s;
        proxy_read_timeout 10s;
        proxy_next_upstream error timeout http_503;
        proxy_next_upstream_tries 2;
        proxy_buffering off;
    }
}
```

NGINX's exact directives and defaults depend on version and edition; the test records the installed version. Explicitly specifying HTTP/1.1 avoids relying on a changing default. Understand which failures count toward passive exclusion and when retry is permitted before copying configuration. [NGINX upstream reference](#).

## 11. A dangerous retry: the payment succeeded but the response failed

At counter A, a cashier records a payment, then the receipt printer fails. Sending the same instruction to counter B can charge twice. Network retry is not a business undo button.

The lab's POST endpoint deliberately increments its effect counter and returns 503. Although the proxy can retry selected failed GET requests, it must not blindly replay this non-idempotent request on another backend after sending it. The test checks that the total effect count across A and B is one. This verifies a proxy policy boundary; it does not implement a durable payment service.

For real payments, use the durable scoped key and reconciliation protocol from Notebook 31. Account for all retry layers: browser, mobile SDK, gateway, service client and worker. Two retries at several layers can multiply downstream traffic. A deadline is an overall budget; a connect timeout or an idle read timeout is only one phase's rule. Cancellation on one side does not prove the remote business operation stopped.

**Predict:** the client times out while A is still charging the card. Should the user click “new payment”? No. Preserve the operation identity and query/reconcile its status. A load balancer can redirect future traffic; it cannot infer the missing business outcome.

## 12. WebSockets, SSE, HTTP/2 and streaming AI

An ordinary short HTTP response is like a letter. An SSE stream or WebSocket is more like a telephone call: once connected, moving later requests elsewhere does not move the existing call. Long-lived connections consume file descriptors, buffers and backend capacity. New servers receive new connections; old connections may remain unevenly distributed.

For SSE, unwanted proxy buffering can delay visible events. Configure buffering and caching deliberately, flush events, send suitable heartbeats and design a durable resume cursor when missed events matter. The real lab proves the first event reaches the client before the backend is allowed to produce the second. It does not use an arbitrary “it felt fast” assertion. [NGINX buffering behavior](#).

For classic HTTP/1.1 WebSocket upgrade proxying, explicitly forward the relevant Upgrade/Connection semantics; these hop-by-hop headers are not simply forwarded like arbitrary application headers. Configure timeouts and heartbeat behavior intentionally. [NGINX WebSocket documentation](#).

Reconnecting a stream needs authentication, replay policy, duplicate handling and a clear “history expired” response. A sticky session cannot reconstruct lost messages. A token expiring during a stream needs a defined closure/renewal policy. The existing real-time notebook and lab cover application-level replay and logout; this NGINX lab adds actual SSE proxy evidence, not a new claim of tested WebSocket tunnelling.

For streaming AI, measure time to first token separately from time to final answer. A responsive first token does not prove completion, correctness or fair GPU scheduling. Propagate disconnects where supported and bound abandoned work.



## 13. Kubernetes Service, Ingress, Gateway and service mesh

A Service provides a stable service-facing abstraction over changing endpoints. A LoadBalancer Service generally needs an implementation/provider to provision or integrate external balancing; writing the YAML alone does not create a working cloud endpoint everywhere. Ingress needs a controller. Gateway API introduces a richer resource model, but also needs a compatible implementation. [Kubernetes Service concepts](#).

Think of Service as a stable office number and endpoints as current staff. Traffic policies, readiness, locality and session affinity influence delivery. A persistent connection can remain associated with one backend; do not expect perfectly equal HTTP request counts merely because three pods are Ready.

A service mesh can provide east-west traffic policies, identity and telemetry. It adds configuration, resource and failure complexity. It cannot make an unsafe debit retry safe. Choose it for concrete requirements that justify operating it, not because a diagram looks more advanced with sidecars.

Trace the actual path: cloud frontend → ingress/gateway implementation → service/backend endpoints → pod. Avoid unnecessary proxy layers with conflicting timeouts and duplicate retries. When investigating imbalance, inspect connection reuse, endpoint membership, sticky policy, zone locality and measured request cost before changing algorithms.

## 14. Autoscaling and cold starts: more counters take time to open

Balancing distributes existing eligible capacity. Autoscaling changes capacity. They are related, but one does not substitute for the other. New pods need scheduling, image availability, initialization, readiness and often cache warm-up before they can help.

CPU utilization can be a poor scaling signal for a service blocked on a database or serving many idle sockets. Consider request concurrency, queue age, latency and dependency limits. Scaling a connection pool from ten replicas to a hundred can exhaust a database even if each replica keeps the same small pool.

Use minimum capacity, stabilization, appropriate resource requests and controlled scale-down. Choose a target below overload and test the reaction delay against expected bursts. A ten-second burst is not solved by a two-minute capacity startup unless buffering/rejection protects the system in the meantime.

**Worked estimate:** normal demand is 1,200 requests/second and each instance is comfortable at 200. Six instances fit steady demand, but seven are needed to retain 1,200 after losing one, before extra headroom and deployment overhead. If readiness takes a minute, capacity planning must cover that minute too.

## 15. Draining, rolling releases and graceful shutdown

To close a counter politely, stop sending new visitors, let current visitors finish within a deadline, and handle unfinished work explicitly. Removing an endpoint and stopping its process are not necessarily simultaneous at every proxy and client.

```
# lab: DrainTheCounter
state={'accepting':True,'active':2}
def admit():
    if not state['accepting']: return False
    state['active']+=1;return True
state['accepting']=False
assert admit() is False
state['active']-=1;assert state['active']==1
state['active']-=1;assert state['active']==0
print('No new admissions after drain; existing work reaches zero before stop.')
```

This state-machine illustration does not emulate a proxy's drain implementation. Production checks must include propagation delay, idle keepalive connections, long-lived streams, worker leases, termination grace periods and

hard deadline behavior. A pre-stop delay alone is not proof every layer stopped routing traffic.

During a rolling release, ensure old and new code understand overlapping API/event/database schemas. Expand, migrate, switch, then contract. If new code deletes a column the old version needs, an application rollback may be impossible. A canary compares a small eligible population against a baseline; compare errors, latency and business outcomes, not just pod readiness. Blue/green routing permits a quick traffic switch but still shares the database compatibility problem.

## 16. The balancer can fail too: DNS, zones and regions

Two backends behind one unprotected proxy still depend on that proxy. Managed or replicated balancing can reduce that failure point, but you must understand its control plane, state, zone capacity and recovery behavior. A local lab with several processes does not survive loss of its laptop.

DNS distributes answers with caching, TTLs and client-specific behavior. Changing a DNS record does not instantly move every established connection or cached client. Anycast advertises an address from multiple locations and depends on network routing; it is not a guarantee that every request selects the lowest application latency. Global traffic policies must coordinate with where data can safely be read or written.

Active/passive designs can simplify single-writer state but require verified promotion and fencing. Active/active can improve locality and availability but introduces conflict and consistency design. Do not send writes to two independently promoted primaries unless the data model explicitly supports it. A regional endpoint becoming reachable is not proof its database is current enough to accept financial writes.

## 17. Forwarded headers, rate limits and abuse boundaries

Behind a proxy, the immediate network peer is often the proxy itself. Applications sometimes use forwarded headers to recover the original client address or scheme. Only trust headers set or sanitized by a known, protected proxy path. A client-supplied X-Forwarded-For: 127.0.0.1 is not proof of localhost privilege.

The lab overwrites X-Forwarded-For at its single edge and verifies a forged value is replaced. A multi-proxy chain needs an explicit trusted-hop policy rather than blindly dropping or trusting every address. Authentication and tenant authorization must not be delegated to an arbitrary header.

Rate limits differ from concurrency limits. A tenant sending a few extremely slow requests can occupy all workers while staying under requests-per-second limits. Bound body/header sizes, connection duration and per-dependency concurrency where appropriate. Preserve protocol correctness; custom HTTP parsers and inconsistent proxy/backend parsing can create request-smuggling risks. Use maintained implementations and test the configured chain.

## 18. Observability: explain the queue without guessing

Collect request rate, eligible backend count, outstanding work, errors, rejection counts, queue time, backend connection time, response time and end-to-end latency. Separate 4xx due to client behavior from proxy transport failures and backend 5xx. Averages hide tail latency; inspect p95/p99 alongside volume and sample size.

```
# lab: PercentileTrap
times=[10]*95+[1000]*5
mean=sum(times)/len(times)
ordered=sorted(times)
def nearest_rank(p):
    import math
    return ordered[max(0,math.ceil(p*len(ordered))-1)]
assert mean==59.5 and nearest_rank(.99)==1000
print('Mean:',mean,'ms; p99:',nearest_rank(.99),'ms')
```

Record which percentile definition and interval you use. Do not average percentiles from different servers as if that produced the combined percentile. Do not put raw user IDs, prompts or request IDs into unlimited metric labels; use bounded labels and traces/logs with appropriate retention and privacy controls.

When B gets little traffic, follow a diagnostic order: is it ready and present? Does it have a lower weight? Is affinity pinning traffic? Are existing connections persistent? Is passive health exclusion active? Does locality constrain selection? Are metrics measuring connections or requests? Only then decide whether the algorithm itself is wrong.

## 19. Interview challenge and answer key

Design Mira's ticket API for a sudden 10× burst. It has short catalogue reads, a payment POST and live status streams. One server dies halfway through a payment; another deployment begins while streams remain open.

Award two points each for: workload-specific routing; bounded queues/deadlines; spare capacity and autoscaling delay; durable idempotency/reconciliation; correct stream resume; shared state/authorization; readiness versus liveness; drain/rollback compatibility; trusted headers/TLS; evidence-driven metrics. A strong answer scores at least 16/20 and never treats an ambiguous payment timeout as a guaranteed failure.

Worked outline: cache authorized/public catalogue data appropriately; use healthy L7 backends for HTTP policy; keep payment intent in durable storage; retry only within the business contract; retain status events with cursors; avoid relying on sticky sessions for correctness; reserve failure capacity; drain old instances with a deadline and resume policy; compare canary business outcomes; inspect tail latency and saturation.

Your answer can choose different products or algorithms if its assumptions, invariants and failure behavior are clear. The goal is to explain why the system remains understandable when something breaks, not to draw the largest collection of boxes.

## 20. What was explained, simulated and executed

Six notebook cells explore weighted scheduling, sampled choices/EWMA, rendezvous hashing, queue behavior, draining and percentiles. They are deterministic teaching experiments. The NGINX lab separately tests a real local reverse proxy and two backend processes; consult its report for actual results. TLS deployment, L4 balancing, public ingress, WebSocket tunnelling and independent-host failure are explained but not newly certified by that test.

Continue with Notebook 33 for the cloud deployment acceptance plan, AI-quality scorecards, database plan exercises, accessibility scenarios, identity rotation, disaster recovery and a personal interview schedule. This distinction between understanding, simulation and executed evidence is itself an interview skill.

### Additional executed transport checks

The [TLS/WebSocket report](#) now records an actual secure NGINX tunnel: trusted localhost certificate accepted, untrusted certificate rejected, missing authorization and hostile Origin rejected, and bidirectional messages continuing through reload. Follow the [acceptance runbook](#) to reproduce it. A private test certificate does not replace public DNS, certificate renewal or cloud ingress acceptance.

## Deep workshop - Use queueing to explain a slow load balancer

A backend can complete 100 requests per second for a measured workload. Sending 99 leaves little room for bursts and slow requests. Waiting can grow sharply before errors appear. A load balancer redistributes arrivals; it does not create capacity.

## Calculate in-flight work

For a stable system, Little's law states average in-flight work  $L = \text{arrival rate } \lambda * \text{average time } W$ . At 80 requests/second and 0.25 seconds,  $L=20$ . Keep units consistent. This is average accounting under stability assumptions, not a formula for every latency percentile.

## Algorithm and workload

Round robin shares counts, not necessarily work. A 5 ms request and a 5 second request differ radically. Least-connections may be distorted by long-lived idle connections. Least-active-request or latency-aware choices need reliable measurements and careful admission of newly healthy instances.

## Retry safety

A backend can commit a POST and lose the response. Proxy retry can repeat the effect. Consider method semantics, stable operation IDs, and uncertain outcomes. A healthy socket is not proof of a successful transaction. Would consistent hashing fix a hot tenant? **Not necessarily**: stable placement may preserve the hotspot. Limits, repartitioning, cache replication, or changed ownership may be necessary. Explain affinity benefits and capacity risks together.

# Notebook 33 - The launch room: cloud, AI quality, databases and recovery

Mira's cinema now has a working load balancer. The team celebrates-until someone asks a harder question: "What evidence would make us comfortable opening ticket sales tomorrow?" A diagram is not enough. Neither is one green test. This notebook turns each previously pending area into a detailed learning journey and an acceptance exercise.

You will move through the launch room, the AI review desk, the database investigation, an accessibility session and a recovery drill. Each lesson follows a problem, a mechanism, a worked decision and a question to answer. Five executable Python cells make important ideas tangible. Cloud deployment, independent factual review, physical-device testing and your personal interview score remain unexecuted where the required environment or participation is absent. Adding explanations does not fabricate those results.

## 1. Four labels that prevent false confidence

Give every claim one of four labels: **explained**, **simulated**, **executed locally**, or **accepted in the target environment**. A local three-process Kafka test is executed locally; it is not an independent-host network-partition test. A diagram of an AWS load balancer is explained; it is not an AWS deployment. A browser viewport is a simulation of width, not a physical iPhone.

For every experiment, record the question, configuration/version, inputs, expected result, observations, failure evidence and limits. Avoid "it works" without naming what "it" means. If an earlier failure led to a correction, retain both the failure and the new result. Otherwise the history can look more reliable than the system actually was.

Mira's acceptance register has seven columns: requirement, owner, environment, procedure, evidence, outcome and next action. Outcome can be PASS, FAIL or NOT RUN. NOT RUN is useful information; it tells the next engineer what must happen rather than making them rediscover missing proof.

**Exercise:** classify "NGINX sent requests to B after process A was stopped." Answer: a local process-failure result. It says nothing by itself about a broken network link, shared disk failure or a region outage.

## 2. Cloud architecture: map responsibilities before product names

Build the smallest topology that meets explicit needs. Start with request volume, latency, data sensitivity, availability objective, recovery objectives, team capacity and spending limit. Then map responsibilities to services: ingress/TLS, compute, durable database, cache, queue, object storage, identity, secrets, telemetry and backups.

```
Public DNS + certificate
  -> managed ingress/load balancer
  -> application replicas in selected failure domains
  -> private database / cache / broker endpoints

Background workers -> durable job queue -> authorized data/model services
Deployment identity -> artifact registry + narrowly scoped deployment permissions
Telemetry -> collector/storage with retention and access policy
Backups -> separate recovery destination + tested restore procedure
```

This provider-neutral diagram is not an instruction to run every service publicly. Decide which components need inbound internet access and which should be reachable only from the application. Control outbound access too, especially for agents that consume untrusted documents and might otherwise call arbitrary destinations.

In one possible AWS mapping, an Application Load Balancer fronts a managed container workload, with a managed relational database and a secret manager. That is an example, not a claim that an account was provisioned or a recommendation independent of your requirements. Other platforms can satisfy the same responsibilities. Compare operational behavior and team knowledge, not only icon similarity.

**Predict:** every load-balancer target fails its health check. Will traffic always stop? Not necessarily. Some products have fail-open behavior; AWS ALB documents one such condition. Test the exact provider policy instead of importing assumptions from a local proxy. [ALB health-check behavior](#).

### 3. Infrastructure as code, identity and cost controls

Infrastructure as code records desired resources and their relationships. A reviewed plan exposes intended changes; application creates or updates real resources. Neither step proves the running application is healthy. Keep state protected and concurrency controlled. Separate environments with explicit ownership rather than depending on a naming convention alone.

Use a deployment identity scoped to the selected environment. Prefer short-lived federation/workload credentials where supported over copied long-lived keys. Treat a CI secret as sensitive even when logs mask it: malicious build code can still try to transmit values elsewhere. Pull-request trust, action dependencies and repository permissions matter. [GitHub secret concepts](#).

Budgeting starts with an estimate of steady usage, peaks, storage growth, backups, network egress and retained logs. An alert often warns rather than imposing a hard cap. Set ownership tags, alert recipients and a cleanup plan. Do not assume stopping a compute instance deletes its volumes, snapshots, addresses or load balancer.

For the actual deployment exercise, fill in: account/project, region, environment name, spending limit, identity, domain ownership, approved data, desired lifetime and teardown owner. These values are still needed before a real cloud rollout. The library contains local runtime evidence and this deployment procedure; it does not invent credentials or a spending decision.

### 4. A deployment sequence you can explain under pressure

First build an immutable artifact and record its digest. Run appropriate unit, integration, migration and dependency checks. Prepare runtime secrets without placing them in image layers. Review infrastructure changes and confirm the intended account. Deploy the smallest staging slice, then verify DNS, certificate identity, routes, health and authorization from the intended client network.

Next test business behavior: create an authenticated job, retry its idempotency key, submit a conflicting payload, stream status, reconnect, terminate a worker and inspect the final durable outcome. Verify telemetry contains correlation without credentials. Exercise backup restoration into a separate destination. Only then decide whether the rollout meets the declared acceptance bar.

| Gate          | Concrete evidence                                          | Failure response                                 |
|---------------|------------------------------------------------------------|--------------------------------------------------|
| Artifact      | Exact digest and passing build report                      | Do not deploy an unidentified image              |
| Migration     | Old/new compatibility and rehearsal                        | Stop before destructive schema change            |
| Network/TLS   | Expected certificate, routes and private boundaries        | Correct routing/trust, not disabled verification |
| Identity      | Valid login, denied cross-tenant access, revocation checks | Keep protected functions unavailable             |
| Business flow | Durable successful/retried/failed outcomes                 | Repair or roll back within data compatibility    |
| Recovery      | Timed restore and reconciliation                           | Revise recovery claims and procedure             |

For canaries, choose a small eligible population and compare meaningful metrics against a baseline over a representative interval. A low request count may hide a severe failure. A rollback of code must respect data already written by the new version. Stop conditions and decision ownership should be agreed before the incident.

## 5. Service levels: translate a promise into a measurement

An SLI is a measurement, an SLO is its target, and an SLA is an agreement with consequences under specified terms. For a booking API, “successful eligible requests completed within two seconds” is more specific than “the server is healthy.” Define exclusions, observation point and what counts as a successful business outcome.

For a request-based 99.9% success SLO across 100,000 eligible requests, the error budget is 100 failed requests. That is not automatically 43.2 minutes of downtime: a request-based budget and a time-based availability calculation use different denominators.

```
# lab: RequestErrorBudget
eligible=100_000; target=.999; failed=140
allowed=round(eligible*(1-target))
assert allowed==100
burn=failed/allowed
print('Allowed failures:',allowed,'; observed:',failed,'; budget used:',round(100*burn),'%')
assert burn>1
```

Fast burn over a short window may justify immediate action; a longer window helps distinguish sustained trouble from noise. A budget is a decision aid for reliability investment, not permission to ignore individual harmful failures. An incorrect payment can matter even when aggregate HTTP availability looks excellent.

Ask: which user journey is missing from the dashboard? A homepage probe can be green while checkout fails. Add synthetic journeys and business reconciliation metrics where appropriate, and keep their test data clearly separated.

## 6. AI evaluation: the answer can be fluent and wrong

Mira adds an assistant that answers refund questions from policy documents. It retrieves the right page and produces a beautifully phrased answer-but changes “within fourteen days” into “within thirty days.” Retrieval succeeded; grounded answer generation failed.

Evaluate stages separately. Ingestion checks detect missing/stale documents and permission metadata. Retrieval metrics check whether needed evidence appears. Reranking and context assembly checks detect dropped or truncated support. Generation checks assess factual claims, completeness, justified abstention and instruction resistance. End-to-end checks include identity, latency, cost, timeouts and user-visible recovery.

Create a versioned case record containing question, authorized identity, source snapshot/hash, expected claims, forbidden claims, acceptable abstention conditions, category and adjudication notes. Keep a development set for iteration and an untouched holdout for a later decision. Once you inspect a failing holdout and tune to it, that case becomes development evidence; choose a genuinely new set for the next independent estimate.

The existing eight-case generation regression passed six cases. This notebook does not relabel it. One unnecessary abstention and one timeout remain failures under that test. Source-only fallback improves availability during generation failure, but does not magically improve every model answer.



## 7. Build a scorecard that cannot hide failures

Report factual correctness, answer coverage, unsupported-claim rate, permission violations, justified abstentions, inappropriate abstentions, latency/timeouts and cost separately. If you report correctness only among answered questions, also show how many questions received answers. A system that refuses everything can avoid unsupported claims while being useless.

```
# lab: HonestAnswerScorecard
# Synthetic outcomes for learning metric denominators, not new model measurements.
cases=[
    {'answered':True,'correct':True,'timeout':False},
    {'answered':True,'correct':True,'timeout':False},
    {'answered':True,'correct':False,'timeout':False},
    {'answered':False,'correct':False,'timeout':False},
    {'answered':False,'correct':False,'timeout':True},
]
answered=sum(x['answered'] for x in cases)
correct=sum(x['correct'] for x in cases)
coverage=answered/len(cases); conditional=correct/answered
assert coverage==.6 and correct/len(cases)==.4
assert sum(x['timeout'] for x in cases)==1
print('Coverage:',coverage,'; correctness among answered:',round(conditional,3),'; correct over
all:',correct/len(cases))
```

A citation should support the claim attached to it. Check whether the cited source is authorized, current, actually retrieved and sufficient for the assertion. Citation existence is not entailment. Human reviewers should assess claim-level support against the source, record disagreements and resolve them through a stated rubric. An LLM judge can assist triage, but it can share biases, be manipulated by answer text or disagree with domain experts.

For a release decision, separate zero-tolerance cases from averages. One unauthorized disclosure should not disappear inside a 98% overall score. Set thresholds before looking at results, document who adjudicates ambiguous cases and retain unsuccessful runs.

## 8. Small samples: six out of eight is not certainty

An observed pass rate is an estimate for the sampled cases, not a guarantee for all future users. A simple confidence interval illustrates sampling uncertainty, but it cannot fix an unrepresentative dataset. Eight nearly identical questions reveal little about a different language, tenant boundary or source conflict.

```
# lab: SmallSampleUncertainty
import math
def wilson(successes,total,z=1.96):
    if not 0<=successes<=total or total<=0: raise ValueError('invalid counts')
    p=successes/total; denom=1+z*z/total
    center=(p+z*z/(2*total))/denom
    radius=z*math.sqrt(p*(1-p)/total+z*z/(4*total*total))/denom
    return center-radius,center+radius
low,high=wilson(6,8)
assert low<.5 and high>.9
print('Observed 0.75; approximate 95% Wilson interval:',tuple(round(x,3) for x in (low,high)))
```

This interval assumes a suitable sampling model; correlated hand-picked development examples do not become an independent population sample merely because we computed it. Use stratified categories, fresh source families and real failure diversity. Pairwise comparisons on the same cases can help isolate model/configuration changes, but avoid repeatedly tuning to the final test set.

Interesting review exercise: one model gets all easy policy questions right but exposes a private document once. Another misses two answerable questions but never crosses the access boundary. Do not collapse that decision into a single undifferentiated accuracy percentage.



## 9. Adversarial RAG and agent boundaries

Build cases for instructions hidden in documents, forged system messages, malicious URLs, conflicting sources, stale policy dates, unsupported premises, no evidence, ambiguous entities, multilingual requests, oversized contexts and attempted cross-tenant retrieval. Test whether permissions are applied before retrieval expansion, graph summaries, caching and generation-not merely when rendering the final answer.

Treat retrieved text as data. Give tools narrow schemas and allowlisted capabilities. Validate tool arguments outside the model, obtain identity from authentication, impose step/time/cost budgets and require the application-defined approval boundary for consequential actions. A model's statement "approved" is not a human approval record.

For agent evaluation, inspect the trace: what evidence led to which action, which policy allowed it, whether retries duplicated an effect, and what survived a checkpoint restart. Separate task success from safe execution. A correct final answer does not excuse an unauthorized intermediate read.

An independent evaluation exercise should use a reviewer who did not tune the prompt, with a frozen source snapshot and a predeclared rubric. Include enough examples in each failure category to make the results useful. This library's authored cases are development evidence; an outside adjudication has not been invented.

## 10. Database detective work: reproduce the slow query

Mira sees "database slow" on a dashboard. Before creating an index, capture the query shape, bind types, data distribution, execution frequency, returned row count, concurrency, isolation level, waits and plan. Determine whether time is spent waiting for a lock, scanning rows, sorting, doing random I/O or waiting for a connection pool. An index does not solve every one of those problems.

Use a disposable schema and representative synthetic data. The supplied `labs/database-plan-workshop/mysql.sql` and `oracle.sql` create clearly named teaching tables, seed a dataset and show before/after plan procedures. They do not contain credentials or delete an existing schema. Run them only in a chosen lab schema and record the actual engine/version and output. The MySQL recipe has now run on MySQL 8.4.11, including a verified backup restore. Oracle Free 23.26.3 also passed its estimated and executed plan exercises. The reports contain raw plans and exact versions; Oracle Free is not Oracle 19c certification.

The schema models tenant, ticket ID, status, creation time and price. A candidate index `(tenant_id, status, created_at, id)` fits a particular equality-plus-ordering query. Test another query that omits status and observe how that changes available ordering/access paths. The same index need not serve both optimally.

Do not compare a cold first run of one query with a warmed tenth run of another and call the difference an index benefit. Record cache conditions, repeat controlled samples, check result equivalence and examine writes too. Indexes trade read work against storage and maintenance.

## 11. MySQL: chosen key, actual work and misleading hints

EXPLAIN describes the optimizer's selected plan. Inspect the access type, selected key, estimated rows, filtering and sorting. `possible_keys` lists candidates; it is not proof that an index was used. EXPLAIN ANALYZE on supported statements executes the statement and adds observed timing/row/loop information. [MySQL EXPLAIN reference](#).

After adding the composite index, inspect whether the intended prefix is useful for the actual predicates. A range condition can change how later key parts participate. An implicit conversion, function applied to a column or a

low-selectivity predicate can alter the chosen path. Keep statistics current and understand collation/type effects. A full scan on a small table is not automatically a defect.

Exercise sequence: run the unindexed filtered/order query; create the candidate index; gather the relevant statistics; run the same query/parameters; compare actual examined work and sorting; then remove the tenant predicate from the query without changing the index. Explain why the result may differ. Do not use FORCE INDEX as a substitute for understanding the optimizer's cost decision.

For pagination, verify that the cursor's unique tie-breaker matches the ORDER BY and index design. A query that returns ten rows may still examine millions. Small response size is not proof of a cheap database operation.

## 12. Oracle: estimates versus the executed child cursor

EXPLAIN PLAN produces a proposed plan. For actual execution analysis, identify the relevant SQL\_ID and child cursor, collect execution statistics deliberately, fetch the result fully where appropriate, and inspect DBMS\_XPLAN.DISPLAY\_CURSOR. Compare estimated and actual rows, starts, predicates and buffer work. Child cursors and bind-sensitive behavior are reasons to identify the execution precisely rather than assuming the last similar SQL text is the right one.

Gather optimizer statistics through a controlled process suitable for the table and workload. Histograms and correlated columns can matter, but adding them indiscriminately is not a universal improvement. Check stale statistics, data skew and parameter distribution before forcing a plan. [Oracle statistics guidance](#).

The workshop includes a distinctive query comment to help locate its cursor. It needs appropriate access to the diagnostic views; do not grant broad production privileges just to follow a tutorial. The plan's presence of an index is only the beginning: an inefficient index path with many row visits may be worse than a scan.

**Explain to a child:** the estimated plan is the route you draw before leaving home; execution statistics show where you actually walked and waited. A shorter line on the map is not always a shorter journey when a bridge is closed.

## 13. Transactions, migrations and database recovery

Protect business invariants at the database boundary. Unique constraints resolve races more reliably than two independent “does it exist?” checks. Understand transaction isolation for your actual engine: dirty reads, non-repeatable reads, phantoms and write-skew discussions depend on the operation and implementation. Do not transfer SQLite/H2 behavior to MySQL or Oracle without verification.

For a migration, define old/new compatibility, locking impact, expected duration, rollback/forward-repair options and monitoring. Additive schema changes can still be operationally expensive on a large table. Backfill in bounded batches with durable progress and a way to pause. Keep destructive cleanup until readers no longer need the old representation.

Replication is not a backup. An accidental deletion can replicate perfectly. A backup is not a successful restore until you have restored and verified it. Use engine-supported snapshot/backup methods rather than copying an active database file and hoping all pages represent one consistent moment.

The next exercise uses SQLite's backup API and verifies business rows in a separate file. It teaches verification mechanics; it is not a MySQL/Oracle recovery certification.

```
# lab: RestoreAndReconcile
import sqlite3, tempfile
from pathlib import Path
with tempfile.TemporaryDirectory() as folder:
    folder=Path(folder)
    live=sqlite3.connect(folder/'live.db')
```

```

live.execute('CREATE TABLE ledger(id INTEGER PRIMARY KEY,cents INTEGER NOT NULL)')
live.executemany('INSERT INTO ledger VALUES(?,?)',[(1,1200),(2,800)])
live.commit()
backup=sqlite3.connect(folder/'backup.db');live.backup(backup);backup.close()
live.execute('DELETE FROM ledger');live.commit()
restored=sqlite3.connect(folder/'restored.db');saved=sqlite3.connect(folder/'backup.db')
saved.backup(restored)
assert restored.execute('PRAGMA integrity_check').fetchone()[0]=='ok'
assert restored.execute('SELECT COUNT(*),SUM(cents) FROM ledger').fetchone()==(2,2000)
saved.close();restored.close();live.close()
print('A separate restore passed structural integrity and business reconciliation checks.')

```

Counts and sums are useful but can miss compensating errors; compare identifiers, constraints and domain-specific reconciliations too. Record which acknowledged operations occurred after the recovery point and how they are replayed or resolved.

## 14. Physical accessibility: follow a person, not a score

Meet Arun, who navigates by keyboard, and Leela, who uses a screen reader on a phone. Can they discover the chapter, understand its headings, reach code examples, submit a form, recover from an error and understand a streamed answer? A page can look attractive and pass an automated rule set while still making those journeys difficult.

Use semantic headings, landmarks, labels and native controls where possible. Keep focus visible, names meaningful and state changes understandable without relying only on color. Test zoom/reflow and touch targets in the real browser. A desktop WebKit engine is useful evidence, but not a complete substitute for Safari on an actual iPhone with its keyboard, viewport and assistive technology. [Accessibility evaluation overview](#).

Use the existing MANUAL-ACCEPTANCE worksheet to record device model, OS, browser, assistive technology, task, expected behavior, actual observation and retest. Leave unexecuted rows NOT RUN. A tutor cannot simulate a person's experience by writing “passed” into that table.

For code blocks and tables, check that scrolling is possible without trapping focus and that headers identify relationships. A visually obvious comparison can be confusing when read linearly. Explain a diagram in surrounding prose so its meaning survives when the reader cannot see the arrows.

## 15. Accessible errors, focus and streaming status

Imagine a form says “Invalid input” in red at the top while focus remains on a distant button. Leela hears nothing and submits repeatedly. Associate the error with the field, describe the repair, keep the user's valid input, and provide a predictable focus strategy for the context. Success and pending status also need to be perceivable. [WAI form notifications](#).

For a streaming AI answer, announcing every token can overwhelm a screen reader. Consider a stable status such as “Generating answer,” a way to stop, and a final completion announcement while keeping the answer available for normal navigation. Do not forcibly move focus every time content arrives. Let the user control reading rather than chasing a moving focus target.

Manual scenario: start an answer, move to another control, interrupt the connection, reconnect, then log out. Check announcements, focus, duplicated text, stale-state labels and whether protected content stops updating. Repeat at enlarged text size and on the actual mobile device. Automated axe checks and browser assertions supplement these observations; they do not replace them.

An acceptance bug report should quote the actual announcement or observed behavior, name the device/version and supply repeatable steps. “Accessibility is bad” is hard to fix; “the error is not announced and is not programmatically associated with the amount field” is actionable.

## 16. Multi-host failure: separate the failure domains

Three broker processes on one laptop can demonstrate leader election after a process kill. They cannot demonstrate surviving loss of that laptop. Separate hosts, zones, disks, networks and control-plane dependencies according to the failure you intend to test.

Build a fault matrix: process crash, slow dependency, connection reset, one-way packet loss, total partition, disk full, stale credential, corrupted state and whole-zone outage. For each, identify expected detection, availability, data safety, recovery path and evidence. Inject one well-defined fault at a time before combining failures; otherwise you may not know which assumption failed.

A distributed system can have a healthy-looking process on the wrong side of a partition. Fencing prevents an obsolete owner from continuing writes after a new owner takes over. A lease needs a clear authority, expiration model and enforcement at the resource being protected; a timestamp in memory alone may not stop an old worker's external effect.

Exercise: a worker pauses for a long garbage collection while holding a lease. Another worker acquires the expired lease. The first resumes. How will the database or external operation reject obsolete work? Discuss fencing tokens, operation IDs and downstream cooperation, not only "we use a lock."

## 17. Endurance, open-loop load and coordinated omission

A closed-loop client waits for each response before sending the next request. When the server slows down, the client sends less work. That can hide the queue growth a real independent arrival process would cause. An open-loop generator schedules arrivals independently, subject to its own explicit limits, and can reveal overload more clearly.

Record offered load, achieved throughput, concurrency, errors, timeouts, rejection counts, latency distribution and generator saturation. A generator that runs out of CPU is not measuring the server's maximum capacity. Avoid coordinated omission: if your test stops issuing requests during a stall, it may underrepresent the waiting time real users would have experienced.

For endurance, include realistic data growth, token expiry/refresh, cache eviction, log rotation, scheduled jobs, connection churn and a deployment. Sample memory, handles/file descriptors, queue age, storage and database connections. Warm-up, steady state and recovery should be distinguishable. The existing ten-minute soak is real evidence for its bounded workload; it is not a multi-day leak proof.

Longer tests require a target duration tied to the suspected failure. If a leak appears after daily rotation, a longer but still ten-minute test will never exercise the cause. Choose a workload and observation window from the hypothesis rather than simply maximizing request count.

## 18. Identity rotation: three different keys, three different problems

Separate a client secret used by an application to authenticate to an identity provider, a signing key used to sign tokens, and a data-encryption key used to protect stored values. Rotating one does not automatically rotate or revoke the others. Access tokens, refresh tokens, application sessions and provider sessions also have distinct lifecycles.

A signing-key rotation typically needs a publication/overlap strategy so valid outstanding tokens can still be verified while newly issued tokens use the new key. Clients may cache key sets; refresh behavior and unknown key IDs need bounded handling. A client-secret cutover needs provider-supported overlap or coordinated rollout. Never assume updating one secret object updates every running process and connection pool immediately.

For refresh tokens, secure issuance/storage, replay resistance and appropriate rotation or sender-constraining mechanisms matter. Revocation policy must cover compromised credentials and reuse. Match the identity-provider capabilities and client type rather than treating every token as an interchangeable JWT. [OAuth security best current practice](#).

Zero-downtime rotation acceptance should test old/new application instances concurrently, fresh and existing sessions, failed callbacks, background refresh, expired tokens, key-cache refresh, an unavailable instance, rollback and final rejection of retired credentials. Record what still succeeds during overlap and exactly when old material stops being accepted. Existing local logout tests cover a narrower, documented network-outage scenario.

## 19. Revocation across an unavailable instance

Think of a head teacher announcing that a visitor pass is cancelled. An absent classroom cannot hear the announcement immediately. A durable delivery journal helps when it reconnects, but there is still a policy question: may that classroom admit visitors while its revocation information is stale?

Options include a shared authoritative session store, short-lived access with an explicit stale-information limit, synchronous checks where availability permits, or a durable revocation feed with readiness reconciliation. Each has latency and availability costs. A queue alone does not guarantee a message is useful after its signed event expires.

Define a rejoin protocol. Before an instance receives protected traffic, establish its identity, refresh necessary trust material, reconcile relevant revocations and check its progress against an authoritative watermark. If the required information cannot be obtained, choose the documented fail-closed or limited-function behavior rather than silently accepting stale privileges.

The library's real Spring test observed a temporarily valid session at an isolated receiver, then successful revocation after durable retry. That observation is valuable because it exposes the gap instead of hiding it. It does not prove every long-outage or shared-session design is solved.

## 20. Disaster recovery: RPO, RTO and the missing last orders

RPO describes the acceptable recovery-point loss under the defined scenario. RTO describes the target time to restore the required service. They must come from business needs and a concrete recovery strategy; having backups alone does not establish either objective. [Disaster-recovery planning](#).

```
# lab: RecoveryTimeline
# Synthetic drill times, seconds since incident start.
incident=0; detected=45; restored=420; validated=540; last_recoverable_commit=-90
service_recovery=validated-incident
point_gap=incident-last_recoverable_commit
assert service_recovery==540 and point_gap==90
assert service_recovery>restored # Restore completion alone is not validated service recovery.
print('Recovery through validation:',service_recovery,'s; recoverable point lag:',point_gap,'s')
```

A warm standby, pilot-light environment and backup/restore strategy have different cost and recovery-time tradeoffs. Include dependency configuration, certificates, secrets, deployment artifacts, schemas, queues and network routes in the plan. Restoring only the main database may leave background work inconsistent.

Reconcile external effects after restore. A bank may have captured a payment after your last recoverable database commit. Blindly replaying every “missing” payment can duplicate charges. Use provider operation IDs and durable audit evidence to resolve ambiguity. Fence the old primary before promoting a new writer, and test the eventual fallback path too.

A recovery drill ends when the defined user journey and data reconciliation are accepted, not when a storage console says “restore complete.” Record detection, decision, restoration, validation and traffic-switch times separately.

## 21. Incident command, runbooks and post-incident learning

During an incident, establish one coordination channel, a decision owner, a timeline and a bounded mitigation plan. Separate investigation from repeated uncontrolled changes. Preserve useful evidence while protecting sensitive data. Communicate impact and uncertainty plainly: “checkout fails for this population” is more useful than “everything is unstable.”

A runbook should state symptoms, prerequisites, commands or actions, expected observations, stop conditions, rollback, validation and escalation. Avoid commands that delete broadly named resources. Scope actions to known owned infrastructure and verify the target environment. If a rollback cannot restore schema compatibility, say so before it is needed.

Afterward, explain contributing conditions and why existing safeguards did not catch them. Assign improvements with owners and verification criteria. “Engineer was careless” rarely explains why one ordinary action could cause broad failure. A useful correction might be a safer default, an invariant check, better observability or a rehearsed recovery procedure.

Mira's example action: “Add an integration test that verifies the payment status after a lost upstream response using the original operation ID.” That is more testable than “make retries safer.”

## 22. GitHub CI: a green check answers a specific question

The hosted run for commit bc63933215c492cfe2e2a9edd2f2cd1e87111443 completed successfully: [recorded GitHub run](#). This closes the earlier “latest CI not checked” item for that commit. A future commit has its own status; do not transfer a green result across changed code.

The workflow builds and tests defined paths. It does not certify public cloud, every physical browser, MySQL/Oracle plans or personal interview skill. Preserve exact commit, environment, job status and relevant reports. A skipped job is not a passed test; an allowed failure should not be summarized as unconditional success.

Keep test stages purposeful: quick deterministic checks first, then component integrations, then expensive end-to-end or deployment checks where justified. Do not give untrusted pull-request code production deployment credentials. Pin and review dependencies/actions according to your maintenance policy, and scope job permissions to what each step needs.

The new load-balancing lab is also suitable for CI on a Linux runner with nginx installed. A passing local report remains local evidence until the corresponding hosted job has actually run.

## 23. A practice programme that reveals understanding

Reading recognition feels easier than recall. Close the notebook and explain the mechanism in your own words. Then change one assumption: the retry arrives concurrently, the cursor's sort key changes, the model has no evidence, or the identity receiver has been offline longer than token validity.

Use four rounds: a 15-minute explanation round, a 30-minute coding round, a 30-minute debugging round and a 45-minute system-design round. Grade correctness, invariant preservation, failure reasoning, test quality and communication separately. Keep your original unaided answer, elapsed time, feedback and a fresh follow-up answer. Repeating the same memorized question is not the same as transfer.

Example coding round: implement a tenant-scoped idempotency record with payload conflicts and tests.  
 Debugging round: a transaction annotation fails only through one call path-trace the proxy boundary. Design  
 round: safely process ticket payments with bursts, restarts, duplicated broker delivery and uncertain remote  
 responses. Explain why your chosen consistency boundary is sufficient.

Personal readiness remains ungraded until you submit answers. You can use the following scoring sheet  
 yourself, but a completed reading checkbox does not assign points automatically.

## 24. A scored launch-room challenge

You have three application instances, an asynchronous payment provider, a database, a RAG assistant and live  
 ticket-status streams. Demand doubles while one zone fails. An identity secret is being rotated and the last  
 backup is ninety seconds old. Design the incident response and the safe next deployment.

| Criterion           | Full-credit reasoning                                                                       | Points |
|---------------------|---------------------------------------------------------------------------------------------|--------|
| Traffic/capacity    | Distinguishes existing connections, queues, eligible capacity and overload rejection        | 4      |
| Payment correctness | Keeps operation identity; reconciles ambiguous provider effects; fences obsolete writers    | 5      |
| State/recovery      | States RPO/RTO boundary; restores and reconciles; does not confuse replicas with backups    | 4      |
| Identity            | Distinguishes key types, overlap, revocation lag and rejoin checks                          | 4      |
| AI                  | Uses authorized evidence, justified abstention, independent evaluation and visible fallback | 4      |
| Database            | Examines actual query/plan/waits; checks migration compatibility                            | 3      |
| Accessibility       | Includes keyboard/screen-reader recovery and controlled status announcements                | 3      |
| Operations          | Names stop conditions, owner, evidence and rollback limits                                  | 3      |

Total 30. A practice target is 24 with no unauthorized-data or duplicate-payment safety error. The answer need  
 not use every product mentioned in the library. A simpler architecture with explicit boundaries is stronger than a  
 complicated one whose failure behavior cannot be explained.

Answer outline: reduce nonessential load; preserve status visibility; use existing durable operation identities;  
 avoid promoting competing writers; check actual recovery-point lag; reconcile the unavailable zone's identity  
 state before admission; pause unsafe rotation/cutover changes; provide source-only or unavailable AI output  
 where needed; keep interaction errors accessible; validate restored data and core journeys before reopening  
 traffic; document what remains uncertain.



## 25. Reading coverage versus acceptance work

| Previously pending area | Detailed learning coverage now                                                                                            | Evidence still required for final acceptance                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Load balancing          | Notebook 32: layers, algorithms, queues, health, sessions, TLS, streams, Kubernetes, draining, global failure and metrics | Provider-specific/multi-host/TLS checks beyond the scoped local NGINX report |
| Cloud deployment        | Lessons 2-5: topology, identity, cost, infrastructure and acceptance gates                                                | Authorized account/project, region, budget and actual deployment             |
| AI accuracy             | Lessons 6-9: case design, denominators, confidence, adversarial cases and agent traces                                    | Fresh independent adjudication; existing 6/8 failures remain                 |
| Accessibility           | Lessons 14-15: human journeys, forms, focus, streaming and recorded manual checks                                         | Actual devices and assistive technology observations                         |
| MySQL / Oracle          | Lessons 10-13 plus separate engine-specific scripts                                                                       | Execution on those engines with recorded actual plans                        |
| Production reliability  | Lessons 16-21: failure domains, endurance, rotation, revocation, disaster recovery and incidents                          | Target-environment fault and restore evidence                                |
| Interview readiness     | Lessons 23-24 plus earlier workbooks                                                                                      | Your unaided responses, grading and a fresh follow-up round                  |
| GitHub CI               | Lesson 22 and the linked successful prior run                                                                             | Each new commit's own hosted result                                          |

You can now study these topics without chasing another book to understand the basic mechanism, tradeoffs and exercises. Optional official links support version checks; executing external systems still requires their runtimes and access. No finite notebook can include every future research result or guarantee every interview. The useful target is clear reasoning, reproducible experiments and honest evidence.

### Read the latest drill results

The [acceptance runbook](#) connects these lessons to actual database, secure WebSocket, process-crash, secret-rotation, evaluation and accessibility experiments. In the MySQL run, the original query inspected a 1,000-row table and sorted matches. The composite index supplied the requested order and let the query stop after 20 returned rows. Removing the status equality brought sorting back. This is the difference between memorizing that indexes are fast and explaining why a particular query benefits.

The accessibility drill found that visually scrollable code was not keyboard-focusable. The shared renderer now exposes these regions to keyboard users and shows focus clearly. We test arrow-key scrolling and Tab leaving the region. A zero automated-violation count still needs the physical-device observations from the manual worksheet.

The new 16-case model evaluation passed 12. One role-spoofing example returned the attacker-requested value. That is a concrete reason to keep the default extractive path and keep model-selected actions constrained; more polished prose does not repair a trust boundary. The externally labelled evaluation uses a separate fixed SQuAD subset and keeps its own denominator. Its first completed run passed 5 of 12, including three timeouts and four incorrect answers to unanswerable questions. That small public subset is not a representative benchmark or an estimate of overall model accuracy.



## A rejected draft must never reach the reader

The optional model adapter now buffers its bounded draft until completion and citation-format checks pass. Invalid citations, interrupted streams and malformed events no longer leave partial model text in the answer. A provider failure produces explicitly labelled, authorized source excerpts. This trades immediate token display for the ability to withhold a failed draft. A valid citation can still accompany a false statement, so released drafts visibly require factual review. The [acceptance runbook](#) explains the eight regression tests and the separate controlled Spring session-race experiment. Neither mechanical validation nor a reproduced possible failure mechanism proves general model accuracy or the historical login root cause.

## A bookmark cannot stop you paying twice

Imagine a bookmark showing where you stopped reading a recipe. That bookmark is a checkpoint. If the next instruction is “pay the shop,” reopening the book does not tell the shop whether it already received your money. An idempotency key is a receipt number the shop recognizes: the same payment request with the same number must not create a second charge. The bookmark and receipt solve different problems.

Our real model once said a checkpoint “ensures” idempotent effects, despite citing a source that required separate keys. The citation was valid; the claim was wrong. The adapter now explicitly asks the model to preserve limitations and distinguish separate safeguards. That is a prompt improvement, not a mathematical guarantee. The source-only mode remains the default.

Practice the same reasoning elsewhere: replication copies accidental deletions too, so it does not replace historical backups. Encryption keeps a message private, while authorization decides who may act. A cache TTL says nothing about a payment timeout unless the evidence explicitly connects them. Read the [live semantic regression report](#) for the actual answers and review criteria. These are development cases, not an independent benchmark.

## Deep workshop - Replace a green checklist with an evidence ledger

“Tested” must name what ran, where, and what was observed. Three local processes are useful evidence, but not equivalent to independent-machine, zone, or provider failure.

### State one claim precisely

Claim: worker crash does not duplicate a charge. Setup: one API, durable job store, and idempotent receiver. Intervention: kill the worker after charge commit but before completion save. Observation: recovery returns the same receipt and one charge exists. Limitation: shared database failure and regional partition were not tested.

### Separate four labels

A design explains a mechanism. A runnable example supplies an experiment. A passing test establishes assertions in its setup. Production acceptance checks agreed requirements in the actual environment. Moving a recipe directly to “verified” skips the necessary evidence.

### Apply this to AI

Define questions and criteria before reviewing predictions. Keep failures, timeouts, abstentions, and malformed results in the denominator. Tuning on those questions makes them development cases; fresh independent evaluation is needed for generalization.

### Oral exercise

Explain one failure, the smallest causal fix, the regression that catches it, and a remaining uncertainty. **Answer standard:** Clear boundaries make the accomplishment more credible. Recognizing unsupported claims is part of engineering expertise, not lack of confidence. Human device testing and your unaided interview answers cannot be inferred from automated success.

# Experiments without sliders

These examples provide the essential reasoning of the HTML experiments. PDF and EPUB do not execute browser code. Notes and progress are not synchronized across formats.

## Predict a gradient step

For points (1,2) and (2,4), start  $w=0$  and  $b=0$  in  $\text{prediction}=w*x+b$ . Mean squared loss is 10. The gradients are -10 and -6. At learning rate 0.1,  $w$  becomes 1 and  $b$  becomes 0.6. Predictions become 1.6 and 2.6; loss becomes 1.06. At rate 0.6,  $w=6$  and  $b=3.6$ ; predictions are 9.6 and 15.6, so loss is 96.16. A larger step can overshoot. Neither result establishes performance on unseen data.

## Trace binary search

Array [2,4,4,9,13], target 4. Maintain [lo,hi). Start [0,5). Midpoint 2 has value 4: keep it by setting  $hi=2$ . Midpoint 1 has 4: set  $hi=1$ . Midpoint 0 has 2: set  $lo=1$ . Result is index 1. For target 14 the result is index 5, a no-match position; do not dereference it.

## Predict a retry

The worker stores pending operation PAY-17. The receiver commits one payment. The worker crashes before recording completion, then retries PAY-17. With atomic receiver deduplication, it receives the old receipt and charges remain 1. Without deduplication, charges can become 2. A workflow checkpoint is not a payment receipt.

## Calculate average in-flight work

For a stable system,  $L=\lambda*W$ . At 80 arrivals per second and average time 0.25 seconds, average in-flight work is 20. At 0.5 seconds it is 40. This average accounting identity is not a prediction of latency percentiles or proof of stability.

## Seven questions before moving on

1. What problem does this solve? Give an everyday example.
2. Can I define each prerequisite and technical term?
3. Can I trace the inputs, intermediate states, and outputs?
4. Why does it work, and under which assumptions?
5. Can I explain implementation, edge cases, failures, and debugging?
6. When is an alternative better?
7. Can I solve a fresh exercise and explain the answer without hints?

The checklist is an acceptance standard, not proof that every topic has already passed editorial review. Executable lab results, classroom examples, and production acceptance are different evidence categories. This edition preserves those distinctions.